Blockchain Technologies

By E.M.Roopa Devi

Unit-I introduces the basic concepts of **distributed computing** based on which blockchain technology works.

It also covers history, definitions, features, types and benefits of blockchains along with **consensus mechanisms** that are at the core of blockchain technology.

Unit-II **Decentralization** covers the concepts of decentralization and its relationship with blockchain technology. Various methods and platforms that can be used to decentralize a process or system have also been introduced.

Cryptography and Technical Foundations introduces the theoretical foundations and cryptography which is necessary to fully understand blockchain technology. Concepts such as public and private key cryptography with practical examples are included. Finally, an introduction to financial markets is also included as there are many interesting use cases for blockchain technology in the financial sector.

Unit-III **Bitcoin** is the first and largest blockchain and introduces technical concepts related to bitcoin cryptocurrency in detail.

Alternative coins introduces alternative cryptocurrencies that were introduced after the invention of Bitcoin. It also presents examples of different altcoins, their properties and how they have been developed and implemented.

In-depth discussion on history, definition of **smart contracts**, Ricardian contracts, Oracles and the theoretical aspect of smart contracts are provided.

Unit-IV **Ethereum 101** introduces the design and architecture of the Ethereum blockchain in detail. It covers various technical concepts related to the Ethereum blockchain that explains the underlying principles, features and components of this platform in depth.

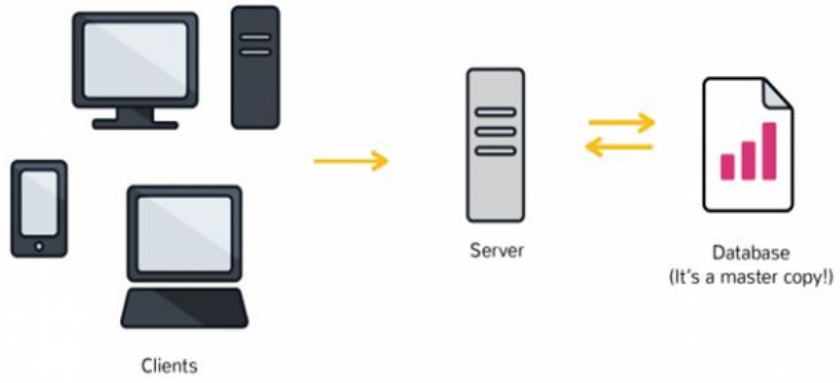
Ethereum Development provides a detailed practical introduction to development of decentralized applications and smart contracts using the Ethereum blockchain.

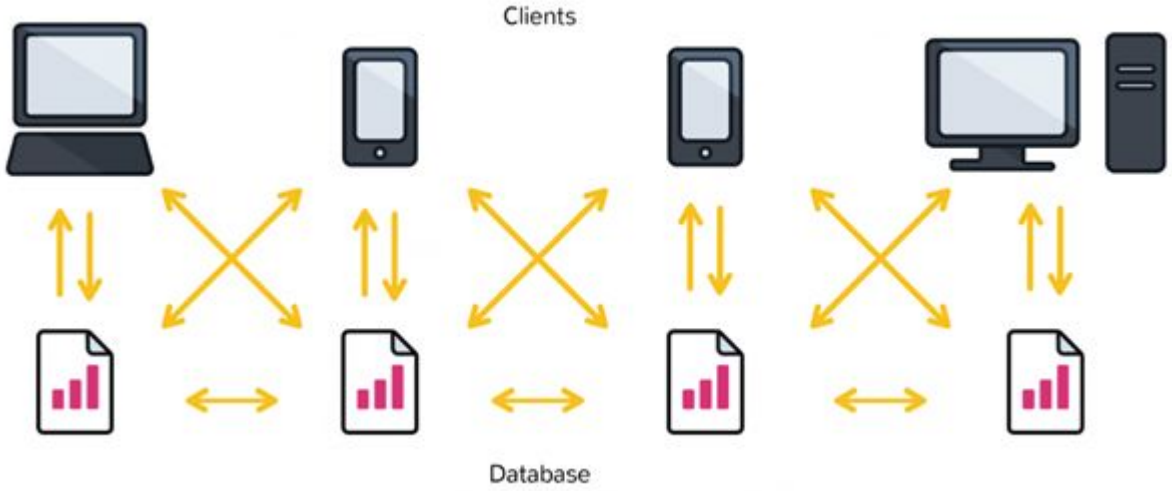
Unit-V presents a discussion about the **hyperledger** project from the Linux foundation, which includes different blockchain projects introduced by its members.

Blockchain-Outside of Currencies provides a practical and detailed introduction to applications of blockchain technology in fields other than cryptocurrencies including **Internet of Things**, government, media and finance.

UNIT-I

- **Distinction between databases and blockchain ledgers**
  - *It begins with architecture*

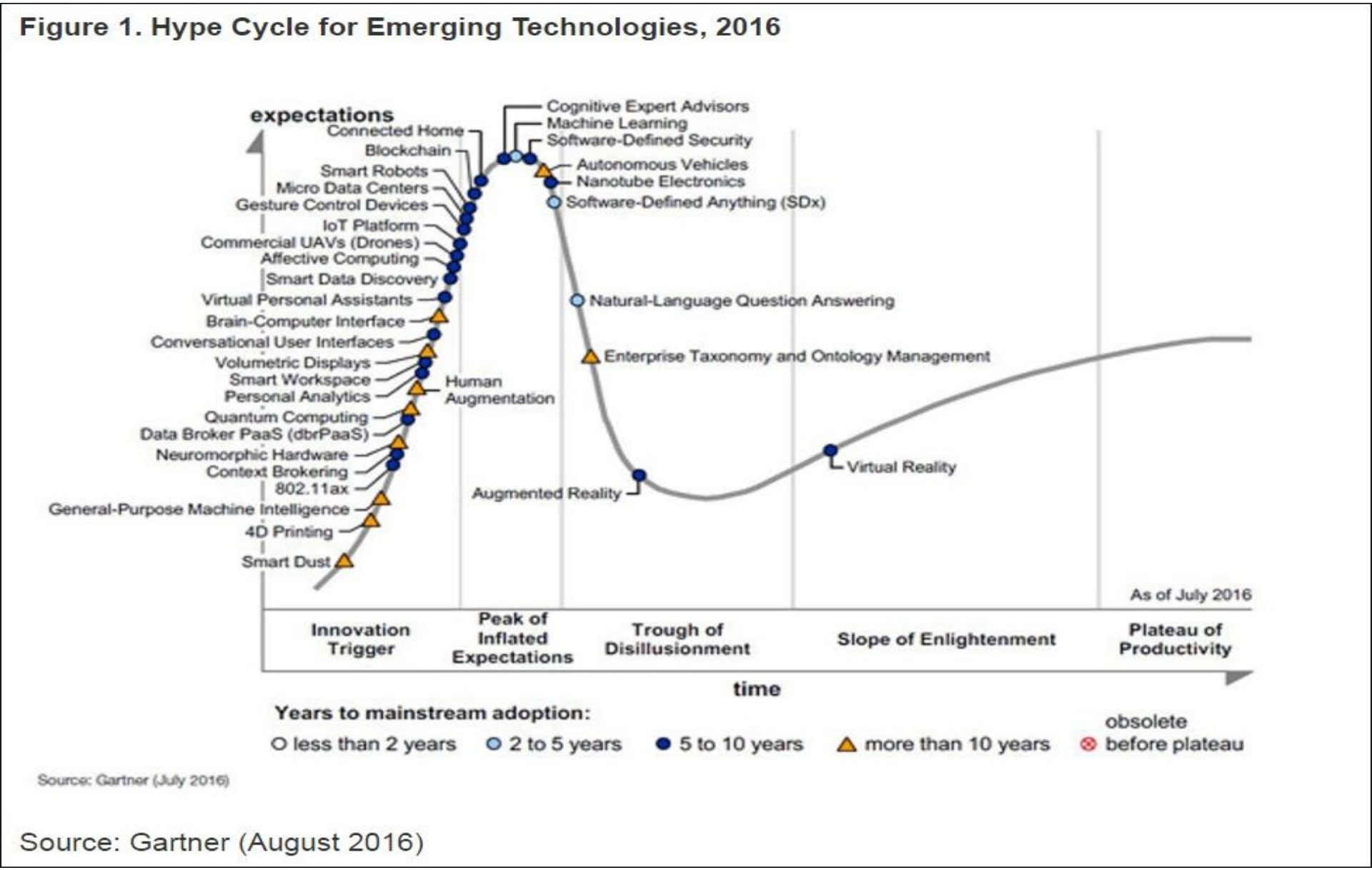




Distinction between databases and blockchain ledgers

Databases have admins & centralized control	No on is the admin or in-charge
Only entities with rights can access database	Anyone can access (public) blockchain
Only entities entitled to read or write can do so	Anyone with right proof of work can write on the blockchain
Databases are fast	Blockchains are slow
No history of records & ownership of digital records	History of records & ownership of digital records

Gartner's hype cycle for emerging technologies



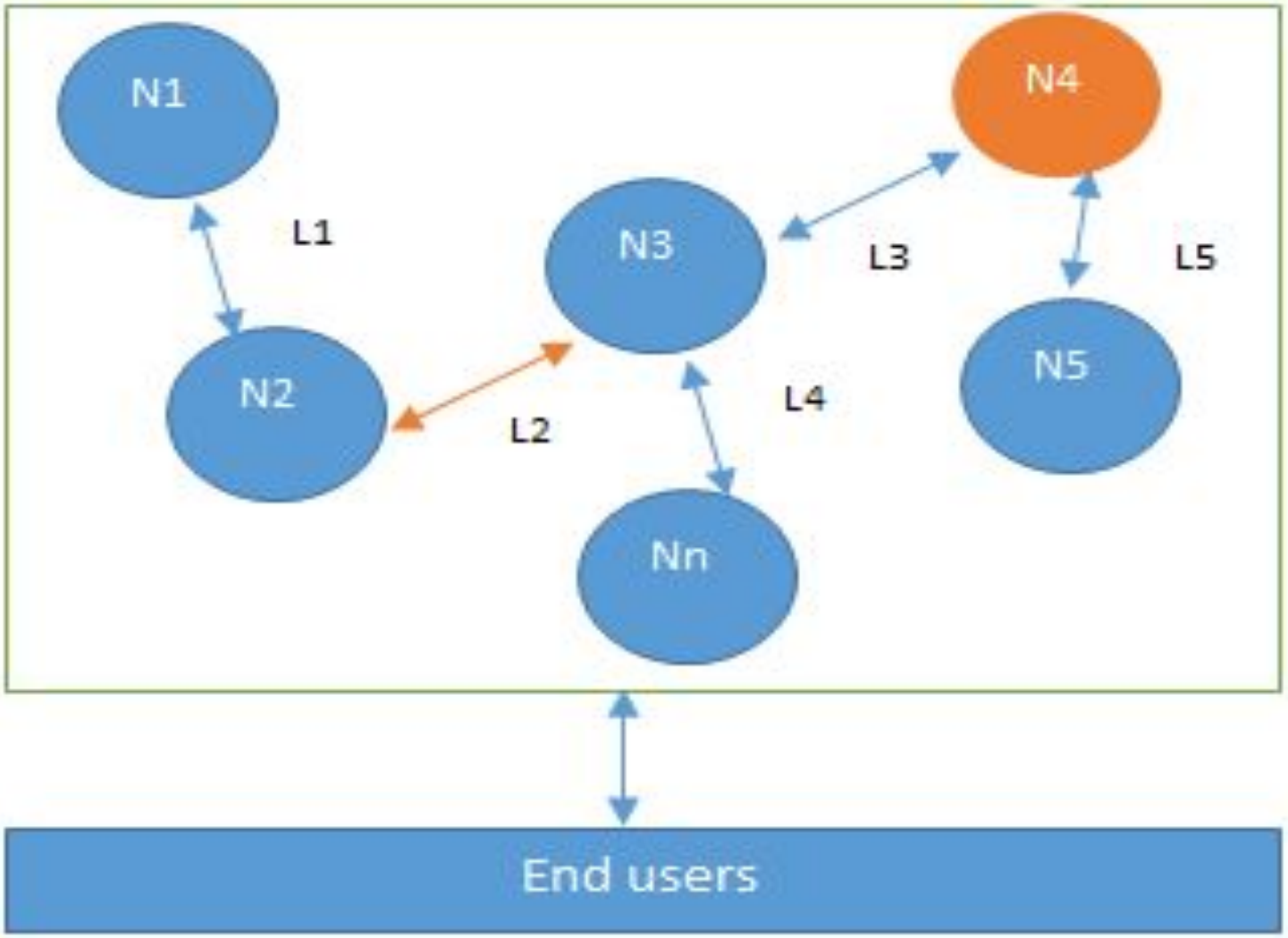
Source: Gartner (August 2016)

- Benefits of Blockchain technology are decentralized trust, cost savings, transparency and efficiency.
- Challenges are scalability and privacy

Distributed systems

- Blockchain is a decentralized distributed system
- Distributed systems are a computing paradigm whereby two or more nodes work with each other in a coordinated fashion in order to achieve a common outcome and it is modeled in such a way that end users see it as a single logical platform

Design of a Distributed System



- Any unexpected behaviour of a node on the network can be categorized as Byzantine
- The main challenge in distributed system design is coordination between nodes and fault tolerance
- Another challenge that CAP states that a distributed system cannot have all desired properties (consistency, availability and partition tolerance) simultaneously

CAP theorem

- Consistency is a property that ensures that all nodes in a distributed system have a single latest copy of data
- Availability means that the system is up, accessible for use and is accepting incoming requests and responding with data without any failures as and when required
- Partition tolerance ensures that if a group of nodes fails the distributed system still continues to operate correctly

- To achieve fault tolerance, replication is used.
- Consistency is achieved using consensus algorithms to ensure that all nodes have the same copy of data. This is also called state machine replication.

  Faulty nodes are of two types:
- One is simply crashed and another is malicious or performing inconsistent behaviour which is difficult to detect and correct.

Byzantine Generals problem

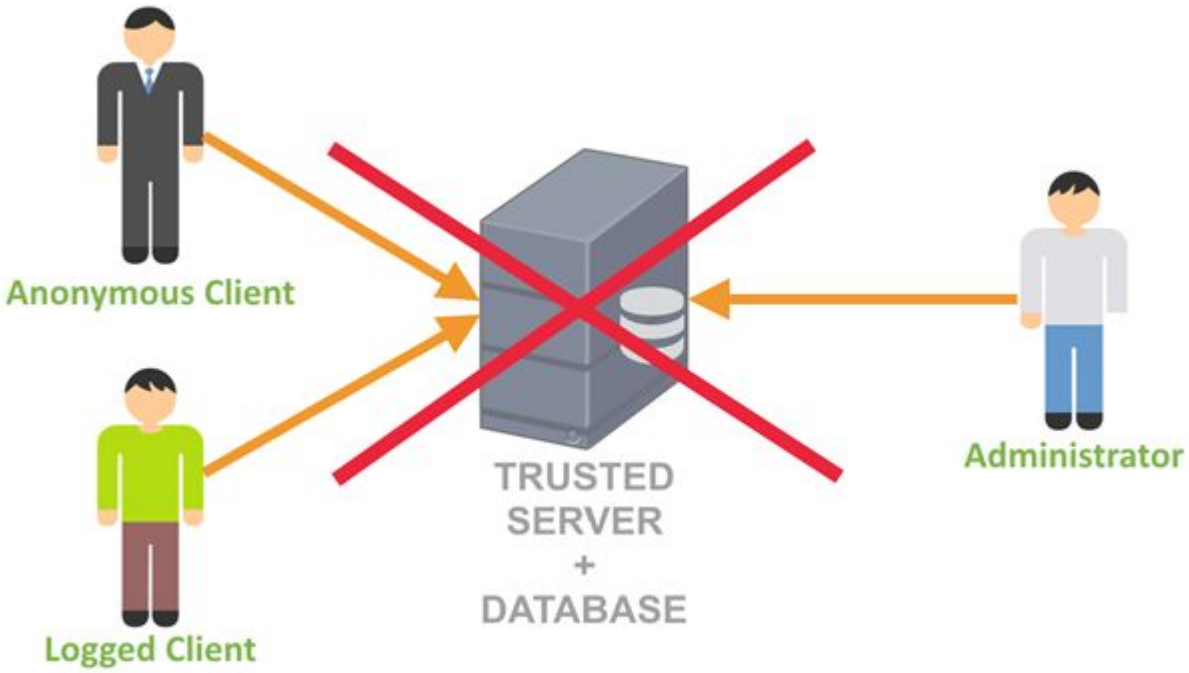
- Nodes are generals, Byzantine nodes are traitors and communication channel between the nodes is messenger
- Practical Byzantine Fault Tolerance (PBFT) algorithm is used to perform agreement between the generals in the presence of traitors
- Proof of Work (PoW) algorithm was developed as a mechanism to achieve consensus

Consensus

- Consensus is a process of agreement between distrusting nodes on a final state of data
- Agreement between two nodes (client-server systems)
- Agreement between multiple nodes (distributed systems)
- Achieving consensus between multiple nodes is known as distributed consensus

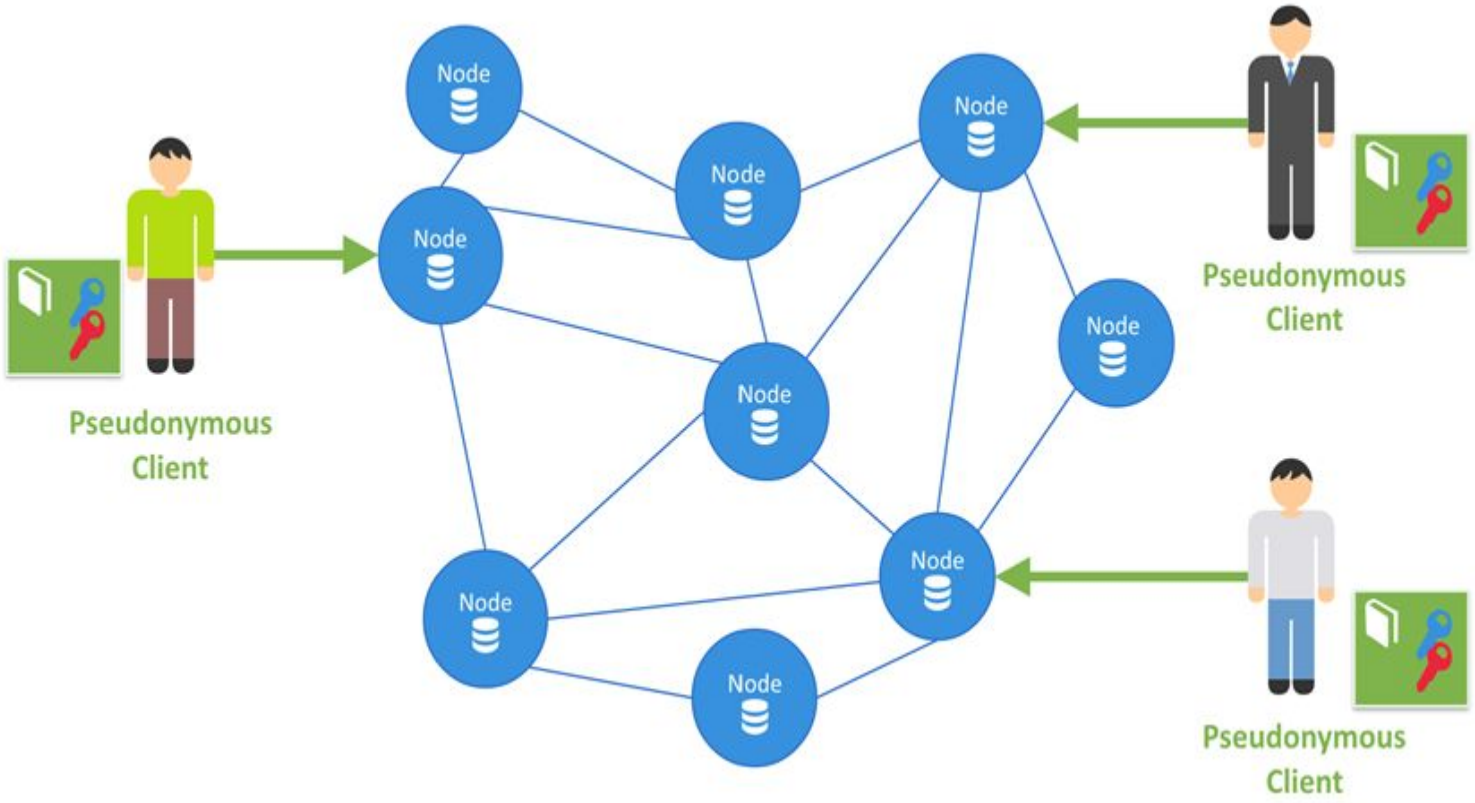
Consensus components

- **Blockchain structure**
  - No more client/server architecture with name roles



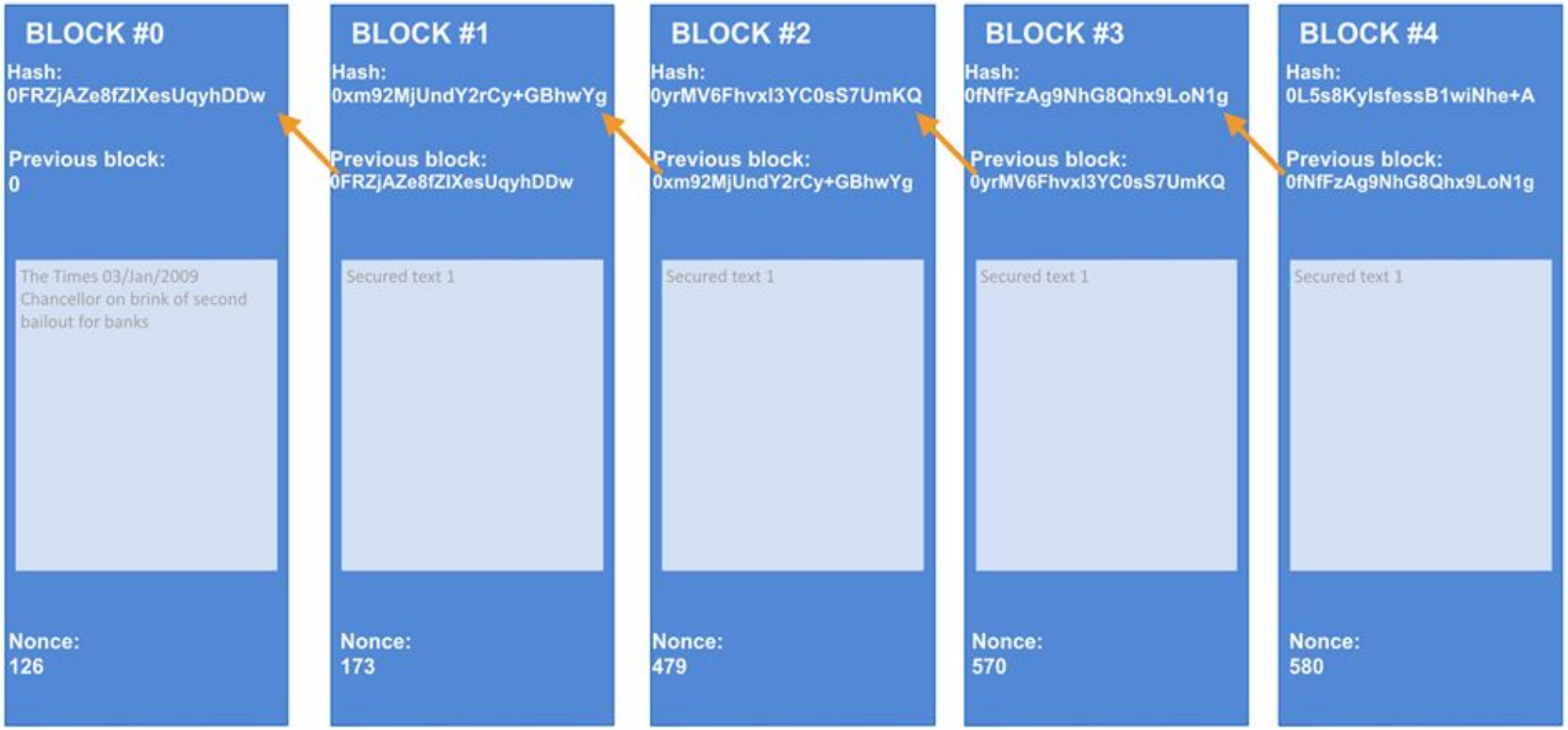
Consensus components

- **Blockchain structure**
  - Peer-to-peer Architecture with pseudonymous client bearing key pairs. Each node as a database copy.



Consensus components

- **Blockchain structure**
  - Data structure:



Consensus components

- **Blockchain structure**
  - Blocks of data:

```
yallet@tyler:~/.bitcoin/blocks$ find . -name 'blk*.dat' -mtime -7 -ls
26610095 130688 -rw-------   1 yallet   yallet   133819048 Nov 23 20:37 ./blk00688.dat
26610563 130556 -rw-------   1 yallet   yallet   133682935 Nov 25 16:30 ./blk00690.dat
26611820 130992 -rw-------   1 yallet   yallet   134128511 Nov 24 17:53 ./blk00689.dat
26609041 131076 -rw-------   1 yallet   yallet   134217422 Nov 22 21:51 ./blk00687.dat
26610902 130840 -rw-------   1 yallet   yallet   133975212 Nov 21 20:41 ./blk00686.dat
26612258 130460 -rw-------   1 yallet   yallet   133583976 Nov 26 13:46 ./blk00691.dat
26611825 114692 -rw-------   1 yallet   yallet   117440512 Nov 28 09:34 ./blk00693.dat
26611491 130112 -rw-------   1 yallet   yallet   133230159 Nov 27 14:49 ./blk00692.dat
yallet@tyler:~/.bitcoin/blocks$ hexdump -C blk00691.dat | head -n 15
00000000  f9 be b4 d9 53 3a 0f 00  00 00 00 20 f3 48 e2 80  |....S:..... .H..|
00000010  bb 89 03 22 dd e9 93 ad  9e bc fd 7e 53 14 45 7a  |..."........~S.Ez|
00000020  b5 f2 97 00 00 00 00 00  00 00 00 00 1f 5b e2 c0  |.............[..|
00000030  d1 7d cb 96 9a 37 86 21  c4 a8 af 5a ad a0 ad 0b  |.}...7.!...Z....|
00000040  b2 d2 ef 15 75 c3 3a c6  67 6e 46 0e de 58 38 58  |....u.:.gnF..X8X|
00000050  d4 e6 03 18 3e c5 4e e3  fd 45 0b 01 00 00 00 01  |....>.N..E......|
00000060  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000080  ff ff ff ff 49 03 d0 b8  06 2f 48 61 6f 42 54 43  |....I..../HaoBTC|
00000090  2f e7 94 bb e5 9b be e7  9c 81 e8 af 86 e6 98 a5  |/...............|
000000a0  e9 a3 8e e9 9d a2 ef bc  8c e7 8e af e4 bd a9 e7  |................|
000000b0  a9 ba e5 bd 92 e6 9c 88  e5 a4 9c e9 ad 82 e3 80  |................|
000000c0  82 2f 06 74 7d 3d e3 b3  1d 9c f7 99 01 00 ff ff  |./.t}=..........|
000000d0  ff ff 01 4b 1d d3 4e 00  00 00 00 19 76 a9 14 bf  |...K..N.....v...|
000000e0  d3 eb b5 48 5b 49 a6 cf  16 57 82 46 23 ea d6 93  |...H[I...W.F#...|
yallet@tyler:~/.bitcoin/blocks$
```

Consensus Mechanisms

- A consensus mechanism is a set of steps that are taken by all or most nodes in order to agree on a proposed state or value
- The various requirements which must be met in order to provide the desired results in a consensus mechanism are:
- **Agreement:** All honest nodes decide on the same value
- **Termination:** All honest nodes terminate execution of the consensus process and eventually reach a decision

- **Validity:** The value agreed upon by all honest nodes must be the same as the initial value proposed by atleast one honest node
- **Fault tolerant:** The consensus algorithm should be able to run in the presence of faulty or malicious nodes
- **Integrity:** This is a requirement whereby any node make decisions only once in a single consensus cycle

Types of Consensus mechanism

- Byzantine fault tolerance-based
- Leader-based consensus mechanisms
- Paxos nodes are assigned various roles such as Proposer, Acceptor and Learner
- RAFT assigned with three states, Follower, Candidate or Leader to the nodes.

History of Blockchain

Bitcoin-2008(Satoshi Nakamoto)

Blockchain-2009

Electronic cash

- Theoretical concepts of consensus algorithms in distributed systems provides the basis for practical implementation of Proof of Work algorithms in bitcoin
- Different electronic cash schemes paved the way for invention of cryptocurrencies, specifically bitcoin

The concept of Electronic Cash

- Issues in e-cash - accountability and anonymity addressed using cryptographic operations, blind signatures and secret sharing
- Blind signatures allow signing a document without actually seeing it
- Secret sharing is a concept that allows the detection of using the same e-cash token twice (double spending)
- Protocols such as Chaum, Fiat and Naor (CFN) for detection of anonymity and double spending

- Brand’s e-cash system improved on CFN, more efficient and introduced the concept of security reduction to prove statements about e-cash scheme
- Security reduction is a technique used in cryptography to prove that a certain algorithm is secure by using another problem as a comparison.
- Hashcash is a PoW system introduced to control e-mail spam.

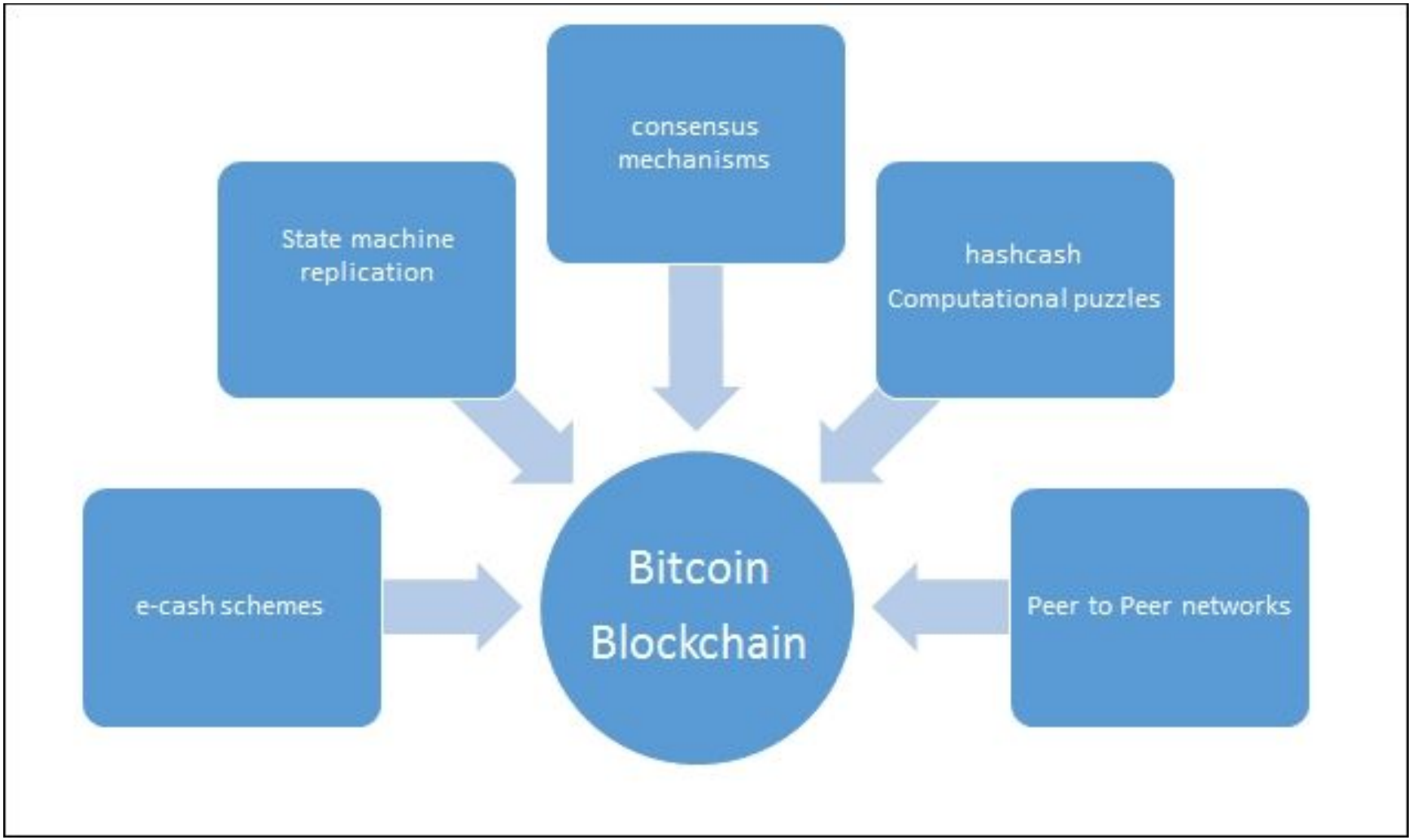
- if legitimate users want to send e-mails, then compute a hash as a proof thus a reasonable amount of computing resources is spent before sending the e-mail.
- Eventhough generating hashcash is a compute intensive process, it is useful for legitimate user as they normally send less number of messages.
- But if the spammer sends the e-mails, usually thousands in number, it is infeasible to compute hashcash for all e-mails, thus making the spamming effort expensive, used to overcome e-mail spamming.
- Hashcash takes a considerable amount of computing resources to compute but is easy and quick to verify.

- Verification is performed by the user who receives the e-mail.
- Hashcash is popularized by its use in the bitcoin mining process.
- This idea of using computational puzzles or pricing function to prevent e-mail spam was introduced in 1992.
- Pricing function was the name given to the hard functions that are required to be computed before access to a resource can be granted.
- Later, Adam Back invented hashcash independently in 1997, which introduced the usage of computing hash functions as PoW.

- In 1998, b-money was introduced by Wei Dai proposed the idea of creating money through solving computational puzzles such as hashcash.
- It is based on a peer-to-peer network where each node maintains its own list of transactions.
- Another idea by Nick Szabo called BitGold was introduced in 2005 and also proposed solving computational puzzles to mint digital currency.
- In 2005, Hal Finney introduced the concept of cryptographic currency by combining ideas from b-money and hashcash puzzles but it still relied on a centralized trusted authority.

- There were multiple issues range from no clear solution of disagreements between nodes to rely on a central trusted third party and trusted timestamping.
- In 2009, bitcoin introduction solves the problem of distributed consensus in a trustless network. It uses public key cryptography with hashcash as PoW to provide a secure, controlled and decentralized method of minting digital currency.
- The key innovation is the idea of an ordered list of blocks composed of transactions and cryptographically secured by the PoW mechanism
- Thus various concepts from e-cash schemes and distributed systems were combined together to invent bitcoin known as blockchain.

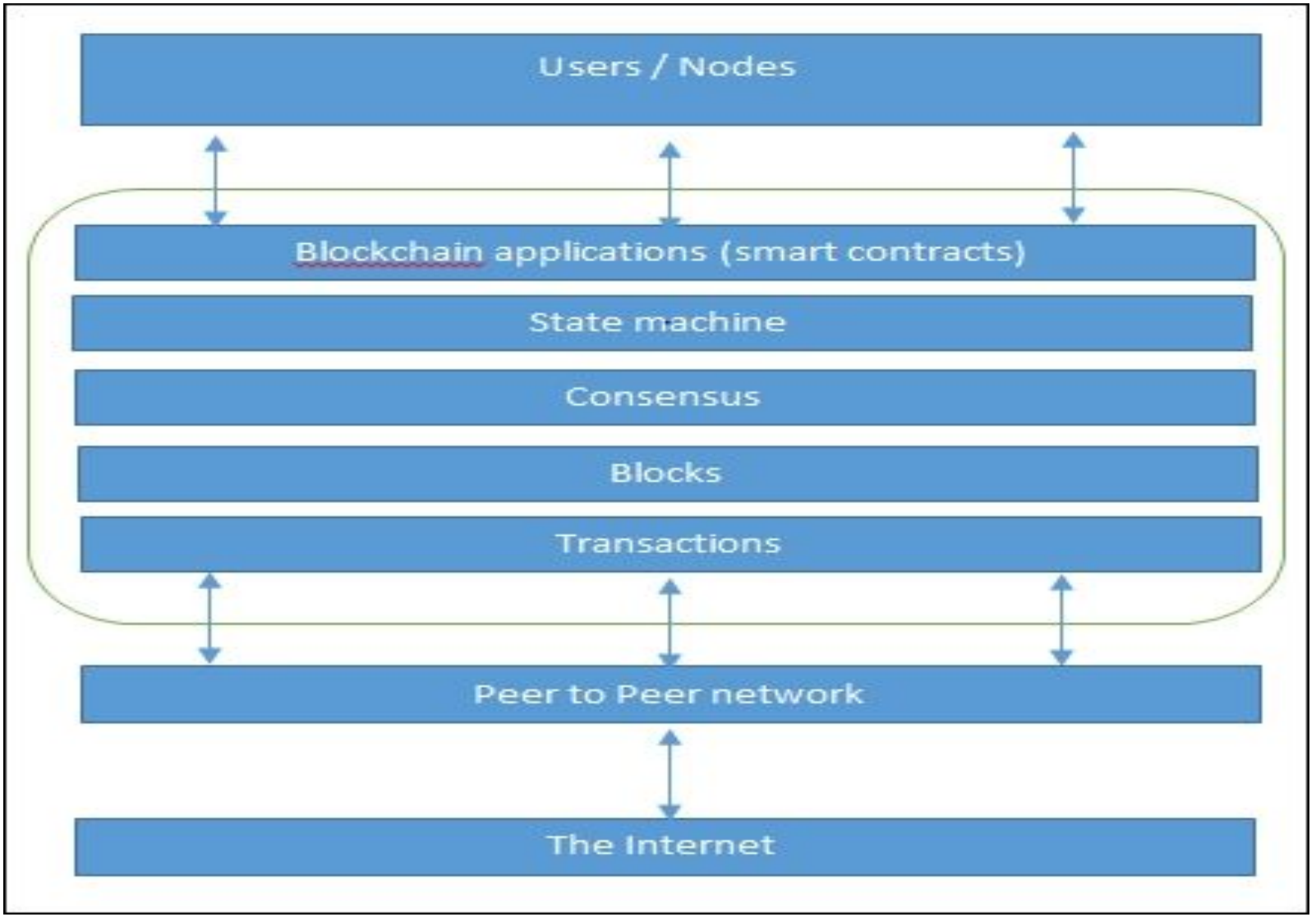
Invention of bitcoin blockchain using various concepts



Introduction to blockchain

- Blockchain is a peer-to-peer distributed ledger that is cryptographically secure, append-only, immutable and updateable only through consensus or agreement among peers.
- Blockchain is a layer of a distributed peer-to-peer network running on top of the Internet similar to SMTP, HTTP or FTP running on top of TCP/IP

Network view of a blockchain



- From a business point of view a blockchain can be defined as a platform whereby peers can exchange values using transactions without the need for a central trusted arbitrator.

- This allows blockchain to be a decentralized consensus mechanism where no single authority is in charge of the database.

- A block is a selection of transactions bundled together in order to organize them logically. It is made up of transactions and its size is variable depending on the type and design of the blockchain in use.



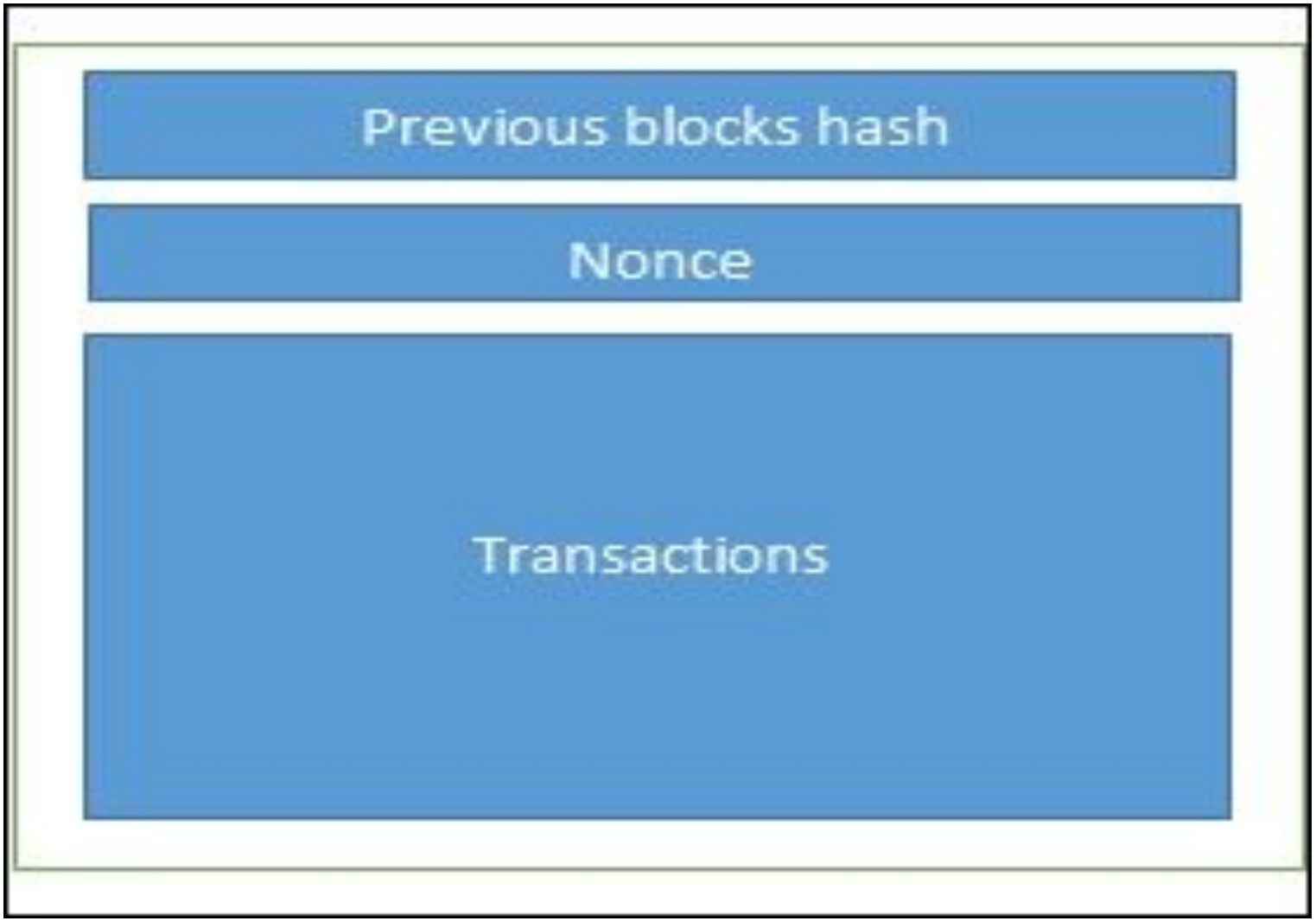
Source: KPMG Research

- A reference to a previous block is also included in the block unless it is a genesis block.

- A genesis block is the first block in the blockchain that was hardcoded at the time the blockchain was started.

- The structure of a block is also dependent on the type and design of a blockchain, but there are few attributes essential to the functionality of a block such as the block header, pointers to previous blocks, timestamp, nonce, transaction counter, transactions and other attributes.

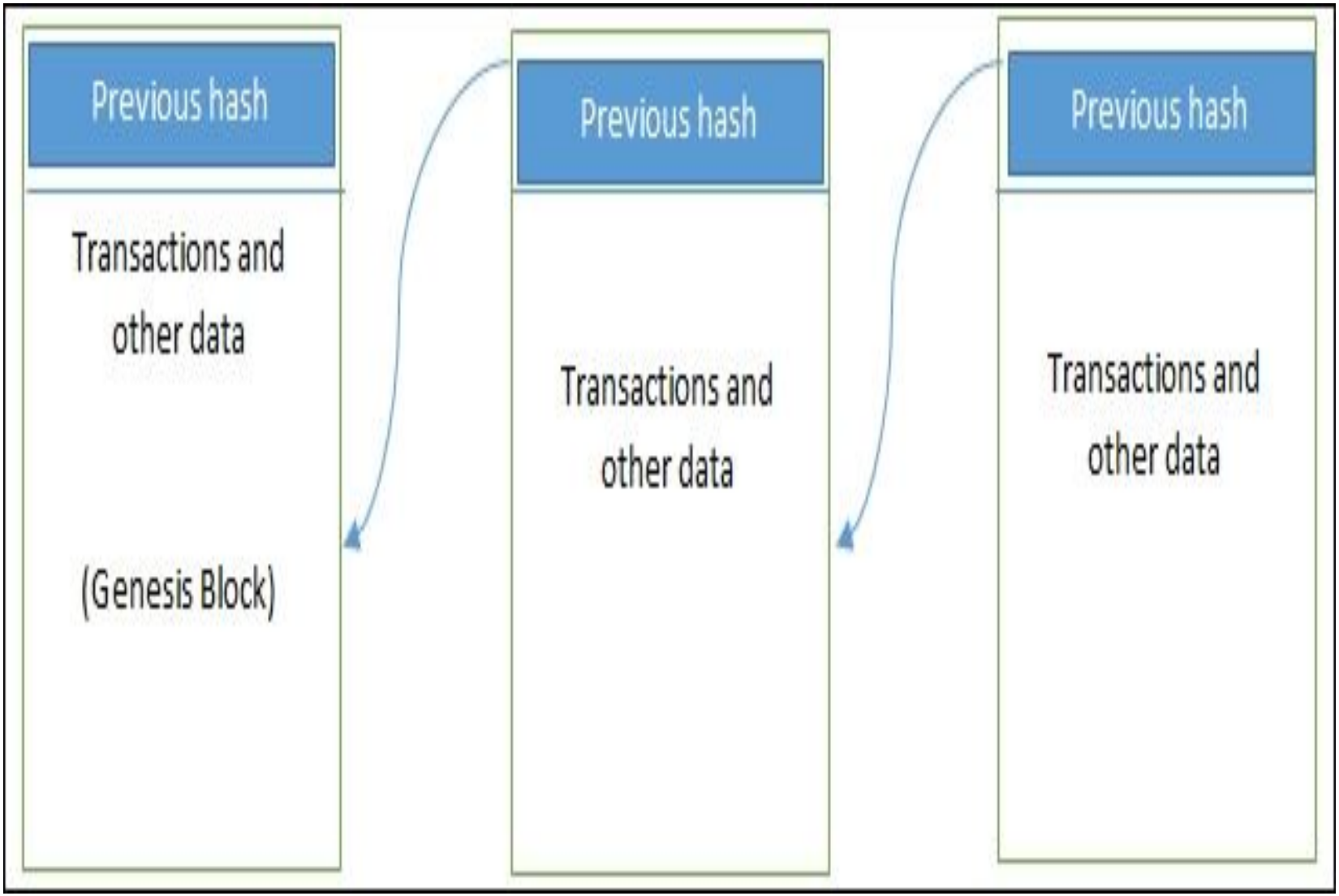
Structure of a block

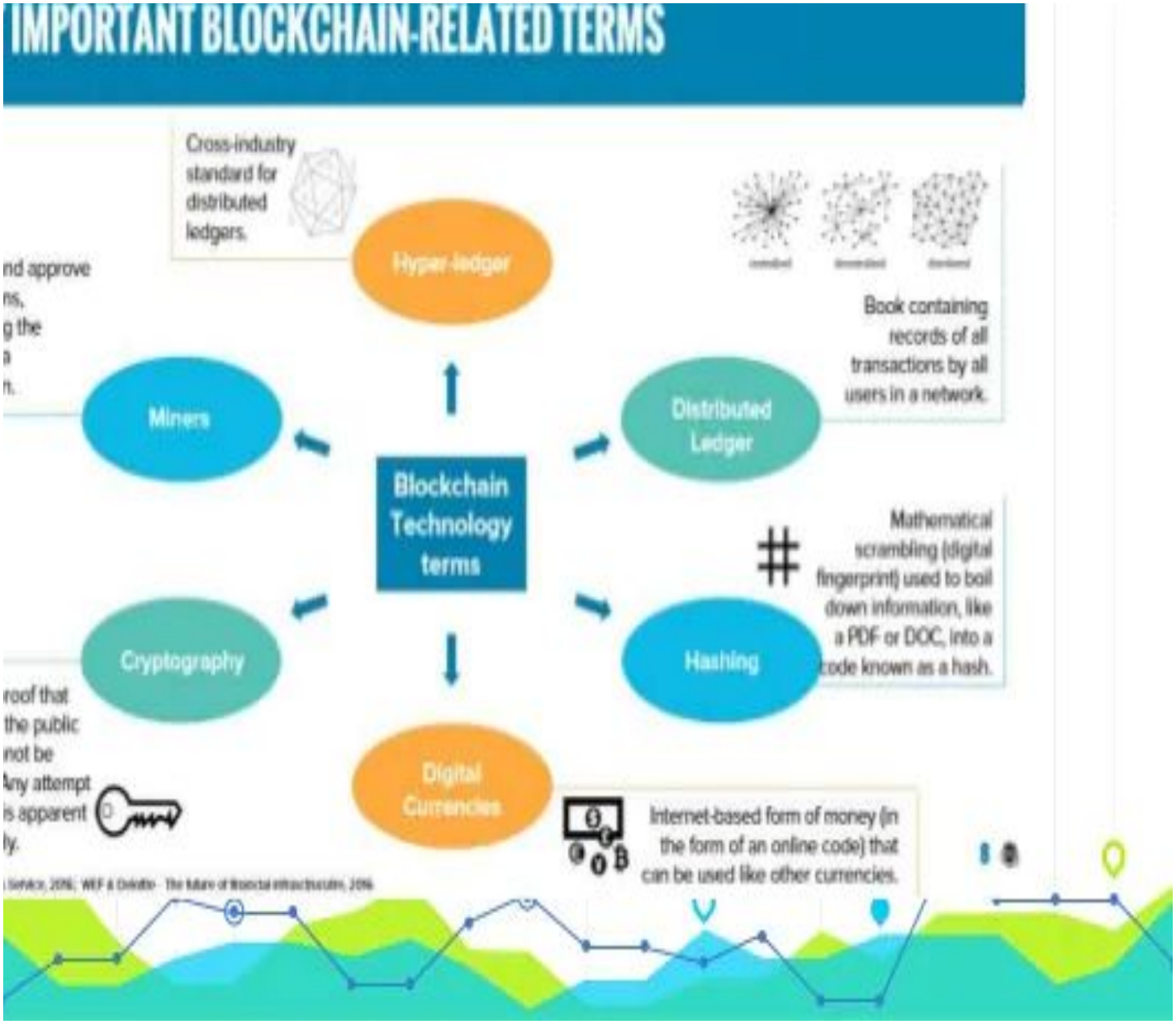


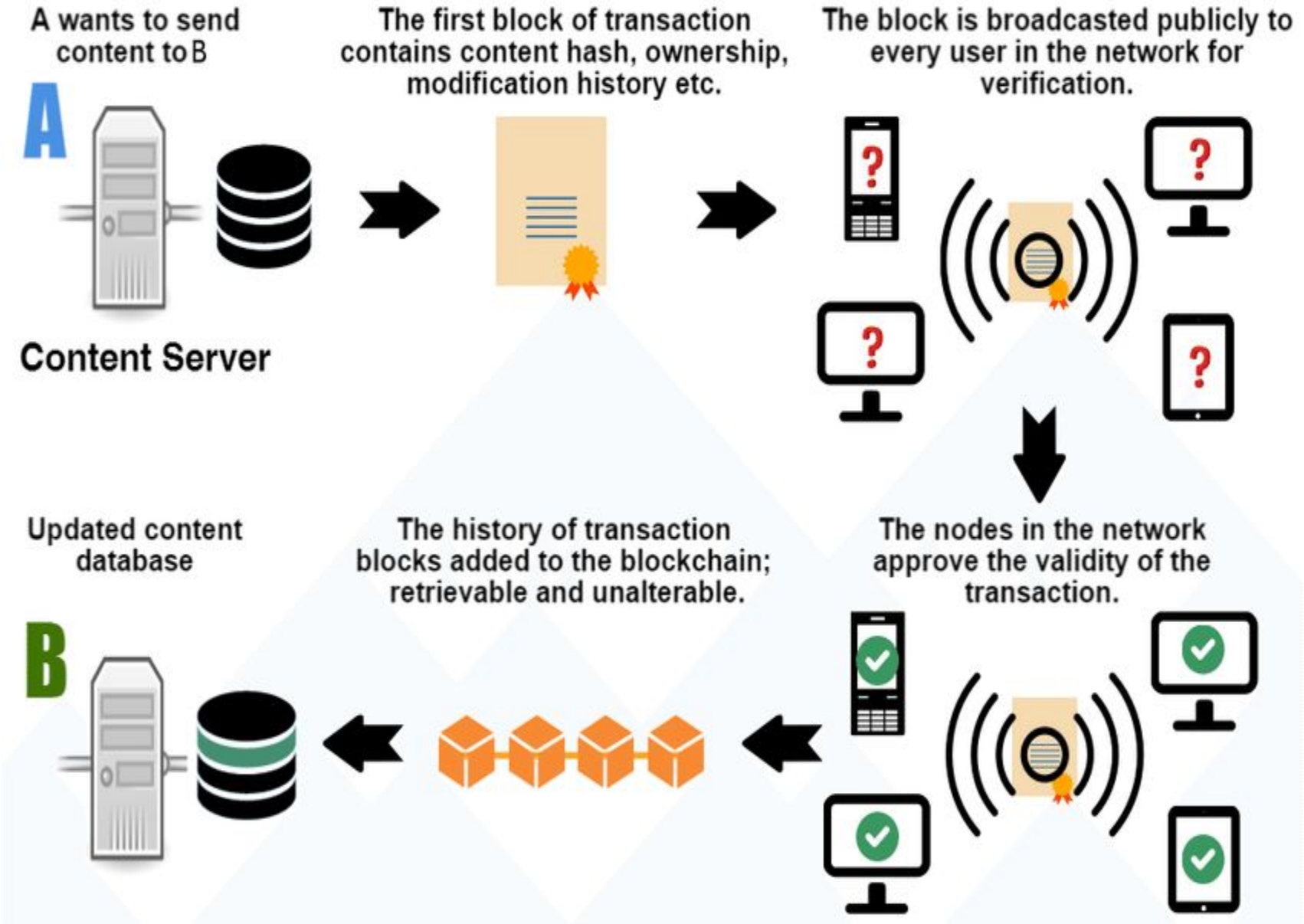
Various technical definitions of blockchains

- Blockchain is a decentralized consensus mechanism. In a blockchain, all peers come to an agreement regarding the state of a transaction
- Blockchain is a distributed shared ledger in which transactions are ordered and grouped into blocks.
- Blockchain is a data structure, a linked list uses hash pointers instead of normal pointers. Hash pointers are used to point to the previous block.

Generic structure of a blockchain




IMPORTANT BLOCKCHAIN-RELATED TERMS
Cross-industry standard for distributed ledgers.
Hyper-ledger
Book containing records of all transactions by all users in a network.
Distributed Ledger
Miners
Blockchain Technology terms
Mathematical scrambling (digital fingerprint) used to boil down information, like a PDF or DOC, into a code known as a hash.
Hashing
Cryptography
Digital Currencies
Internet-based form of money (in the form of an online code) that can be used like other currencies.


A wants to send content to B
A
Content Server
The first block of transaction contains content hash, ownership, modification history etc.
The block is broadcasted publicly to every user in the network for verification.
The nodes in the network approve the validity of the transaction.
The history of transaction blocks added to the blockchain; retrievable and unalterable.
Updated content database
B

Generic elements of a blockchain

Addresses

- Addresses are unique identifiers that are used in a transaction on the blockchain to denote senders and recipients.
- An address is a public key or derived from a public key. While addresses can be reused by the same user, addresses themselves are unique.
- In practise, a single user may not use the same address again and generate a new one for each transaction. This newly generated address will be unique.
- Bitcoin is in fact a pseudonymous system. End users are usually not directly identifiable but some research in de-anonymizing bitcoin users have shown that users can be identified successfully.
- In practise, users need to generate new address for each transaction in order to avoid linking transactions to the common owner, thus avoiding identification.

Transaction

- A transaction is the fundamental unit of a blockchain.
- A transaction represents a transfer of value from one address to another.

Block

- A block is composed of multiple transactions and some other elements such as the previous block hash (hash pointer, timestamp and nonce)

Peer-to-peer network

- This is a network topology whereby all peers can communicate with each other and send and receive messages.

Scripting or programming language

- Transaction scripts are predefined sets of commands for nodes to transfer tokens from one address to another and perform various other functions.
- Turing complete programming language is a desirable feature of blockchains but the security of language is an issue.

Virtual machine

- This is an extension of a transaction script. A virtual machine allows Turing complete code to be run on a blockchain (as smart contracts) whereas a transaction script can be limited in its operation.
- Virtual machines are not available on all blockchains. However, various blockchains use virtual machines to run programs for example, Ethereum Virtual Machine (EVM) and Chain Virtual Machine (CVM).

State machine

- A blockchain can be viewed as a state transition mechanism whereby a state is modified from its initial form to the next and eventually to a final form as a result of a transaction execution and validation process by nodes

Nodes

- A node in blockchain network performs various functions depending on the role it takes.
- A node can propose and validate transactions and perform mining to facilitate consensus and secure the blockchain.
- This is done by following a consensus protocol.
- Nodes can also perform other functions such as simple payment verification, validators and many other functions depending on the type of the blockchain used and the role assigned to the node.

Smart contracts

- These programs run on top of the blockchain and encapsulate the business logic to be executed when certain conditions are met.
- The smart contract feature is not available in all blockchains but is now becoming a very desirable feature due to the flexibility and power it provides to the blockchain applications.


Smart Contract
Immutability
Currency
Features of BC
Decentralized
Consensus Mechanism
Traceability

Features of a blockchain

Distributed Consensus

- This enables a blockchain to present a single version of truth that is agreed upon by all parties without the requirement of a central authority

Transaction Verification

- Any transactions posted from nodes on the blockchain are verified based on a predetermined set of rules and only valid transactions are selected for inclusion in a block

Platforms for Smart Contracts

- A blockchain is a platform where programs can run that execute business logic on behalf of the users.
- Not all blockchains have a mechanism to execute smart contracts

Transferring value between peers

- Blockchain enables the transfer of value between its users through tokens. Tokens -> carrier of value

Generating Cryptocurrency

- This is an optional feature depending on the type of blockchain used.
- A blockchain can generate cryptocurrency as an incentive to its miners who validate the transactions and spend resources in order to secure the blockchain.

Smart Property

- By linking a digital or physical asset to the blockchain in an irrevocable manner such that it cannot be claimed by anyone else, full control of the asset and cannot be double spent or double owned.
- Compare it with a digital music file, for example, which can be copied many times without any control on a blockchain however no one can claim it unless it is transferred.
- This feature is implemented in Digital Rights Management (DRM) and electronic cash systems where double spent detection is a key requirement and first solved in bitcoin.

Provider of security

- Blockchain is based on proven cryptographic technology that ensures the integrity and availability of data.
- Generally, confidentiality is not provided due to the requirements of transparency.
- This has become a main and barrier for its adaptability by financial institutions and other industries that need privacy and confidentiality of transactions.

- Other security services such as non-repudiation and authentication are provided by securing with the help of private keys and digital signatures

Immutability

- Records once added onto the blockchain are immutable.
- There is the possibility of rolling back the changes but requires computing resources.
- For example, if a malicious user wants to alter the previous blocks, PoW has to been computed again for all blocks already added to the blockchain.

- This difficulty makes the records on a blockchain practically immutable.

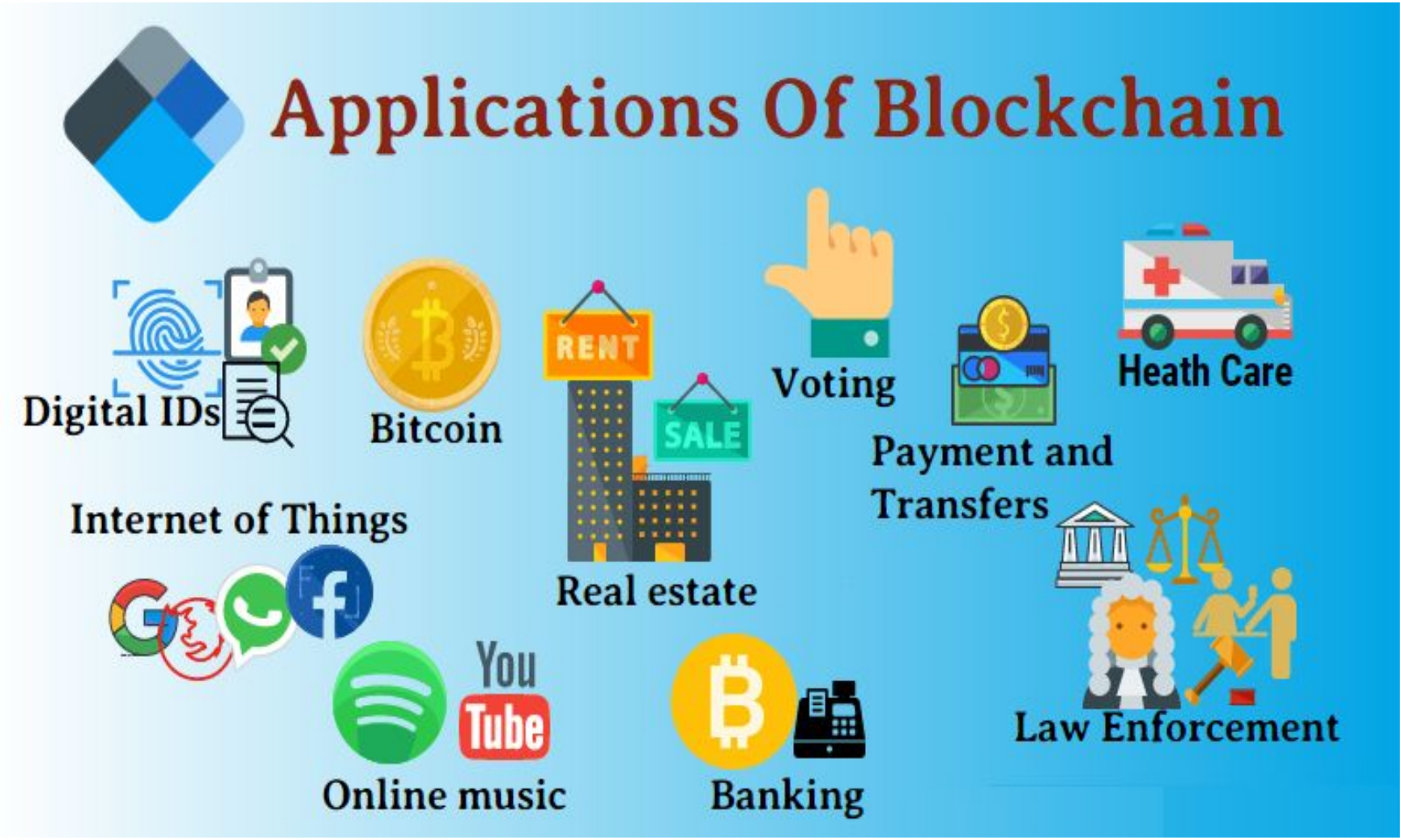
Uniqueness

- Every transaction is unique and has not been spent already.
- This is related to cryptocurrencies where detection and avoidance of double spending are the key requirement

Smart Contracts

- Blockchain provides a platform to run smart contracts. These are automated autonomous programs that reside on the blockchain and encapsulate business logic and code in order to execute a required function when certain conditions are met.
- This is a revolutionary feature of blockchain control of actions that users of blockchain need to perform according to their specific business requirements.

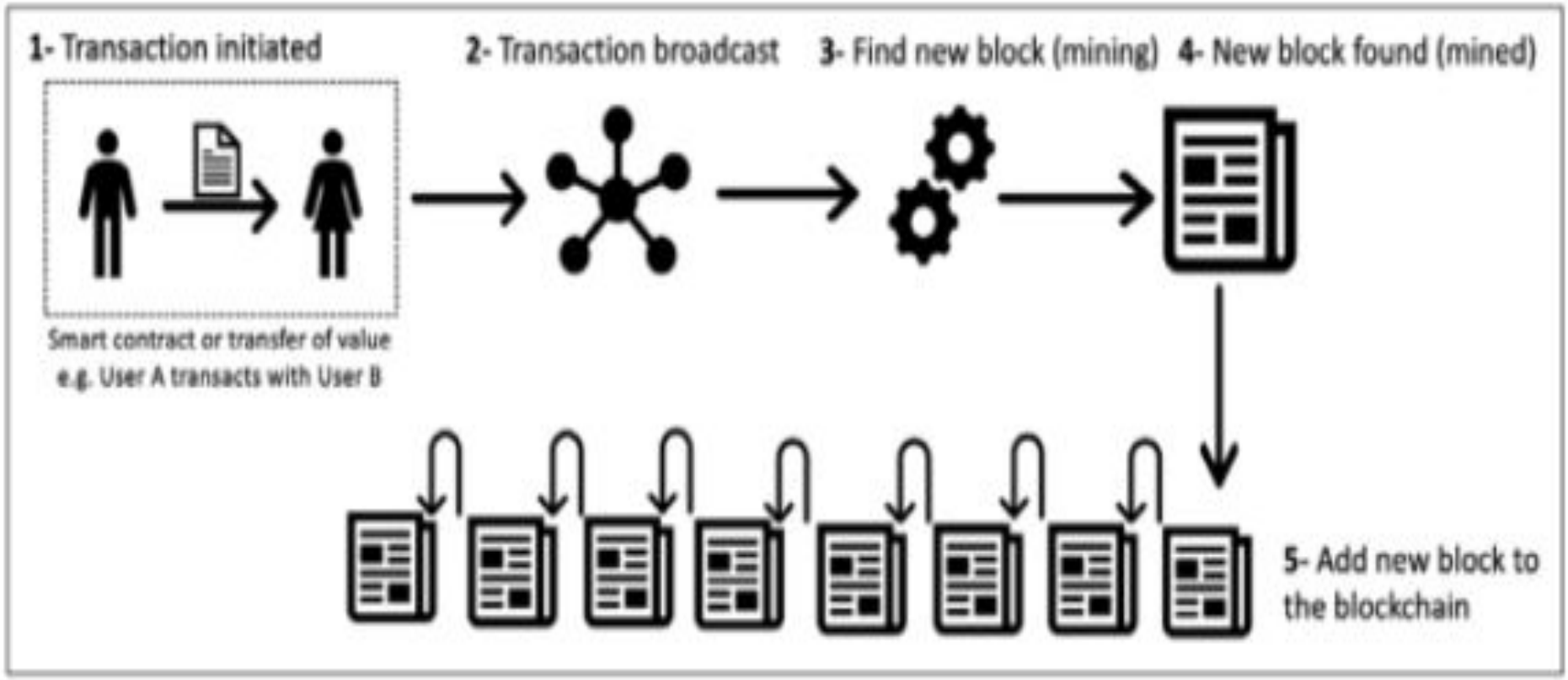
Applications of Blockchain Technology



How blockchain accumulates blocks

1. A node starts a transaction by signing it with its private key.
2. The transaction is propagated (flooded) by using much desirable Gossip (information propagation) protocol to peers, which validates the transaction based on pre-set criteria. Usually, more than one node is required to validate the transactions.
3. Once the transaction is validated, it is included in a block, which is then propagated on to the network. At this point, the transaction is considered confirmed.
4. The newly created block now becomes part of the ledger and the next block links itself cryptographically back to this block. This link is a hash pointer. At this stage, the transaction gets its second confirmation and the block gets its first.
5. Transactions are then reconfirmed every time a new block is created. Usually, six confirmations in the bitcoin network are required to consider the transaction final.

Steps 4 and 5 can be considered non-compulsory as the transaction itself is finalized in step 3; however, block confirmation and further transaction reconfirmations, if required, are then carried out in steps 4 and 5.


1- Transaction initiated
2- Transaction broadcast
3- Find new block (mining)
4- New block found (mined)
Smart contract or transfer of value
e.g. User A transacts with User B
5- Add new block to the blockchain

Tiers of Blockchain Technology

- Blockchain is evolving, and this versioning shows different tiers of evolution and usage of blockchain technology.

- In fact, all blockchain platforms, with limited exceptions, support these functionalities and applications.

- In addition to Tier 1, Tier 2 and Tier 3, or Tier X in the future, the following represents my own vision of what blockchain technology eventually could become as this technology advances:

Blockchain 1.0

- ★ This tier was introduced with the invention of Bitcoin, and it is primarily used for cryptocurrencies.
- ★ Also, as Bitcoin was the first implementation of cryptocurrencies, it makes sense to categorize this first generation of blockchain technology to include only cryptographic currencies.
- ★ All alternative cryptocurrencies as well as Bitcoin fall into this category.
- ★ It includes core applications such as payments and applications. This generation started in 2009 when Bitcoin was released and ended in early 2010.

Blockchain 2.0

- This second blockchain generation is used by financial services and smart contracts.
- This tier includes various financial assets, such as derivatives, options, swaps, and bonds.
- Applications that go beyond currency, finance, and markets are incorporated at this tier.
- Ethereum, Hyperledger, and other newer blockchain platforms are considered part of Blockchain 2.0.

Blockchain 3.0

- This third blockchain generation is used to implement applications beyond the financial services industry and is used in government, health, media, the arts, and justice.

- Again, as in Blockchain 2.0, Ethereum, Hyperledger, and newer blockchains with the ability to code smart contracts are considered part of this blockchain technology tier.

- This generation of blockchain emerged around 2012 when multiple applications of blockchain technology in different industries were researched.

Blockchain X.0

- This generation represents a vision of blockchain singularity where one day there will be a public blockchain service available that anyone can use just like the Google search engine.

- It will provide services for all realms of society.

- It will be a public and open distributed ledger with general-purpose rational agents {Machina economicuĩ) running on a blockchain, making decisions, and interacting with other intelligent autonomous agents on behalf of people, and regulated by code instead of law or paper contracts.

Types of blockchain

Public blockchains (permission-less ledgers)

- These blockchains are open to the public and anyone can participate as a node in the decision-making process.
- Users may or may not be rewarded for their participation.
- These ledgers are not owned by anyone and are publicly open for anyone to participate in.
- All users of the permissionless ledger maintain a copy of the ledger on their local nodes and use a distributed consensus mechanism in order to reach a decision about the eventual state of the ledger.

Semi-private blockchains

- Part of the blockchain is private and part of it is public.
- The private part is controlled by a group of individuals whereas the public part is open for participation by anyone.

Sidechains (pegged sidechains)

- Coins can be moved from one blockchain to another and moved back.
- Common uses include the creation of new altcoins (alternative cryptocurrencies) whereby coins are burnt as a proof of adequate stake. There are two types of sidechain.
- Burnt or burning the coins means that the coins are sent to an address that is unspendable and makes coins irrecoverable.
- This mechanism is used to bootstrap a new currency or introduce scarcity, which results in the increased value of the coin.
- The example provided above for burning coins is applicable to a one-way pegged sidechain.
- The second type is called a two-way pegged sidechain, which allows the movement of coins from the main chain to the sidechain and back to the main chain when required.

- **Rootstock** enables smart contract development for bitcoin and results in faster throughput.

Permissioned ledger

- A permissioned ledger is a blockchain whereby the participants of the network are known and already trusted.
- Permissioned ledgers do not need to use a distributed consensus mechanism, instead an agreement protocol can be used to maintain a shared version of truth about the state of the records on the blockchain.
- There is also no requirement for a permissioned blockchain to be private as it can be a public blockchain but with regulated access control.

Distributed ledger

- This ledger is distributed among its participants and spread across multiple sites or organizations. This type can either be private or public.
- The key idea is that, unlike many other blockchains, the records are stored contiguously instead of sorted into blocks. This concept is used in Ripple which is blockchain and cryptocurrency-based global payment network

Shared ledger

- This is generic term that is used to describe any application or database that is shared by the public or a consortium.

Fully private and proprietary blockchains

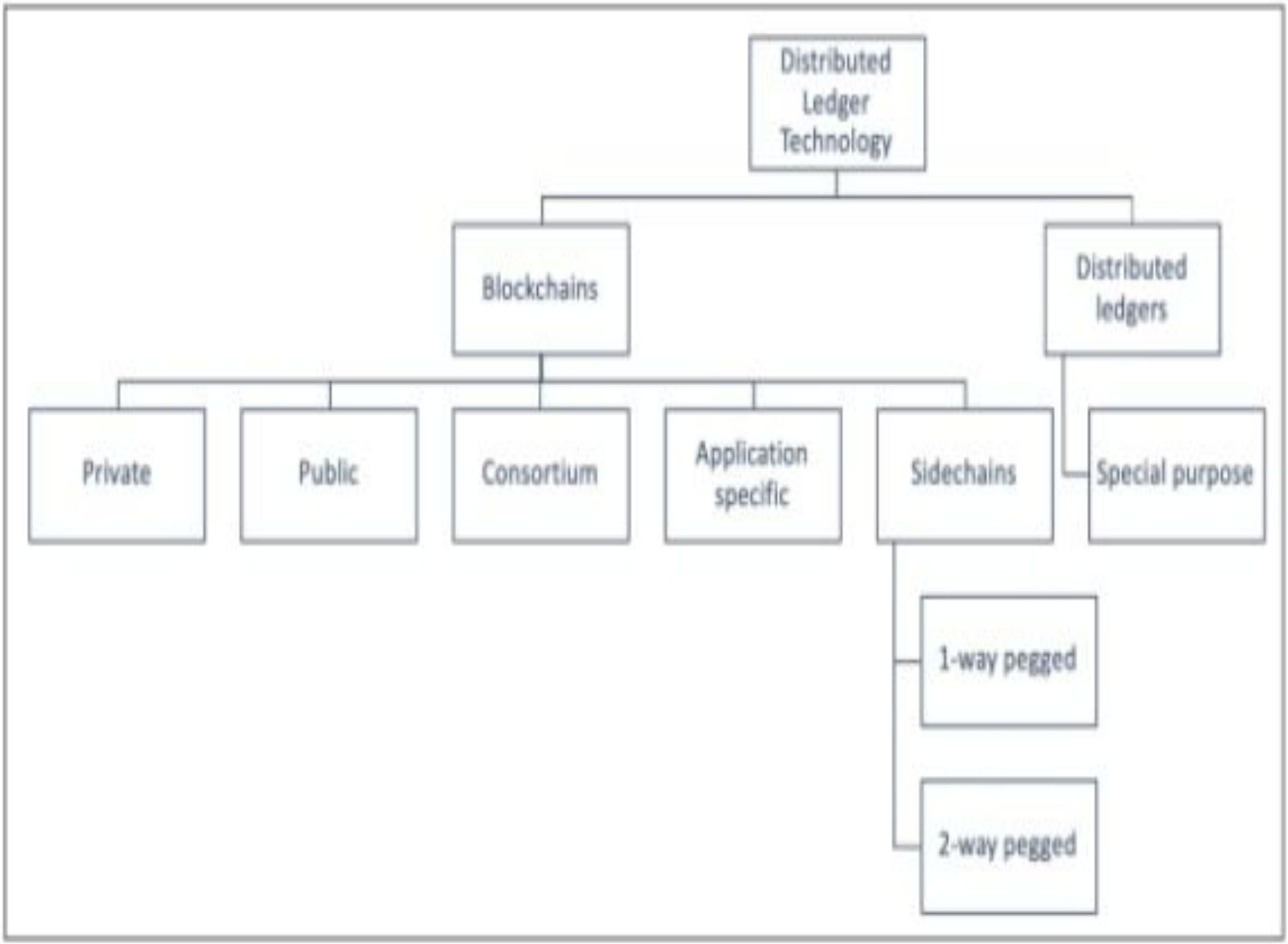
- These blockchains have no mainstream application as they deviate from the core idea of decentralization in blockchain technology.
- Nonetheless in specific private settings within an organization there might be a need to share data and provide some level of guarantee of the authenticity of the data.
- These blockchains could be useful in the scenario, for example, for collaboration and sharing data between various government departments.

Tokenized blockchains

- These blockchains are standard blockchains that generate cryptocurrency as a result of a consensus process via mining or via initial distribution. ex. bitcoin and ethereum

Tokenless blockchains

- These are probably not real blockchains because they lack the basic unit of transfer of value but are still valuable in situations where there is no need to transfer value between nodes and only sharing some data among various already trusted parties is required.
- Consensus is the backbone of a blockchain and provides decentralization of control as a result through an optional process known as mining.
- The choice of consensus algorithm is also governed by the type of blockchain in use.
- Not all consensus mechanisms are suitable for all types of blockchains. For example, in public permissionless blockchains it would make sense to use PoW instead of some basic agreement mechanism that perhaps is based on proof of authority.
- Therefore it is essential to choose a consensus algorithm appropriately for a blockchain project. ex. hyperledger fabric and quorum


Distributed Ledger Technology
Blockchains
Distributed ledgers
Private
Public
Consortium
Application specific
Sidechains
Special purpose
1-way pegged
2-way pegged

Consensus in blockchain

Consensus is basically a distributed computing concept that has been used in blockchain in order to provide a means of agreeing to a single version of truth by all peers on the blockchain network. The following two categories of consensus mechanism exist:

1. Proof-based, leader-based, or the Nakamoto consensus whereby a leader is elected and proposes a final value. (decentralized/permission-less). Bitcoin and Ethereum uses PoW. scalable but slow.
2. Byzantine fault tolerance-based, which is a more traditional approach based on rounds of votes. (consortium/permissioned). faster but not scalable.

Proof of Work

- This type of consensus mechanism relies on proof that enough computational resources have been spent before proposing a value for acceptance by the network.
- This is used in bitcoin and other cryptocurrencies. Currently, this is the only algorithm that has proven astonishingly successful against Sybil attacks.

Proof of Stake

- This algorithm works on the idea that a node or user has enough stake in the system.
- For example, the user has invested enough in the system so that any malicious attempt would outweigh the benefits of performing an attack on the system.
- This idea was first introduced by Peercoin and is going to be used in the Ethereum blockchain.
- Another important concept in Proof of Stake (PoS) is coin age, which is derived from the amount of time and the number of coins that have not been spent.
- In this model, the chances of proposing and signing the next block increase with the coin age.

Delegated Proof of Stake

- Delegated Proof of Stake (DPOS) is an innovation over standard PoS whereby each node that has stake in the system can delegate the validation of a transaction to other nodes by voting.
- This is used in the bitshares blockchain.

Proof of Elapsed Time

- Introduced by Intel, it uses Trusted Execution Environment (TEE) to provide randomness and safety in the leader election process via a guaranteed wait time.
- It requires the Intel SGX (Software Guard Extensions) processor in order to provide the security guarantee and for it to be secure. (details in Hyperledger)

Deposit-based consensus

- Nodes that wish to participate on the network have to put in a security deposit before they can propose a block.

Proof of importance

- This idea is important and different from Proof of Stake.
- Proof of importance not only relies on how much stake a user has in the system but it also monitors the usage and movement of tokens by the user to establish a level of trust and importance. This is used in Nemcoin.

Federated consensus or federated Byzantine consensus

- Used in the stellar consensus protocol, nodes in this protocol keep a group of publicly trusted peers and propagates only those transactions that have been validated by the majority of trusted nodes.

Reputation-based mechanisms

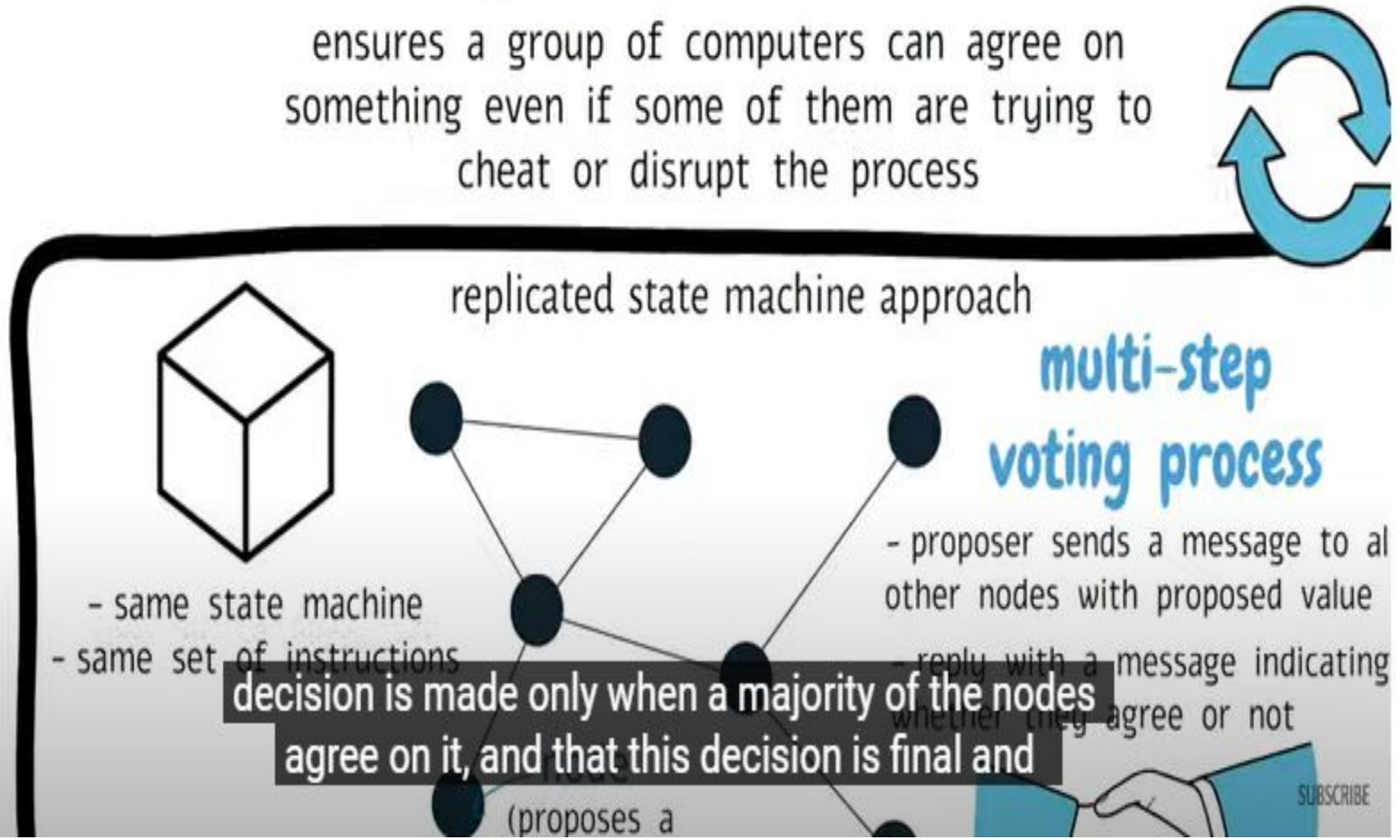
- A leader is elected on the basis of the reputation it has built over time on the network.
- This can be based on the voting from other members.

Practical Byzantine Fault Tolerance

- Practical Byzantine Fault Tolerance (PBFT) achieves state machine replication, which provides tolerance against Byzantine nodes.
- Various other protocols, including but are not limited to PBFT, PAXOS, RAFT, and Federated Byzantine Agreement (FBA), are also being used or have been proposed for use in many different implementations of distributed systems and blockchains.

Practical Byzantine Fault Tolerance (PBFT)

ensures a group of computers can agree on something even if some of them are trying to cheat or disrupt the process



Proof of Activity

- stakeholder is selected in a pseudorandom and uniform manner
- combination of PoW and PoS
- PoW is used in the first stage and PoS is used in remaining stages for energy efficiency
- achieve consensus and good level of security

Proof of Capacity

- Uses hard disk space as a resource to mine the blocks unlike in PoW, uses CPU resources
- So called hard drive mining introduced in BurstCoin cryptocurrency

Proof of Storage

- outsourcing of storage capacity
- variations, Proof of Replication, Proof of Data Possession, Proof of Space and Proof of Space-time

Proof of Authority (PoA)

- utilizes the identity of the participants called validators as a stake on the network
- validators have the authority to propose new blocks and validate as per blockchain rules. PoA algorithms are Clique and Aura

CAP theorem and blockchain

- CAP theorem is violated in blockchain, and especially in the most successful implementation: bitcoin, but this is not the case.
- In blockchains consistency is sacrificed in favor of availability and partition tolerance. In this scenario, Consistency (C) on the blockchain is not achieved simultaneously with Partition tolerance (P) and Availability (A), but it is achieved over time.
- This is called eventual consistency, where consistency is achieved as a result of validation from multiple nodes over time.
- For this purpose, the concept of mining was introduced in bitcoin; this is a process that facilitates the achievement of consensus by using a consensus algorithm called PoW.
- At a higher level, mining can be defined as a process that is used to add more blocks to the blockchain.

Benefits and limitations of blockchain

Decentralization

- This is a core concept and benefit of blockchain.
- There is no need for a trusted third party or intermediary to validate transactions; instead a consensus mechanism is used to agree on the validity of transactions.

Transparency and trust

- As blockchains are shared and everyone can see what is on the blockchain, this allows the system to be transparent and as a result trust is established.
- This is more relevant in scenarios such as the disbursement of funds or benefits where personal discretion should be restricted.

Immutability

- Once the data has been written to the blockchain, it is extremely difficult to change it back.
- It is not truly immutable but, due to the fact that changing data is extremely difficult and almost impossible, this is seen as a benefit to maintaining an immutable ledger of transactions.

High availability

- As the system is based on thousands of nodes in a peer-to-peer network, and the data is replicated and updated on each and every node, the system becomes highly available.
- Even if nodes leave the network or become inaccessible, the network as a whole continues to work, thus making it highly available.

Highly secure

All transactions on a blockchain are cryptographically secured and provide integrity.

Simplification of current paradigms

- The current model in many industries such as finance or health is rather disorganized, wherein multiple entities maintain their own databases and data sharing can become very difficult due to the disparate nature of the systems.
- But as a blockchain can serve as a single shared ledger among interested parties, this can result in simplifying this model by reducing the complexity of managing the separate systems maintained by each entity.

Faster dealings

In the financial industry, especially in post-trade settlement functions, blockchain can play a vital role by allowing the quicker settlement of trades as it does not require a lengthy process of verification, reconciliation, and clearance because a single version of agreed upon data is already available on a shared ledger between financial organizations.

Cost saving

As no third party or clearing houses are required in the blockchain model, this can massively eliminate overhead costs in the form of fees that are paid to clearing houses or trusted third parties.

Platform for smart contracts

- A blockchain is a platform on which programs can run that execute business logic on behalf of the users.
- This is very useful feature but not all blockchains have a mechanism to execute smart contracts; however, this is a very desirable feature.
- It is available on newer blockchain platforms such as Ethereum and Multichain, but not on Bitcoin.

Challenges and limitations of blockchain technology

- As with any technology there are challenges that need to be addressed in order to make a system more robust, useful, and accessible.
- Blockchain technology is no exception; in fact a lot of effort is being made in Academia and Industry to overcome the challenges posed by blockchain technology.
- A selection of the most sensitive challenges are presented as follows:
  - Scalability
  - Adoptability
  - Regulation
  - Relatively immature technology
  - Privacy

- Scalability
  - blockchain networks are not as scalable as required. ex. current financial networks
- Adoption
  - to increase adoption, make easier use of blockchain networks and improve scalability
- Regulation
  - barrier to adoption because no regulatory authority and control exists unlike traditional one where consumers have a certain level of confidence that if something goes wrong they can hold someone accountable due to the existence of regulatory authorities

- Relatively immature technology
  - requires research to achieve maturity
- Privacy
  - in public blockchain (Bitcoin), privacy is a concern as everyone see every transaction
  - but this transparency is not desirable in financial, law or medical sectors, so research going on to drive mass adoption

UNIT-II

Decentralization

- Decentralization is not a new concept; it has been used in strategy, management, and governance for a long time.
- The basic idea of decentralization is to distribute control and authority to peripheries instead of one central authority being in full control of the organization.
- This results in several benefits for organizations such as increased efficiency, quicker decision making, better motivation, and a reduced burden on top management.

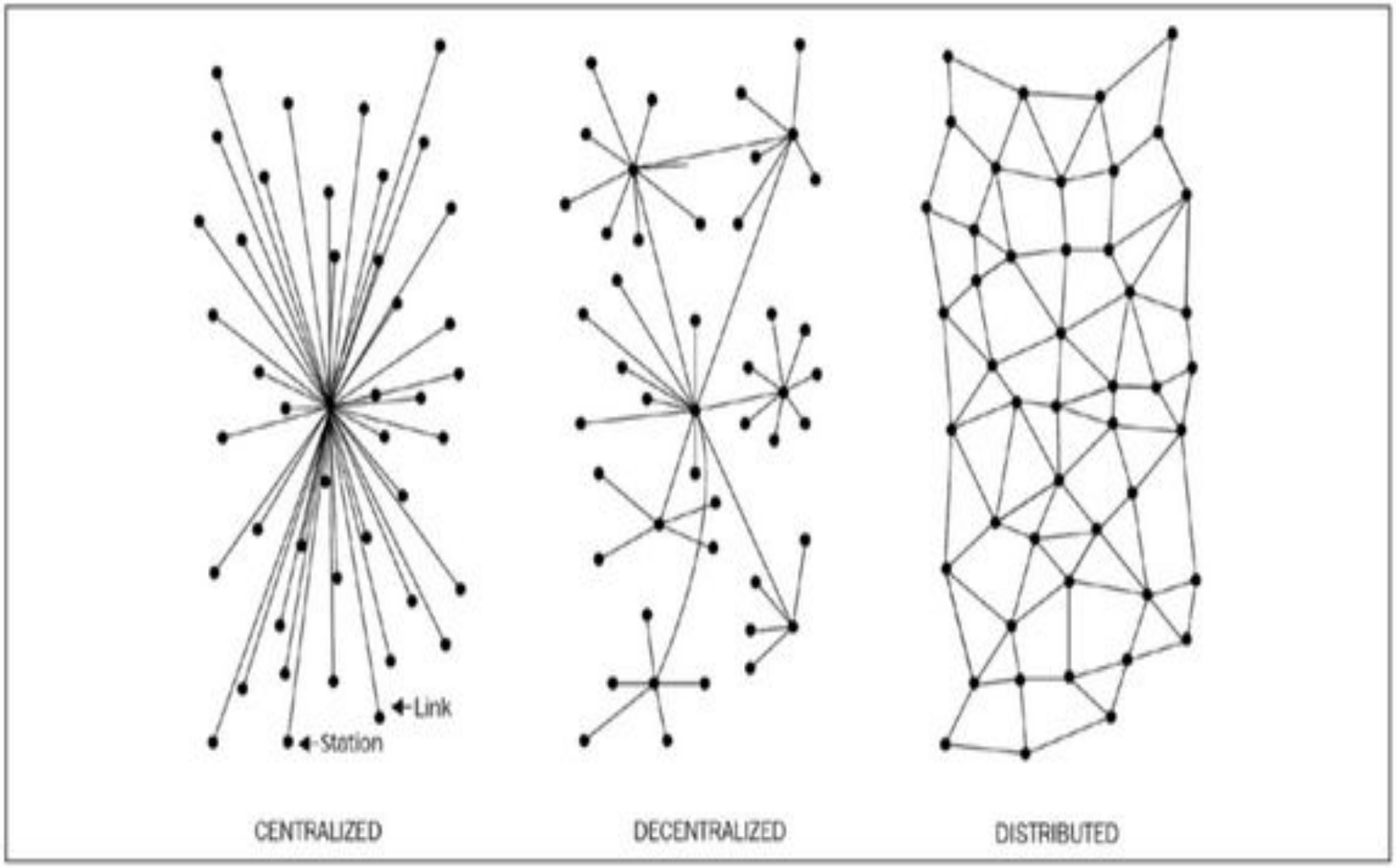
Decentralization using Blockchain

- With PoW, anyone can gain authority to validate the transaction.

Information and Communication Technology

- Traditionally uses centralized system

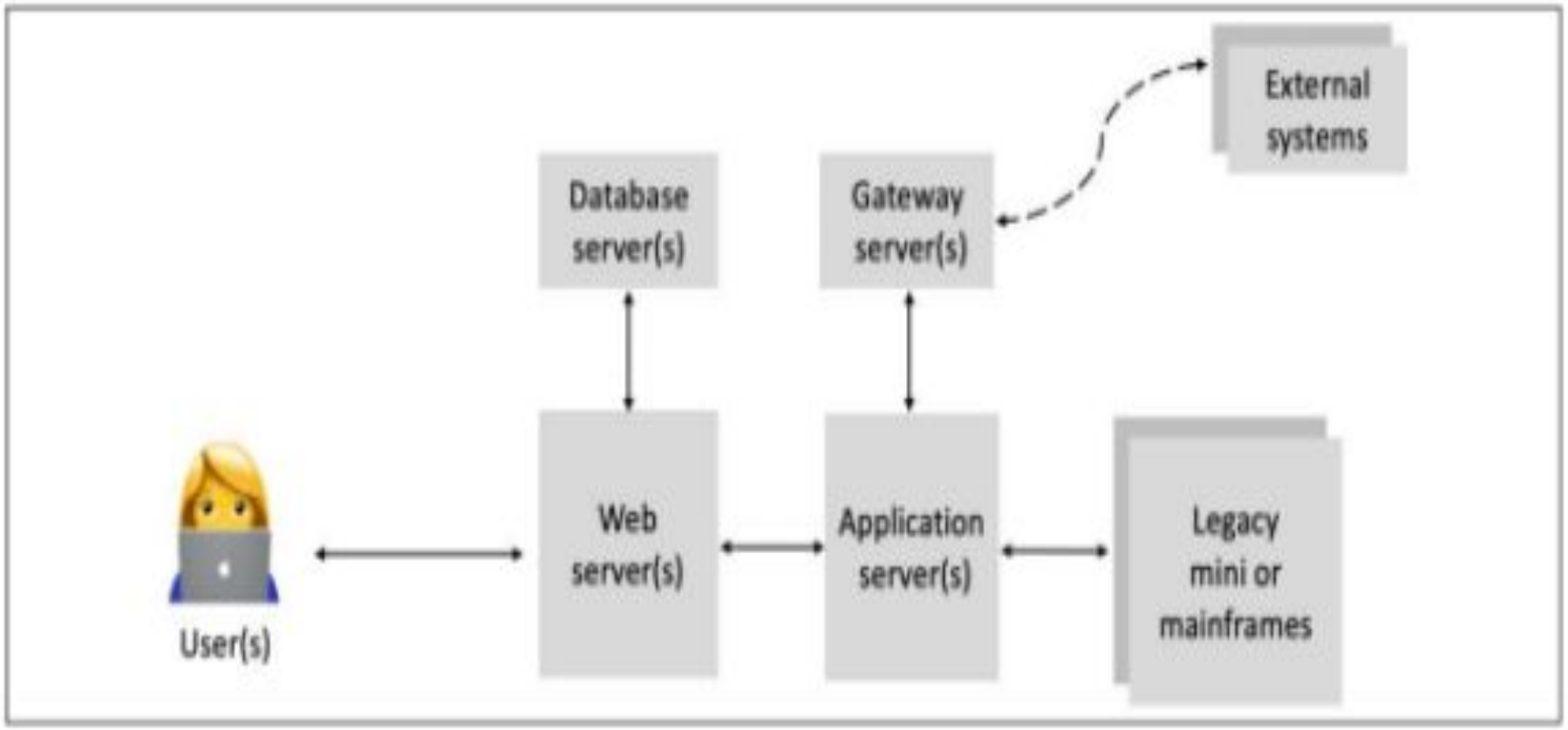
- With the advent of bitcoin and blockchain technology, decentralized system is used


Link
Station
CENTRALIZED
DECENTRALIZED
DISTRIBUTED

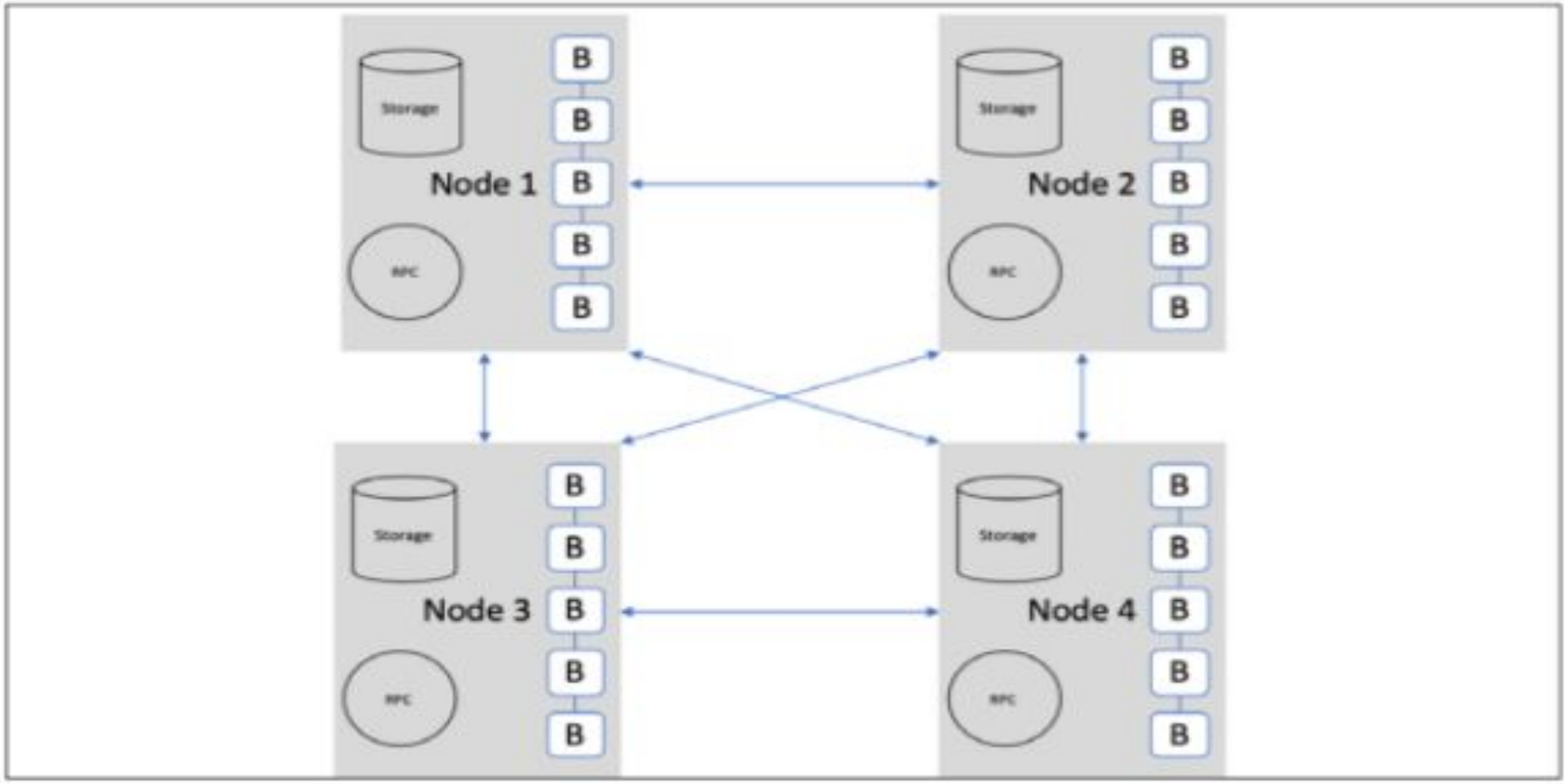
- **Centralized system** used by Google, Amazon, eBay and Apple's App Store
- **Distributed system** (central authority exists)
  - parallel computing platforms are used in weather research and forecasting, simulation and financial modeling
  - computation not in parallel and data is replicated across multiple nodes as a single, coherent system
  - variation of both these models achieve fault tolerance and speed
- **Decentralized system** (no central authority/trusted third party/intermediary/service provider exists) -ex.different departments in an organization

Centralized	Distributed	Decentralized
		 Notice no hand means no central controller / authority

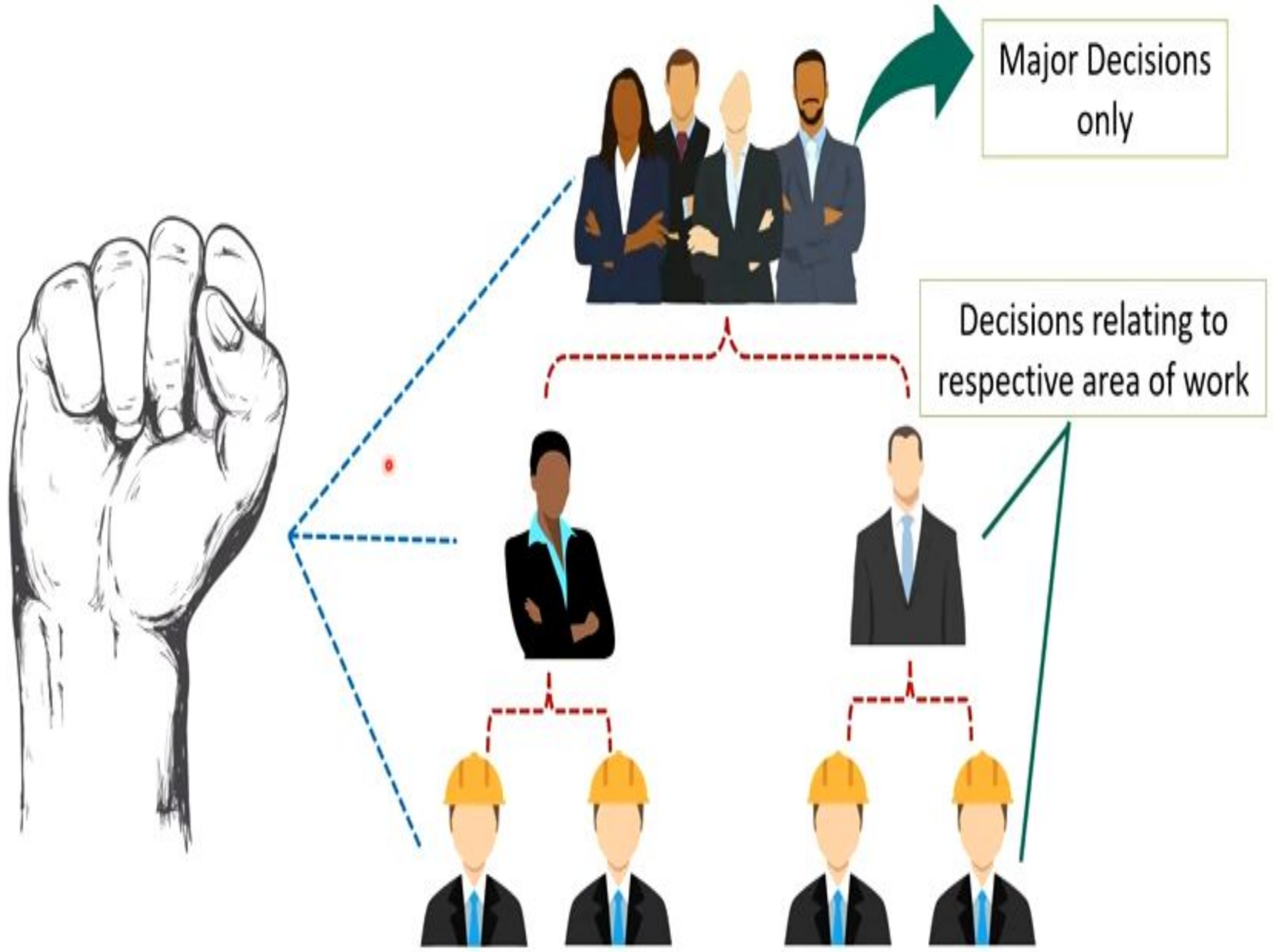
A traditional distributed system comprises many servers performing different roles



A decentralized system (based on blockchain) where an exact replica of the applications and data is maintained across the entire network on each participating node



Feature	Centralized	Decentralized
Ownership	Service provider	All users
Architecture	Client/server	Distributed, different topologies
Security	Basic	More secure
High availability	No	Yes
Fault tolerance	Basic, single point of failure	Highly tolerant, as service is replicated
Collusion resistance	Basic, because it's under the control of a group or even single individual	Highly resistant, as consensus algorithms ensure defense against adversaries
Application architecture	Single application	Application replicated across all nodes on the network
Trust	Consumers have to trust the service provider	No mutual trust required
Cost for consumer	Higher	Lower


Major Decisions only
Decisions relating to respective area of work

Methods of decentralization

Disintermediation

- Imagine you want to send money to your friend in another country.
- You go to a bank that will transfer your money to the bank in the country of your choice for a fee.

- In this case, the bank keeps a central database that is updated, confirming that you have sent the money.

- With blockchain technology, it is possible to send this money directly to your friend without the need for a bank.

- All you need is the address of your friend on the blockchain.

- This way, the intermediary is no longer required and decentralization is achieved by disintermediation.
- However, it is debatable how practical decentralization is in the financial sector by disintermediation due to heavy regulatory and compliance requirements. Nevertheless, this model can be used not only in finance but also in many other different industries such as health, law and public sector.

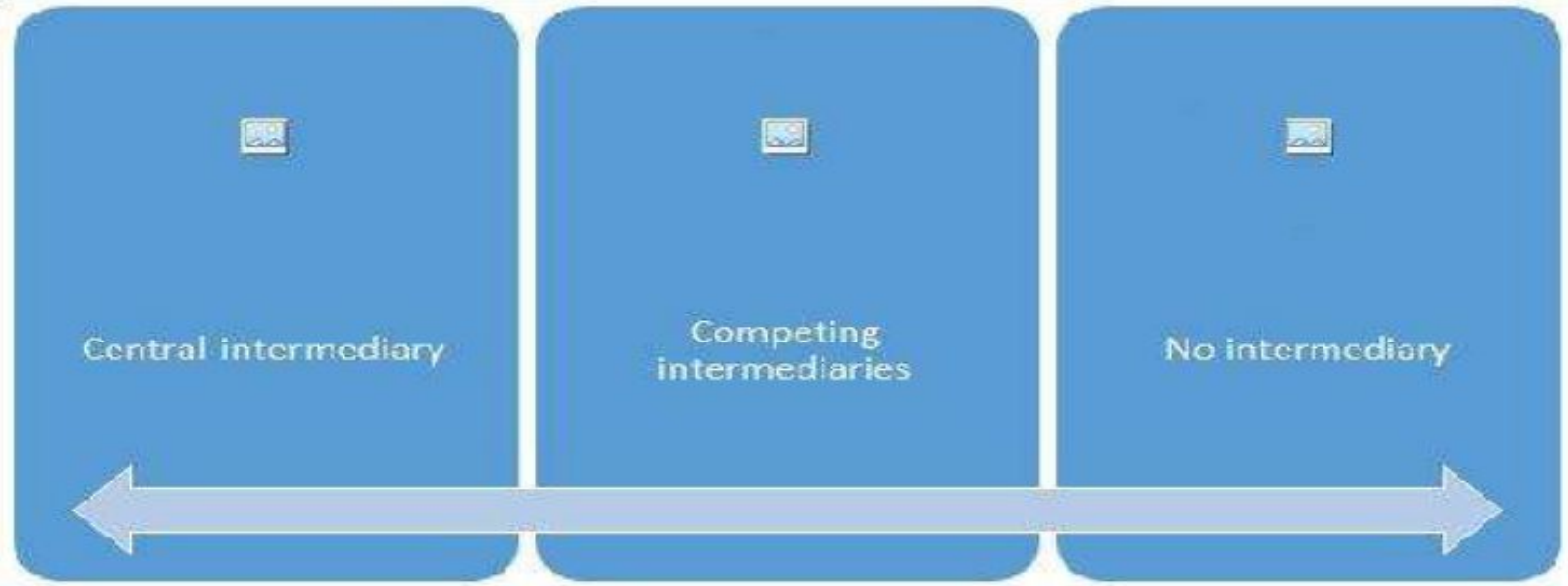
- In the health industry, where patients instead of relying on a trusted third party (hospital record system) can be in full control of their own identity and their data that they can share directly with only those entities that they trust.
- Blockchain can serve as a decentralized health record management system where health records can be exchanged securely and directly between different entities (hospitals, pharmaceutical companies, patients) globally without any central authority.

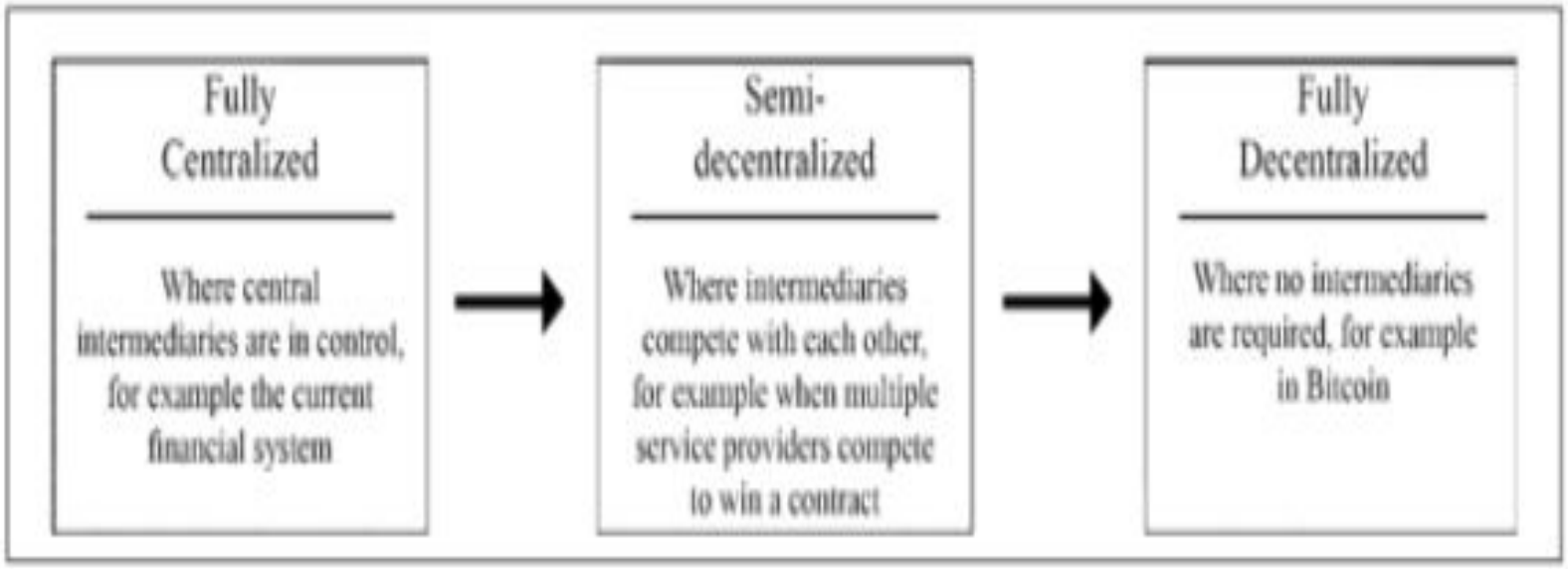

If the project gets fully funded, the contract automatically passes the money to the creator

Through competition (contest-driven decentralization)

- In this method, a group of service providers compete with each other in order to be selected for the provision of services by the system.
- This paradigm does not achieve complete decentralization, but to a certain degree ensures that an intermediary or service provider is not monopolizing the service.
- In the context of blockchain technology, a system can be envisioned in which smart contracts can choose an external data provider from a large number of providers based on their reputation, previous score, reviews, and quality of service.
- This will not result in full decentralization, but it allows smart contracts to make a free choice based on the criteria mentioned earlier.
- This way, an environment of competition is cultivated among service providers, whereby they compete with each other to become the data provider of choice.
- In the following diagram, varying levels of decentralization are shown. On the left-hand side, there is a conventional approach where a central system is in control; on the right-hand side, complete disintermediation is achieved; and in middle, competing intermediaries or service providers are shown.
- In the middle, intermediaries or service providers are selected based on reputation or voting, thus achieving partial decentralization.

Scale of decentralization





Is a blockchain really needed? When is a blockchain required? In what circumstances, is blockchain preferable to traditional databases? Answers:

Question	Yes/No	Recommended solution
Is high data throughput required?	Yes	Use a traditional database.
	No	A central database might still be useful if other requirements are met. For example, if users trust each other, then perhaps there is no need for a blockchain. However, if they don't or trust cannot be established for any reason, blockchain can be helpful.
Are updates centrally controlled?	Yes	Use a traditional database.
	No	You may investigate how a public/private blockchain can help.
Do users trust each other?	Yes	Use a traditional database.
	No	Use a public blockchain.
Are users anonymous?	Yes	Use a public blockchain.
	No	Use a private blockchain.
Is consensus required to be maintained within a consortium?	Yes	Use a private blockchain.
	No	Use a public blockchain.
Is strict data immutability required?	Yes	Use a blockchain.
	No	Use a central/traditional database.

- While there are many benefits of decentralization-including but not limited to transparency, efficiency, cost saving, development of trusted ecosystems, and in some cases privacy and anonymity-some challenges, such as security requirements, software bugs, and human errors, also need to be looked at thoroughly.

- For example, in a decentralized system such as bitcoin or Ethereum, where security is usually provided by private keys, how can it be ensured that a smart property associated with these private keys cannot be rendered useless if, due to a human error, the private keys are lost or if, due to a bug in the smart contract code, the decentralized application is vulnerable to attack by adversaries?

- It is noted that not everything to be decentralized.

Routes to decentralization

- Even though there are systems that existed before bitcoin or blockchain that can be classed as decentralized to a certain degree, such as BitTorrent or Gnutella file sharing, with the advent of the blockchain technology many initiatives are being taken in order leverage this new technology for decentralization.
- Usually, the bitcoin blockchain is the first choice for many as it has proven to be the most resilient and secure blockchain with a market cap of almost 12 billion dollars.
- An alternative approach is to use other blockchains, such as Ethereum, which is currently the tool of choice of many developers for building decentralized applications.

How to decentralize

- Evaluate the decentralization requirements of a variety of things in the context of blockchain technology.

- The framework basically proposes four questions to be answered in order to provide a clear idea as to how a system can be decentralized.

  1. What is being decentralized?
  2. What level of decentralization is required?
  3. What blockchain is used?
  4. What security mechanism is used?

Answers for the above questions

1. This can be any system, for example an Identity system or trading.

2. The level of decentralization required can be based on the scale of decentralization. It can be full disintermediation or partial disintermediation.

3. The blockchain suitable for fitting a particular application can be bitcoin blockchain, Ethereum blockchain, or any other blockchain.

4. Security of a decentralized system can be guaranteed using Atomicity, for example, whereby either the transaction executes in full or does not execute at all. This ensures the integrity of the system. Other mechanisms can include reputation, which allows varying degrees of trust in a system.

Example Application - a money transfer system

1. Money transfer system
2. Disintermediation
3. Bitcoin
4. Atomicity

Blockchain and full ecosystem decentralization

- In order to achieve complete decentralization, it is necessary that the environment around the blockchain is also decentralized.
- Blockchain itself is a distributed ledger that runs on top of conventional systems.
- These elements include storage, communication, and computation.
- The factors, such as Identity and Wealth, traditionally based on centralized paradigms, need to decentralize in order to achieve a fully decentralized ecosystem.

Storage

- Data can be stored directly in a blockchain, thus decentralization is achieved.
- But drawback is that blockchain is not suitable for storing large amounts of data by design.
- Stores transactions containing arbitrary data but not suitable for storing images or large blobs of data, as in case of traditional database systems.
- A better alternative is to use **distributed hash tables (DHTs).**
- DHTs were originally used in peer-to-peer file sharing software, such as BitTorrent, Napster, Kazaa and Gnutella.

- DHT research was made popular by CAN, Chord, Pastry, and Tapestry projects.
- BitTorrent is the most scalable and fast network, but there is no incentive for users to keep the files indefinitely.
- Users do not usually keep files permanently, and if nodes leave the network that has data required by someone, there is no way to retrieve it except having the required nodes rejoin the network again so that the files become available once more.
- Two main requirements are high availability and link stability, which means that data should be available when required and network links should always be accessible.

- **Inter Planetary File System (IPFS) by** *Juan Benet* satisfies above two properties and the vision is to provide a decentralized World Wide Web by replacing the HTTP protocol.
- IPFS uses Kademlia DHT and merkle **DAG (Directed Acyclic Graph)** to provide the storage and searching functionality, respectively.
- The incentive mechanism is based on a protocol known as Filecoin that pays incentives to nodes that store data using the BitSwap mechanism.
- The BitSwap mechanism allows nodes to keep a simple ledger of bytes sent or bytes received under a one-to-one relationship.
- Also, a Git-based version control mechanism is used in IPFS to provide structure and control over the versioning of data.

- There are other alternatives, such as Ethereum swarm, storj, and maidsafe.
- Ethereum has its own decentralized and distributed ecosystem that uses Swarm for storage and the whisper protocol for communication.
- Maidsafe is aiming to provide a decentralized World Wide Web.
- BigChainDB is another storage layer decentralization project aimed at providing a scalable, fast, and linearly scalable decentralized database as opposed to a traditional filesystem.
- BigChainDB complements decentralized processing platforms and file systems such as Ethereum and IPFS.

Communication

- The communication layer in blockchain is decentralized.
- The original vision of the Internet was to develop a decentralized system.
- Services such as e-mail and online storage in which service provider is in control and users trust them to give them access to the service when required.
- This model is based on the trust of the central authority (the service provider) and users are not in control of their data; even passwords are stored on trusted third-party systems.

- There is a need to provide control to individual users in such a way that access to their data is guaranteed and is not dependent on a single third party.

- Access to the Internet (the communication layer) is based on Internet service providers (ISPs) that act as a central hub for Internet users.

- If the ISP is shut down for political or any other reasons, then no communication is possible in this model.

- An alternative is to use mesh networks.

- Even though they are limited in functionality as compared to the Internet, they still provide a decentralized alternative where nodes can talk directly to each other without a central hub such as an ISP.

- An example of a mesh network is **Firechat**, which allows iPhone users to communicate with each other directly in a peer-to-peer fashion without an Internet connection.

- Now imagine a network that allows users to be in control of their communication: no one can shut it down for political or censorship reasons.
- This could be the next step toward decentralizing communication networks in the blockchain ecosystem.
- It must be noted that this model may only be required in a jurisdiction where the Internet is censored and controlled by the government.
- the original vision of the Internet was to build a decentralized network; however, over the years, with the advent of large-scale service providers such as Google, Amazon, and eBay, the control is shifting toward the big players.

- For example, e-mail is a decentralized system at its core; anyone can run an e-mail server with minimal effort and can start sending and receiving e-mails, but there is a better alternative available that is already providing a managed service for end users, so there is a natural inclination toward selecting a centralized service as it is more convenient and free.
- Free services, however, are being offered at the cost of valuable personal data and many users are not aware of this fact.
- This is one example that shows how the Internet has moved toward centralization.
- Blockchain has once again given this vision of decentralization to the world and now concerted efforts are being made to harness this technology and gain the benefits that it can provide.

Computation (computing power and decentralization)

- Decentralization of computing or processing is achieved by a blockchain technology such as Ethereum, where smart contracts with embedded business logic can run on the network.
- Other blockchain technologies also provide similar processing layer platforms where business logic can run over the network in a decentralized manner.
- **Decentralized ecosystem overview**



- Internet or Meshnets provides a decentralized communication layer
- Storage layer uses technologies such as IPFS and BigChainDB to enable decentralization
- Blockchain serves as a decentralized processing layer, provide a storage layer too but affects speed and capacity of the system
- Therefore, other solutions such as IPFS and BigChainDB are more suitable to store large amounts of data in a decentralized way

- At the top, the Identity and Wealth layers in which bitAuth and OpenID have provided authentication and identification services with varying degrees of decentralization and security assumptions
- Blockchain is capable of providing solutions to various problems.
- A concept relevant to Identity known as *Zooko's Triangle* requires that a naming system in a network protocol be secure, decentralized, and meaningful to humans.

- A system can have only two of these properties simultaneously, but with the advent of blockchain, in the form of Namecoin, this problem was resolved
- However, challenges such as reliance on users to store and maintain private keys securely becomes obstacle to suitability of decentralization in every scenario unlike centralized systems
- With the advent of the blockchain technology, software versions of traditional physical organizations in the context of decentralization are being developed

- Decentralization may not be appropriate for every scenario.
- Centralized systems with well-established reputations tend to work better in many cases.
- For ex.,email platforms from reputable companies such as Google or Microsoft would provide a better service than a scenario where individual email servers are hosted by users on the Internet.

- For ex., Swarm and Whisper are developed to provide decentralized storage and communication for Ethereum.

- With the advent of blockchain technology, it is now possible to build software versions of traditional physical organizations in the form of Decentralized Organizations (DOs) and other similar constructs

Terminologies concerned to decentralization and its applications

Smart contract

- A smart contract can be thought of as a small decentralized program.
- Smart contracts do not necessarily need a blockchain to run; however, due to the security benefits that the blockchain technology provides, it is now becoming almost a standard to use blockchain as a decentralized execution platform for smart contracts.
- A smart contract usually contains some business logic and a limited amount of data.
- Actors or participants in the blockchain use these smart contracts or they run autonomously on behalf of the network participants.
- These small programs reside on the blockchain and execute business logic if some specific criteria are met.

Autonomous agents

- An autonomous Agent (AA) is an artificially intelligent software entity that acts on behalf of its owner to achieve some desirable goals without requiring any or minimal intervention from its owner.

Decentralized organization (DOs)

- **Decentralized organization (DOs)** are software programs that run on a blockchain and are based on the idea of real human organizations with people and protocols.
- Once a DO, in the form of a smart contract or a set of smart contracts, is added to the blockchain, it becomes decentralized and parties interact with each other based on the code defined within the DO software.

Decentralized autonomous organization (DAO)

- Just like DOs, a **Decentralized autonomous organization (DAO)** is also a computer program than runs on top of a blockchain and embedded within it are governance and business logic rules.

- DAO and DO: the main difference is that DAOs are autonomous, which means that they are fully automated and contain artificially intelligent logic, whereas DOs lack this feature and rely on human input in order to execute business logic.

- Ethereum blockchain introduced DAOs for the first time.

- In DAO, the code is considered the governing entity rather than humans or paper contracts.

- A *Curator,* however, is a human entity that participates as someone who maintains this code and acts as a proposal evaluator for the community.
- DAOs are capable of hiring external *Contractors* if enough input is received from the token holders (participants).
- The DAO project raised 168 million US dollars in its crowd-funding phase.
- The DAO project was designed to be a venture capital fund which was aimed at providing a decentralized business model with no single entity as an owner.
- Unfortunately, this was hacked due to a bug in the DAO code and millions of dollars' worth of **Ether currency (ETH)** were siphoned out of the DAO into a child DAO created by the hackers.

- It required a hard fork on the Ethereum blockchain to reverse the impact of the hack and initiate the recovery of the funds.

- This incident opened up a debate on the security, quality, and the need for thorough testing of the code in smart contracts in order to ensure integrity and adequate control.

- There are projects underway, especially in Academia, that are looking to formalize smart contract coding.

- Currently, DAOs do not have any legal status even though they may contain some intelligent code that enforces some protocols and conditions, but these rules have no value in the current real-world legal system.


$
Lay's
Honey
Barbecue

DAO

RUN BY CODE → SMART CONTRACT → DO TASK

(SELF-SUSTAINABLE)

- One day, perhaps an *autonomous agent* that is commissioned and permissioned by a law enforcement agency or a regulator containing rules and regulations could be embedded in a DAO, to ensure the integrity of the DAO from a legal and compliance perspective.

- An **Autonomous Agent (AA)** is a piece of code that runs without human intervention.

- The fact that DAOs are purely decentralized entities makes it possible to run them in any physical jurisdiction.

- Therefore, they raise a big question as to how a current legal system would work with such a varied mix of different jurisdictions and geographies.

Decentralized autonomous corporations

- **DAOs, Decentralized autonomous corporations (DACs)** are a similar concept but are considered a smaller subset of DAOs.
- DACs and DAOs: difference is that DAOs are usually considered to be nonprofit, whereas DACs can make money via shares offered to the participants and by paying dividends.
- These corporations can run a business automatically without human intervention based on the logic programmed within them.

Decentralized autonomous societies

- **Decentralized autonomous societies (DASs)** are a concept whereby entire societies can function on a blockchain with the help of multiple complex smart contracts and a combination of DAOs and **Decentralized applications (DAPPs)** running autonomously.

- This model does not mean an outlaw approach, nor is it based on a totally libertarian ideology; instead, many services that a government offers can be delivered via blockchain, such as Government Identity Card systems, passport issuance, and records of deeds, marriages, and births.

- Another theory is that, if a government is corrupt and central systems do not provide the satisfactory levels of trust that a society needs, then the society can start its own virtual society on a blockchain that is driven by decentralized consensus and is transparent.

- This might be seen as a libertarian or cypherpunk dream but is entirely possible on a blockchain.

Decentralized applications

- All DAOs, DACs, and DOs are basically decentralized applications that run on top of a blockchain in a peer-to-peer network.
- This is the latest advancement in technology with regard to decentralization.
- Decentralized applications or DAPPs are software programs that can run on their own blockchain, use another already existing established blockchain, or use only protocols of an existing blockchain solution.
- These are called Type I, Type II, and Type III DAPPs.

Requirements of a decentralized application

In order for an application to be considered a decentralized application, it must meet the following criteria.

1. The DAPP should be fully open source and autonomous and no single entity should be in control of a majority of its tokens. All changes to the application must be consensus-driven based on the feedback given by the community.
2. Data and records of operations of the application must be cryptographically secured and stored on a public, decentralized blockchain in order to avoid any central points of failure.
3. A cryptographic token must be used by the application in order to provide access and rewards to those who contribute value to the applications, for example, miners in bitcoin.
4. The tokens must be generated by the decentralized application according to a standard cryptographic algorithm. This generation of tokens acts as a proof of the value to contributors (for example, miners).

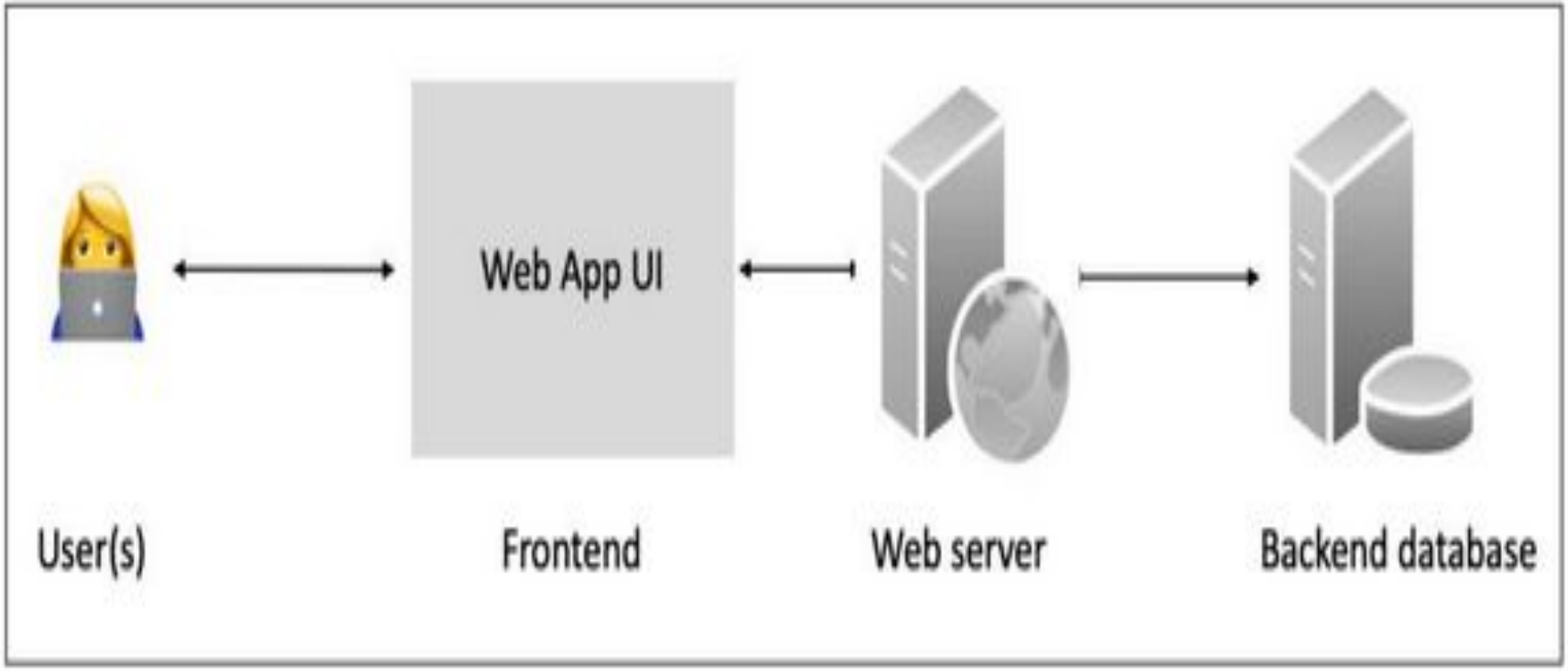
Operations of a DAPP

- Establishment of consensus by a DAPP can be achieved using consensus algorithms such as Proof of Work and Proof of Stake.
- So far, only PoW has been found to be incredibly resistant to 51% attacks, as is evident from bitcoin.
- Furthermore, a DAPP can distribute tokens (coins) via mining, fundraising, and development.

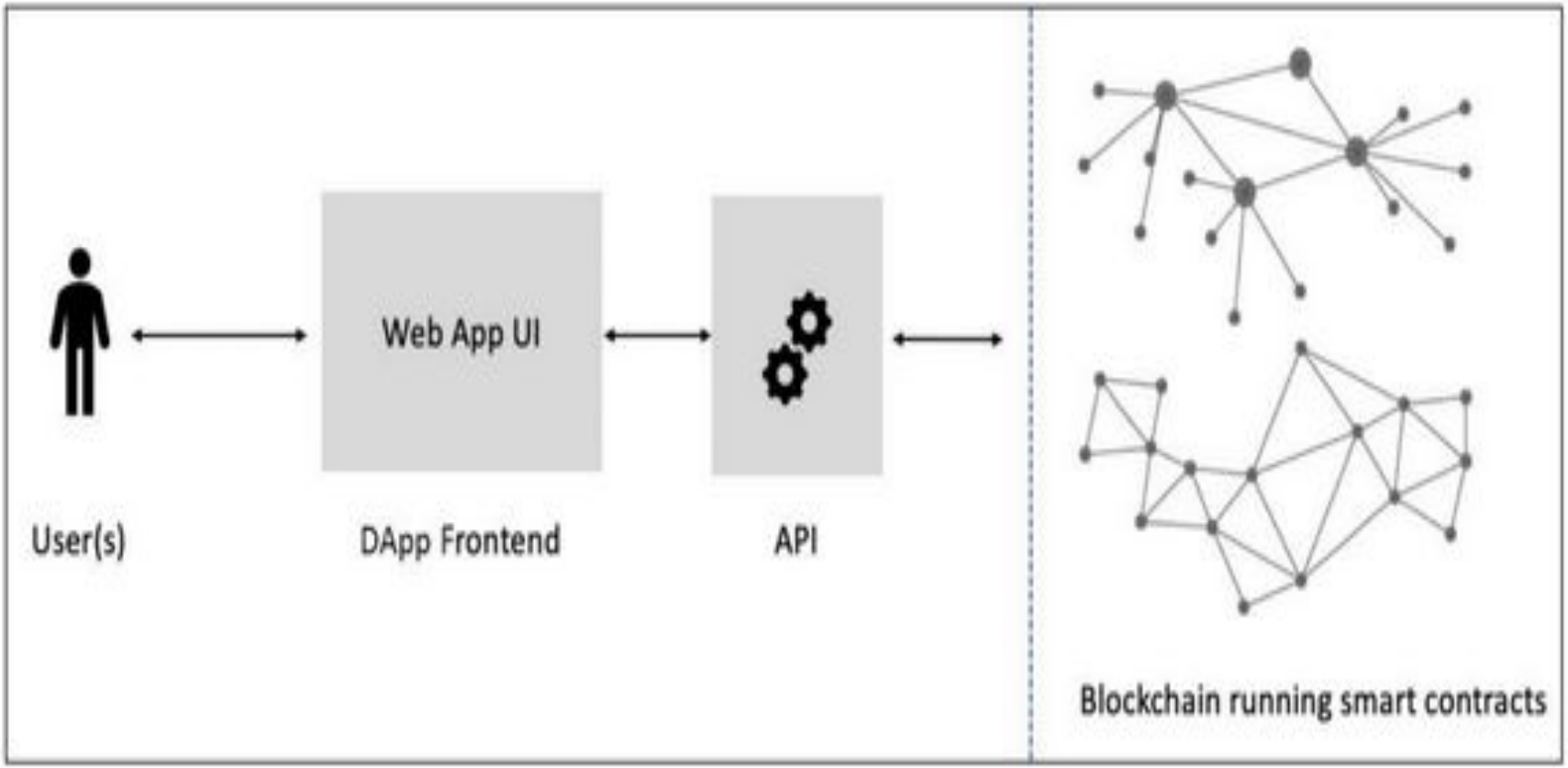
Design of a DAPP

- DApp is a software application that runs on a decentralized network such as a distributed ledger.
- Recently, it become very popular due to the development of various decentralized platforms such as Ethereum, EOS and Tezos

Traditional Application Architecture (generic client/server)



Generic DApp architecture



- A DApp has a blockchain as a backend.
- The key element that plays a vital role in the creation of DApp is a smart contract that runs on the blockchain and has business logic embedded within it.
- The frontend in DApp or app architecture can be either a thick client, a mobile app or a web frontend (a web user interface).
- A web frontend is written using a JavaScript framework such as React or Angular.

Entity	Autonomous?	Software?	Owned?	Capital?	Legal status	Cost
DO	No	No	Yes	Yes	Yes	High
DAO	Yes	Yes	No	Yes	Unsettled	Low
DAC	Yes	Yes	Yes	Yes	Unsettled	Low
DAS	Yes	Yes	No	Possible	Unsettled	Low
DApp	Yes	Yes	Yes	Optional tokens	Unsettled	Use case dependent

Examples of some decentralized DAPP applications built on Ethereum blockchain

KYC-Chain

- This application provides a facility to manage **Know Your Customer (KYC)** data in a secure and convenient way based on smart contracts.

Open Bazaar

- This is a decentralized peer-to-peer network that allows commercial activities directly between sellers and buyers instead of relying on a central party, as opposed to conventional providers such as eBay and Amazon.
- It should be noted that this system is not built on top of a blockchain; instead, distributed hash tables are used in a peer-to-peer network in order to enable direct communication and data sharing between peers.
- It makes use of bitcoin as a payment network, however.

Lazooz

- This is a decentralized equivalent of Uber.
- It allows peer-to-peer ride sharing and users can be incentivized by *proof of movement* and can earn Zooz coins.

Platforms for decentralization

Many platforms for decentralization are introduced to make distributed application development easy, accessible, and secure for users.

Ethereum

- Ethereum tops the list as being the first blockchain that introduced a Turing-complete language and the concept of a virtual machine.
- This is in contrast to the limited scripting language in bitcoin and many other cryptocurrencies.
- With the availability of this Turing-complete language called Solidity, endless possibilities have opened for the development of decentralized applications.
- This was proposed in 2013 by *Vitalik Buterin* and provides a public blockchain to develop smart contracts and decentralized applications.
- Currency tokens on Ethereum are called Ethers.

Maidsafe

- Maidsafe provides a SAFE (Secure Access for Everyone) network that is made up of unused computing resources, such as storage, processing power, and the data connections of its users.
- The files on the network are divided into small chunks of data that are encrypted and distributed throughout the network randomly.
- This data can only be retrieved by its respective owner. One key innovation is that duplicate files are automatically rejected on the network, which helps reduce the need for additional computing resources to manage the load.
- It uses Safecoin as a token to incentivize its contributors.

Lisk

- Lisk is a blockchain application development and cryptocurrency platform.
- It allows developers to use JavaScript to build decentralized applications and host them in their own respective sidechains.
- Lisk uses the **Delegated Proof of Stake (DPOS)** mechanism for consensus whereby 101 nodes can be elected to secure the network and propose blocks.
- It uses the Node.js and JavaScript backend whereas the frontend allows the use of standard technologies, such as CSS3, HTML5, and JavaScript.
- Lisk uses LSK coin as a currency on the blockchain.
- Another derivative of Lisk is Rise, which is a Lisk-based decentralized application and digital currency platform.
- It has more focus on the security of the system.

Cryptography and Technical Foundations

On Ubuntu Linux distribution, OpenSSL is usually already available or it can be installed using the command: $ sudo apt-get install openssl

Introduction

- Cryptography is the science of making information secure in the presence of adversaries.
- It provides a means of secure communication in the presence of adversaries with assumed limitless resources.
- Ciphers are used to encrypt data so that if intercepted by an adversary, the data is meaningless to them without decryption, which requires the secret key.
- Cryptography is generally used to provide a confidentiality service. On its own, it cannot be considered a complete solution but serve as a crucial building block within a larger security system to address a security problem.
- Cryptography provides various security services, such as **Confidentiality, Integrity, Authentication,** (Entity Authentication and Data origin authentication) and **nonrepudiation.**
- Additionally, accountability is also required in various security systems.

Mathematics

- A set is a collection of distinct objects, for example, *X*= *{1, 2, 3, 4, 5}.*

GROUP

- A group is a commutative set with one operation that combines two elements of the set.
- The group operation is closed and associated with an identity element defined. Additionally, each element in the set has an inverse.
- Closure (closed) means that if, for example, elements A and B are in the set, then the resultant element after performing operation on the elements is also in the set.
- Associative means that the grouping of elements does not affect the result of the operation.

FIELD

- A field is a set that contains both additive and multiplicative groups.
- More precisely, all elements in the set form an additive and multiplicative group. It satisfies specific axioms for addition and multiplication.
- For all group operations, the distributive law is also applied.
- The law dictates that the same sum or product will be produced even if any terms or factors are reordered.

A FINITE FIELD

- A finite field is a field with a finite set of elements.
- Also known as Galois fields, these structures are of particular importance in cryptography as they can be used to produce accurate and error-free results of arithmetic operations.
- For example, prime finite fields are used in elliptic curve cryptography to construct discrete logarithm problem.

ORDER

- This is the number of elements in a field.
- It is also known as the cardinality of the field.

PRIME FIELDS

- This is a finite field with a prime number of elements.
- It has specific rules for addition and multiplication, and each nonzero element in the field has an inverse.
- Addition and multiplication operations are performed modulo *p.*

RING

- If more than one operation can be defined over an abelian group, that group becomes a ring.
- There are also certain properties that need to be satisfied.
- A ring must have closure and associative and distributive properties.

A CYCLIC GROUP

- A cyclic group is a type of group that can be generated by a single element called the group generator.
- In other words, if the group operation is repeatedly applied to a particular element in the group, then all elements in the group can be generated.

AN ABELIAN GROUP

- An abelian group is formed when the operation on the elements of a set is commutative.
- Commutative law basically means that changing the order of the elements does not affect the result of the operation, for example, $A \; X \; B = B \; X \; A$.

MODULAR ARITHMETIC

- Also known as clock arithmetic, numbers in modular arithmetic wrap around when they reach a certain fixed number.
- This fixed number is a positive number called modulus and all operations are performed with regard to this fixed number.
- In an analogy to a clock, there are number from 1 to 12.
- When it reaches 12, the number 1 starts again.
- In other words, this arithmetic deals with the remainders after the division operation.
- For example, 50 mod 11 is 6 because 50/11 leaves a remainder of 6.

Cryptography

Confidentiality

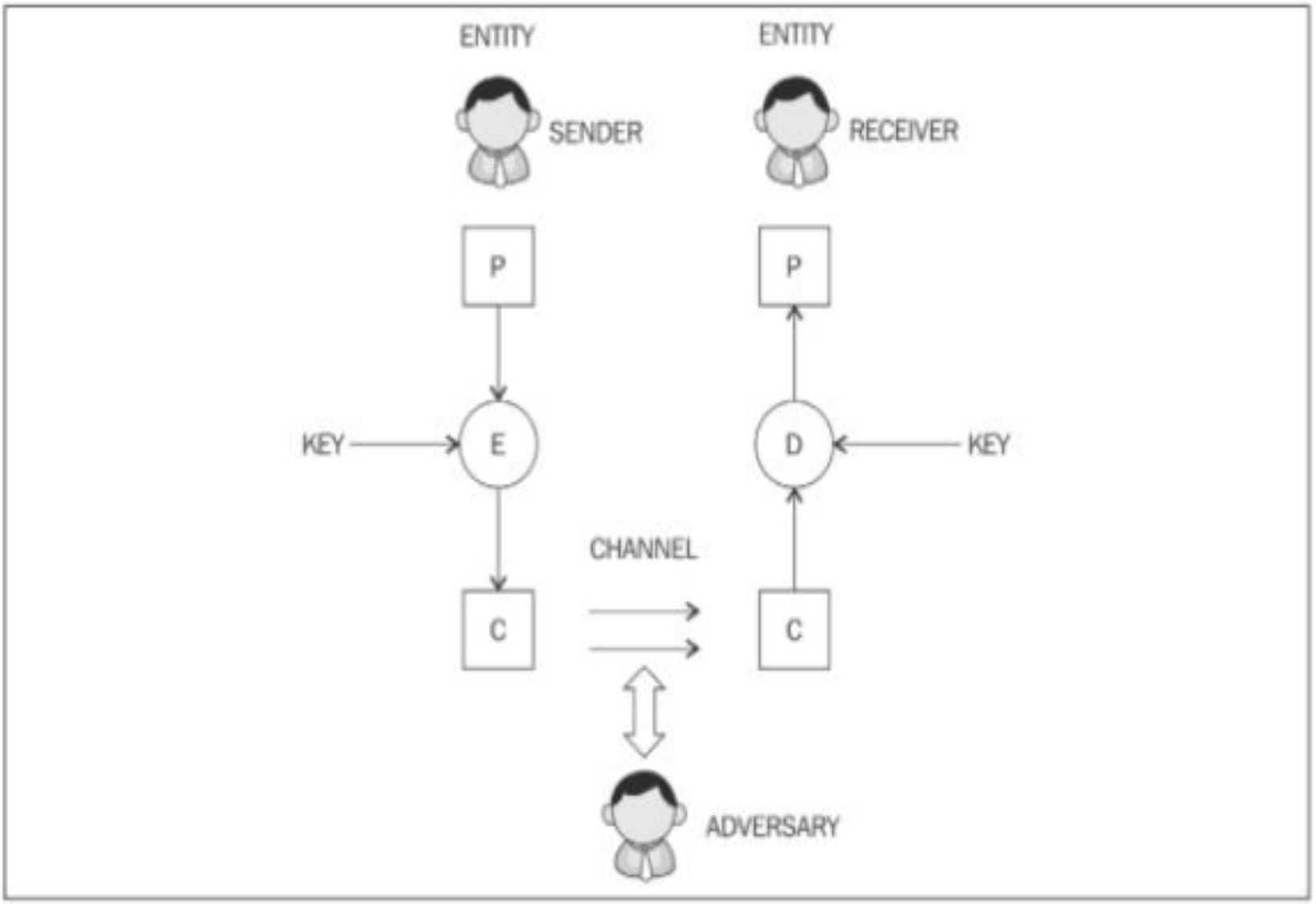
- Confidentiality is the assurance that information is only available to authorized entities.

Integrity

- Integrity is the assurance that information is modifiable only by authorized entities.

Authentication

- Authentication provides assurance about the identity of an entity or the validity of a message. There are two types of authentications,



A model of the generic encryption and decryption model

ENTITY AUTHENTICATION

- Entity authentication is the assurance that an entity is currently involved and active in a communication session.
- Traditionally, users are issued a username and password, which are used to gain access to the platforms they are using.
- This is called single factor authentication as there is only one factor, namely *something you know,* that is, the password and username.
- This type of authentication is not very secure due to various reasons, such as password leakage; therefore, additional factors are now commonly used to provide better security.
- The use of additional techniques for user identification is known as multifactor authentication or two-factor authentication if only two methods are used.
- If more than two factors are used for authentication, that is called multifactor authentication.

Various factors are:

1. The first factor : hardware token or smart card.
   - In this case, a user can use a hardware token in addition to login credentials to gain access to a system.
   - This provides protection by requiring two factors of authentication.
   - A user who has access to the hardware token and knows the log-on credentials will be able to access the system.
   - Both factors should be available in order to gain access to the system, thus making this method a two-factor authentication mechanism.

2. The second factor : uses biometric features in order to identify the user.

   - In this method, a user uses fingerprint, retina, iris, or hand geometry to provide an additional factor for authentication.
   - It can be ensured that a user was indeed present during the authentication mechanism as biometric features are unique to an individual.
   - However, careful implementation is required in order to ensure a high level of security as some research has suggested that biometric systems can be circumvented in certain scenarios.

DATA ORIGIN AUTHENTICATION

- Also known as message authentication, this is an assurance that the source of information is verified.
- Data origin authentication implies data integrity because if a source is corroborated, then data must not have been altered.
- Various methods, such as **Message Authentication Codes (MACs)** and digital signatures are most commonly used.

Non-repudiation

- Non-repudiation is the assurance that an entity cannot deny a previous commitment or action by providing unforgeable evidence.
- It is a security service that provides unforgeable evidence that a particular action has occurred.
- This property is very necessary in disputable situations whereby an entity has denied actions performed, for example, placing an order on an e-commerce system.
- This service produces cryptographic evidence in electronic transactions so that in case of disputes,
- it can be used as a confirmation of an action.
- Non-repudiation has been an active research area for many years.
- Disputes in electronic transactions are a common issue and there is a need to address them in order to increase the confidence level of consumers in the service.

- The non-repudiation protocol usually runs in a communication network and is used to provide evidence that an action has been taken by an entity (originator or recipient) on the network.

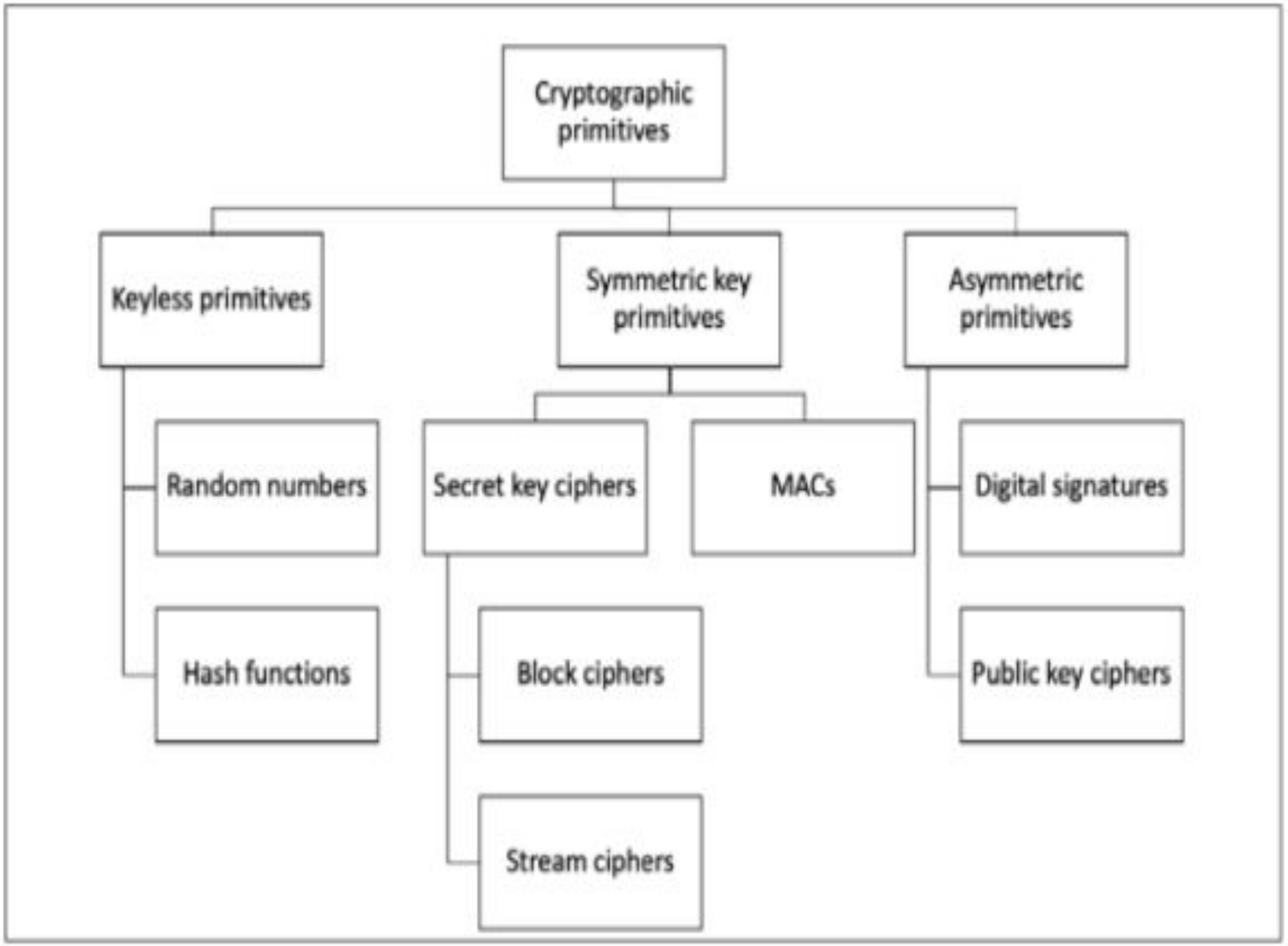
- In this context, there are two communication models that can be used to transfer messages from originator *A* to recipient *B:*
  1. Message is sent directly from originator ***A*** to recipient ***B.***
  2. Message is sent to a delivery agent from originator ***A,*** which then delivers the message to recipient ***B.***
- The main requirements of a non-repudiation protocol are fairness, effectiveness, and timeliness.
- In many scenarios, there are multiple participants involved in a transaction as opposed to only two parties.
- For example, in electronic trading systems, there can be many entities, such as clearing agents, brokers, and traders that can be involved in a single transaction.
- In this case, two-party non-repudiation protocols are not appropriate.
- To address this problem, **Multi-party nonrepudiation protocols (MPNR)** has been developed.

Accountability

- Accountability is the assurance that actions affecting security can be traced to the responsible party.
- This is usually provided by logging and audit mechanisms in systems where a detailed audit is required due to the nature of the business, for example, in electronic trading systems.
- Detailed logs are vital to trace an entity's actions, for example, when a trade is placed in an audit record with the date and time stamp and the entity's identity is generated and saved in the log file.
- This log file can optionally be encrypted and can be part of the database or a standalone ASCII text log file on a system.

Cryptographic primitives

- **A security protocol** is a set of steps taken in order to achieve required security goals by utilizing appropriate security mechanisms.
- Various types of security protocols are in use, such as **authentication protocols, non-repudiation protocols,** and **key management protocols.**



Cryptographic primitives

- **Entity:** It is either a person or a system that sends, receives, or performs operations on data
- **Sender:** Sender is an entity that transmits the data
- **Receiver:** Receiver is an entity that takes delivery of the data
- **Adversary:** This is an entity that tries to circumvent the security service
- **Key:** A key is some data that is used to encrypt or decrypt data
- **Channel:** Channel provides a medium of communication between entities

Keyless primitives

1. *Randomness*

- It is used in the generation of keys and in encryption algorithms
- Private keys must be generated randomly so that they are unpredictable and not easy to guess
- *Two types:* Random Number Generator (RNGs), Pseudorandom Random Number Generator (PRNGs)

a. RNGs

- Ex. Real randomness: Temperature variations, thermal noises
- Sources based on key strokes or disk movements are examples of non-practical generation of randomness
- The measure of randomness is called entropy

b. PNGs

- Using deterministic function, random initial value as the seed, random number is generated
- Ex. Blum-Blum-Shub Generator
- It is an alternative to RNGs in terms of reliability and deterministic nature

2. Hash Functions

- For any arbitrary message length, produces fixed digest
- It provides integrity service
- It is produced using dedicated and iterated hash construction techniques

Symmetric cryptography

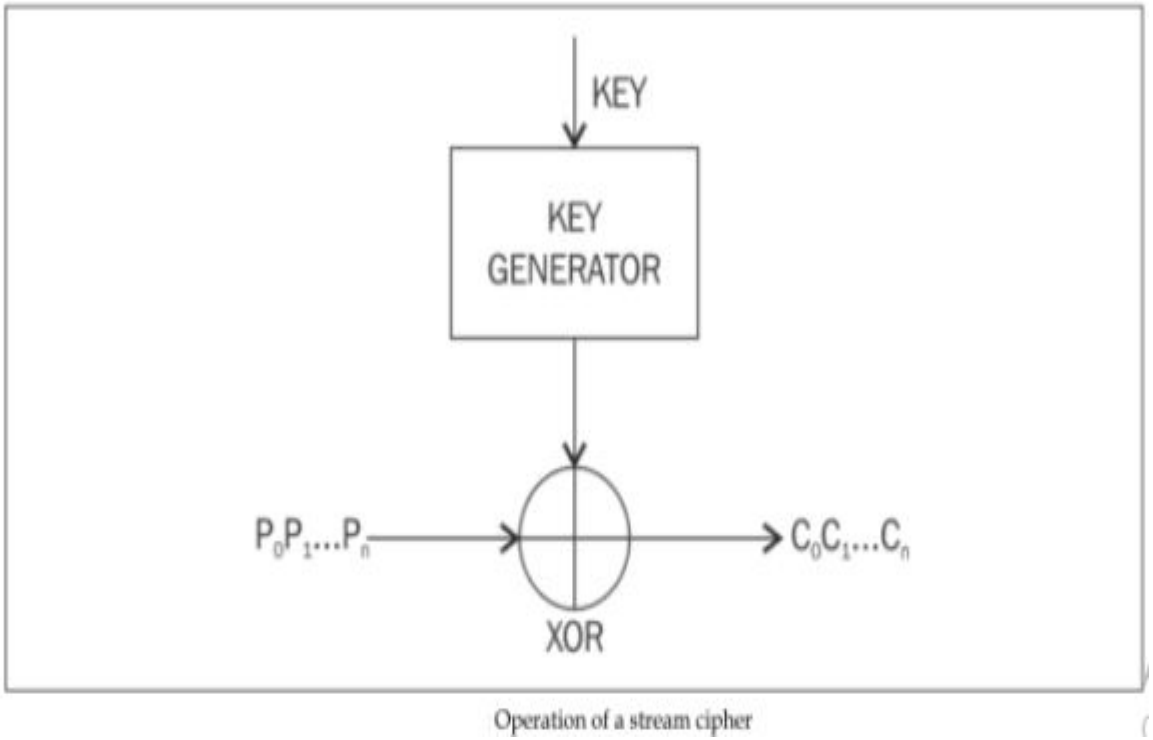
There are two types of symmetric ciphers, stream ciphers and block ciphers. **Data Encryption Standard (DES)** and **Advanced Encryption Standard (AES)** are common examples of block ciphers, whereas RC4 and A5 are commonly used stream ciphers.

STREAM CIPHERS

- These ciphers are encryption algorithms that apply encryption algorithms on a bit-by-bit basis to plain text using a key stream.
- There are two types of stream ciphers: synchronous and asynchronous.
- Synchronous stream ciphers are ones where key stream is dependent only on the key, whereas asynchronous stream ciphers have a key stream that is also dependent on the encrypted data.

- In stream ciphers, encryption and decryption are basically the same function because they are simple modulo 2 additions or XOR operation.

- The key requirement in stream ciphers is the security and randomness of key streams.

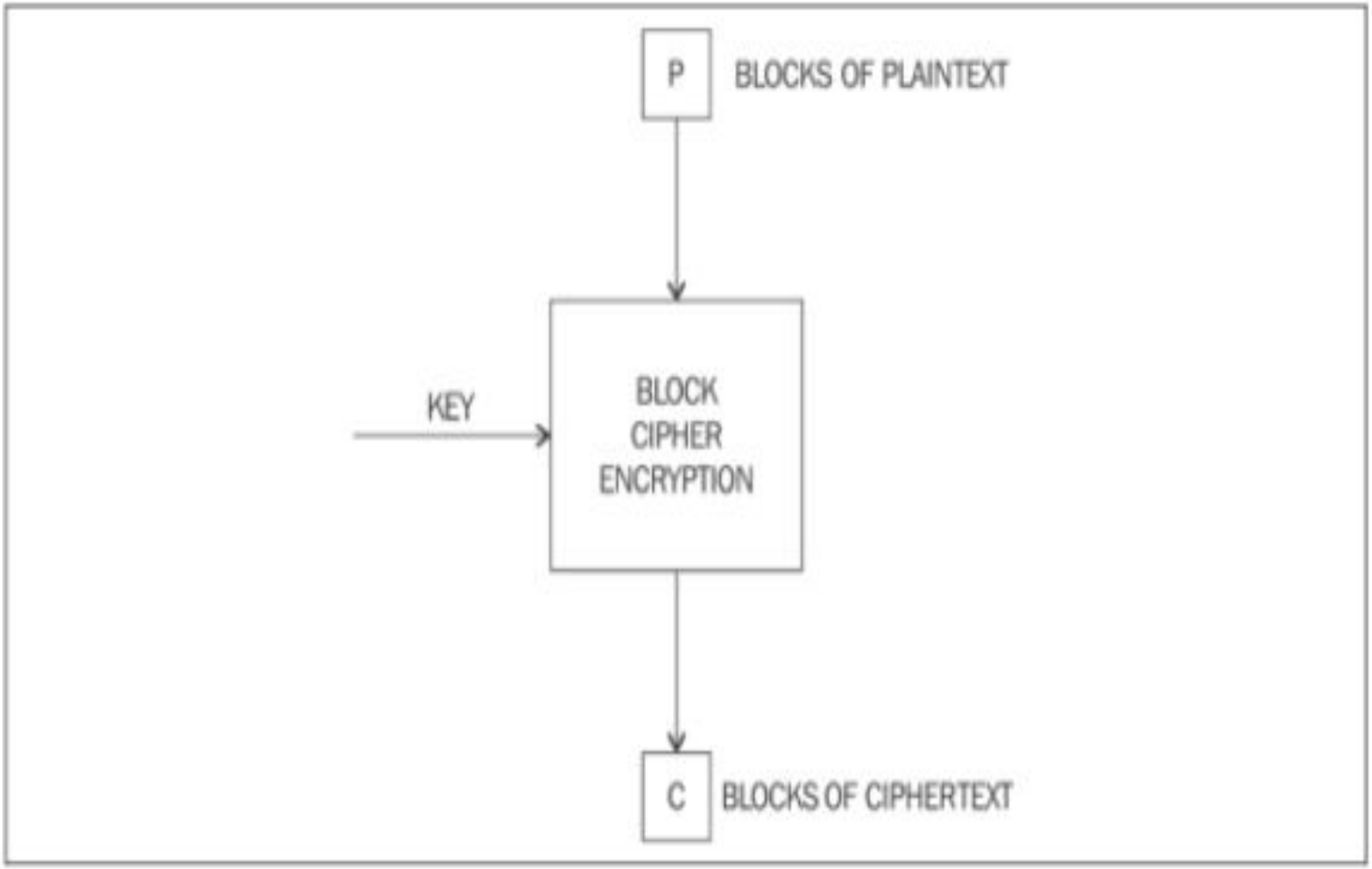


Operation of a stream cipher

Operation of a stream cipher

Block ciphers

- The text to be encrypted (plain text) is divided into blocks of fixed length and apply encryption block by block.
- Block ciphers are usually built using a design strategy known as Fiestel cipher.
- Recent block ciphers, such as AES (Rijndael) have been built using a combination of substitution and permutation called **substitution-permutation network (SPN)**

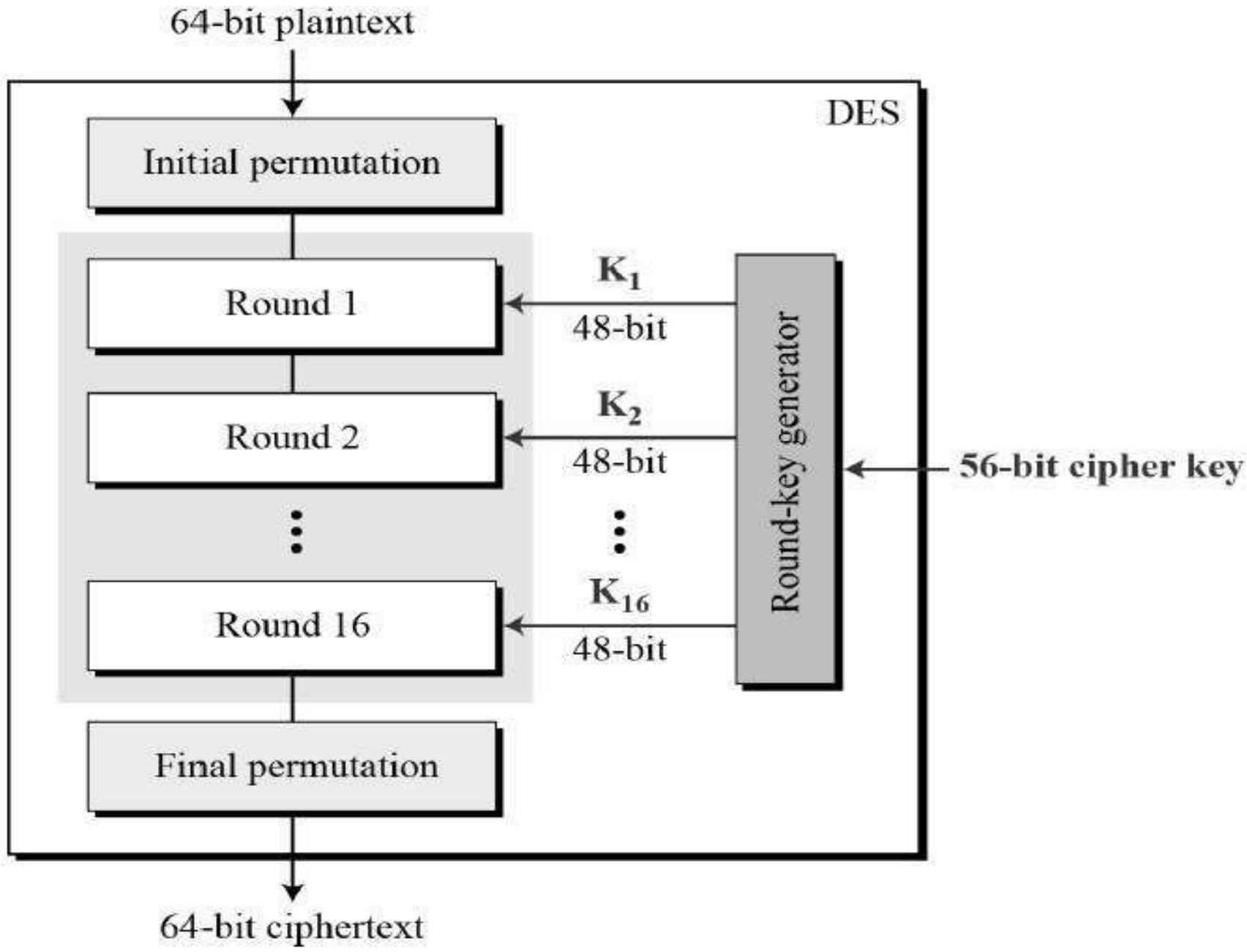


Simplified operation of a block cipher

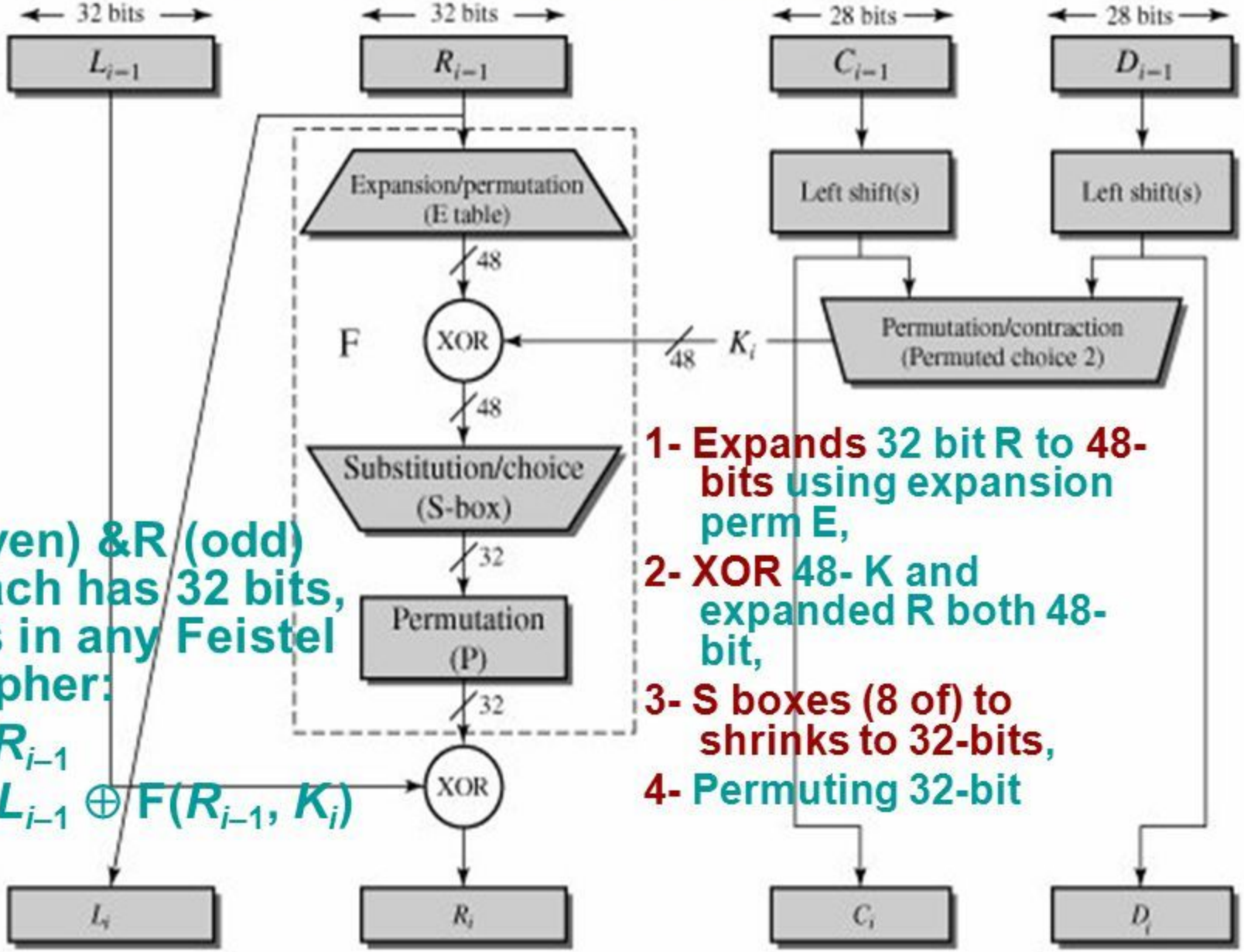
- Fiestel ciphers are based on the Fiestel network, which is a structure developed by *Horst Fiestel.*
- This structure is based on the idea of combining multiple rounds of repeated operations to achieve desirable cryptographic properties known as confusion and diffusion.
- Fiestel networks operate by dividing data into two blocks (left and right) and process these blocks via keyed round functions.

- Confusion makes the relationship between the encrypted text and plaintext complex.
- This is achieved by substitution in practice. For example, 'A' in plain text is replaced by 'X' in encrypted text.
- In modern cryptographic algorithms, substitution is performed using lookup tables called S-boxes. The diffusion property spreads the plain text statistically over the encrypted data, which ensures that even if a single bit is changed in the input text, it results in changing at least half (on average) of the bits in the cipher text.
- Confusion is required to make finding the encryption key very difficult even if many encrypted and decrypted data pairs are created using the same key.
- In practice, this is achieved by transposition or permutation.

- A key advantage of using Fiestel cipher is that encryption and decryption operations are almost identical and only require a reversal of the encryption process in order to achieve decryption.
- DES is a prime example of Fiestel-based ciphers


64-bit plaintext
DES
Initial permutation
Round 1
Round 2
Round 16
Final permutation
K_1
48-bit
K_2
48-bit
K_{16}
48-bit
Round-key generator
56-bit cipher key
64-bit ciphertext

DES Round Structure



L (even) &R (odd) each has 32 bits, as in any Feistel cipher:

$L_i = R_{i-1}$

$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$

1- Expands 32 bit R to 48-bits using expansion perm E,

2- XOR 48- K and expanded R both 48-bit,

3- S boxes (8 of) to shrinks to 32-bits,

4- Permuting 32-bit

- Various modes of operation for block ciphers are **Electronic Code Book (ECB), Cipher block chaining (CBC), Output Feedback Mode (OFB),** or **Counter mode (CTR).**
- These modes are used to specify the way in which an encryption function would be applied to the plain text.

Block encryption mode

In this mode, plaintext is divided into blocks of fixed length depending on the type of cipher used and then the encryption function is applied on each block.

Keystream generation modes

In this mode, the encryption function generates a keystream that is then XORed with the plaintext stream in order to achieve encryption.

Message authentication modes

- In this mode, a message authentication code is computed as a result of an encryption function. MAC is basically a cryptographic checksum that provides an integrity service.

- The most common method to generate MAC using block ciphers is CBC-MAC, where some part of the last block of the chain is used as a MAC.

Cryptographic Hashes

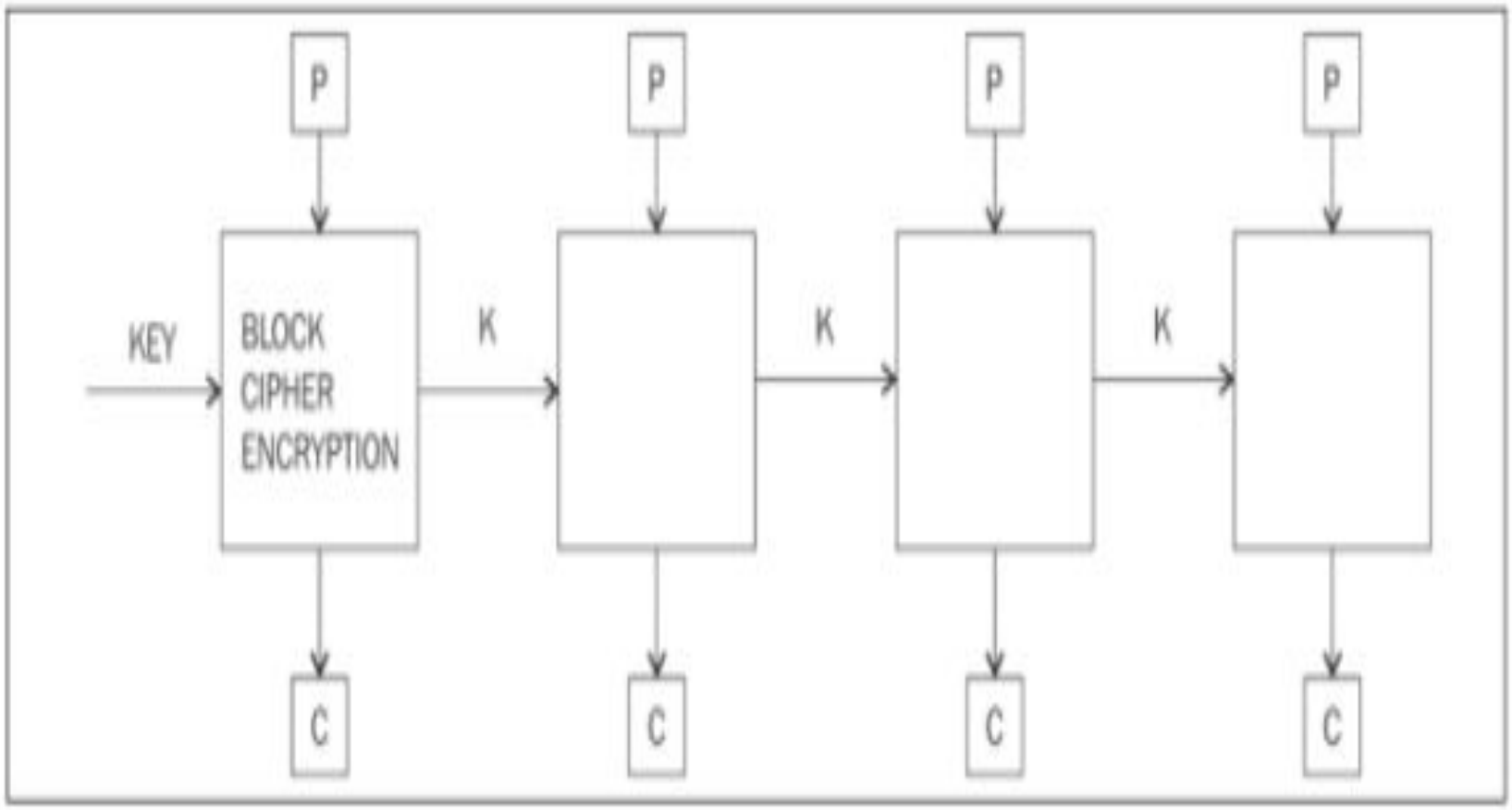
- Hash functions are basically used to compress a message to a fixed length digest.

- In this mode, block ciphers are used as a compression function to produce a hash of plain text.

Electronic code book

- This is a basic mode of operation in which the encrypted data is produced as a result of applying the encryption algorithm one by one separately to each block of plain text.

- This is the simplest mode but should not be used in practice as it is insecure and can reveal information

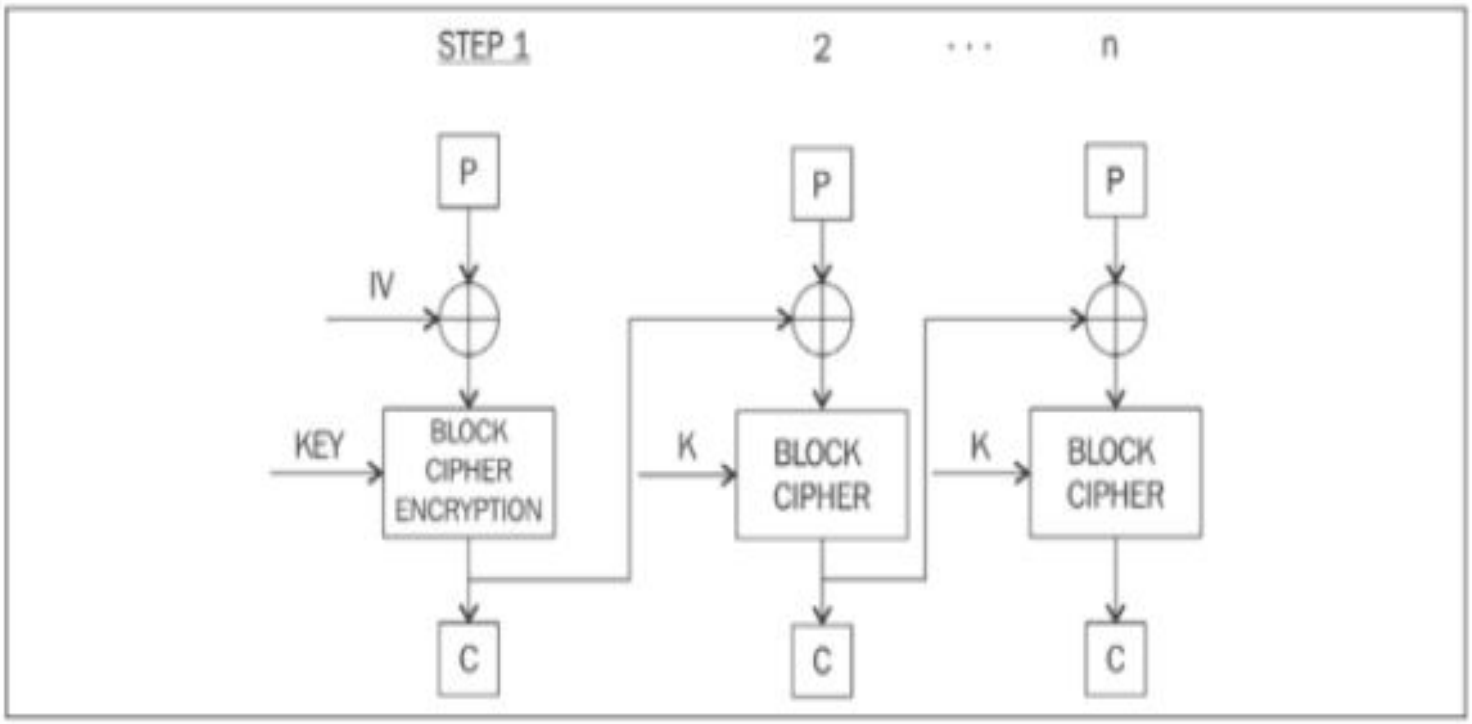


Electronic codebook mode for block ciphers

Electronic code book mode for block ciphers

Cipher block chaining

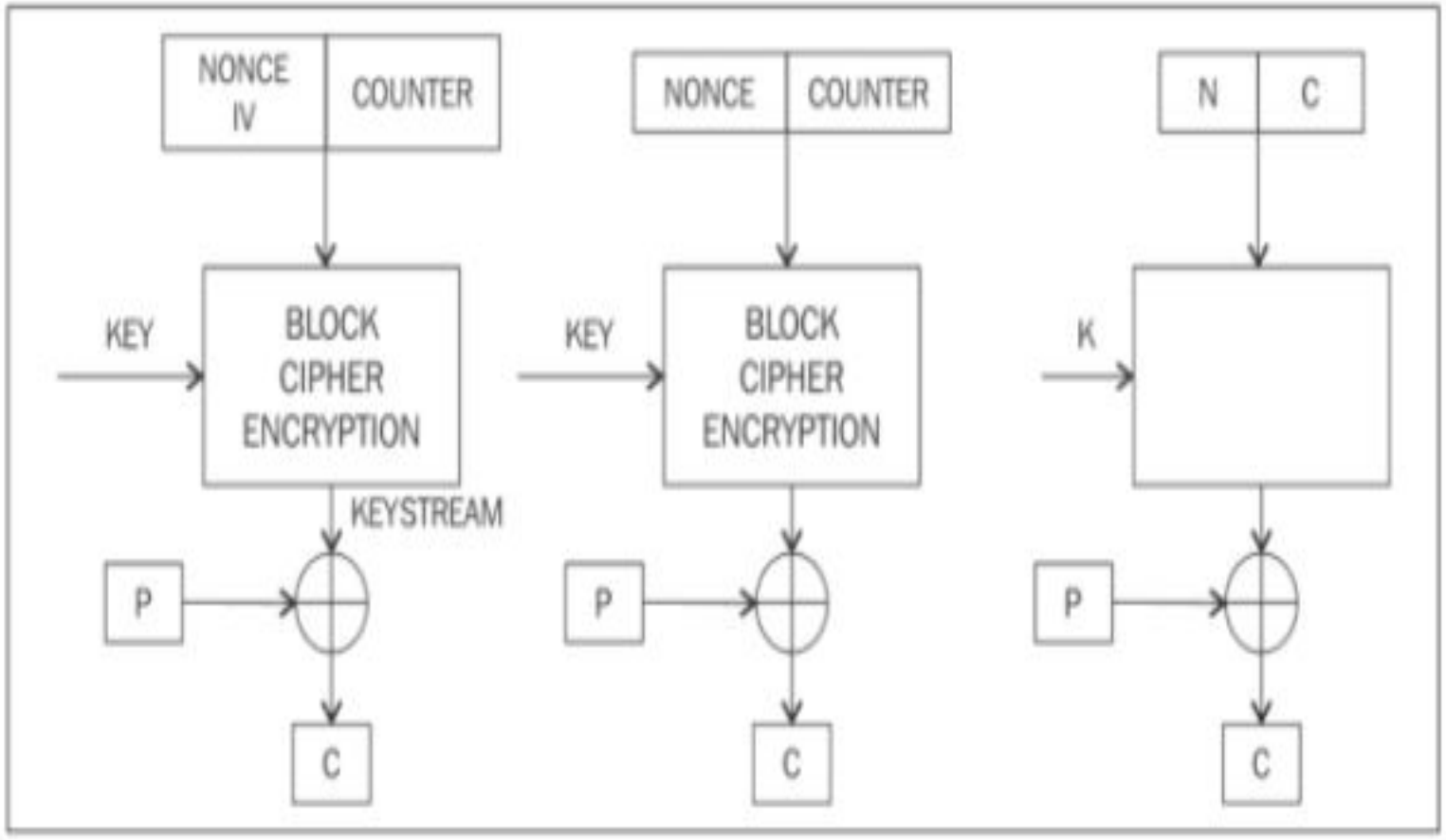
- In this mode, each block of plain text is XORed with the previous encrypted block.
- The CBC mode uses initialization vector IV to encrypt the first block.
- It is recommended that IV be randomly chosen.



Cipher block chaining mode

Counter mode

- The CTR mode effectively uses a block cipher as a stream cipher.
- In this case, a unique nonce is supplied that is concatenated with the counter value in order to produce a key stream.



Counter mode

There are other modes, such as **Cipher Feedback mode (CFB), Galois Counter mode (GCM),** and Output Feedback mode, which are also used in various scenarios.

Data Encryption Standard (DES)

- DES uses a key of only 56 bits, which has raised some concerns.
- This problem was addressed with the introduction of **Triple DES (3DES),** which proposed the usage of a 168-bit key using three 56-bit keys and the same number of executions of the DES algorithm, thus making brute force attacks almost impossible.
- But other limitations, such as slow performance and 64-bit block size, are not desirable.

Advanced Encryption Standard (AES)

- Original Rijndael allows different key and block sizes of 128-bit, 192-bit, and 256-bits, but in the AES standard, only a 128-bit block size is allowed.
- However, key sizes of 128-bit, 192-bit, and 256-bit are allowed.

AES STEPS

- During the AES Algorithm processing, a 4 by 4 array of bytes knows as *state* is modified using multiple rounds.
- Full encryption requires 10 to 14 rounds depending on the size of the key.

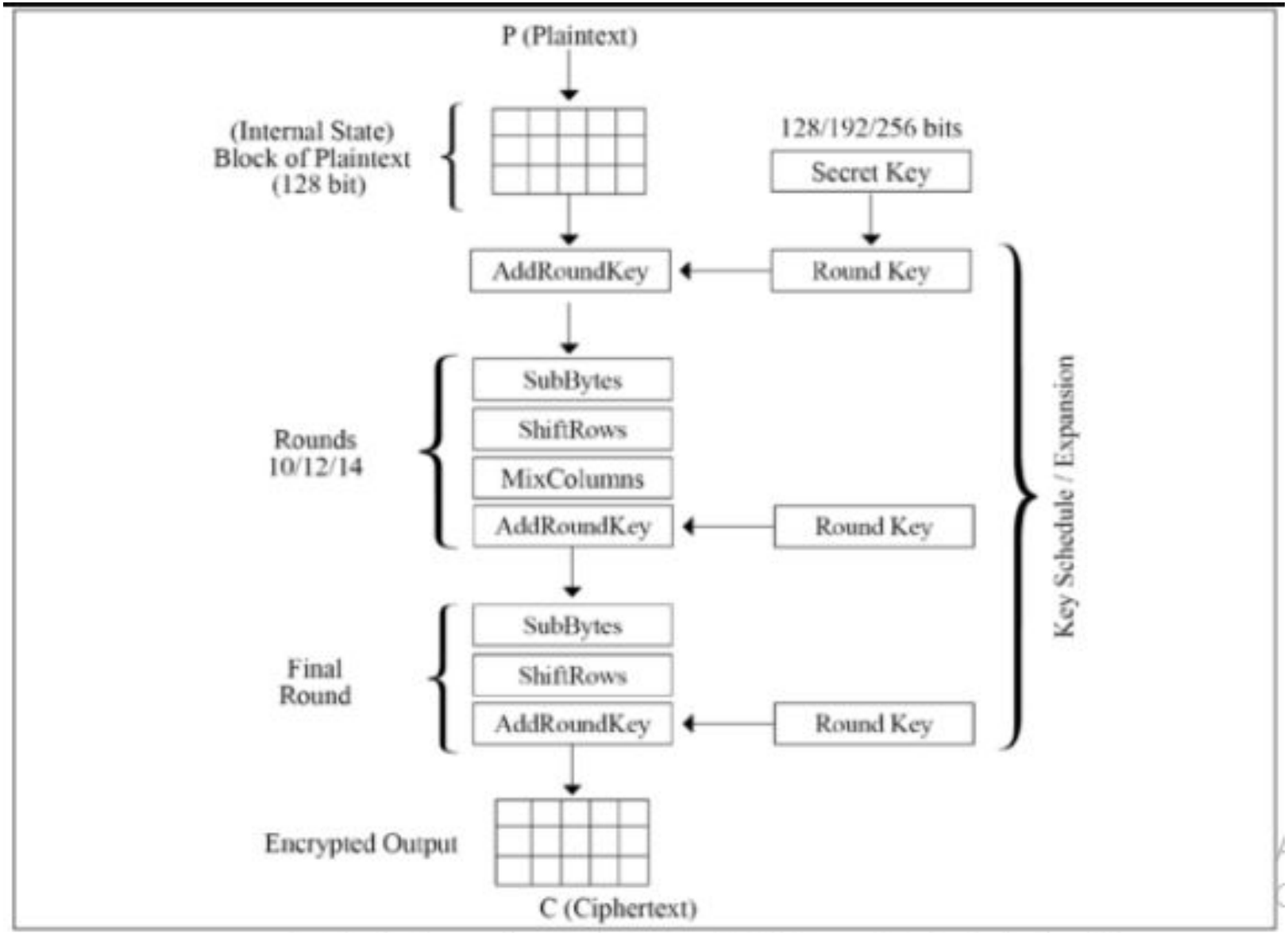
Key size	Number of rounds required
128-bit	10 rounds
192-bit	12 rounds
256-bit	14 rounds

- Once the state is initialized with the input to the cipher, four operations are performed in four stages in order to encrypt the input.
- These stages are AddRoundKey, SubBytes, ShiftRows, and MixColumns:

1. In the AddRoundKey step, the state array is XORed with a subkey, which is derived from the master key.
2. This is the substitution step where a lookup table (S-box) is used to replace all bytes of the state array.
3. This step is used to shift each row except the first one in the state array to the left in a cyclic and incremental manner.
4. Finally, all bytes are mixed in this step in a linear fashion column-wise.

- The preceding steps describe one round of AES.

- In the final round (either 10,12, or 14 depending on the key size), stage 4 is replaced with Add round key to ensure that the first three steps cannot be simply inverted back.



AES block diagram, showing the first round of AES encryption. In the final round,

- Various cryptocurrency wallets use AES encryption to encrypt locally-stored data.
- Bitcoin wallets use AES-256 in CBC mode to encrypt the private keys.
- In Ethereum wallets, AES-128 CTR is used. AES-128 bit in counter mode is used to encrypt the private key.
- Peers in Ethereum also use AES in counter mode (AES CTR) to encrypt their Peer to Peer (P2P) communications.

Asymmetric cryptography

- Asymmetric cryptography refers to a type of cryptography whereby the key that is used to encrypt the data is different from the key that is used to decrypt the data.
- Also known as public key cryptography, it uses public and private keys in order to encrypt and decrypt data, respectively.
- Various asymmetric cryptography schemes are in use, such as RSA, DSA, and El-Gammal, ECC
- Using private key for encryption and public key for decryption ->authentication, non-repudiation and integrity
- Using public key for encryption and private key for decryption -> confidentiality
- Key establishment and management schemes must be followed
- In order to identify the entity involved in a transaction, digital signatures and challenge-response protocols are combined

Integer factorization

Large integers are very hard to factor. Ex.RSA

Discrete logarithm

For example, consider the following equation:

$3^2 \bmod 10 = 9$

Given 9, finding 2 and the exponent of the generator 3 is very hard. This hard problem is commonly used in **Diffie-Hellman** key exchange and digital signature algorithms.

Elliptic curves

$$y^2 = x^3 + ax + b$$

a, *b* are integers that can have various values and are elements of the field on which the elliptic curve is defined.

- For cryptographic purposes, elliptic curve over prime finite fields is used instead of real numbers
- Cryptosystems based on elliptic curves are Elliptic Curve Digital Signatures Algorithm (ECDSA) and Elliptic Curve Diffie-Hellman (ECDH) key exchange.

Pubic and private keys

- Private keys is a randomly generated number that is kept secret and held privately by the users
- In RSA, instead of 1024 bits, key length of 2048 bits is used
- A public key is available publicly and published by the private key owner
- Modulus generation:
  - Select p and q very large primes
  - Multiply p and q , n=p.q to generate modulus n

- Generate co-prime:
  - Assume a number called e. It should be greater than 1 and less than ***(p-1) (q-1).***
  - In other words, e must be such a number that no number other than 1 can be divided into e and ***(p-1) (q-1).*** This is called co-prime, that is, e is the co-prime of ***(p-1)(q-1).***
- Generate public key:
  - Modulus generated in step 1 and e generated in step 2, is a public key which can be shared but p and q should be secret
- Generate private key:
  - private key, d is inverse of e modulo (p-1)(q-1), calculated using extended euclidean algorithm. Use large primes for p and q, so that it is computationally infeasible to find n.

RSA Encryption $C = P^e \bmod n$

Decryption $P = C^d \bmod n$

ELLIPTIC CURVE CRYPTOGRAPHY (ECC)

- ECC is based on the discrete logarithm problem that is based on elliptic curves over finite fields (Galois fields).
- The main benefit of ECC over other types of public key algorithms is that it needs a smaller key size while providing the same level of security as, for example, RSA.
- As ECC needs less space to operate, it is becoming very popular on embedded platforms or in systems where storage resources are limited
- The same level of security can be achieved in ECC by only using 256-bit operands as compared to 3072-bits in RSA.

Mathematics behind ECC

- Elliptic curve groups consist of points on the curve over a finite field.
- An elliptic curve can be defined as: $y^2 = x^3 + Ax + B \bmod p$

Point Addition: $P + Q$

Point doubling: $2P = P + P$

$$x_3 = s^2 - x_1 - x_2 mod\ p$$

$$y_3 = s(x_1 - x_3) - y_1 mod\ p$$

$$S = \frac{(y_2 - y_1)}{(x_2 - x_3)} mod\ p$$

Discrete logarithm problem

- The discrete logarithm problem in ECC is based on the idea that under certain conditions, all points on an elliptic curve form a cyclic group.
- A private key is a randomly chosen integer, whereas the public key is a point on the curve.

Hash functions are typically used to provide data integrity services.

- These can be used as one-way functions and to construct other cryptographic primitives, such as MACs and digital signatures.
- Some applications used hash functions as a means of generating **pseudo random numbers (PRNGs).**

ECC

Global Public Elements

$E_q(a, b)$	elliptic curve with parameters a, b, and q, where q is a prime or an integer of the form 2^m
G	point on elliptic curve whose order is large value n

User A Key Generation

Select private n_A	$n_A < n$
Calculate public P_A	$P_A = n_A \times G$

User B Key Generation

Select private n_B	$n_B < n$
Calculate public P_B	$P_B = n_B \times G$

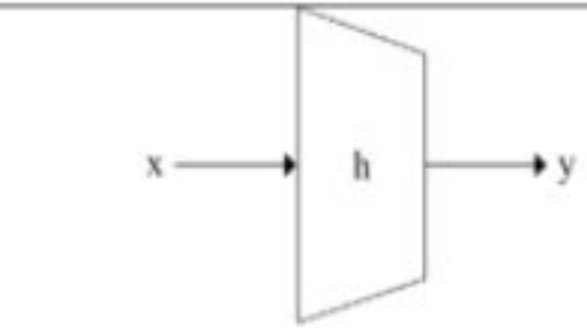
Calculation of Secret Key by User A

$K = n_A \times P_B$

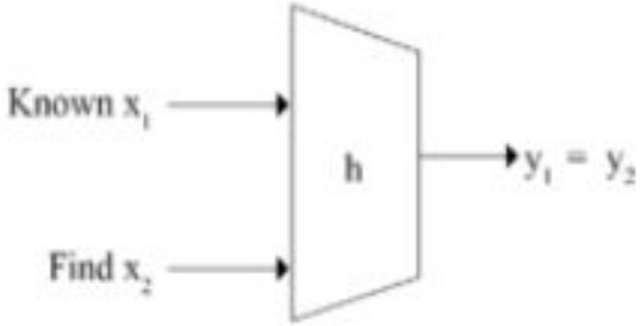
Calculation of Secret Key by User B

$K = n_B \times P_A$

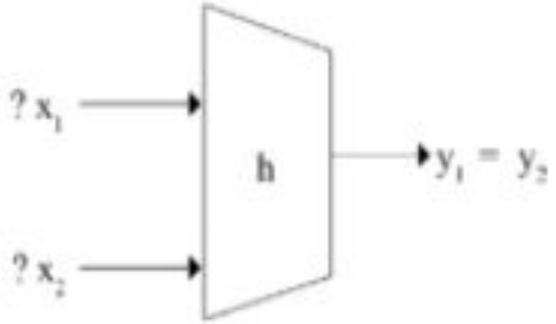
Figure 10.7 ECC Diffie-Hellman Key Exchange



1 - PRE-IMAGE RESISTANCE



2 - SECOND PRE-IMAGE RESISTANCE



3 - STRONG COLLISION RESISTANCE

Three security properties of hash functions

Properties:

- pre-image resistance (one-way property)
  h(x)=y

- second pre-image resistance (weak collision resistance)
  Given x and h(x), it is impossible to find any other message m

- collision resistance (strong collision resistance)
  Two different input messages should not hash to the same output

❑ Avalanche effect specifies that a small change, even a single character change in the input text, will result in a totally different hash output.

❑ MD5 is a 128-bit hash function that was commonly used for file integrity checks.

❑ SHA-1 : 160-bit hash function is used commonly in SSL and TLS

❑ SHA-2: SHA-224, SHA-256, SHA-384 and SHA-512

❑ SHA-3: SHA3-224, SHA3-256, SHA3-384 and SHA3-512

RIPEMD: RIPEMD is *RACE Integrity Primitives Evaluation Message Diges which is* build based on MD4. 128-bit, 160-bit, 256-bit, and 320-bit are available

Whirlpool: This is based on a modified version of Rijndael cipher known as W. and uses the Miyaguchi-Preneel compression function, which is a type of one-way function used for the compression of two fixed length inputs into a single fixed length output.

It is a single block length compression function.

- Hash functions have many practical applications ranging from simple file integrity checks and password storage to be used in cryptographic protocols and algorithms.
- They are used in hash tables, distributed hash tables, bloom filters, virus fingerprinting, peer-to-peer P2P file sharing, and many other applications.
- In bitcoin, Proof of work function uses SHA-256 twice in order to verify the computational effort spent by miners.
- RIPEMD 160 is used to produce bitcoin addresses
- Ethereum doesn't use SHA-3 but Keccak, after some modifications such as increase in the number of rounds and simpler message padding compared to SHA-3

SHA-256 algorithm:

- Pre-processing:

    1. Padding of the message, which is used to make the length of a block to 512-bits if it is smaller than the required block size of 512-bits.

    2. Parsing the message into message blocks that ensure that the message and its padding is divided into equal blocks of 512-bits.

    3. Setting up the initial hash value, which is the eight 32-bit words obtained by taking the first 32-bits of the fractional parts of the square roots of the first eight prime numbers. These initial values are randomly chosen in order to initialize the process and gives a level of confidence that no backdoor exists in the algorithm.

- Hash computation:

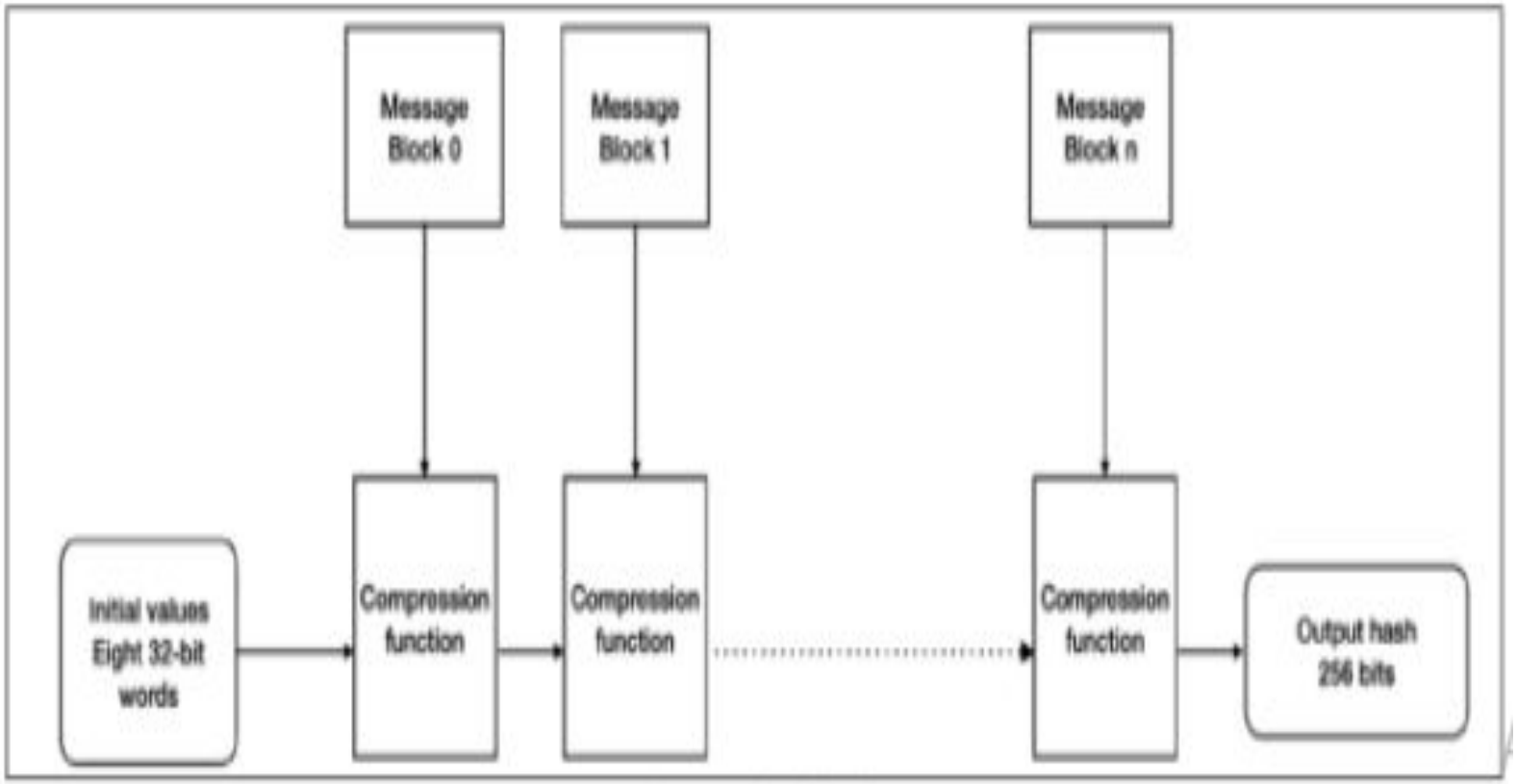
    1. Each message block is processed in a sequence and requires 64 rounds to compute the full hash output. Each round uses slightly different constants to ensure that no two rounds are the same.

    2. First, the message schedule is prepared.

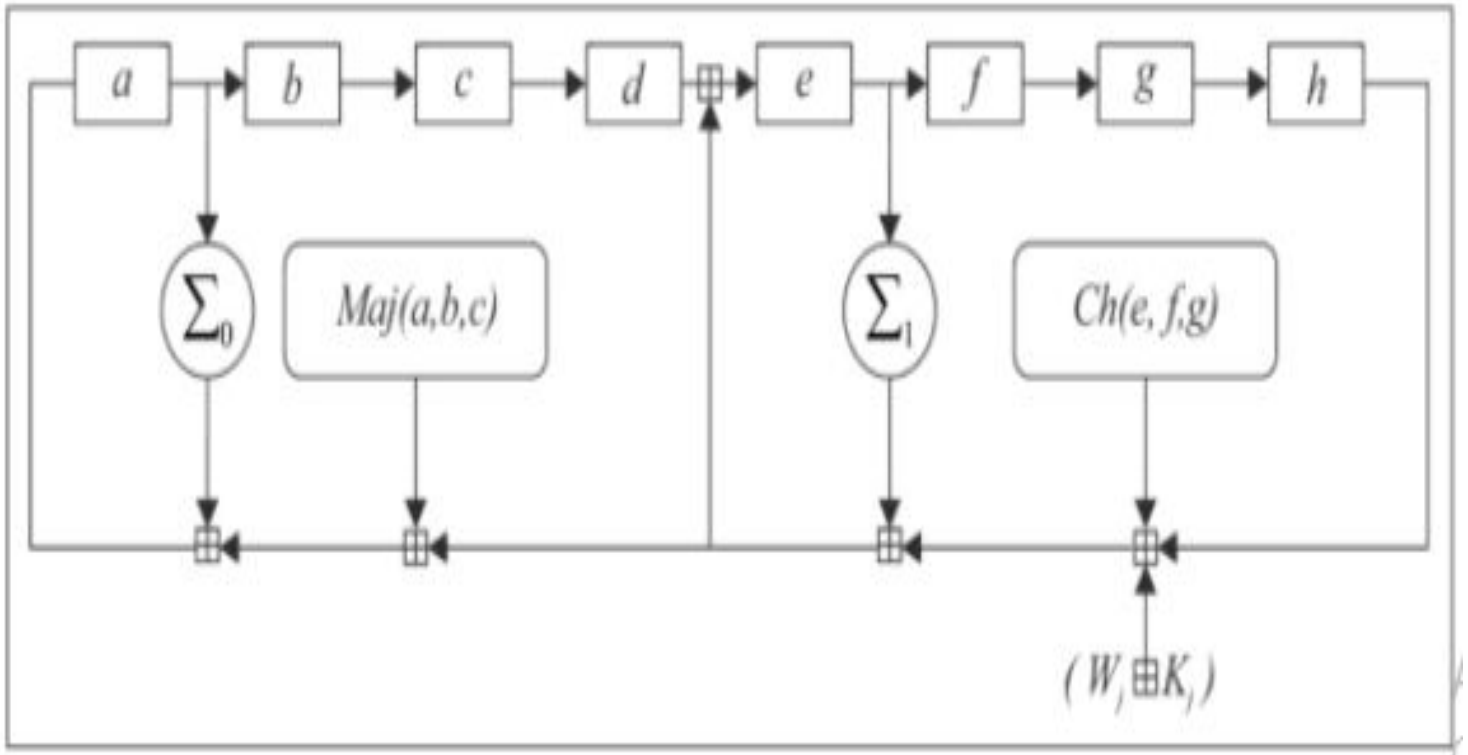
    3. Then, eight working variables are initialized.

    4. Then, the intermediate hash value is calculated.

    5. Finally, the message is processed and the output hash is produced.

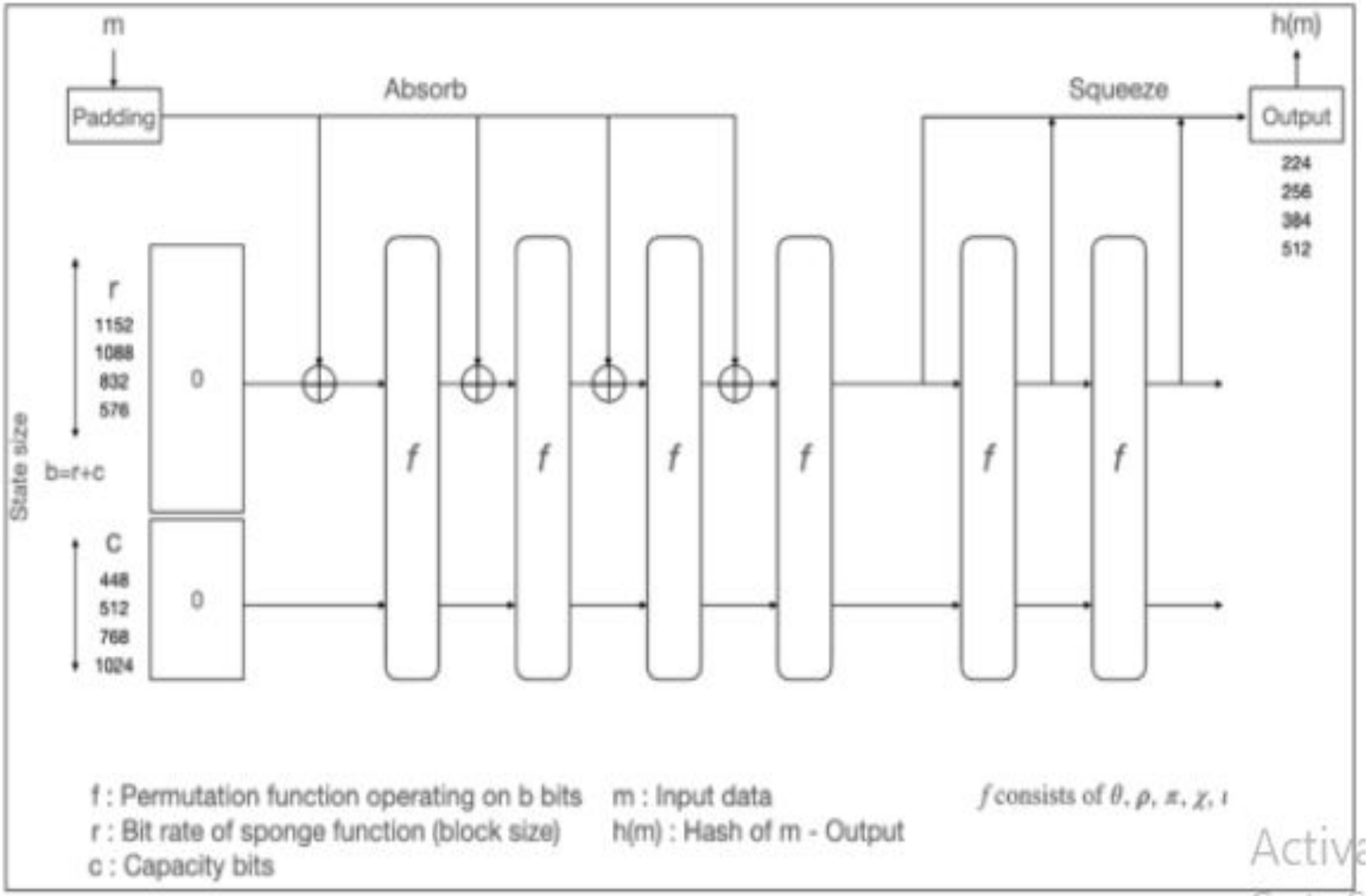


SHA-256 high level overview



One round of an SHA-256 compression function

- A newer approach called sponge and squeeze construction is used in Keccak, which is basically a random and unkeyed permutation model.
- Different variants of SHA3 such as SHA3-224, SHA3-256, SHA3-384, SHA3-512, SHAKE128, and SHAKE256. SHAKE128 and SHAKE256 allow the output to be extended to any desired length.
- SHA-256 is used in Bitcoin's PoW algorithm



The SHA-3 absorbing and squeezing function in SHA3

r (block size)	c (capacity)	Output hash size
1152	448	224
1088	512	256
832	768	384
576	1024	512

θ – Theta: XOR bits in the state, used for mixing

ρ – Rho: Diffusion function performing rotation of bits

π – Pi: Diffusion function

χ – Chi: XOR each bit, bitwise combine

ι – Iota: Combination with round constants

Sponge and squeeze model:

- First, the data is absorbed into the sponge after applying padding, where it is then changed into a subset of permutation state using XOR and then the output is squeezed out of the sponge function that represents the transformed state.
- Rate is the input block size of a sponge function, whereas capacity determines the generic security level.

Message Authentication codes (MACs)

- MACs are sometimes called keyed hash functions and can be used to provide message integrity and data origin authentication.
- MACs can be constructed using block ciphers or hash functions.

MACs using block ciphers

- In this approach, block ciphers are used in the **Cipher block chaining mode (CBC mode)** in order to generate a MAC.
- Any block cipher-for example, AES in the CBC mode-can be used.
- The MAC of the message is in fact the output of the last round of the CBC operation.
- The length of the MAC output is the same as the block length of the block cipher used to generate MAC.
- MACs are verified simply by computing the MAC of the message and comparing it with the received MAC.
- If they are the same, then the message integrity is confirmed; otherwise, the message is considered altered.
- MACs work like digital signatures, but they cannot provide the nonrepudiation service due to their symmetric nature.

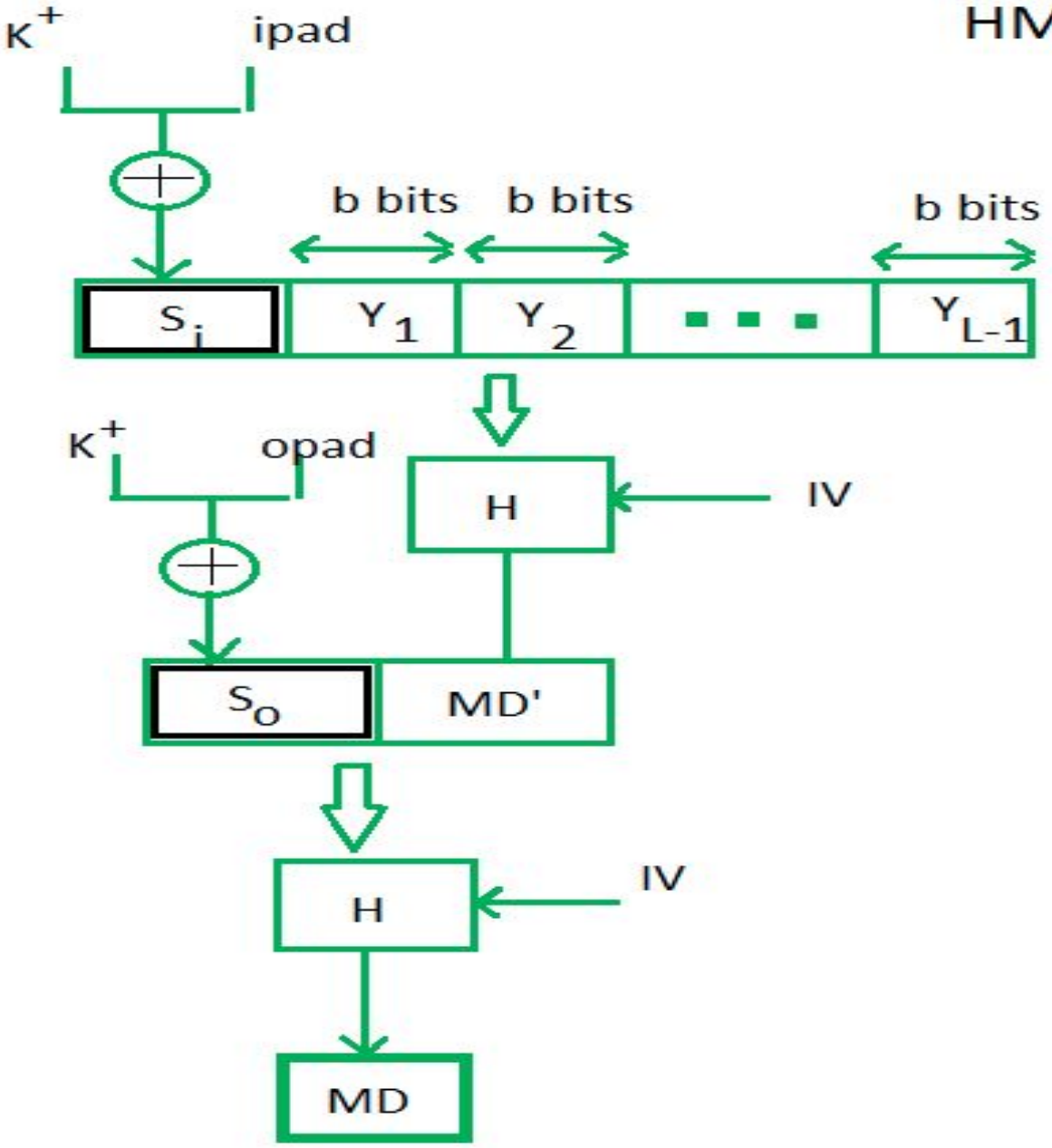
HMACs (hash-based MACs)

- Similar to the hash function, they produce a fixed length output and take an arbitrarily long message as the input.
- In this scheme, the sender signs a message using MAC and the receiver verifies it using the shared key.
- The key is hashed with the message using either of the two methods known as secret prefix or the secret suffix method.
- In the first method, the key is concatenated with the message, that is, the key comes first and the message comes after, whereas in the latter method, the key comes after the message:

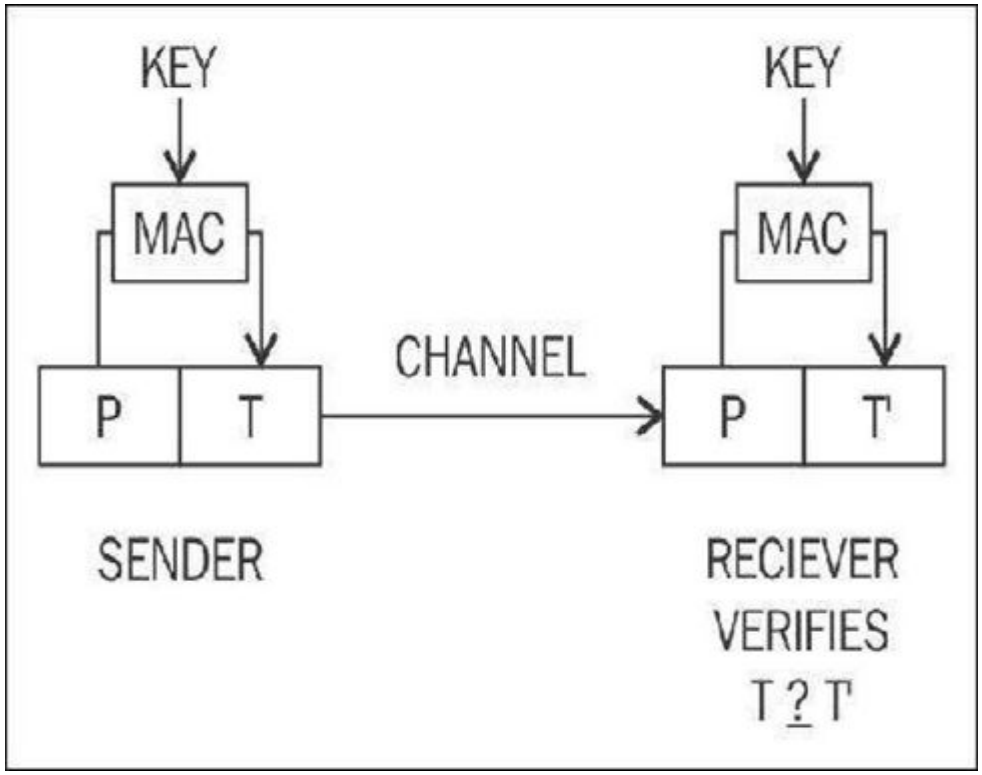
Secret prefix: M = MACk(x) = h(k\\x)

Secret suffix: M=MACk(x) = h(x\\k)

HMAC construct



- HMAC constructions schemes that use various techniques, such as **ipad** and **opad** (inner padding and outer padding)



RSA digital signature algorithms

Two steps:

1. Calculate the hash value of the data packet

2. Sign the hash value with the signer's private key

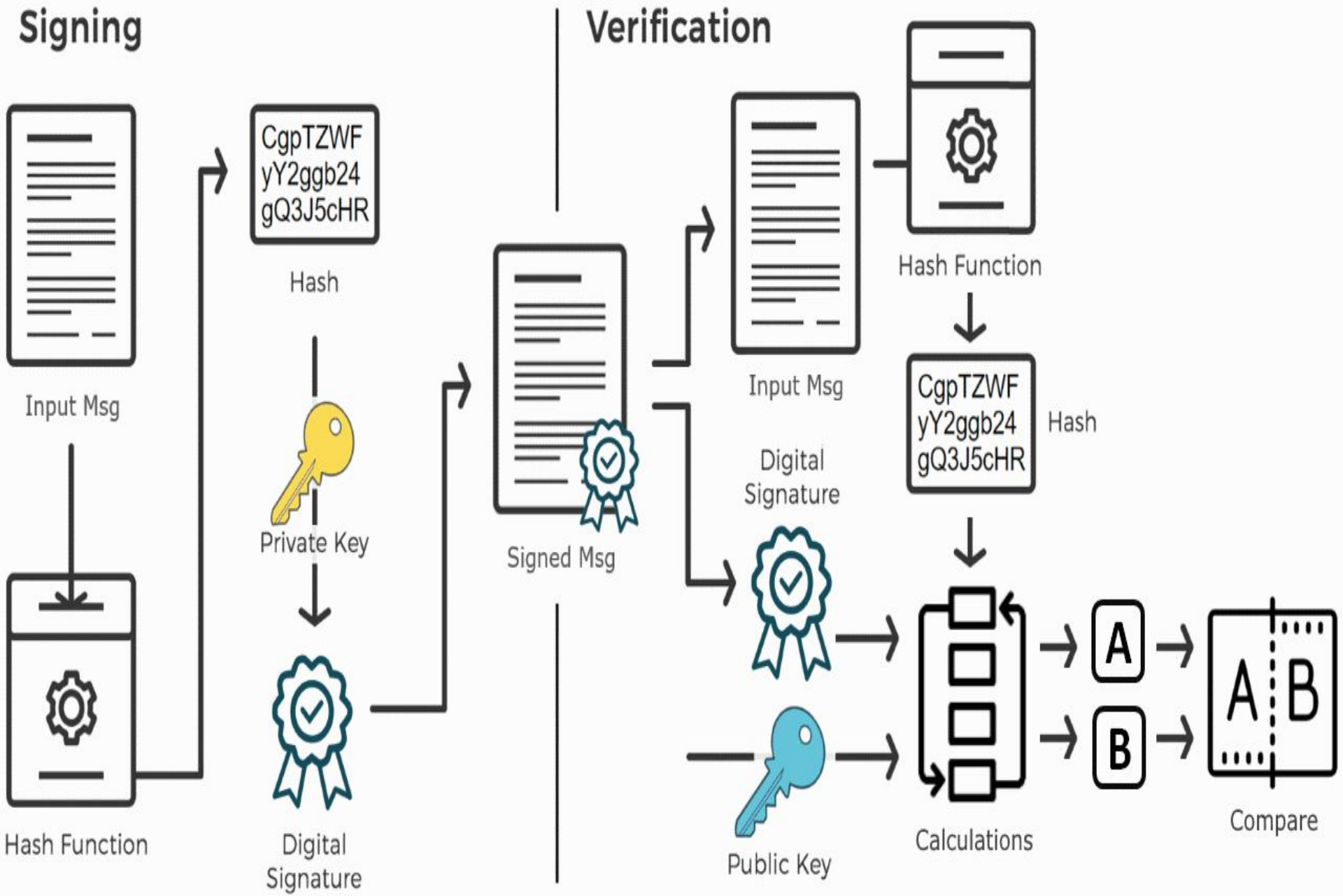
Properties:

a. authenticity

b. unforgeability

c. non-reusability

RSA key size (bits)	Elliptic curve key size (bits)
1,024	160
2,048	224
3,072	256
7,680	384
15,360	521


Signing
Verification
Input Msg
Hash Function
CgpTZWF
yY2ggb24
gQ3J5cHR
Hash
Private Key
Digital
Signature
Signed Msg
Input Msg
Hash Function
CgpTZWF
yY2ggb24
gQ3J5cHR
Hash
Digital
Signature
Public Key
Calculations
A
B
A B
Compare

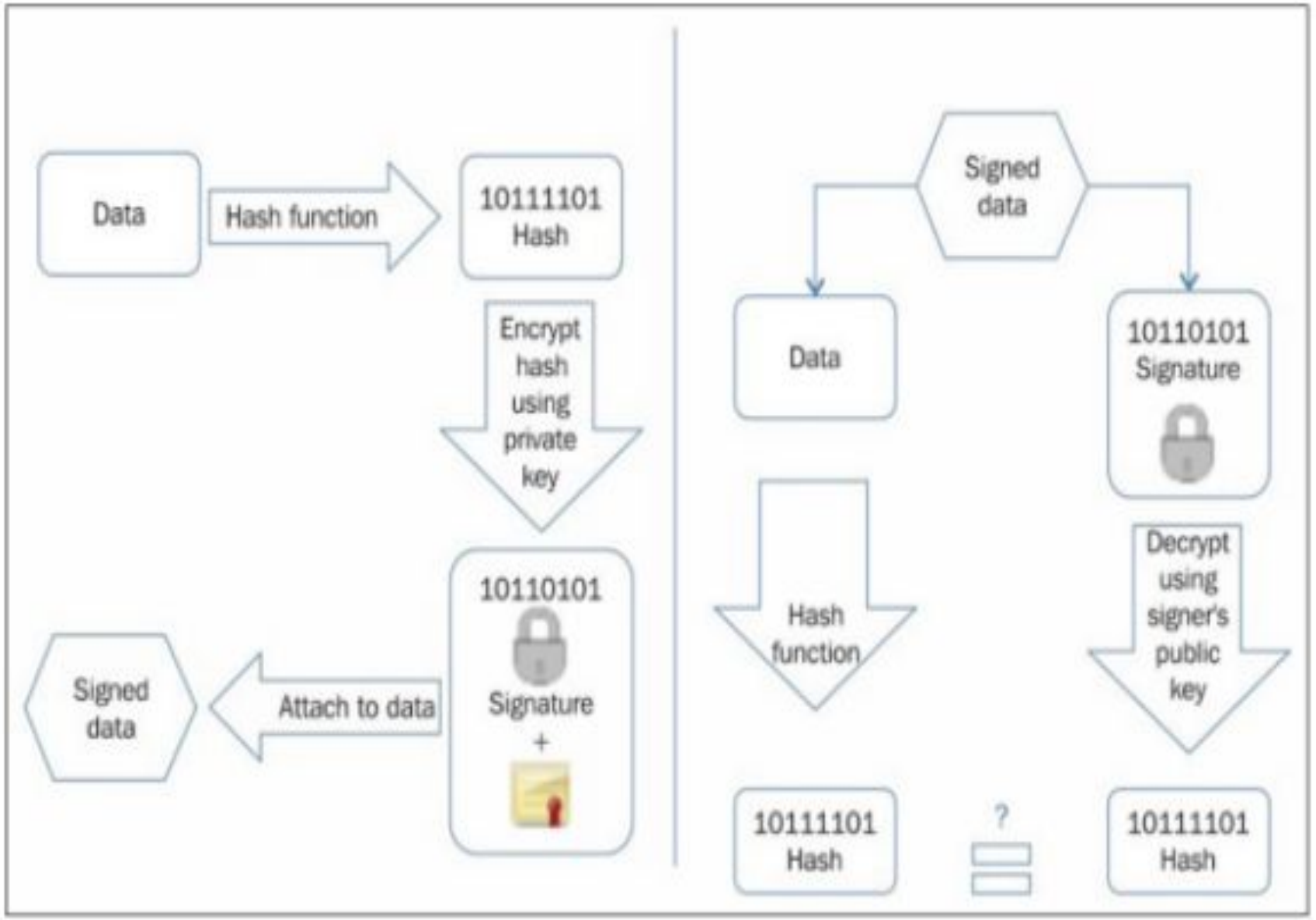
Sign then encrypt

The sender digitally signs the data using private key and appends the signature to the data and then encrypts the data and the digital signature using receiver's public key. This scheme is more secure than encrypt then sign scheme.

Encrypt then sign

The sender encrypts the data using receiver's public key and then digitally signs the encrypted data.

In practice, a digital certificate that contains the digital signature is issued by a Certificate Authority (CA) that associates a public key with an identity.



Digital signing (left) and verification process (right) (example of RSA digital signatures)

Elliptic curve digital signature algorithm

ECDSA is a DSA based on elliptic curves. It is based on modular exponentiation and discrete logarithm problem. It is used on the Bitcoin and Ethereum blockchain platforms to validate messages and provide data integrity services.

Algorithm steps:

1. Key Generation
2. Signing
3. Verifying

ECDSA signing - How are signatures created?

The ECDSA signing algorithm takes a message and a private key and produces a signature, a pair of integers (r, s).

The **ECDSA signing** algorithm works by:

1. Calculating the message **hash**, using a cryptographic hash function e.g. SHA-256

$$h = hash(msg)$$

2. Generating securely a **random** number ***k***
3. Calculating the random point $R = k * G$ and take its x-coordinate: $r = R.x$
4. Calculating the signature proof s using the formula:

$$s = k^{-1} * (h + p * r) \bmod n$$

where p is the signer's private key, and the order n

5. Return the signature (r, s).

ECDSA Signature Verification

The **ECDSA signature verification** algorithm works by converting s back to R (R') using the public key and message hash. The recovered R's x-coordinate r' is compared with r from the signature:

1. Calculating the message hash, with the same hash function used when signing
2. Calculating the modular inverse $s1$ of the signature proof:

$$s1 = s^{-1} \pmod{n}$$

3. Recovering the random point used during the signing:

$$R' = (h * s1) * G + (r * s1) * pubKey$$

4. Retrieving r' from R': $r' = R'.x$
5. Calculating the signature validation result by comparing whether $r' == r$

1. First, define an elliptic curve E with the following:
    - Modulus P
    - Coefficients a and b
    - Generator point A that forms a cyclic group of prime order q
2. An integer d is chosen randomly so that $0 < d < q$.
3. Calculate public key **B** so that **B = d A**.
    - The public key is a sextuple in the form shown here:

$$Kpb = (p,a,b,q,A,B)$$

    - The private key is a randomly chosen integer d in step 2:

$$Kpr = d$$

Now, the signature can be generated using the private and public key.

4. An ephemeral key K_e is chosen, where $0 < K_e < q$. It should be ensured that K_e is truly random and that no two signatures have the same key; otherwise, the private key can be calculated.
5. Another value R is calculated using $R = K_e A$; that is, by multiplying A (the generator point) and the random ephemeral key.
6. Initialize a variable r with the x coordinate value of point R so that $r = xR$.
7. The signature can be calculated as follows:

$$S = (h(m) + dr)K_e{}^{-1} mod\ q$$

Here, m is the message for which the signature is being computed, and $h(m)$ is the hash of the message m.

8. Signature verification is carried out by following this process:
    - Auxiliary value w is calculated as $w = s\text{-}1 mod\ q$
    - Auxiliary value $u1 = w.\ h(m)\ mod\ q$
    - Auxiliary value $u2 = w.\ r\ mod\ q$
    - Calculate point P, $P = u1A + u2B$
9. Verification is carried out as follows:
    - r, s is accepted as a valid signature if the x-coordinate of point P calculated in *Step 4* has the same value as the signature parameter $r\ mod\ q$; that is:

$$Xp = r\ mod\ q\ means\ valid\ signature$$

$$Xp\ != r\ mod\ q\ means\ invalid\ signature$$

Merkle Trees

- Merkle trees allow secure and efficient verification of large data sets.
- It is a binary tree in which first, the inputs are placed at the leaves (node with no children), and then values of pairs of child nodes are hashed together in order to produce a value for the parent node (internal node) until a single hash value known as Merkle root is achieved.

PATRICIA TREES

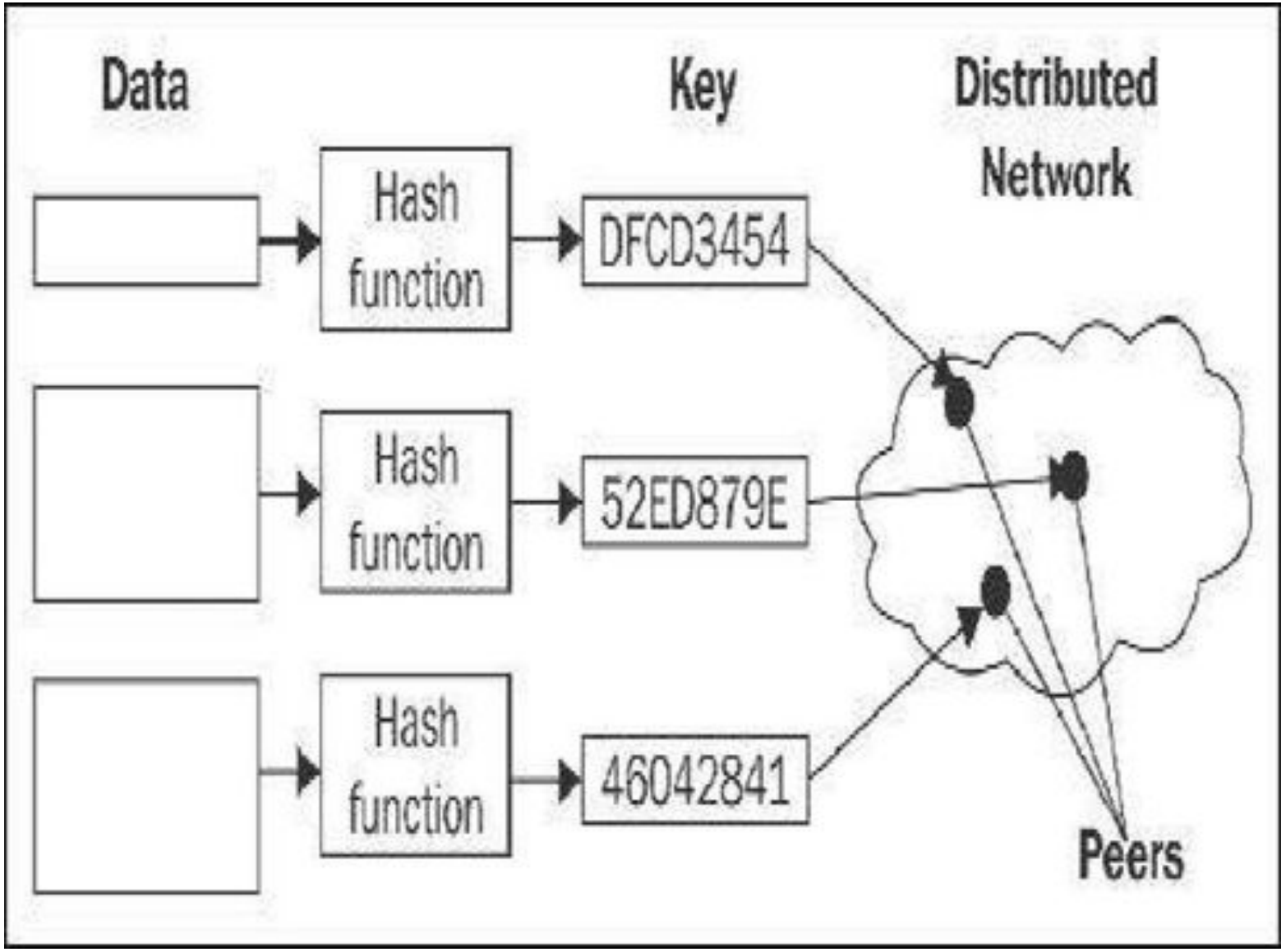
- **A** trie or a digital tree is an ordered tree data structure used to store a dataset.
- **Practical Algorithm to Retrieve Information Coded in Alphanumeric (Patricia),** also known as Radix tree, is a compact representation of a trie in which a node that is the only child of a parent is merged with its parent.
- **Merkle-Patricia tree,** based on the definitions of Patricia and Merkle, is a tree that has a root node that contains the hash value of the entire data structure.

DISTRIBUTED HASH TABLES (DHTS)

- A hash table is a data structure that is used to map keys to values.
- Internally, a hash function is used to calculate an index into an array of buckets, from which the required value can be found.
- Buckets have records stored in them using a hash key and are organized in a particular order.
- Distributed hash table is a data structure where data is spread across various nodes and nodes are equivalent to buckets in a peer-to-peer to network.

Working of DHT:

- First, data is passed through a hash function, which results in generating a compact key.
- This key is then linked with the data (values) on the peer-to-peer network.
- When users on the network request the data (via the filename), the filename can be hashed again to produce the same key and any node on the network can then be requested to find the corresponding data.
- DHTs provides decentralization, fault tolerance, and scalability.


Data
Key
Distributed Network
Hash function
DFCD3454
Hash function
52ED879E
Hash function
46042841
Peers

UNIT-III

Introduction

- Bitcoin has started a revolution with the introduction of the very first fully decentralized digital currency, and one that has proven to be extremely secure and stable.
- In 1982, *David Chaum* proposed a scheme that used blind signatures to build untraceable digital currency.
- In this scheme, a bank would issue digital money by signing a blind and random serial number presented to it by the user.
- The user could then use the digital token signed by the bank as currency.
- The limitation in this scheme was that the bank had to keep track of all used serial numbers.
- This was a central system by design and required to be trusted by the users.

- Later on in 1990, *David Chaum* proposed a refined version named e-cash that not only used blinded signature, but also some private identification data to craft a message that was then sent to the bank.
- This scheme allowed the detection of double spending but did not prevent it.
- If the same token was used at two different locations, then the identity of the double spender would be revealed, e-cash could only represent a fixed amount of money.
- *Adam Back's* hashcash, introduced in 1997, was originally proposed to thwart e-mail spam.
- The idea behind hashcash was to solve a computational puzzle that was easy to verify but comparatively difficult to compute.

- The idea was that for a single user and a single e-mail, extra computational effort was not noticeable, but someone sending a large number of spam e-mails would be discouraged as the time and resources required to run the spam campaign would increase substantially.
- B-money was proposed by *Wei Dai* in 1998, which introduced the idea of using Proof of Work to create money.
- A major weakness in the system was that an adversary with higher computational power could generate unsolicited money without allowing the network to adjust to an appropriate difficulty level.
- The system lacked details on the consensus mechanism between nodes and some security issues such as Sybil attacks were also not addressed.

- At the same time, *Nick Szabo* introduced the concept of BitGold, which was also based on the Proof of Work mechanism but had the same problems as b-money with the exception that the network difficulty level was adjustable.
- *Tomas Sander* and *Ammon TaShama* introduced an e-cash scheme in 1999 that, for the first time, used Merkle trees to represent coins and zero knowledge proofs to prove the possession of coins.
- In the scheme, a central bank was required that kept a record of all used serial numbers.

- This scheme allowed users to be fully anonymous albeit at a computational cost. **RPOW (Reusable Proof of Work)** was introduced by *Hal Finney* in 2004 and used the hashcash scheme by *Adam Back* as a proof of computational resources spent to create the money.
- This was also a central system that kept a central database to keep track of all used POW tokens.
- This was an online system that used remote attestation made possible by a **trusted computing platform (TPM hardware).**

Bitcoin

- Bitcoin is a peer-to-peer electronic cash that does need an intermediary bank to transfer payments between peers.
- Bitcoin is built on decades of cryptographic research such as the research in Merkle trees, hash functions, public key cryptography, and digital signatures.
- Aforesaid technologies are combined in bitcoin to create the world's first decentralized currency.
- The key issue that has been addressed in bitcoin is an elegant solution to the Byzantine Generals problem along with a practical solution of the double-spend problem.

Bitcoin definition

- Bitcoin can be defined in various ways; it's a protocol, a digital currency, and a platform.
- It is a combination of peer-to-peer network, protocols, and software that facilitate the creation and usage of the digital currency named bitcoin.
- Decentralization of currency was made possible for the first time with the invention of bitcoin.
- Moreover, the double spending problem was solved in an elegant and ingenious way in bitcoin.
- Double spending problem arises when, for example, a user sends coins to two different users at the same time and they are verified independently as valid transactions.

KEYS AND ADDRESSES

- Elliptic curve cryptography is used to generate public and private key pairs in the Bitcoin network.
- The bitcoin address is created by taking the corresponding public key of a private key and hashing it twice, first with the SHA256 algorithm and then with RIPEMD160.
- The resultant 160-bit hash is then prefixed with a version number and finally encoded with a Base58Check encoding scheme.
- The bitcoin addresses are 26-35 characters long and begin with digit 1 or 3. A typical bitcoin address looks like a string shown here:

  IANAguGG8bikEv2fYsTBnRUmx7QUcK58wt

- This is also commonly encoded in a QR code for easy sharing.

- Currently, there are two types of addresses, the commonly used P2PKH and another P2SH type, starting with 1 and 3, respectively.
- In the early days, bitcoin used direct Pay-to-Pubkey, which is now superseded by P2PKH.
- However, direct Pay-to-Pubkey is still used in bitcoin for coinbase addresses.
- Addresses should not be used more than once; otherwise, privacy and security issues can arise.
- Avoiding address reuse circumvents anonymity issues to an extent, bitcoin has some other security issues as well, such as transaction malleability, which requires different approaches to resolve.

PUBLIC KEYS IN BITCOIN

- In public key cryptography, public keys are generated from private keys.
- Bitcoin uses ECC based on the SECP256K1 standard.
- A private key is randomly selected and is 256-bit in length.
- Public keys can be presented in an uncompressed or compressed format.
- Public keys are basically *x* and y coordinates on an elliptic curve and in an uncompressed format and are presented with a prefix of 04 in a hexadecimal format.
- *X* and *Y* coordinates are both 32-bit in length. In total, the compressed public key is 33 bytes long as compared to 65 bytes in the uncompressed format.
- The compressed version of public keys basically includes only the *X* part, since the *Y* part can be derived from it.
- The reason why the compressed version of public keys works is that the bitcoin client initially used uncompressed keys, but starting from bitcoin core client 0.6, compressed keys are used as the standard.

PRIVATE KEYS IN BITCOIN

- Private keys are basically 256-bit numbers chosen in the range specified by the SECP256K1 ECDSA recommendation.
- Any randomly chosen 256-bit number from Oxl to OxFFFF FFFF FFFF FFFF FFFF FFFF FFFE BAAE DCE6 AF48 A03B BFD2 5E8C D036 4140 is a valid private key.
- Private keys are usually encoded using **Wallet Import Format (WIF)** in order to make them easier to copy and use.
- WIF can be converted into private key and vice versa.
- Also, **Mini Private Key Format** is sometimes used to encode the key in under 30 characters in order to allow storage where physical space is limited,
- For example, etching on physical coins or damage-resistant QR codes.
- The bitcoin core client also allows the encryption of the wallet that contains the private keys.

BITCOIN CURRENCY UNITS

The smallest bitcoin denomination is the Satoshi.

BASE58CHECK ENCODING

- Bitcoin addresses are encoded using the Base58check encoding.
- This encoding is used to limit the confusion between various characters, such as OOII as they can look the same in different fonts.
- The encoding basically takes the binary byte arrays and converts them into human-readable strings.
- This string is composed by utlilizing a set of 58 alphanumeric symbols

VANITY ADDRESSES

- As bitcoin addresses are based on base58 encoding, it is possible to generate addresses that contain human-readable messages.
- Public address encoded in QR
- Vanity addresses are generated using a purely brute-force method.

Transactions

- Transactions can be as simple as just sending some bitcoins to a bitcoin address, or it can be quite complex depending on the requirements.
- Each transaction is composed of at least one input and output.
- Inputs can be thought of as coins being spent that have been created in a previous transaction and outputs as coins being created.
- If a transaction is minting new coins, then there is no input and therefore no signature is needed.
- If a transaction is to sends coins to some other user (a bitcoin address), then it needs to be signed by the sender with their private key and a reference is also required to the previous transaction in order to show the origin of the coins.
- Coins are, in fact, unspent transaction outputs represented in Satoshis.
- Transactions are not encrypted and are publicly visible in the blockchain.
- Blocks are made up of transactions and these can be viewed using any online blockchain explorer.

The transaction life cycle

1. A user/sender sends a transaction using wallet software or some other interface.
2. The wallet software signs the transaction using the sender's private key.
3. The transaction is broadcasted to the Bitcoin network using a flooding algorithm.
4. Mining nodes include this transaction in the next block to be mined.
5. Mining starts once a miner who solves the Proof of Work problem broadcasts the newly mined block to the network. Proof of Work is explained in detail later in this chapter.
6. The nodes verify the block and propagate the block further, and confirmation starts to generate.
7. Finally, the confirmations start to appear in the receiver's wallet and after approximately six confirmations, the transaction is considered finalized and confirmed. However, six is just a recommended number; the transaction can be considered final even after the first confirmation. The key idea behind waiting for six confirmations is that the probability of double spending is virtually eliminated after six confirmations.

The transaction structure

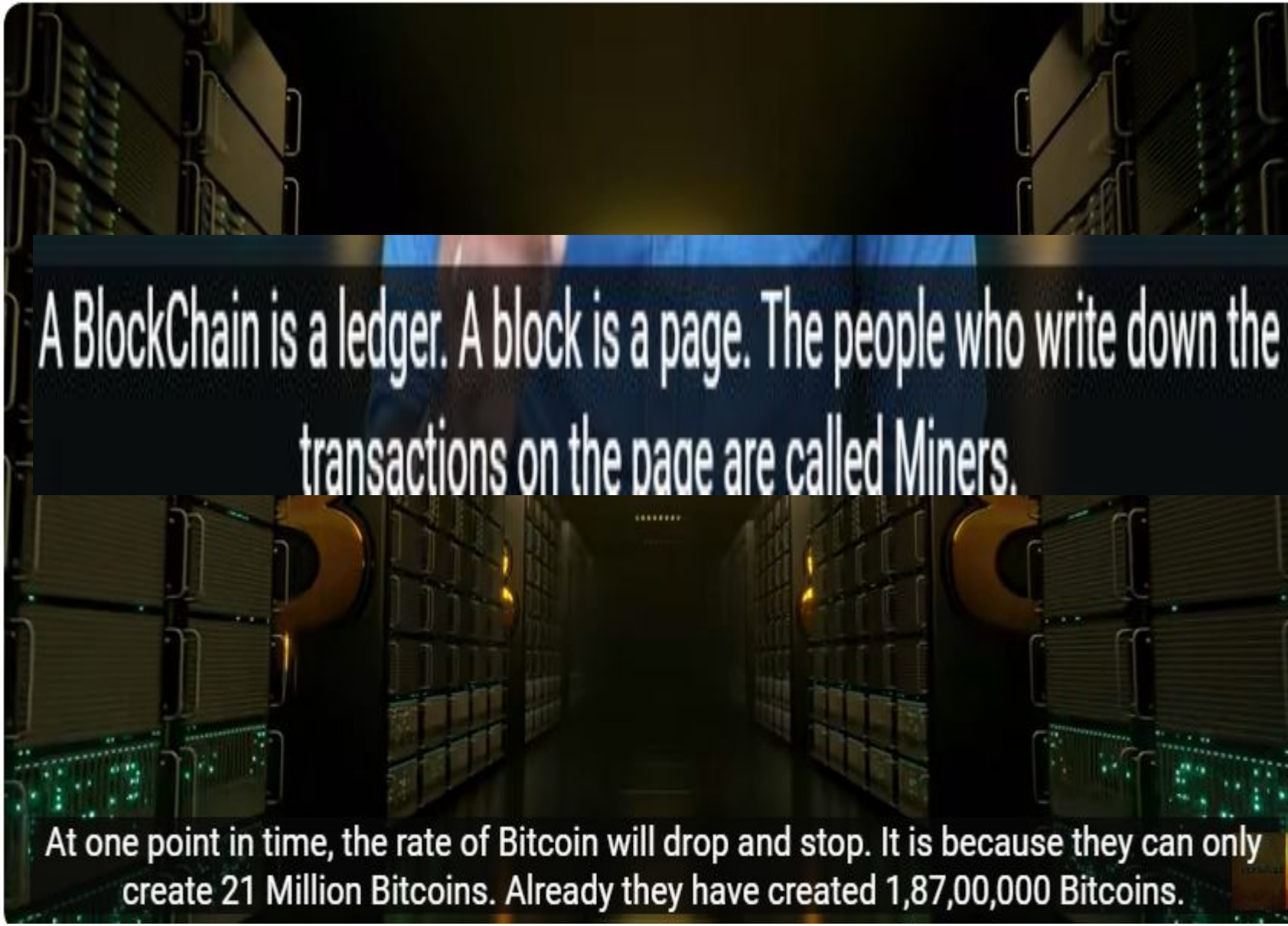
A transaction at a high level contains metadata, inputs, and outputs. Transactions are combined to create a block.

- **MetaData:** This part of the transaction contains some values such as the size of the transaction, the number of inputs and outputs, the hash of the transaction, and a lock_time field. Every transaction has a prefix specifying the version number.
- **Inputs:** Generally, each input spends a previous output. Each output is considered an **Unspent Transaction Output (UTXO)** until an input consumes it.
- **Outputs:** Outputs have only two fields, and they contain instructions for the sending of bitcoins. The first field contains the amount of Satoshis, whereas the second field is a locking script that contains the conditions that need to be met in order for the output to be spent.

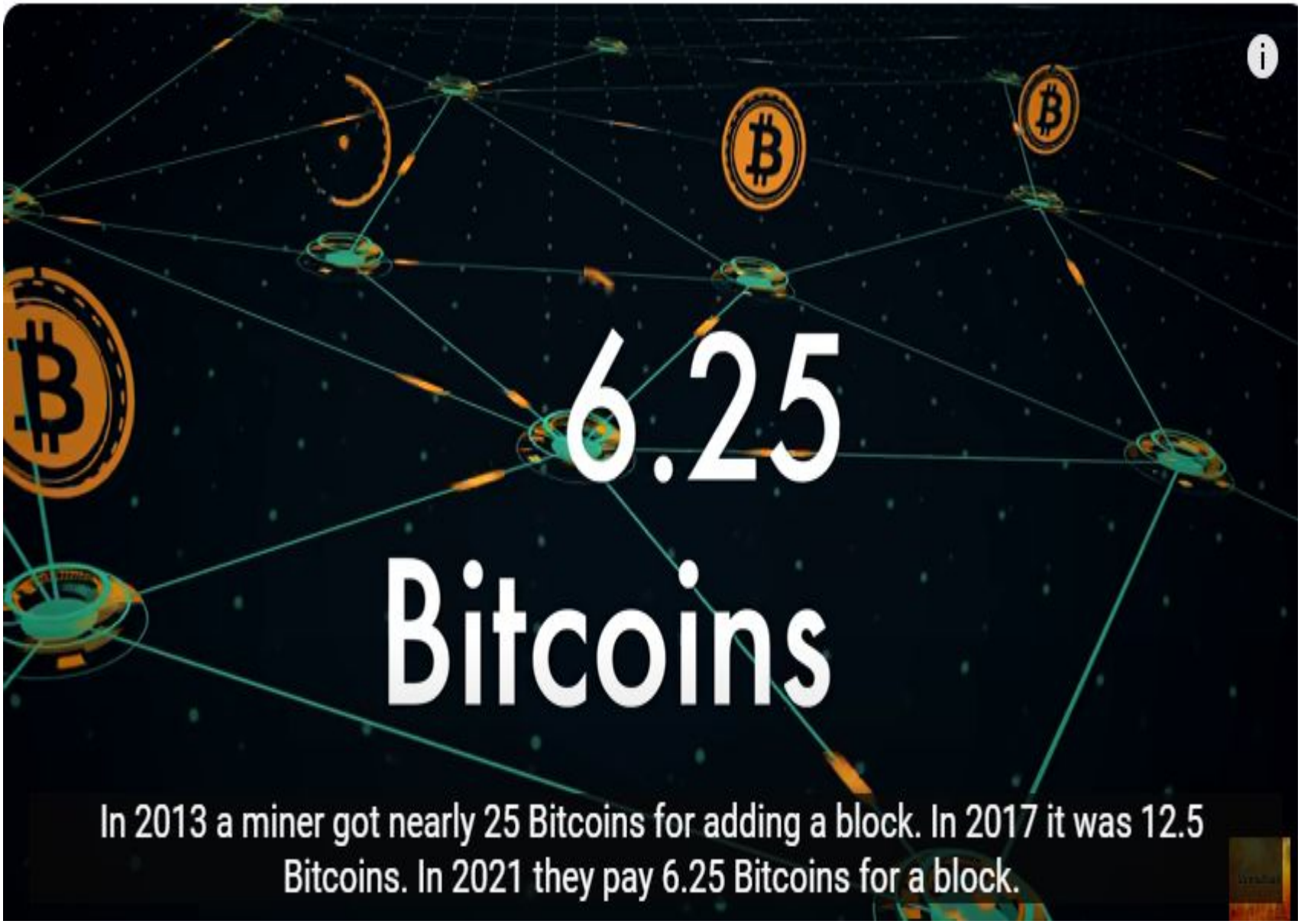
  More information on transaction spending using locking and unlocking scripts and producing outputs is discussed later in this section.
- **Verification:** Verification is performed using bitcoin's scripting language.

THE SCRIPT LANGUAGE

- Bitcoin uses a simple stack-based language called *script* to describe how bitcoins can be spent and transferred.
- It is not Turing complete and has no loops to avoid any undesirable effects of long running/hung scripts on the bitcoin network.
- This scripting language is based on a Forth-like syntax and uses a reverse polish notation in which every operand is followed by its operators.
- It is evaluated from the left to the right using a **Last in First Out (LIFO)** stack.
- Scripts use various Opcodes or instructions to define their operation.
- Opcodes are also known as words, commands, or functions.
- Earlier versions of the bitcoin node had a few Opcodes that are no longer used due to bugs discovered in their design.


A BlockChain is a ledger. A block is a page. The people who write down the transactions on the page are called Miners.
At one point in time, the rate of Bitcoin will drop and stop. It is because they can only create 21 Million Bitcoins. Already they have created 1,87,00,000 Bitcoins.


900
Bitcoin
For each block, they get 6.25 Bitcoins. So 144 Block x 6.25 = 900 Bitcoins. By 2140 the Bitcoin will stop adding blocks.


6.25
Bitcoins
In 2013 a miner got nearly 25 Bitcoins for adding a block. In 2017 it was 12.5 Bitcoins. In 2021 they pay 6.25 Bitcoins for a block.

- The various categories of the scripting Opcodes are constants, flow control, stack, bitwise logic, splice, and arithmetic, cryptography, and lock time.
- A transaction script is evaluated by combining ScriptSig and ScriptPubKey. ScriptSig is the unlocking script, whereas ScriptPubKey is the locking script.
- This is how a transaction is evaluated to be spent; first, it is unlocked and then it is spent.
- ScriptSig is provided by the user who wishes to unlock the transaction.
- ScriptPubkey is part of the transaction output and specifies the conditions that need to be fulfilled in order to spend the output.
- In other words, outputs are locked by the ScriptPubkey (Locking script) that contains the conditions, when met will unlock the output, and coins can then be redeemed.

Types of transaction

- There are various scripts available in bitcoin to handle the value transfer from the source to the destination.
- These scripts range from very simple to quite complex depending upon the requirements of the transaction.
- Standard transactions are evaluated using IsStandardQ and IsStandardTxQ tests and only standard transactions that pass the test are generally allowed to be mined or broadcasted on the bitcoin network.
- However, nonstandard transactions are valid and allowed on the network.

- **Pay to Public Key Hash (P2PKH):** P2PKH is the most commonly used transaction type and is used to send transactions to the bitcoin addresses. The format of the transaction is shown as folows:

ScriptPubKey: OP_DUP 0P_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG

ScriptSig: <sig> <pubKey>

The ScriptPubKey and ScriptSig parameters are concatenated together and executed.

- **Pay to Script Hash (P2SH):** P2SH is used in order to send transactions to a script hash (that is, the addresses starting with 3) and was standardized in BIP16. In addition to passing the script, the redeem script is also evaluated and must be valid. The template is shown as follows:

ScriptPubKey: 0P_HASH160 <redeemScriptHash> OP_EQUAL ScriptSig:

[<sig>...<sign>] <redeemScript>

- **MultiSig (Pay to MultiSig):** M of n multisignature transaction script is a complex type of script where it is possible to construct a script that required multiple signatures to be valid in order to redeem a transaction. Various complex transactions such as escrow and deposits can be built using this script. The template is shown here:

ScriptPubKey: <m> <pubKey> [<pubKey> ...] <n> OP_CHECKMULTISIG

ScriptSig: 0 [<sig > ... <sign>]

Raw multisig is obsolete, and multisig is usually part of the P2SH redeem script, mentioned in the previous bullet point.

- **Pay to Pubkey:** This script is a very simple script that is commonly used in coinbase transactions. It is now obsolete and was used in an old version of bitcoin. The public key is stored within the script in this case, and the unlocking script is required to sign the transaction with the private key.

The template is shown as follows:

<PubKey> OP_CHECKSIG

- **Null data/OP_RETURN:** This script is used to store arbitrary data on the blockchain for a fee. The limit of the message is 40 bytes. The output of this script is unredeemable because 0P_RETURN will fail the validation in any case. ScriptSig is not required in this case.

The template is very simple and is shown as follows:

0P_RETURN <data>

- All transactions are eventually encoded into the hex before transmitting over the bitcoin network.

COINBASE TRANSACTIONS

- A coinbase transaction or generation transaction is always created by a miner and is the first transaction in a block.
- It is used to create new coins.
- It includes a special field, also called *coinbase,* which acts as an input to the coinbase transaction.
- This transaction also allows up to 100 bytes of arbitrary data that can be used to store arbitrary data.

WHAT IS UTXO?

Unspent Transaction Output (UTXO) is an unspent transaction output that can be spent as an input to a new transaction.

Transaction fee

- Transaction fees are charged by the miners.
- The fee charged is dependent upon the size of the transaction.
- Transaction fees are calculated by subtracting the sum of the inputs and the sum of the outputs.
- The fees are used as an incentive for miners to encourage them to include a user transaction in the block the miners are creating.
- All transactions end up in the memory pool, from where miners pick up transactions based on their priority to include them in the proposed block.
- The calculation of priority is introduced later in this chapter; however, from a transaction fee point of view, a transaction with a higher fee will be picked up sooner by the miners.
- There are different rules based on which fee is calculated for various types of actions, such as sending transactions, inclusion in blocks, and relaying by nodes.
- Fees are not fixed by the Bitcoin protocol and are not mandatory; even a transaction with no fee will be processed in due course but may take a very long time.

Contracts

- Contracts are basically transactions that use the bitcoin system to enforce a financial agreement.
- This is a simple definition but has far-reaching consequences as it allows users to design complex contracts that can be used in many real-world scenarios.
- Contracts allow the development of a completely decentralized, independent, and reduced risk platform.
- Various contracts, such as escrow, arbitration, and micropayment channels, can be built using the bitcoin scripting language.
- The current implementation of a script is very limited, but various types of contracts are still possible to develop.
- For example, the release of funds only if multiple parties sign the transaction or perhaps the release of funds only after a certain time has elapsed.
- Both of these scenarios can be realized using multiSig and transaction lock time options.

Transaction malleability

- Transaction malleability in bitcoin was introduced due to a bug in the bitcoin implementation.
- Due to this bug, it becomes possible for an adversary to change the Transaction ID of a transaction, thus resulting in a scenario where it would appear that a certain transaction has not been executed.
- This can allow scenarios where double deposits or withdrawals can occur. In other words, this bug allows the changing of the unique ID of a bitcoin transaction before it is confirmed.
- If the ID is changed before confirmation, it would seem that the transaction did not happen at all, which can then allow double deposits or withdrawal attacks.

Transaction pools

- Also known as *memory pools,* these pools are basically created in local memory by nodes in order to maintain a temporary list of transactions that are not yet confirmed in a block.
- Transactions are included in a block after passing verification and based on their priority.

Transaction verification

This verification process is performed by bitcoin nodes.

1. Check the syntax and ensure that the syntax of the transaction is correct.

2. Verify that inputs and outputs are not empty.
3. Check whether the size in bytes is less than the maximum block size, which is 1 MB currently.
4. The output value must be in the allowed money range (0 to 21 million BTC).
5. All inputs must have a specified previous output, except for coinbase transactions, which should not be relayed.
6. Verify that nLockTime must not exceed 31-bits. For a transaction to be valid, it should not be less than 100 bytes. Also, the number of signature operands in a standard signature should be less than or not more than 2.
7. Reject ***nonstandard*** transactions; for example, ScriptSig is allowed to only push numbers on the stack. ScriptPubkey not passing the isStandardQ checks.
8. A transaction is rejected if there is already a matching transaction in the pool or in a block in the main branch.

9. The transaction will be rejected if the referenced output for each input exists in any other transaction in the pool.
10. For each input, there must exist a referenced output transaction. This is searched in the main branch and the transaction pool to find whether the output transaction is missing for any input, and this will be considered an orphan transaction. It will be added to the orphan transactions pool if a matching transaction is not in the pool already.
11. For each input, if the referenced output transaction is the coinbase, it must have at least 100 confirmations; otherwise, the transaction will be rejected.
12. For each input, if the referenced output does not exist or has been spent already, the transaction will be rejected.
13. Using the referenced output transactions to get input values, verify that each input value, as well as the sum, is in the allowed range of 0-21 million BTC.
14. Reject the transaction if the sum of input values is less than the sum of output values.
15. Reject the transaction if the transaction fee would be too low to get into an empty block.

Blockchain

- Blockchain is a public ledger of a timestamped, ordered, and immutable list of all transactions on the bitcoin network.
- Each block is identified by a hash in the chain and is linked to its previous block by referencing the previous block's hash.

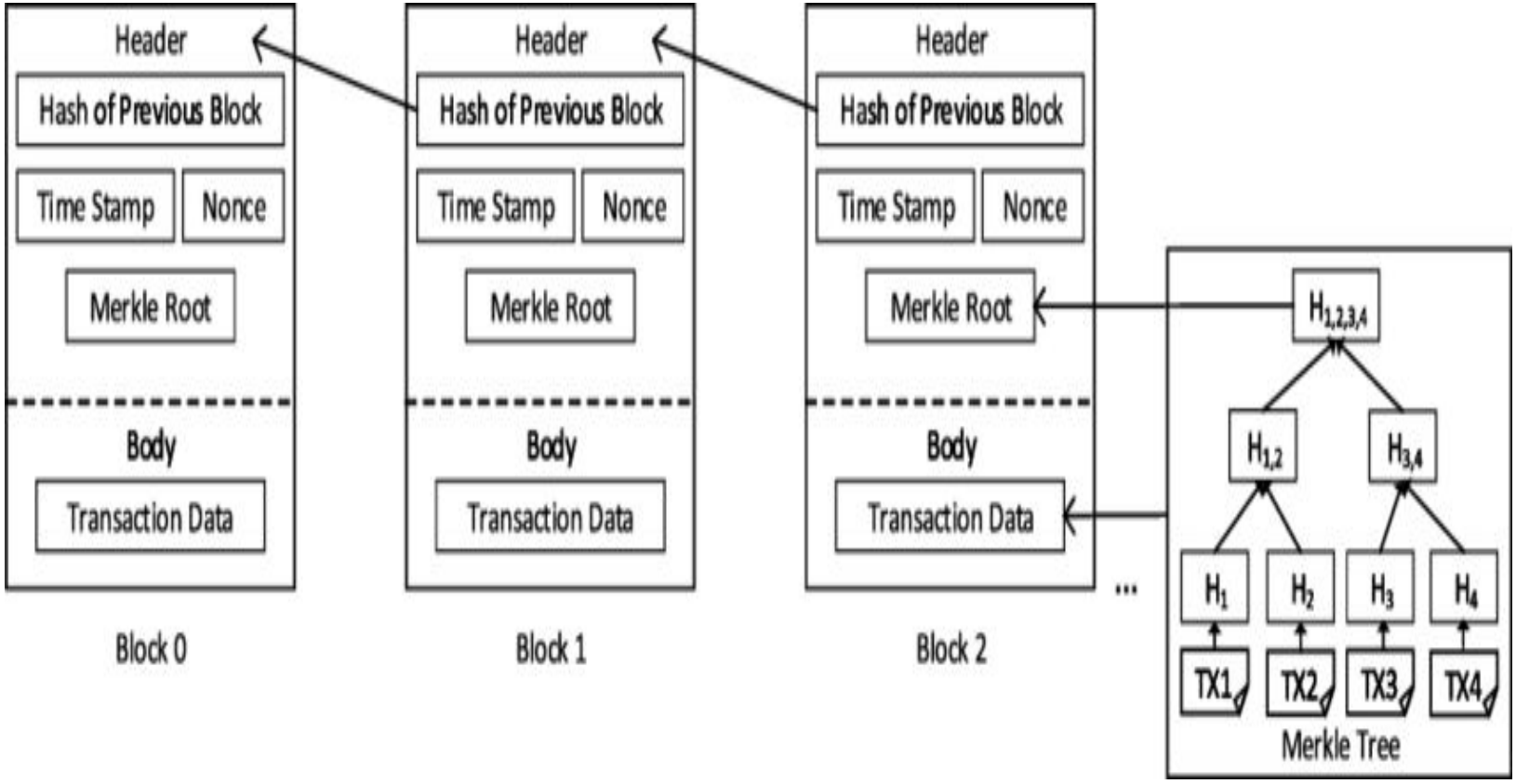
The structure of a block

Bytes	Name	Description
80	Block header	This includes fields from the block header described in the next section.
variable	Transaction counter	The field contains the total number of transactions in the block, including the coinbase transaction.
variable	Transactions	All transactions in the block.

The structure of a block header

Bytes	Name	Description
4	Version	The block version number that dictates the block validation rules to follow.
32	Previous block header hash	This is a double SHA256 hash of the previous block's header.
32	Merkle root hash	This is a double SHA256 hash of the merkle tree of all transactions included in the block.
4	Timestamp	This field contains the approximate creation time of the block in the Unix epoch time format. More precisely, this is the time when the miner has started hashing the header (the time from the miner's point of view).
4	Difficulty target	This is the difficulty target of the block.
4	Nonce	This is an arbitrary number that miners change repeatedly in order to produce a hash that fulfills the difficulty target threshold.

A visualization of blockchain, block, block header, transaction and script



- Blockchain is a chain of blocks where each block is linked to its previous block by referencing the previous block header's hash.
- This linking makes sure that no transaction can be modified unless the block that records it and all blocks that follow it are also modified.
- The first block is not linked to any previous block and is known as the genesis block.

The genesis block

- This is the first block in the bitcoin blockchain.
- The genesis block was hardcoded in the bitcoin core software
- Bitcoin provides protection against double spending by enforcing strict rules on transaction verification and via mining.
- Blocks are added in the blockchain only after strict rule checking and successful Proof of Work solution.

- Block height is the number of blocks before a particular block in the blockchain.
- The current height (at the time of writing this) of the blockchain is 434755 blocks.
- Proof of Work is used to secure the blockchain. Each block contains one or more transactions, out of which the first transaction is a coinbase transaction.
- There is a special condition for coinbase transactions that prevent them to be spent until at least 100 blocks in order to avoid a situation where the block may be declared stale later on.
- Stale blocks are created when a block is solved and every other miner who is still working to find a solution to the hash puzzle is working on that block.
- As the block is no longer required to be worked on, this is considered a stale block.

- Orphan blocks are also called detached blocks and were accepted at one point in time by the network as valid blocks but were rejected when a proven longer chain was created that did not include this initially accepted block.
- They are not part of the main chain and can occur at times when two miners manage to produce the blocks at the same time.
- Because of the distributed nature of bitcoin, network forks can occurs naturally.
- In cases where two nodes simultaneously announce a valid block can result in a situation where there are two blockchains with different transactions.
- This is an undesirable situation but can be addressed by the bitcoin network only by accepting the longest chain.
- In this case, the smaller chain will be considered orphaned.
- If an adversary manages to gain 51% control of the network hashrate (computational power), then they can impose their own version of transaction history.

- Forks in blockchain can occur with the introduction of changes in the Bitcoin protocol.
- In case of *soft fork,* only previous valid blocks are no longer acceptable, thus making soft fork backward compatible.
- In case of soft fork, only miners are required to upgrade to the new client software in order to make use of the new protocol rules.
- Planned upgrades do not necessarily create forks because all users should have updated already.
- A hard fork, on the other hand, invalidates previously valid blocks and requires all users to upgrade.
- New transaction types are sometimes added as a soft fork, and any changes such as block structure change or major protocol changes results in hard fork.

- New blocks are added to the blockchain approximately every 10 minutes and network difficulty is adjusted dynamically every 2016 blocks in order to maintain a steady addition of new blocks to the network.
- **Network difficulty** is calculated using,

$$Target = Previous\ target * Time/2016 * 10\ minutes$$

- Previous target represents the old target value, and time is the time spent to generate previous 2016 blocks.
- Network difficulty basically means how hard it is for miners to find a new block, that is, how difficult the hashing puzzle is now.

MINING

- Mining is a resource-intensive process by which new blocks are added to the blockchain.
- Blocks contain transactions that are validated via the mining process by mining nodes and are added to the blockchain.
- This process is resource-intensive in order to ensure that the required resources have been spent by miners in order for a block to be accepted.
- New coins are minted by the miners by spending the required computing resources.
- This also secures the system against frauds and double spending attacks while adding more virtual currency to the bitcoin ecosystem.

- Roughly one new block is created (mined) every 10 minute.
- Miners are rewarded with new coins if and when they create new blocks and are paid transaction fees in return of including transactions in their blocks.
- New blocks are created at an approximate fixed rate.
- Also, the rate of creation of new bitcoins decreases by 50%, every 210,000 blocks, roughly every 4 years.
- When bitcoin was initially introduced, the block reward was 50 bitcoins; then in 2012, this was reduced to 25 bitcoins.
- In July 2016, this was further reduced to 12.5 coins (12 coins) and the next reduction is estimated to be on July 4, 2020.
- This will reduce the coin reward further down to approximately six coins.

- Approximately 144 blocks, that is, 1,728 bitcoins are generated per day.
- The number of actual coins can vary per day; however, the number of blocks remains at 144 per day.
- Bitcoin supply is also limited and in 2140, almost 21 million bitcoins will be finally created and no new bitcoins can be created after that.
- Bitcoin miners, however, will still be able to profit from the ecosystem by charging transaction fees.

TASK OF MINERS

- Once a node connects with the bitcoin network, there are several tasks that a bitcoin miner performs.

SYNCHING UP WITH THE NETWORK

- Once a new node joins the bitcoin network, it downloads the blockchain by requesting historical blocks from other nodes

- **Transaction validation:** Transactions broadcasted on the network are validated by full nodes by verifying and validating signatures and outputs.
- **Block validation:** Miners and full nodes can start validating blocks received by them by evaluating them against certain rules. This includes the verification of each transaction in the block along with verification of the nonce value.
- **Create a new block:** Miners propose a new block by combining transactions broadcasted on the network after validating them.
- **Perform Proof of Work:** This task is the core of the mining process and this is where miners find a valid block by solving a computational puzzle. The block header contains a 32-bit nonce field and miners are required to repeatedly vary the nonce until the resultant hash is less than a predetermined target.
- **Fetch reward:** Once a node solves the hash puzzle, it immediately broadcasts the results, and other nodes verify it and accept the block. There is a slight chance that the newly minted block will not be accepted by other miners due to a clash with another block found at roughly the same time, but once accepted, the miner is rewarded with 12.5 bitcoins (as of 2016) and any associated transaction fees.

PROOF OF WORK

- This is a proof that enough computational resources have been spent in order to build a valid block.
- **Proof of Work (PoW)** is based on the idea that a random node is selected every time to create a new block.
- In this model, nodes compete with each other in order to be selected in proportion to their computing capacity.
- The following equation sums up the Proof of Work requirement in bitcoin:

$$H (N \,||\, P_hash \,||\, Tx\backslash\backslash Tx\backslash\backslash \ldots\ Tx) < Target$$

Where N is a nonce, P_hash is a hash of the previous block, Tx represents transactions in the block, and Target is the target network difficulty value.

- This means that the hash of the previously mentioned concatenated fields should be less than the target hash value.
- The only way to find this nonce is the brute force method.
- Once a certain pattern of a certain number of zeroes is met by a miner, the block is immediately broadcasted and accepted by other miners.

THE MINING ALGORITHM

- The previous hash block is retrieved from the bitcoin network.
- Assemble a set of potential transactions broadcasted on the network into a block.
- Compute the double hash of the block header with a nonce and the previous hash using the SHA256 algorithm.
- If the resultant hash is lower than the current difficulty level (target), then stop the process.
- If the resultant hash is greater than the current difficulty level (target), then repeat the process by incrementing the nonce. As the hash rate of the bitcoin network increased, the total amount of 32-bit nonces was exhausted too quickly. In order to address this issue, the ***extra nonce*** solution was implemented, whereby the coinbase transaction is used as a source of extra nonce to provide a larger range of nonces to be searched by the miners.
- Mining difficulty increased over time and bitcoins that could be mined by single CPU laptop computers now require dedicated mining centers to solve the hash puzzle. The current difficulty level can be queried using the bitcoin command line interface using the following command:

```
$ bitcoin-cli getdifficulty
```

THE HASHING RATE

- The hashing rate basically represents the rate of calculating hashes per second.
- In early days of bitcoin, it used to be quite small as CPUs were used, but with dedicated mining pools and ASICs now, this has gone up exponentially in the last few years.
- This has resulted in increased difficulty.
- The following hash rate graph shows the hash rate increase over time and is currently measured in Exa hashes.
- This means that in 1 second, bitcoin network miners are computing more than 1 000 000 000 000 000 000 hashes per second.

MINING SYSTEMS

- Over time, bitcoin miners have used various methods to mine bitcoins.
- As the core principle behind mining is based on the double SHA256 algorithm, overtime miners have developed sophisticated systems to calculate the hash faster and faster.

CPU

- CPU mining was the first type of mining available in the original bitcoin client. Users could even use laptop or desktop computers to mine bitcoins.
- CPU mining is no longer profitable and now more advanced mining methods such as ASIC-based mining are used.

GPU

- Due to the increased difficulty of the bitcoin network and general tendency of finding faster methods to mine, miners started to use GPUs or graphics cards available in PCs to perform mining.
- GPUs support faster and parallelized calculations that are usually programmed using the OpenCL language.
- This turned out to be a faster option as compared to CPUs.
- Users also used techniques such as overclocking to gain maximum benefit of the GPU power.
- Also, the possibility of using multiple graphics cards increased the popularity of graphics cards' usage for bitcoin mining.
- GPU mining, however, has some limitations, such as overheating and the requirement for specialized motherboards and extra hardware to house multiple graphics cards.

FPGA

- Even GPU mining did not last long, and soon miners found another way to perform mining using FPGAs.
- **Field Programmable Gate Array (FPGA)** is basically an integrated circuit that can be programmed to perform specific operations.
- FPGAs are usually programmed in **hardware description languages (HDLs),** such as Verilog and VHDL.
- Double SHA256 quickly became an attractive programming task for FPGA programmers and several open source projects started too.
- FPGA offered much better performance as compared to GPUs; however, issues such as accessibility, programming difficulty, and the requirement for specialized knowledge to program and configure FPGAs resulted in a short life of the FPGA era for bitcoin mining.
- Also, the arrival of ASICs resulted in quickly phased out FPGA-based systems for mining. Mining hardware such as X6500 miner, Ztex, and Icarus were developed during the time when FPGA mining was profitable.
- Various FPGA manufacturers, such as Xilinx and Altera, produce FPGA hardware and development boards that can be used to program mining algorithms.

ASICS

- **Application Specific Integrated Circuit (ASIC)** was designed to perform the SHA-256 operation.
- These special chips were sold by various manufacturers and offered a very high hashing rate.
- This worked for some time, but due to the quickly increasing mining difficulty level, single-unit ASICs are no longer profitable.
- Currently, mining is out of the reach of individuals and now professional mining centers using thousands of ASIC units in parallel are offering mining contracts to users to perform mining on their behalf.
- There is no technical limitation, that's why a single user cannot run thousands of ASICs in parallel, but it will require dedicated data centers and hardware and cost for a single individual can become prohibitive.

MINING POOLS

- A mining pool forms when group miners work together to mine a block.
- The *Pool manager* receives the coinbase transaction if the block is successfully mined, which is then responsible for distributing the reward to the group of miners who invested resources to mine the block.
- There are various models that a mining pool manager can use to pay to the miners, such as the pay-per-share model and the proportional model.
- In the pay per share model, the mining pool manager pays a flat fee to all miners who participated in the mining exercise, whereas in the proportional model, the share is calculated based on the amount of computing resources spent to solve the hash puzzle.
- Many commercial pools now exist and provide mining service contracts via the cloud and easy-to-use web interfaces. The most commonly used ones are **AntPool, F2Pool,** and **BW.COM.**

The bitcoin network

- The bitcoin network is a P2P network where nodes exchange transactions and blocks.
- There are different types of nodes on the network.
- There are two main types of nodes, full nodes and SPV nodes.
- Full nodes are implementations of bitcoin core clients performing the wallet, miner, full blockchain storage, and network routing functions.
- SPV nodes or lightweight clients perform only wallet and network routing functionality.

- Some nodes prefer to be full blockchain nodes only and contain complete blockchain and perform network routing functions but do not perform mining or store private keys (the wallet function).
- Another type is solo miner nodes that can perform mining, store full blockchain, and act as a bitcoin network routing node.

- A full node performs four functions: wallet, miner, blockchain, and the network routing node.
- When a bitcoin core node starts up, first, it initiates the discovery of all peers.
- This is achieved by querying DNS seeds that are hardcoded into the bitcoin core client and are maintained by bitcoin community members.
- This lookup returns a number of DNS A records. The bitcoin protocol works on TCP port 8333 by default for the main network and TCP 18333 for testnet.

- First, the client sends a protocol message *Version* that contains various fields, such as version, services, timestamp, network address, nonce, and some other fields.
- The remote node responds with its own version message followed by verack message exchange between both nodes, indicating that the connection has been established.
- After this, **Getaddr** and **addr** messages are exchanged to find the peers that the client do not know.
- Meanwhile, either of the nodes can send a ping message to see whether the connection is still live.

- Now the block download can begin.
- If the node already has all blocks fully synchronized, then it listens for new blocks using the *Inv* protocol message; otherwise, it first checks whether it has a response to *inv* messages and have inventories already.
- If yes, then it requests the blocks using the **Getdata** protocol message; if not, then it requests inventories using the *GetBlocks* message.

- IBD = Initial block download and sync node means the node from where the blocks are being requested from Getblockchaininfo and getpeerinfo RPCs were updated with a new functionality to cater for this change.
- An RPC, *getchaintips,* is used to list all known branches of the blockchain.
- This also includes headers only blocks.
- Getblockchaininfo is used to provide the information about the current state of the blockchain.
- Getpeerinfo is used to list both the number of blocks and the headers that are in common between peers.

Version: This is the first message that a node sends out to the network, advertising its version and block count. The remote node
then replies with the same information and the connection is then established.

Verack : This is the response of the version message accepting the connection request.

Inv: This is used by nodes to advertise their knowledge of blocks and transactions.

Getdata: This is a response to inv, requesting a single block or transaction identified by its hash.

Getblocks: This returns an ***inv*** packet containing the list of all blocks starting after the last known hash or 500 blocks.

Getheaders : This is used to request block headers in a specified range.

Tx : This is used to send a transaction as a response to the getdata protocol message.

Block: This sends a block in response to the ***getdata*** protocol message.

Headers: This packet returns up to 2,000 block headers as a reply to the getheaders request.

Getaddr: This is sent as a request to get information about known peers.

Addr: This provides information about nodes on the network. It contains the number of addresses and address list in the form of IP address and port number.

- **Full client and SPV client:** Full clients are thick clients or full nodes that download the entire blockchain; this is the most secure method of validating the blockchain as a client.
- Bitcoin network nodes can operate in two fundamental modes: full client or lightweight SPV client.
- SPV clients are used to verify payments without requiring the download of a full blockchain.
- SPV nodes only keep a copy of block headers of the current valid longest blockchain.
- Verification is performed by looking at the merkle branch that links the transactions to the original block the transaction was accepted in.
- This is not very practical and requires a more practical approach, which was implemented with BIP37, where bloom filters were used to filter out relevant transactions only.

- Bloom filters: Bloom filter is basically a data structure (a bit vector with indexes) that is used to test the membership of an element in a probabilistic manner.
- It basically provides probabilistic lookup with false positives but no false negatives.
- Elements are added to the bloom filter after hashing them several times and then set the corresponding bits in the bit vector to 1 via the corresponding index.
- In order to check the presence of the element in the bloom filter, the same hash functions are applied and compared with the bits in the bit vector to see whether the same bits are set to 1.
- Not every hash function (such as SHA1) is suitable for bloom filters as they need to be fast, independent, and uniformly distributed.
- The most commonly used hash functions for bloom filters are fnv, mumur, and Jenkins.

- These filters are mainly used by simple payment verification SPV clients to request transactions and the merkle blocks they are interested in.
- A merkle block is a lightweight version of the block, which includes a block header, some hashes, a list of 1-bit flags, and a transaction count.
- This information can then be used to build a merkle tree.
- This is achieved by creating a filter that matches only those transaction and blocks that have been requested by the SPV client.
- Once version messages have been exchanged and connection has been established between peers, the nodes can set filters according to their requirements.
- These probabilistic filters offer a varying degree of privacy or precision depending upon how accurately or loosely they have been set.
- A strict bloom filter will only filter transactions that have been requested by the node but at the expense of the possibility of revealing the user addresses to adversaries who can correlate transactions with their IP addresses, thus compromising privacy.

- On the other hand, a loosely set filter can result in retrieving more unrelated transactions but will offer more privacy.
- Also, for SPV clients, bloom filters allow them to use low bandwidth as opposed to downloading all transactions for verification.
  - **BIP** 37 proposed the bitcoin implementation of bloom filters and introduced three new messages to the Bitcoin protocol.
  - **Filterload:** This is used to set the bloom filter on the connection.
  - **Filteradd:** This adds a new data element to the current filter.
  - **FilterClear:** This deletes the currently loaded filter.

Wallets

- The wallet software is used to store private or public keys and bitcoin address. It performs various functions, such as receiving and sending bitcoins.
- Nowadays, software usually offers both functionalities: bitcoin client and wallet.
- On the disk, the bitcoin core client wallets are stored as the Berkeley DB file:

```
:-/.bitcoin$ file wallet.dat

wallet.dat: Berkeley DB (Btree, version 9, native byte-order)
```

- Private keys can be generated in different ways and are used by different types of wallets.
- Wallets do not store any coins, and there is no concept of wallets storing balance or coins for a user.
- In fact, in the bitcoin network, *coins* do not exist; instead, only transaction information is stored on the blockchain (more precisely, UTXO, unspent outputs), which are then used to calculate the amount of bitcoins.

WALLET TYPES

- In bitcoin, there are different types of wallets that can be used to store private keys.
- As a software program, they also provide some functions to the users to manage and carry out transactions on the bitcoin network.

❖ Non-deterministic wallets

- These wallets contain randomly generated private keys and are also called *Just a Bunch of Key wallets.*
- The bitcoin core client generates some keys when first started and generates keys as and when required.
- Managing a large number of keys is very difficult and an error-prone process can lead to theft and loss of coins.
- Moreover, there is a need to create regular backups of the keys and protect them appropriately in order to prevent theft or loss.

❖ Deterministic wallets

- In this type of wallet, keys are derived out of a seed value via hash functions.
- This seed number is generated randomly and is commonly represented by human-readable *mnemonic code* words. This phrase can be used to recover all keys and makes private key management comparatively easier.

❖ Hierarchical deterministic wallets

- HD wallets store keys in a tree structure derived from a seed.The seed generates the parent key (master key), which is used to generate child keys and, subsequently, grandchild keys.
- Key generation in HD wallets does not generate keys directly; instead, it produces some information (private key generation information) that can be used to generate a sequence of private keys.
- The complete hierarchy of private keys in an HD wallet is easily recoverable if the master private key is known. It is because of this property that HD wallets are very easy to maintain and are highly portable.

❖ Brain wallets

- The master private key can also be derived from the hash of passwords that are memorized.
- The key idea is that this passphrase is used to derive the private key and if used in HD wallets, this can result in a full HD wallet that is derived from a single memorized password. This is known as brain wallet.
- This method is prone to password guessing and brute force attacks but techniques such as *key stretching* can be used to slow down the progress made by the attacker.

❖ Paper wallets

- This is a paper-based wallet with the required key material printed on it. It requires physical security to be stored.
- Paper wallets can be generated online from various service providers, such as https://bitcoinpaperwallet.com/ or bitaddress.org

❖ Hardware wallets

- Another method is to use a tamper-resistant device to store keys.
- This tamper-resistant device can be custom-built or with the advent of NFC-enabled phones, this can also be a **secure element (SE)** in NFC phones.
- Trezor and Ledger wallets (various types) are the most commonly used bitcoin hardware wallets.

❖ Online wallets

- Online wallets, as the name implies, are stored entirely online and are provided as a service usually via cloud.
- They provide a web interface to the users to manage their wallets and perform various functions such as making and receiving payments.
- They are easy to use but imply that the user trust the online wallet service provider.

❖ Mobile wallets

- Mobile wallets, as the name suggests, are installed on mobile devices.
- They can provide various methods to make payments, most notably the ability to use smartphone cameras to scan QR codes quickly and make payments.
- Mobile wallets are available for the Android platform and iOS, for example, breadwallet, copay, and Jaxx.

Bitcoin Payments

- Bitcoin is not recognized as a legal currency in many jurisdictions, but it is increasingly being accepted as a payment method by many online merchants and e-commerce websites.
- Customers can scan the QR barcode with the seller's payment URI in it and pay using their mobile devices.
- Bitcoin URIs allow users to make payments by simply clicking on links or scanning QR codes.
- **URI (Uniform Resource Identifier)** is basically a string that represents the transaction information.

- Various payment solutions, such as xbtterminal and 34 bytes bitcoin POS terminal are available commercially.
- Bitcoin payment processor, offered by many online service providers, allows integration with e-commerce websites.
- Various BIPs have been proposed and finalized in order to introduce and standardize bitcoin payments.
- BIP 70 (secure payment protocol) describes the protocol for secure communication between a merchant and customers.
- This protocol uses X.509 certificates for authentication and runs over HTTP and HTTPS.
- There are three messages in this protocol: PaymentRequest, Payment, and PaymentACK.
- The key features of this proposal is defence against man-in-the-middle attacks and secure proof of payment.

- Man in-the-middle attacks can result in a scenario where the attacker is sitting between the merchant and the buyer, but in fact, the *man in the middle* is interacting with the buyer instead of the merchant.
- This can result in manipulation of the merchant's bitcoin address to defraud the buyer.
- Bitcoin lightning network, a solution for scalable off-chain instant payments, was introduced in early 2016, which allows off-blockchain payments.
- The network makes use of payments channels that run off the blockchain. This allows greater speed and scalability of bitcoin.

Bitcoin investment and buying and selling bitcoins

- There are many online exchanges where users can buy and sell bitcoins.
- This is a big business on the Internet now and it offers bitcoin trading, CFDs, spread betting, margin trading, and various other choices.
- Traders can buy bitcoins or trade by opening long or short positions to make profit when bitcoin's price goes up or down.
- Several other features, such as exchanging bitcoins for other virtual currencies, are also possible, and many online bitcoin exchanges provide this function.
- Advanced market data, trading strategies, charts, and relevant data to support traders is also available.

Alternative Coins

- Bitcoin was released in 2009 and the first alternative coin project (named Namecoin) was introduced in 2011.
- Alternative approaches to bitcoin can be divided broadly into two categories, based on the primary purpose of their development.
- If the primary purpose is to build a decentralized blockchain platform, they are called alternative chains; if the sole purpose of the alternative project is to introduce a new virtual currency, it's called an altcoin.

- However there is no established rule as to which chain becomes the altcoin.
- This has happened recently with Ethereum, where a hard fork caused a new currency **ETC (Ethereum classic)** to come into existence in addition to the **Ethereum (ETH)** currency.
- Ethereum classic is the old chain and Ether is the new chain after the fork.
- Such a contentious hard fork is not desirable for a number of reasons.

- First it is against the true spirit of decentralization as the Ethereum foundation, a central entity, decided to go ahead with the hardfork even though not everyone agreed to the proposition; second it also splits the user community due to disagreement over the hard fork. Altcoins must be able to attract new users, trades, and miners otherwise the currency will have no value.
- Currency gains its value, especially in the virtual currency space, due to the network effect and its acceptability by the community.
- If a coin fails to attract enough users then soon it will be forgotten.
- Users can be attracted by providing an initial amount of coins and can be achieved by using various methods.

Methods of providing an initial number of altcoins are discussed as follows:

- **Create a new blockchain:** Altcoins can create a new blockchain and allocate coins to initial miners but this approach is now unpopular due to many scam schemes or pump and dump schemes where initial miners made a profit with the launch of a new currency and then disappeared.

- **Proof of burn:** Another approach to allocating initial funds to a new altcoin is Proof of burn, also called a one-way peg or price ceiling. In this method users permanently destroy a certain quantity of bitcoins in proportion to the quantity of altcoins to be claimed. For example if 10 bitcoins were destroyed then altcoins can have a value no greater than the amount of bitcoins destroyed. This basically means that bitcoins are being converted into altcoins by burning them.

- **Proof of Ownership:** Instead of permanently destroying bitcoins, an alternative method is to prove that users own a certain number of bitcoins. This proof of ownership can be used to claim altcoins by tethering altcoin blocks to bitcoin blocks. For example, this can be achieved by merged mining in which effectively bitcoin miners can mine altcoin blocks while mining for bitcoin without any extra work.

- **Pegged sidechains:** Sidechains, as the name suggests, are blockchains separate from the bitcoin network but bitcoins can be transferred to them. Altcoins can also be transferred back to the bitcoin network. This concept is called a two-way peg.

Theoretical foundations

Alternatives to Proof of Work

- The **Proof of Work (PoW)** scheme in the context of cryptocurrency was first used in bitcoin and served as a mechanism to provide assurance that a miner had completed the required number of work in order to find a block.
- This in return provided decentralization, security, and stability for the blockchain.
- PoW schemes are required to have a much desired property called *progress freeness,* which basically means that the reward for consuming computational resources should be random and proportional to the contribution made by the miners.
- In this case, some chance of winning the block reward is given to even those miners who have comparatively less computational power.

- Adjustable difficulty ensures that the difficulty target for mining on the blockchain is regulated in response to increased hashing power and the number of users.
- Quick verification is a property which means that mining computational puzzles should be easy and quick to verify.
- Another aspect of the PoW scheme, especially the one used in Bitcoin (Double SHA-256), is that since the introduction of ASICs the power is shifting towards miners or mining pools who can afford to operate large-scale ASIC farms and this challenges the core philosophy of the decentralization of bitcoin.

- The core idea behind this method is that as puzzle solving requires a large amount of memory, it is not feasible to be implemented on ASIC-based systems.
- This technique was initially used in litecoin and Tenebrix where the Scrypt hash function was used as an ASIC-resistant PoW scheme.
- Even though this scheme was initially advertised as ASIC resistant, recently Scrypt ASICs have now become available disproving the original claim by litecoin.

- Another approach to ASIC resistance is where multiple hash functions are required to be calculated in order to provide PoW.
- This is also called a chained hashing scheme.
- This approach provide some resistance to ASIC development

- Perhaps another approach could be to design *self-mutating puzzles* that intelligently or randomly change the PoW scheme or its requirements as a function of time.
- This will make it almost impossible to be implemented in ASICs as it will require multiple ASICs to be designed for each function and also randomly changing schemes would be almost impossible to handle in ASICs.
- At the moment, it is unclear how this can be achieved practically.
- PoW has various drawbacks and the biggest of all is energy consumption.

- It has been proposed that PoW puzzles can be designed in such a way that they serve two purposes.

- First their primary purpose is in consensus mechanisms and second to perform some useful scientific computation.

- This way not only can the schemes be used in mining but they can also help to potentially solve other scientific problems too.

- This proof of useful work has been recently put into practice by Primecoin where the requirement is to find special prime number chains known as Cunningham chains and bi-twin chains.

- As the study of prime number distribution has special significance in scientific disciplines such as physics, by mining Primecoin miners not only achieve the block reward but also help in finding the special prime numbers.

PROOF OF STORAGE

- Also known as proof of irretrievability, this is another type of proof of useful work that requires storage of large number of data.
- Introduced by Microsoft Research, this scheme provides a useful benefit of distributed storage of archival data.
- Miners are required to store a pseudo, randomly-selected subset of large data in order to perform mining.

PROOF OF STAKE

- This proof is also called virtual mining.
- This is another type of mining puzzle that has been proposed as an alternative to traditional PoW schemes.
- It was first proposed in PeerCoin in August, 2012.

- In this scheme, the idea is that users are required to demonstrate possession of a certain amount of currency (coins) thus proving that they have a stake in the coin.
- The simplest form of stake is where mining is made comparatively easier for those users who demonstrably own larger amounts of digital currency.
- The benefits of this scheme are twofold; first acquiring large amounts of digital currency is relatively difficult as compared to buying high-end ASIC devices and second it results in saving computational resources.

Proof of Coinage

- The age of a coin is the time since the coins were last used or held.
- This is a different approach from the usual form of Proof of Stake where mining is made easier for users who have the highest stake in the altcoin.
- In the coin-age-based approach the age of the coin (coinage) is reset every time a block is mined.
- The miner is rewarded for holding and not spending coins for a time period.
- This mechanism has been implemented in Peercoin combined with PoW in a creative way. The difficulty of mining puzzles (PoW) is inversely proportional to the coin-age, meaning that if miners consume some coin-age using *coin-stake* transactions then the PoW requirements are relieved.

Proof of deposit

- The core idea behind this scheme is that newly minted blocks by miners are made unspendable for a certain period of time. More precisely the coins get locked for a set number of blocks during the mining operation.

- The scheme works by allowing miners to perform mining at the cost of freezing a certain number of coins for some time.This is a type of Proof of Stake.

Proof of burn

- As an alternate expenditure to computing power, proof of burn in fact destroys a certain amount of bitcoins in order to get equivalent altcoins. This is commonly used when starting up new coin projects as a means to provide a fair initial distribution.

- This can be considered an alternative mining scheme where the value of the new coins comes from the fact that previously a certain number of coins have been destroyed.

Proof of activity

- This scheme is a hybrid of PoW and Proof of Stake.
- In this scheme, blocks are initially produced by using PoW but then each block randomly assigns three stakeholders that are required to digitally sign it.
- The validity of subsequent blocks is dependent on the successful signing of previously randomly chosen blocks.
- There is, however a possible issue known as the *nothing at stake* problem where it would be trivial to create a fork of the blockchain.
- This is possible because in PoW appropriate computational resources are required to mine whereas in Proof of Stake there is no such requirement; as a result, an attacker can try to mine on multiple chains using the same coin.

NON-OUTSOURCEABLE PUZZLES

Difficulty adjustment and retargeting algorithms

- With the advent of bitcoin and altcoins is difficulty in retargeting algorithms.
- In bitcoin a difficulty target is calculated by the following equation; however other coins have either developed their own algorithms or implemented modified versions of the bitcoin difficulty algorithm:

  *T = Time previous * time actual/2016 *10 min*

- The idea behind difficulty regulation in bitcoin is that a generation of 2016 blocks should take roughly around 2 weeks (inter-block time should be around 10 minutes).

- If it takes longer than 2 weeks to mine 2016 blocks then the difficulty is decreased and if it takes less than two weeks to mine 2016 blocks then the difficulty is increased.
- When ASICs were introduced due to a high block generation rate the difficulty increased exponentially and that is one drawback of PoW algorithms that are not ASIC resistant.
- This leads to mining power centralization.
- This also poses another problem; if a new coin starts now with the same Proof of Work based on SHA256 as bitcoin uses, then it would be easy for a malicious user to use an ASIC miner and control the entire network.

- This attack would be more practical if there is less interest in the new altcoin and someone decides to take over the network by consuming adequately high computing resources.
- This may not be a feasible attack if other miners with similar computing power also join the the altcoin network because then miners will be competing with each other.
- Also, multipools pose a greater threat where a group of miners can automatically switch to the currency that is becoming profitable.
- This phenomenon is known as **pool hopping** and can adversely affect a blockchain, and consequently the growth of the altcoin.

- Pool hopping impacts the network adversely because pool hoppers join the network only when the difficulty is low and they can gain quick rewards; the moment difficulty goes up (or is readjusted) they *hop off* and then come back again when the difficulty is adjusted back.
- For example if a multipool consumes its resources in quickly mining a new coin, the difficulty will increase very quickly; when the multipool leaves the currency network; it becomes almost unusable because of the fact that now the difficulty has increased to such a level that it is no longer profitable for solo miners and can no longer be maintained.
- The only fix for this problem is to initiate a hard fork which is usually undesirable for the community.

- All these algorithms are based on the idea of readjusting various parameters in response to hash rate changes; these parameters include the number of previous blocks, difficulty of previous blocks, ratio of adjustment, and the number by which the difficulty can be readjusted back or up.

KIMOTO GRAVITY WELL

- This algorithm is used in various altcoins to regulate difficulty.
- This was first introduced in Megacoin and used to adaptively adjust difficulty of the network every block.
- The logic of the algorithm is shown as follows:

  *KGW = 1 + (0.7084 *pow((double(PastBlocksMass)/double(144)), -1.228))*

- Basically, the algorithm runs in a loop that goes through a set of predetermined blocks (PastBlockMass) and calculates a new readjustment value.
- The core idea behind this algorithm is to develop an adaptive difficulty regulation mechanism that can readjust the difficulty in response to rapid spikes in hash rates.
- **Kimoto Gravity Well (KGW)** ensures that the time between blocks remains approximately the same.
- In bitcoin the difficulty is adjusted every 2016 blocks but in KGW the difficulty is adjusted at every block.

- This algorithm is vulnerable to time warp attacks, which allow an attacker to temporarily enjoy less difficulty in creating new blocks.
- This attack allows a time window where the difficulty becomes low and the attacker can easily generate many coins at a fast rate.

DARK GRAVITY WAVE

- **Dark Gravity Wave (DGW)** is a new algorithm designed to address certain flaws such as the time warp attack in the KGW algorithm.
- This was first introduced in Dash, previously known as Darkcoin.
- It makes use of multiple exponential moving averages and simple move averages to achieve a smoother readjustment mechanism.
- The formula is shown as follows:

$$2222222/\ (((Difficulty+2600)/9)^{A}2)$$

- This formula is implemented in Dash coin and various other altcoins as a mechanism to readjust difficulty.

DIGISHIELD

- This is another difficulty retargeting algorithm that has recently been used in Zcash
- This algorithm works by going through a fixed number of previous blocks to calculate the time they took to be generated and then readjusts the difficulty to the difficulty of the previous block by dividing the actual time span by averaging the target time.
- In this scheme, the retargeting is calculated much rapidly and also the recovery from a sudden increase or decrease in hashrate is quick.
- This algorithm protects against multipools, which can result in rapid hashrate increases.
- The network difficulty is re-adjusted every block or every minute depending on the implementation.
- The key innovation is faster readjust times as compared to KGW.

MIDAS

- Multi Interval Difficulty Adjustment System (MIDAS) is an algorithm which responds much more rapidly to abrupt changes in hash rates and it is more complex compared to previously discussed algorithms.
- This algorithm also provides protection against time warp attacks.

Limitations in bitcoin

Privacy and anonymity

- As the blockchain is a public ledger of all transactions and is openly available
- Combined with traffic analyses, transactions can be linked back to their source IP addresses, thus possibly revealing a transaction's originator.
- This is a big concern from a privacy point of view.
- Even though in bitcoin it is a recommended and common practice to generate a new address for every transaction, thus allowing some level of unlinkability, this is not enough and various techniques have been developed and successfully used to trace the flow of transactions throughout the network and link them back to their originator.

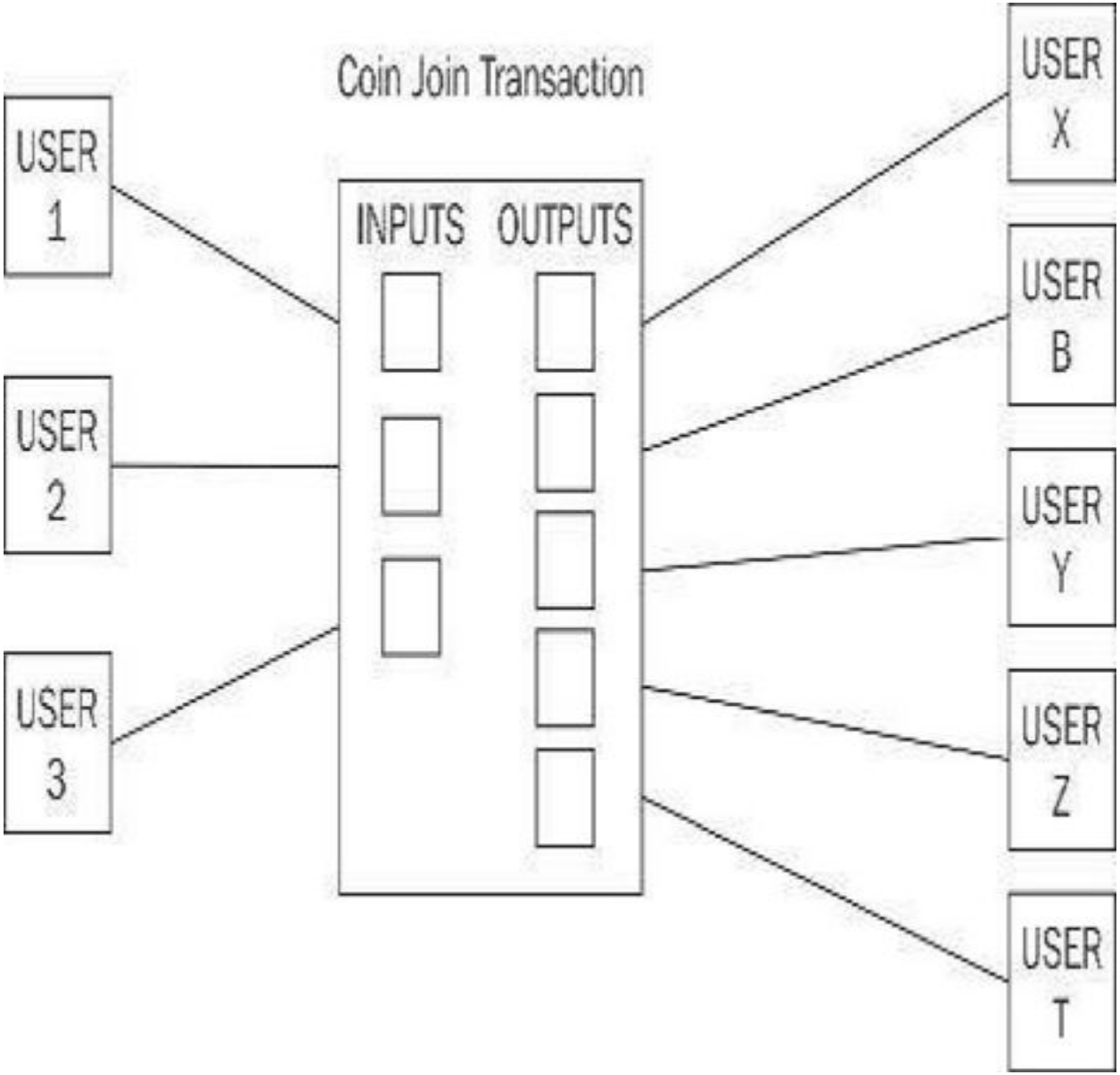
- Various methods to analyse blockchains such as transaction graphs, address graphs, and entity graphs have been used by researchers to link users to the transactions, thus raising privacy concerns.
- The aforementioned analysis techniques can be further enriched by using publicly available information about transactions and linking them to the actual users.
- There are open source block parsers available that can be used to extract transaction information, balances, and scripts from the blockchain database.

- Various proposals have been made to address the privacy issue in bitcoin.
- These proposals fall into three categories: mixing protocols, third-party mixing networks, and inherent anonymity.

MIXING PROTOCOLS

- These schemes are used to provide anonymity to bitcoin transactions.
- In this model, a mixing service provider (an intermediary or a shared wallet) is used.
- Users send coins to this shared wallet as a deposit and the shared wallet then can send some other coins (of the same value deposited by some other users) to the destination.
- Users can also receive coins that were sent by others via this intermediary.
- This way the link between outputs and inputs is no longer there and transaction graph analysis will not be able to reveal the true relationship between senders and receivers.

- CoinJoin is one example of mixing protocols, where two transactions are joined together to form a single transaction while keeping the inputs and outputs unchanged.
- The core idea behind CoinJoin is to build a shared transaction that is signed by all participants.
- This technique improves privacy for all participants involved in the transactions:
- CoinJoin transaction with three users joining their transaction into a single larger CoinJoin transaction


Coin Join Transaction
USER 1
USER 2
USER 3
INPUTS
OUTPUTS
USER X
USER B
USER Y
USER Z
USER T

THIRD-PARTY MIXING PROTOCOLS

- Various third-party mixing services are available but if the service is centralized then it poses the threat of tracing the mapping between senders and receivers, because the mixing service knows about all inputs and outputs.
- In addition to this, fully centralized miners even pose the risk of the administrators of the service stealing the coins.
- Various services, with varying degrees of complexity, such as CoinShuffle, Coinmux, and dark send in Dash (coin) are available that are based on the idea of CoinJoin (mixing) transactions.
- CoinShuffle is a decentralized alternative to traditional mixing services as it does not require a trusted third party.

- CoinJoin-based schemes, however, have some weaknesses, most prominently the possibility of launching a Denial of Service attack by users who committed to signing the transactions initially but now are not providing their signature, thus delaying or stopping joint transaction a altogether.

INHERENT ANONYMITY

- This category includes coins that support privacy inherently and is built into the design of the currency.
- The most popular is Zcash and Other examples include Monero, which makes use of ring signatures to provide anonymous

Extended protocols on top of bitcoin

- Several protocols have been proposed and implemented on top of bitcoin in order to enhance and extend the bitcoin protocol and use for various other purposes instead of just as a virtual currency.

COLORED COINS

- Colored coins is a set of methods that have been developed to represent digital assets on the bitcoin blockchain.
- Coloring a bitcoin refers colloquially to updating it with some metadata representing a digital asset (smart property).
- The coin still works and operates as a bitcoin but additionally carries some metadata that represents some assets.
- This mechanism allows issuing and tracking specific bitcoins.
- Metadata can be recorded using the bitcoins OP_RETURN opcode or optionally in multi-signature addresses.

- This metadata can also be encrypted if required to address any privacy concerns.
- Colored coins can be used to represent a multitude of assets including but not limited to commodities, certificates, shares, bonds, and voting.
- It should also be noted that, in order to work with colored coins, a wallet that interprets colored coins is necessary and normal bitcoin wallets will not work.
- Using this service, any type of digital asset can be created and issued via a colored coin.
- The idea of colored coins is very appealing as it does not require any modification to the existing bitcoin protocol and can make use of the already existing secure bitcoin network.

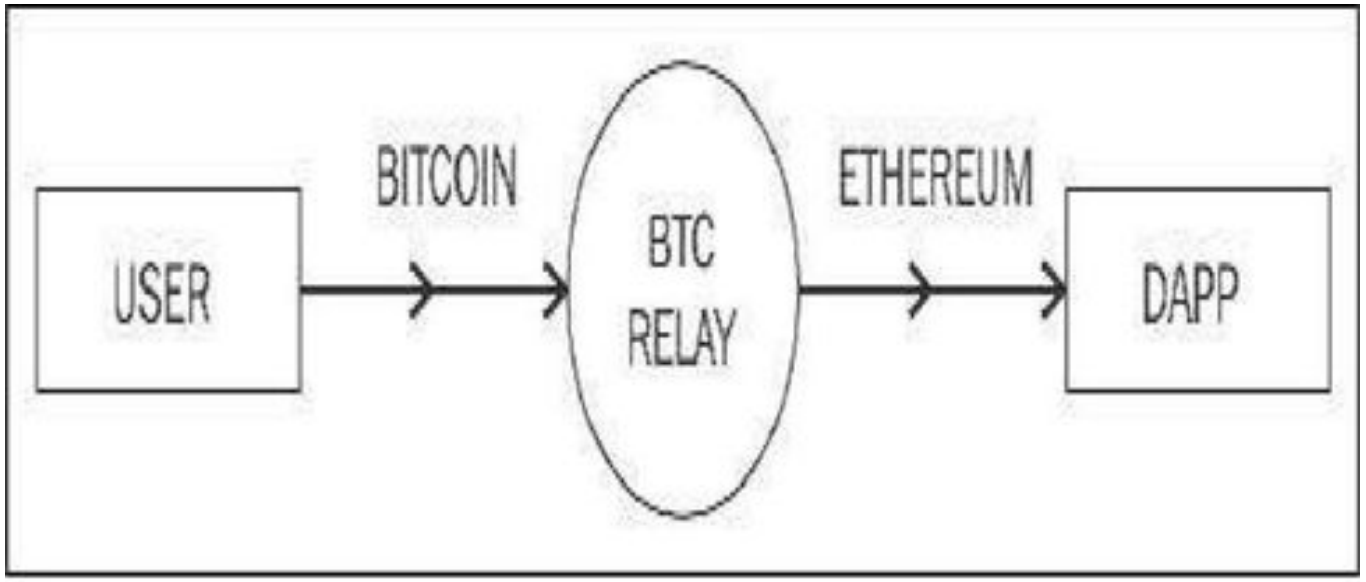
- In addition to the traditional representation of digital assets, there is also the possibility of creating *smart assets* that behave according to the parameters and conditions defined for them.
- These parameters includes time validation, restrictions on transferability, and fees.
- This opens the possibility of creating smart contracts.
- A major use case can be the issuance of financial instruments on the blockchain.
- This will ensure low transaction fees, valid and mathematically secure proof of ownership, fast transferability without requiring an intermediary, and instant dividend payouts to the investors.

COUNTERPARTY

- This is another service that can be used to create custom tokens that act as a cryptocurrency and can be used for various purposes such as issuing digital assets on top of bitcoin blockchain.
- This is a powerful platform and runs on bitcoin blockchains at their core but has developed its own client and other components to support issuing digital assets.
- The architecture consists of a counterparty server, counterblock, counter wallet, and armory_utxsvr.
- Counterparty works based on the same idea as coloured coins by embedding data into regular bitcoin transactions but provides a much richer library and set of powerful tools to support the handling of digital assets.
- This embedding is also called embedded consensus because the counterparty transactions are embedded within bitcoin transactions.

- The currency produced and used by counterparty is known as XCP and is used by smart contracts as the fee for running the contract.
- Counterparty allows the development of smart contracts on Ethereum using solidity language and allows interaction with bitcoin blockchain.
- In order to achieve this, BTC Relay is used as a means to provide interoperability between Ethereum and bitcoin.
- The relayers (nodes that are running BTC Relay) fetch the bitcoin block headers and relay them to a smart contract on the Ethereum network that verifies the PoW.
- This process verifies that a transaction has occurred on the bitcoin network.

- Technically, this is basically an Ethereum contract that is capable of storing and verifying bitcoin block headers just like bitcoin simple payment verification lightweight clients do by using bloom filters.
- The idea can be visualized with the following diagram:



BTC relay concept

Development of altcoins

- Altcoin projects can be started very easily from a coding point of view by simply forking the bitcoin or another coin's source code
- Usually, the code base is written in C++ as like bitcoin but almost any language can be used to develop coin projects, for example Golang or Rust.
- In the case of forking the code of another coin, for example bitcoin, there are various parameters that can be changed to effectively create a new coin.

CONSENSUS ALGORITHMS

- There is a choice of consensus algorithm: **Proof of Work (PoW)** as used in bitcoin or **Proof of Stake (PoS),** as in Peercoin.

HASHING ALGORITHMS

- This is either SHA256, Scrypt, XII, X13, X15, or any other hashing algorithm that is adequate for use as a consensus algorithm.

DIFFICULTY ADJUSTMENT ALGORITHMS

- Various options are available in this category to provide difficulty retargeting mechanisms.
- The most prominent examples are KGW, DGW, Nite's Gravity Wave, and DigiShield.

INTER-BLOCK TIME

- This is the time elapsed between the generation of each block.
- For bitcoin the blocks are generated every 10 minutes, for litecoin it's 2.5 minutes.
- Any value can be used but an appropriate value is usually between a few minutes; if the generation time is too fast it might destabilize the blockchain, if it's too slow it may not attract many users.

BLOCK REWARDS

- A block reward is for the miner who solves the mining puzzle and is allowed to have a Coinbase transaction that contains the reward.
- This used to be 50 coins in bitcoin initially and now many altcoins set this parameter to a very high number; for example in Dogecoin it is 10,000, currently.

REWARD HALVING RATE

- This is another important factor; in bitcoin it is halved every 4 years and now is set to 12.5 bitcoins.
- It's a variable number that can be set to any time period or none at all depending on the requirements.

BLOCK SIZE AND TRANSACTION SIZE

- This is another important factor that determines how high or low the transaction rate can be on the network.
- Block sizes in bitcoin are limited to 1 MB but in altcoins it can vary depending on the requirements.

INTEREST RATE

- This property applies only to PoS systems where the owner of the coins can earn interest at a rate defined by the network in return for the amount of coins that are held on the network as a PoS to protect the network.

COIN AGE

- This parameter defines how long the coin has to remain unspent in order for it to become eligible to be considered stakeworthy.

TOTAL SUPPLY OF COINS

- This number sets the total limit of the coins that can ever be generated.
- For example in bitcoin the limit is 21 million, whereas in Dogecoin it's unlimited.
- This limit is fixed by the block reward and halving schedule

- There are two options to create your own virtual currency: forking existing established cryptocurrency source code or writing a new one from scratch.
- The latter option is less popular but the first option is easier and has allowed the creation of many virtual currencies over the last few years.
- Fundamentally, the idea is that first a cryptocurrency source code is forked and then appropriate changes are made at different strategic locations in the source code to effectively create a new currency.

Namecoin

- Namecoin is the first fork of the bitcoin source code.
- This provides improved decentralization, censorship resistance, privacy, security, and faster decentralized naming.
- Decentralized naming services are intended to provide a response to inherent limitations such as slowness and centralized control in the traditional **Domain Name System (DNS)** protocols used on the Internet.

- Namecoin is also the first solution to Zooko's triangle
- Namecoin is used to provide a service to register a key/value pair.
- One major use case of Namecoin is that it can provide a decentralized **Transport Layer Security (TLS)** certificate validation mechanism, driven by blockchain-based distributed and decentralized consensus.
- It is based on the same technology introduced with bitcoin, but with its own blockchain and wallet software.

- In summary, Namecoin provides the following three services:
  - ❖ Secure storage and transfer of names (keys)
  - ❖ Attachment of some value to the names by attaching up to 520 bytes of data
  - ❖ Production of a digital currency (Namecoin)

- Namecoin also for the first time introduced merged mining, which allows a miner to mine on more than one chain simultaneously.
- The idea is simple but very effective: miners create a Namecoin block and produce a hash of that block.
- Then the hash is added to a bitcoin block and miners solve that block at equal to or greater than the Namecoin block difficulty in order to prove that enough work has been contributed towards solving the Namecoin block.

- More precisely the Coinbase transaction is used to include the hash of the transactions from Namecoin (or any other altcoin).
- The mining task is to solve bitcoin blocks whose Coinbase scripSig contains a hash pointer to Namecoin (or any other altcoin) block.
- If a miner manages to solve a hash at the bitcoin blockchain difficulty level, the bitcoin block is built and becomes part of the bitcoin network.
- In this case, the Namecoin hash is ignored by the bitcoin blockchain.
- On the other hand, if a miner solves a block at Namecoin blockchain difficulty level a new block is created in the Namecoin blockchain.
- The core benefit of this scheme is that all the computational power spent by the miners contributes towards securing both Namecoin and bitcoin:

Obtaining Namecoins

- Even though Namecoins can be mined independently, they are usually mined as part of bitcoin mining by utilizing the merged mining technique
- This way Namecoin can be mined as a byproduct of bitcoin mining.
- Solo mining is no longer profitable as is evident from the following difficulty graph; instead it is recommended to merge-mine, use a mining pool, or even use a cryptocurrency exchange to buy Namecoin.

- This allows a miner to mine primarily bitcoin but also as a result earn Namecoin too.
- Another method that can be used to quickly get some Namecoins is to swap your existing coins with Namecoins, for example, if you already have some bitcoins or an other cryptocurrency that can be used to exchange with Namecoin.
- Online service allows conversion from one cryptocurrency to another, using a simple user-friendly interface.

Generating Namecoin records

- Namecoin records are in the form of key and value pairs
- A name is a lower-case string of the form d/examplename whereas a value is a case-sensitive, UTF-8 encoded JSON object with a maximum of 520 bytes.
- A general namecoin name can be an arbitrary binary string up to 255 bytes long with, 1024-bits of associated identifying information.
- A record on a Namecoin chain is only valid for around 200 days or 36,000 blocks after which it needs to be renewed.
- Namecoin also introduced .bit top level domains that can be registered using Namecoin and can be browsed using specialized Namecoin-enabled resolvers.
- Namecoin wallet software as shown in the following screenshot can be used to register . bit domain names.
- The name is entered and, after the Submit button is pressed, it will ask for configuration information such as DNS, IP, or Identity:

Litecoin

- Litecoin is a fork of the bitcoin source code released in 2011.
- It uses Scrypt as PoW, originally introduced in the Tenebrix coin.
- Litecoin allows for faster transactions as compared to bitcoin due to its faster block generation time of 2.5 minutes.
- Also difficulty readjustment is achieved every 3.5 days roughly due to faster block generation time.
- The total coin supply is 84 million.

- Scrypt is a sequentially memory hard function that is the first alternative to the SHA-256-based PoW algorithm.
- It was originally proposed as a password-based key derivation function PBKDF.
- The key idea is that if the function requires large number of memory to run then custom hardware such as ASICs will require more VLSI area, which would be unfeasible to build.
- The Scrypt algorithm requires a large array of pseudo random bits to be held in memory and a key is derived from this in a pseudo random fashion.
- The algorithm is based on a phenomenon called **Time-Memory Tradeoff (TMTO).**

- If memory requirements are relaxed then it results in increased computational cost.
- Put another way, TMTO shortens the running time of a programme if more memory is given to it.
- This tradeoff makes it unfeasible for an attacker to gain more memory because it's expensive and difficult to implement on custom hardware, or if the attacker chooses to not increase memory, then it results in the algorithm running slowly due to high processing requirements.

Scrypt uses the following parameters to generate a derived key (Kd):

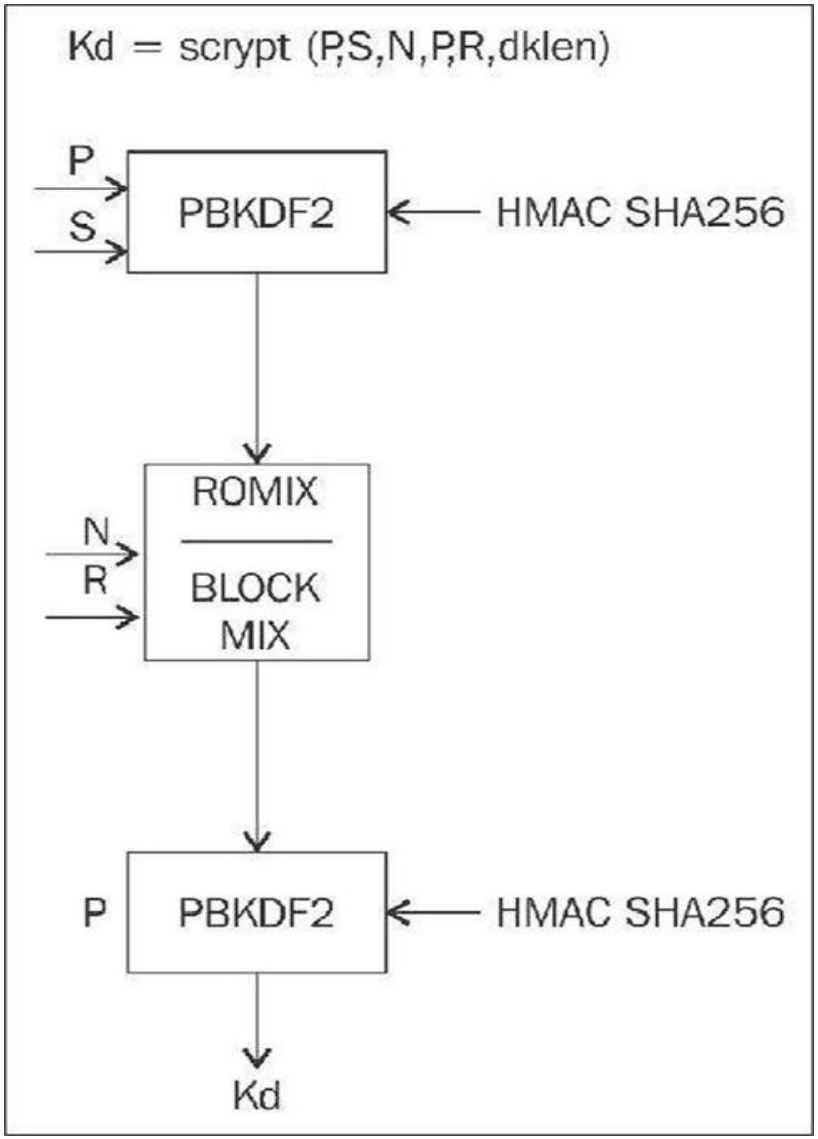
- Passphrase: This is a string of characters to hash
- Salt: This is a random string that is provided to Scrypt functions (generally all hash functions) in order to provide a defence against brute-force dictionary attacks using rainbow tables
- N: This is a memory/CPU cost parameter that must be a power of 2 > 1
- P: The parallelization parameter
- R: The block size parameter
- dkLen: The intended length of the derived key in bytes

Formally, this function can be written as follows:

Kd = scrypt (P, S, N, P, R, dkLen)

- Before applying the core Scrypt function, the algorithm takes P and S as input and applies **PBKDF2** and SHA-256-based HMAC.
- Then the output is fed to an algorithm called **ROMix,** which internally uses the Blockmix algorithm utilizing the Salsa20/8 core stream cipher to fill up the memory which requires large memory to operate, thus enforcing the sequentially memory hard property.
- The output from this step of the algorithm is finally fed to the PBKDF2 function again in order to produce a derived key.

- This process is shown in the following diagram:



Scrypt algorithm

- Scrypt is used in litecoin mining with specific parameters where N= 1024, R = 1, P=I, and S= random 80 bytes producing a 256-bit output.
- Due to the selection of these parameters, the development of ASICs for Scrypt for litecoin mining turned out to be not very difficult.
- In an ASIC for litecoin mining, a sequential logic can be developed that takes the data and nonce as input and applies the **PBKDF2** algorithm with **HMAC-SHA256;** then the resultant bit stream is fed into the SALSA20/8 function which produces a hash that again is fed down to the PBKDF2 and HMAC-256 functions to produce a 256-bit hash output.
- As is the case with bitcoin PoW, in Scrypt also if the output hash is less that the target hash then the function terminates; otherwise, the nonce is incremented and the process is repeated again until a hash is found that is lower than the difficulty target:

- ❖ **Trading Litecoin:** As with other coins, trading litecoin is easily carried out on various online exchanges. The current market cap of litecoin is £161,239,005. The current price of litecoin is E3.25/LTC.
- ❖ **Mining:** Litecoin mining can be carried out solo or in pools. At the moment, ASICs for Scrypt are available that are commonly used to mine litecoin.

- Litecoin mining on a CPU is no longer profitable as is the case with many other digital currencies.
- There are online cloud mining providers and ASIC miners available that can be used to mine litecoin.

Primecoin

- Primecoin is the first digital currency on the market that introduced a useful PoW, as opposed to bitcoin's SHA256-based PoW. Primecoin uses searching prime numbers as a PoW.
- Not all types of prime number meet the requirements to be selected as PoW.
- Three types of prime numbers (known as Cunningham chain of first kind, Cunningham chain of second kind, and bi-twin chains) meet the requirements of a PoW algorithm to be used in cryptocurrencies.
- The difficulty is dynamically adjusted via a continuous difficulty evaluation scheme in Primecoin blockchain.
- The efficient verification of PoW based on prime numbers is also of high importance, because if verification is slow then PoW is not suitable.

- Therefore prime chains are selected as a PoW because finding prime chains gets difficult as the chain increases in length whereas verification remains quick enough to warrant being used as an efficient PoW algorithm.
- It is also important that once a PoW has been verified on a block it must not be reusable on another block.
- This is accomplished in Primecoin by a combination of *Proof of Work certificates* and hashing it with the header of the parent block in the child block.
- The PoW certificate is produced by linking the prime chain to the block header hash.
- It also requires that the block header's origin is divisible by the block header hash.
- If it is, it is divided and after division the quotient is used as a PoW certificate.

- Another property of the adjustable difficulty of PoW algorithms is met by introducing difficulty adjustment every block instead of every 2,016, as is the case with bitcoin.
- This is a *smoother* approach as compared to bitcoin and allows readjustment in the case of sudden increases in hash power.
- Also the total number of coins generated is community-driven and there is no concrete limit on the number of coins Primecoin can generate.

Zcash

- Zcash is the first currency that uses a specific type of zero knowledge proofs known as **zero-knowledge Succinct Non-interactive Arguments of Knowledge (zk-SNARKs)** to provide complete privacy to the user, introduced in 2016.
- These proofs are very short and easy to verify; however, setting up the initial public parameters is a complex process.
- The latter include two keys: the proving key and verifying key.
- The process requires sampling some random numbers in order to construct the public parameters.
- The issue is that these random numbers, also called *toxic waste,* must be destroyed after the parameter generation in order to prevent counterfeiting of Zcash.

- For this purpose, the Zcash team came up with a multi-party computation protocol to generate the required public parameters in a collaborative manner from independent locations to ensure that *toxic waste* is not created.
- Due to the fact that these public parameters are required to be created by the Zcash team, it means that the participants in the ceremony are trusted.
- This is the reason why the ceremony was very open and conducted by making use of a multi-party computation mechanism.
- This mechanism has a property whereby all of the participants in the ceremony will have to be compromised in order to compromise the final parameters.
- When the ceremony is completed all participants physically destroy the equipment used for private key generation.
- This action eliminates any trace of the participants' part of the private key on the equipment.

- zk-SNARKs must satisfy the properties of completeness, soundness, succinctness, and non-interactivity.
- *Completeness* means that there is a definite strategy for a prover to satisfy a verifier that an assertion is true.
- On the other hand, *soundness* means that no prover can convince the verifier that a false statement is true.
- *Succinctness* means that messages passed between the prover and verifier are very small in size.
- Finally, the property non-interactive means that the verification of correctness of an assertion can be carried out without any interaction or very little interaction.
- Also, being a zero knowledge proof, the property of *zero-knowledge* needs to be met too.

- Zcash developers have introduced the concept of a **Decentralized Anonymous Payments scheme (DAP scheme)** that is used in the Zcash network to enable direct and private payments.
- The transactions reveal no information about the origin, destination, and amount of the payments.
- There are two types of addresses available in Zcash, z-addr and t-addr. Z addresses are based on zero knowledge proofs and provide privacy protection whereas T addresses are similar to those of bitcoin.
- Zcash uses an efficient PoW scheme named **Asymmetric PoW (Equihash),** which is based on the Generalized Birthday Problem. It allows very efficient verification.

- It is a memory-hard and ASIC-resistant function.
- A novel idea (initial slow mining) has been introduced with Zcash, which means that the block reward increases gradually over a period of time until it reaches the 20,000th block.
- This allows for initial scaling of the network and experimentation by early miners, and adjustment by Zcash developers if required.
- The slow start did have an impact on price due to scarcity as the price of ZEC on its first day of launch reached roughly 25,000 USD.
- A slightly modified version of the Digishield difficulty adjustment algorithm has been implemented in Zcash.
- The formula is shown as follows:

(next difficulty) = (last difficulty) x SQRT [(150 seconds) / (last solve time)]

Various attributes of Zcash:

Attribute	Value
Name	Zcash
Launch date	28/10/16
Main purpose	Currency
Currency Code	ZEC
Maximum coins	21 million
Block time	10 minutes
Consensus facilitation algorithm	Proof of Work (equihash)
Difficulty adjustment algorithm	DigiShield V3 (modified)
Mining hardware	CPU, GPU
Difficult}' adjustment period	1 block

Smart Contract

- A smart contract is a secure and unstoppable computer program representing an agreement that is automatically executable and enforceable.
- A smart contract is written in a language that a computer or target machine can understand.
- Also, it encompasses agreements between parties in the form of business logic.
- Another key idea is that smart contracts are automatically executed when certain conditions are met.
- They are enforceable, which means that all contractual terms are executed as defined and expected, even in the presence of adversaries.
- Enforcement is a broader term that encompasses traditional enforcement in the form of law, along with implementation of certain measures and controls that make it possible to execute contract terms without requiring any mediation.

- For example, imagine a normal computer programme which is encoded with some logic and executes according to the instruction coded within it, but if the environment it is running in or external factors it relies on deviate from the normal or expected state, the programme may react arbitrarily or simply abort. It is important that smart contracts are immune to this type of issue.
- Secure and unstoppable may well be considered requirements or desirable features but it will provide greater benefits in the long run if security and unstoppable properties are included in the smart contract definition from the beginning.
- This will allow researchers to focus on these aspects from the start and will help to build strong foundations on which further research can then be based.

- For example, various floating point operations calculated by various functions in a variety of programming languages can produce different results in different runtime environments.
- Another example is of some math functions in JavaScript which can produce different results for the same input on different browsers, and which can in turn lead to various bugs.
- This is highly undesirable in smart contracts because, if results are inconsistent between nodes, then consensus will never be achieved.
- A deterministic feature ensures that smart contracts always produce the same output for a specific input.

- In other words, programs once compiled produce a solid and accurate business logic that is completely in line with the requirements programmed in the high level code.
- In summary, a smart contract has the following four properties:
  - Automatically executable
  - Enforceable
  - Semantically sound
  - Secure and unstoppable.

- The first two properties are required as a minimum, whereas the latter two may not be required or implementable in certain scenarios and can be relaxed.
- For example, a derivatives contract does not need to be semantically sound and unstoppable but should at least be automatically executable and enforceable at a basic level.
- On the other hand, a title deed needs to be semantically sound and complete therefore, in order for it to be implemented as a smart contract, the language must be understood by both computers and people.

Unit-IV

Ethereum 101

Introduction

- Ethereum was conceptualized by *Vitalik Buterin* in November 2013.
- The key idea behind Ethereum is the development of a Turing-complete language that allows the development of arbitrary programs (smart contracts) for blockchain and decentralized applications.
- This is in contrast to bitcoin, where the scripting language is very limited and allows basic and necessary operations only.

Ethereum clients and releases

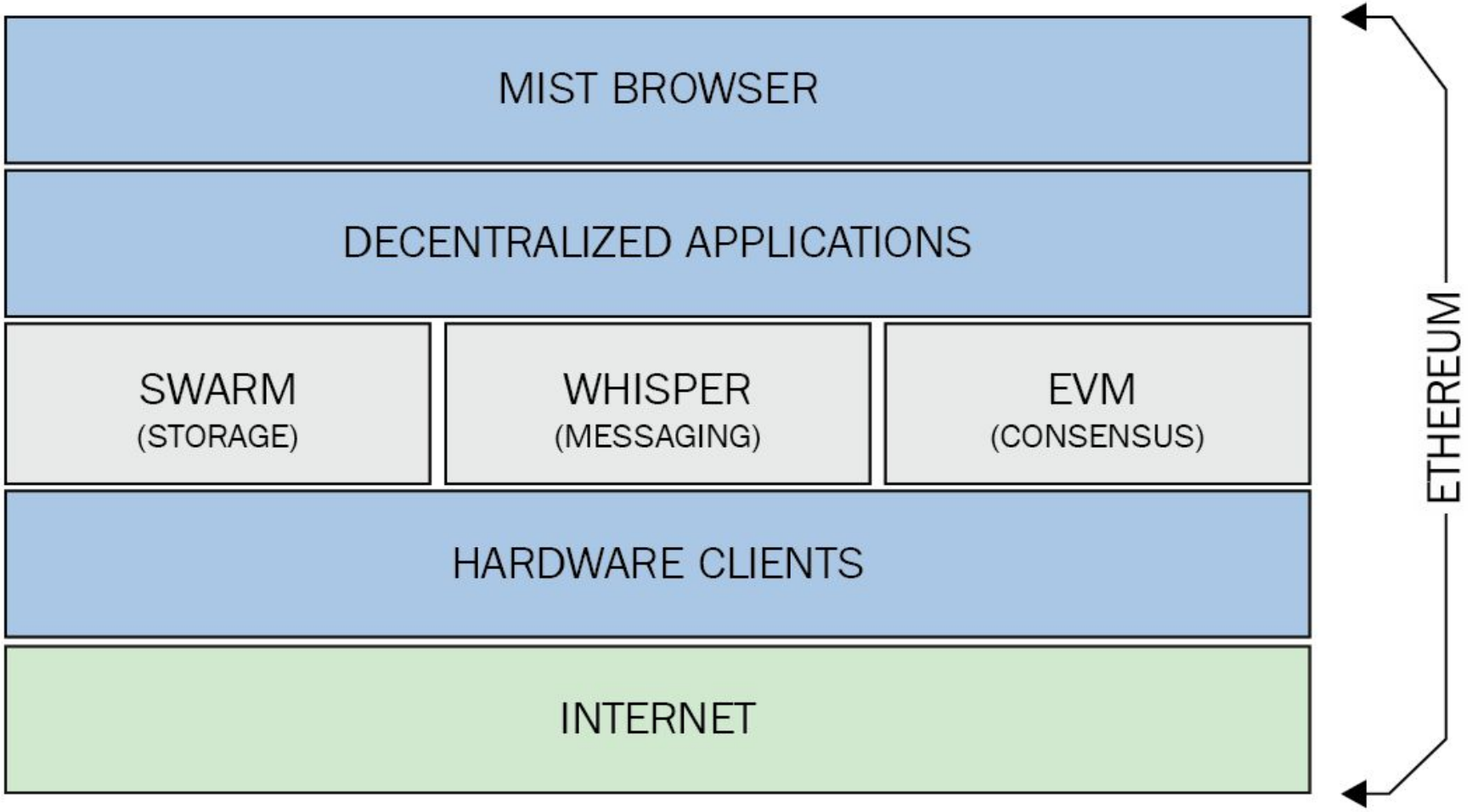
- Currently most popular languages used by ethereum clients are go-Ethereum and parity.
- go-Ethereum (geth) was developed using Golang, whereas parity was built using Rust.
- Mist is a user-friendly **Graphical User Interface (GUI)** wallet that runs geth in the background to sync with the network.

- first release - Frontier
- current release - homestead
- next version - metropolis
- final release - serenity (Proof of Stake algorithm)
- Other areas of research include scalability, privacy and Ethereum virtual machine (EVM) upgrade.
- With the vision of an web 3.0 ecosystem where people, applications, data, and web are all connected together and are able to interact with each other in an intelligent fashion, Ethereum finds continuous improvement
- The services such as DNS, search engines, and identity on the Internet will be decentralized in web 3.0

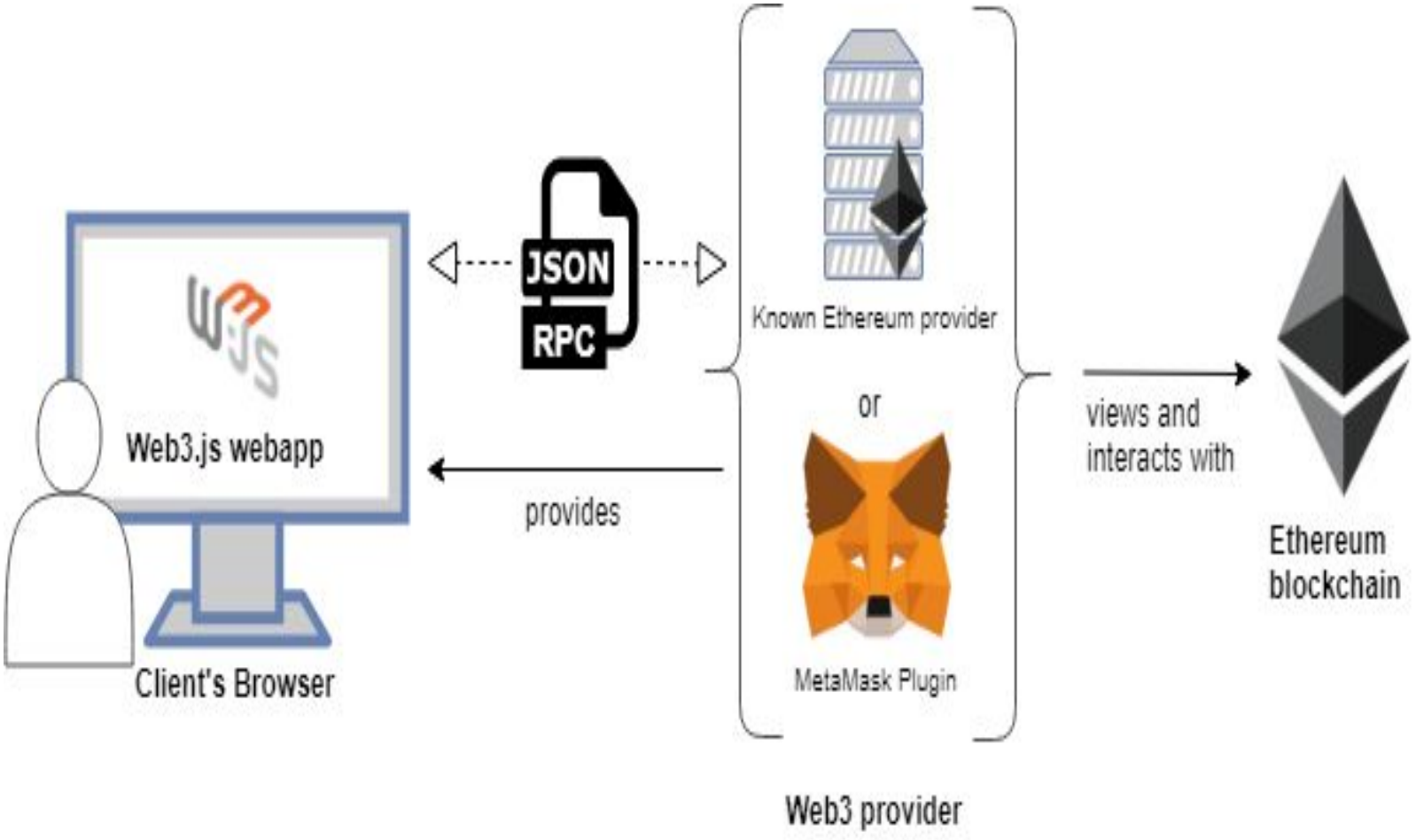
The Ethereum stack

- The Ethereum stack consists of various components.
- At the core, there is the Ethereum blockchain running on the P2P Ethereum network.
- Secondly, there's an Ethereum client (usually geth) that runs on the nodes and connects to the peer-to-peer Ethereum network from where blockchain is downloaded and stored locally.
- It provides various functions, such as mining and account management.
- The local copy of the blockchain is synchronized regularly with the network.
- Another component is the web3.js library that allows interaction with geth via the **Remote Procedure Call (RPC)** interface.

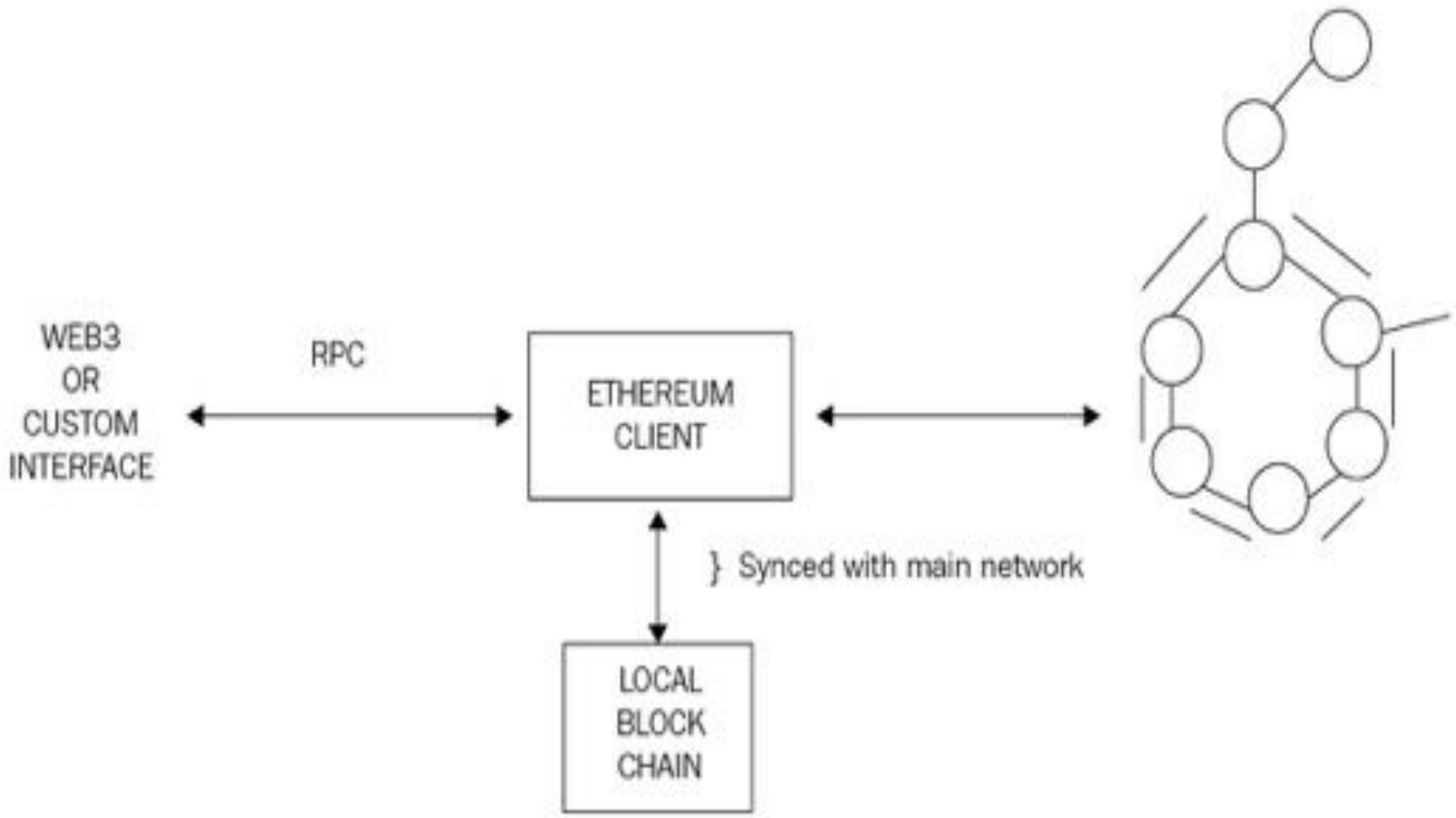
Web 3.0 Tech stack



Ethereum stack exchange



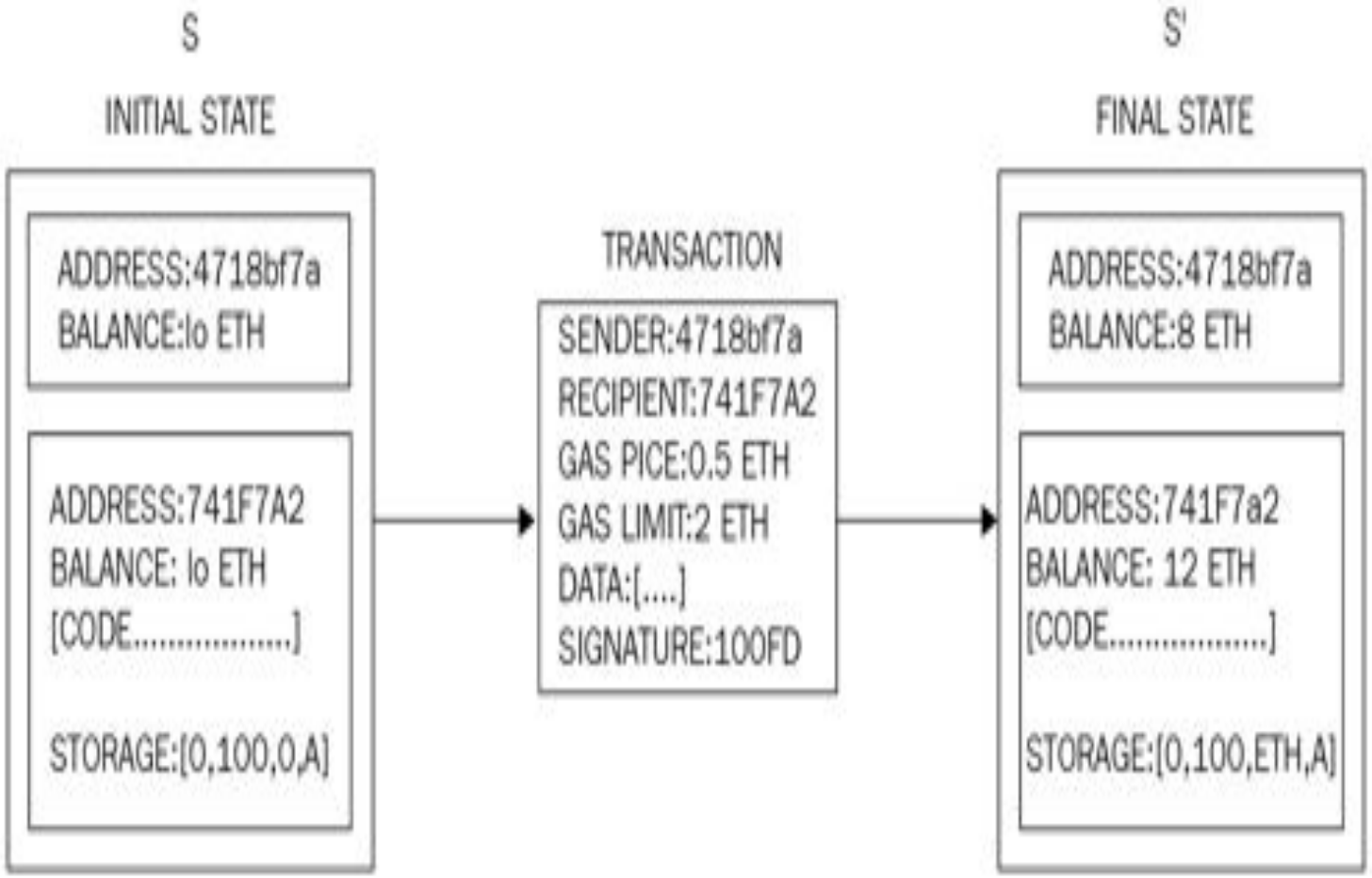
Components of Ethereum ecosystem



Ethereum blockchain

- Ethereum, just like any other blockchain, can be visualized as a transaction-based state machine.
- The idea is that a genesis state is transformed into a final state by executing transactions incrementally.
- The final transformation is then accepted as the absolute undisputed version of the state.
- In the preceding example, a transfer of 2 Ether from **Address 4718bf7a** to **Address 741f7a2** is initiated.
- The initial state represents the state before the transaction execution and the final state is what the morphed state looks like.

Ethereum State transition function



Currency (ETH and ETC)

- As an incentive to the miners, Ethereum also rewards its native currency called Ether (ETH).
- After the DAO hack, a hard fork was proposed in order to mitigate the issue
- For that, two Ethereum blockchains are introduced
- one is called Ethereum classic - currency - ETC is a dedicated community which is a non-forked original version of Ethereum
- other one, hard-forked version is ETH which continues to grow and on which active development is being carried out.
- Here, the focus is mainly on ETH, which is the current, the most active and official Ethereum blockchain.

Forks

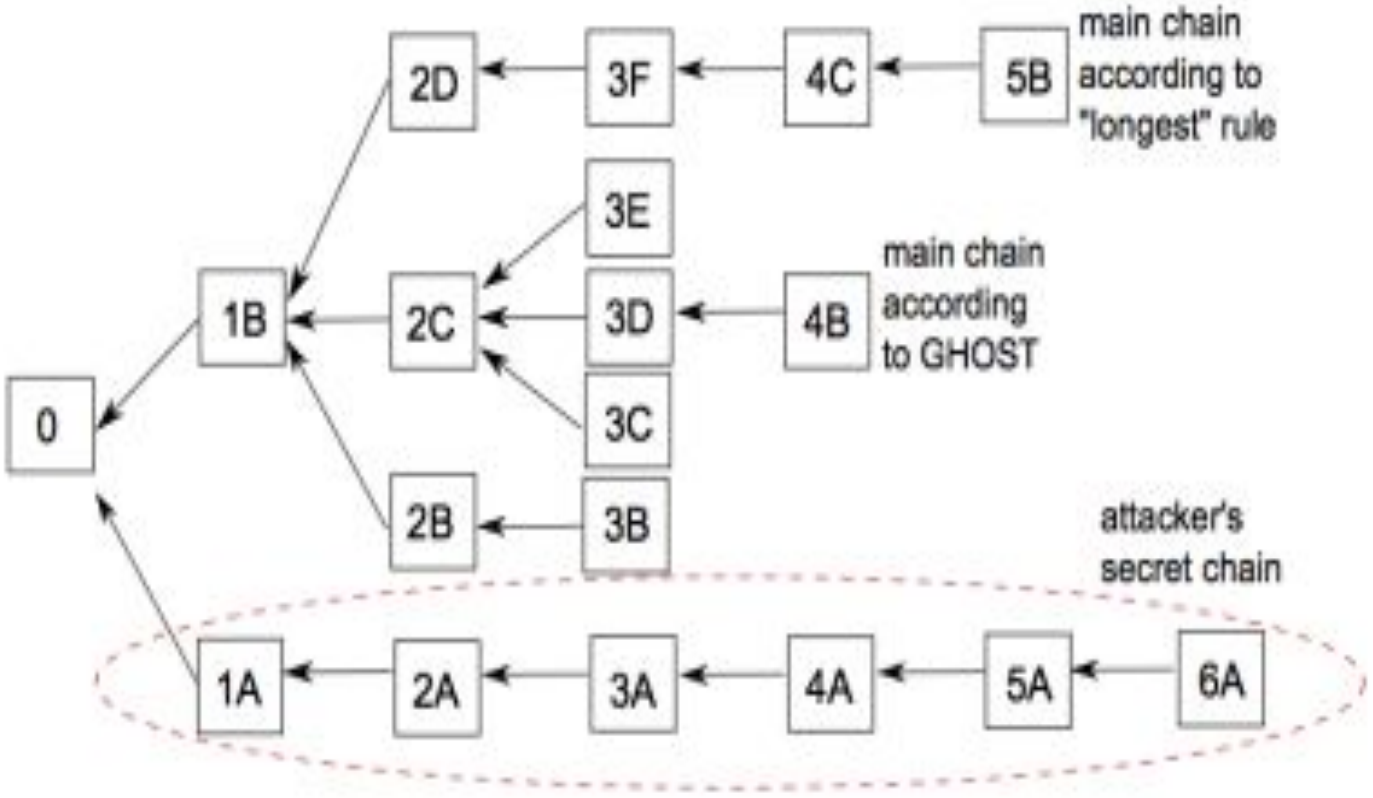
- With the latest release of homestead, due to major protocol upgrades, it resulted in a hard fork.
- The protocol was upgraded at block number 1,150,000, resulting in the migration from the first version of Ethereum known as Frontier to the second version of Ethereum called homestead.
- A recent unintentional fork that occurred in 2016 was due to a bug in the geth client's journaling mechanism.
- Network fork occurred at block number 2,686,351.
- This bug resulted in geth failing to revert empty account deletions in the case of the empty out-of-gas exception.
- This was not an issue in parity (another popular Ethereum client).
- This means that from block number 2686351, the Ethereum blockchain is split into two, one running with parity clients and the other with geth.

- Another key concept in Ethereum is that of gas.
- All transactions on the Ethereum blockchain are required to cover the cost of computation they are performing.
- The cost is covered by something called *gas* or *crypto fuel,* which is a new concept introduced by Ethereum.
- This gas as *execution fee* is paid upfront by the transaction originators.
- The *fuel* is consumed with each operation. Each operation has a predefined amount of gas associated with it.
- Each transaction specifies the amount of gas it is willing to consume for its execution.
- If it runs *out of gas* before the execution is completed, any operation performed by the transaction up to that point is rolled back.
- If the transaction is successfully executed, then any remaining gas is refunded to the transaction originator.

The consensus mechanism

- The consensus mechanism in Ethereum is based on the GHOST protocol.
- Ethereum uses a simpler version of this protocol, where the chain that has most computational effort spent on it in order to build it is identified as the definite version.
- Another way of looking at it is to find the longest chain, as the longest chain must have been built by consuming adequate mining effort.
- **Greedy Heaviest Observed Subtree (GHOST)** was first introduced as a mechanism to alleviate the issues arising out of fast block generation times that led to stale or orphan blocks.
- In GHOST, stale blocks are added in calculations to figure out the longest and heaviest chain of blocks. Stale blocks are called Uncles or Ommers in Ethereum.

Bitcoin longest chain rule



The world state

- The world state in Ethereum represents the global state of the Ethereum blockchain.
- It is basically a mapping between Ethereum addresses and account states.
- The addresses are 20 bytes long.
- This mapping is a data structure that is serialized using **Recursive Length Prefix (RLP).**
- RLP is a specially developed encoding scheme that is used in Ethereum to serialize binary data for storage or transmission over the network and also to save the state in a Patricia tree.
- The RLP function takes an item as an input, which can be a string or a list of items, and produces raw bytes that are suitable for storage and transmission over the network.
- RLP does not encode data; instead, its main purpose is to encode structures

THE ACCOUNT STATE

- The account state consists of four fields: nonce, balance, storageroot and codehash

Nonce

- This is a value that is incremented every time a transaction is sent from the address.
- In case of contract accounts, it represents the number of contracts created by the account.
- Contract accounts are one of the two types of accounts that exist in Ethereum;

Balance

- This value represents the number of Weis which is the smallest unit of the currency (Ether) in Ethereum held by the address.

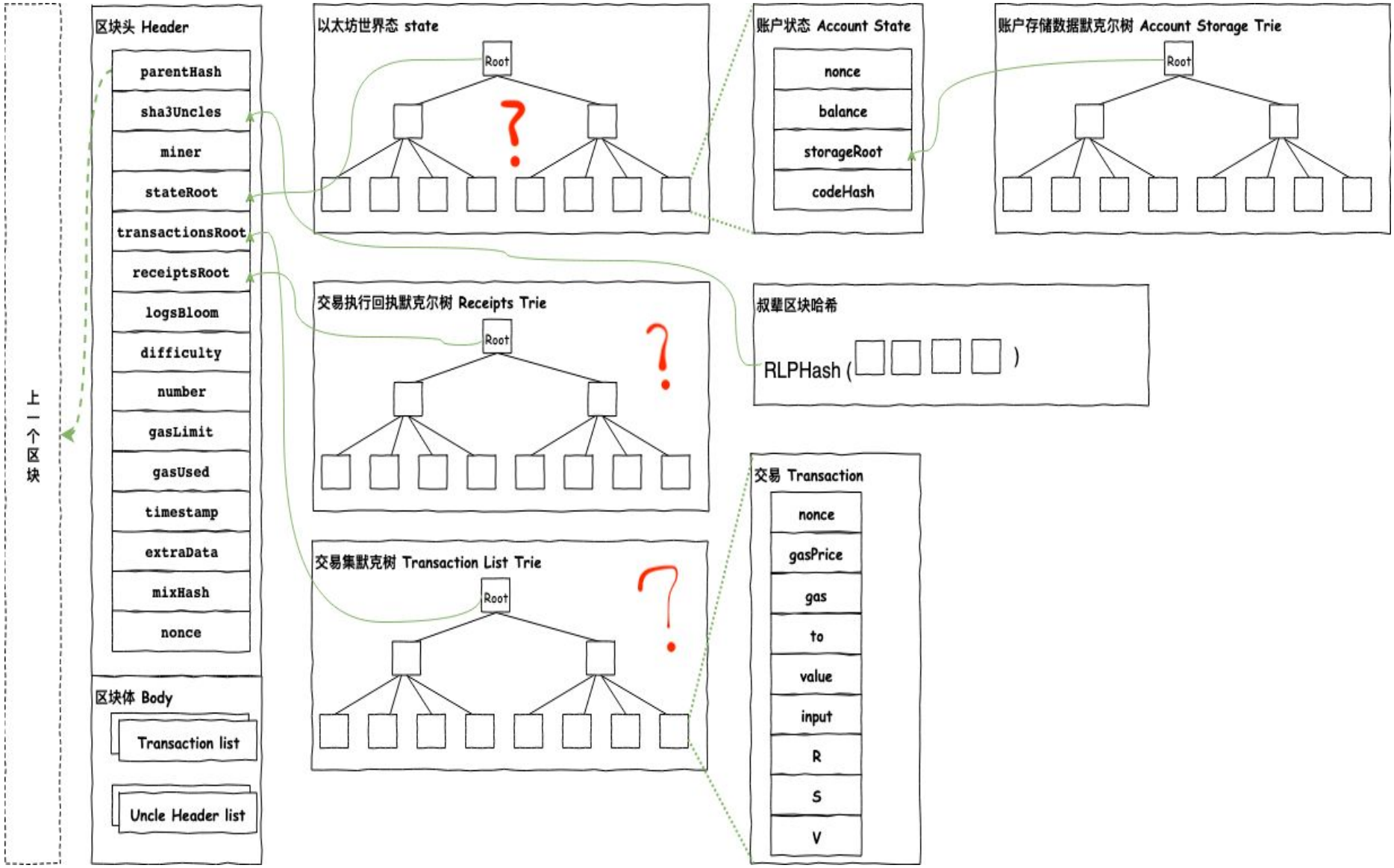
Storageroot

- This field represents the root node of a Merkle Patricia tree that encodes the storage contents of the account.

Codehash

- This is an immutable field that contains the hash of the smart contract code that is associated with the account
- In the case of normal accounts, this field contains the Keccak 256-bit hash of an empty string
- This code is invoked through a message call

The world state and its relationship with accounts trie, accounts, and block header



- The account data structure contains a storage root hash derived from the root node of the account storage trie
- The account data structure is used in the world state trie, which is a mapping between addresses and account states.
- Finally, the root node of the world state trie is hashed using the Keccak 256-bit algorithm and made part of the block header data structure

- Accounts trie (storage contents of account), account tuple, world state trie, and state root hash and their relationship
- Accounts trie is basically a Merkle Patricia tree used to encode the storage contents of an account.
- The contents are stored as a mapping between keccak 256-bit hashes of 256-bit integer keys to the RLP-encoded 256-bit integer values

Transactions

- A transaction in Ethereum is a digitally signed data packet using a private key that contains the instructions that, when completed, either result in a message call or contract creation.
- Transactions can be divided into two types based on the output they produce

- **Message call transactions:** This transaction produces a message call that is used to pass messages from one account to another.
- **Contract creation transactions:** These transactions result in the creation of a new contract. This means that when this transaction is executed successfully, it creates an account with the associated code.

NONCE

- Nonce is a number that is incremented by one every time a transaction is sent by the sender.
- It must be equal to the number of transactions sent and is used as a unique identifier for the transaction.
- A nonce value can only be used once.

Gas Price

- The gasPrice field represents the amount of Wei required in order to execute the transaction.

Gas Limit

- The gasLimit field contains the value that represents the maximum amount of gas that can be consumed in order to execute the transaction.
- This is the amount of fee in Ether that a user (for example, the sender of the transaction) is willing to pay for computation.

TO

- to field is a value that represents the address of the recipient of the transaction.

VALUE

- Value represents the total number of Wei to be transferred to the recipient; in the case of a contract account, this represents the balance that the contract will hold.

SIGNATURE

- Signature is composed of three fields, namely v, *r,* and s
- These values represent the digital signature *(R, S*) and some information that can be used to recover the public key (*V*).
- Also of the transaction from which the sender of the transaction can also be determined.
- The signature is based on ECDSA scheme produces *V,* a single byte value; *R,* a 32 byte value, and S, another 32 byte value.

ECDSASIGN (Message, Private Key) = (V, R, S)

INIT

- The Init field is used only in transactions that are intended to create contracts.
- This represents a byte array of unlimited length that specifies the EVM code to be used in the account initialization process.
- The code contained in this field is executed only once, when the account is created for the first time, and gets destroyed immediately after that.
- Init also returns another code section called *body*, which persists and runs in response to message calls that the contract account may receive.
- These message calls may be sent via a transaction or an internal code execution.

DATA

- If the transaction is a message call, then the data field is used instead of init, which represents the input data of the message call.
- It is also unlimited in size and is organized as a byte array.
- This can be visualized in the following diagram, where a transaction is a tuple of the fields which is then included in a transaction trie (a modified Merkle-Patricia tree) composed of the transactions to be included.
- Finally, the root node of transaction trie is hashed using a Keccak 256-bit algorithm and is included in the block header along with a list of transactions in the block.

- Transactions can be found in either transaction pools or blocks.
- When a mining node starts its operation of verifying blocks, it starts with the highest paying transactions in the transaction pool and executes them one by one.
- When the gas limit is reached or no more transactions are left to be processed in the transaction pool, the mining starts.
- In this process, the block is repeatedly hashed until a valid nonce is found that, once hashed with the block, results in a value less than the difficulty target.
- Once the block is successfully mined, it will be broadcasted immediately to the network, claiming success, and will be verified and accepted by the network.
- This process is similar to Bitcoin's mining process.
- The only difference is that Ethereum's Proof of Work algorithm is ASIC-resistant, known as *Ethash,* where finding a nonce requires large memory.

Relationship between transaction, transaction trie and block header

Contract creation transaction

There are a few essential parameters that are required when creating an account.

- Sender
- Original transactor
- Available gas
- Gas price
- Endowment, which is the amount of ether allocated initially
- A byte array of arbitrary length
- Initialization EVM code
- Current depth of the message call/contract-creation stack (current depth means the number of items that are already there in the stack)

- Addresses generated as a result of contract creation transaction are 160-bit in length.
- 160-bits of the Keccak hash of the RLP encoding of the structure containing only the sender and the nonce.
- Initially, the nonce in the account is set to zero.
- The balance of the account is set to the value passed to the contract.
- Storage is also set to empty.
- Code hash is Keccak 256-bit hash of the empty string.
- The account is initialized when the EVM code (Initialization EVM code) is executed.
- In the case of any exception during code execution, such as not having enough gas, the state does not change.
- If the execution is successful, then the account is created after the payment of appropriate gas costs.
- The current version of Ethereum (homestead) specifies that the result of contract transaction is either a new contract with its balance, or no new contract is created with no transfer of value.
- This is in contrast to previous versions, where the contract could be created regardless of the contract code deployment being successful or not due to an out-of-gas exception.

Message call transaction

A message call requires several parameters for execution,

- Sender
- The transaction originator
- Recipient
- The account whose code is to be executed
- Available gas
- Value
- Gas price
- Arbitrary length byte array
- Input data of the call
- Current depth of the message call/contract creation stack

❖ Message calls result in state transition.

❖ Message calls also produce output data, which is not used if transactions are executed.

❖ In cases where message calls are triggered by VM code, the output produced by the transaction execution is used.

Elements of the Ethereum blockchain

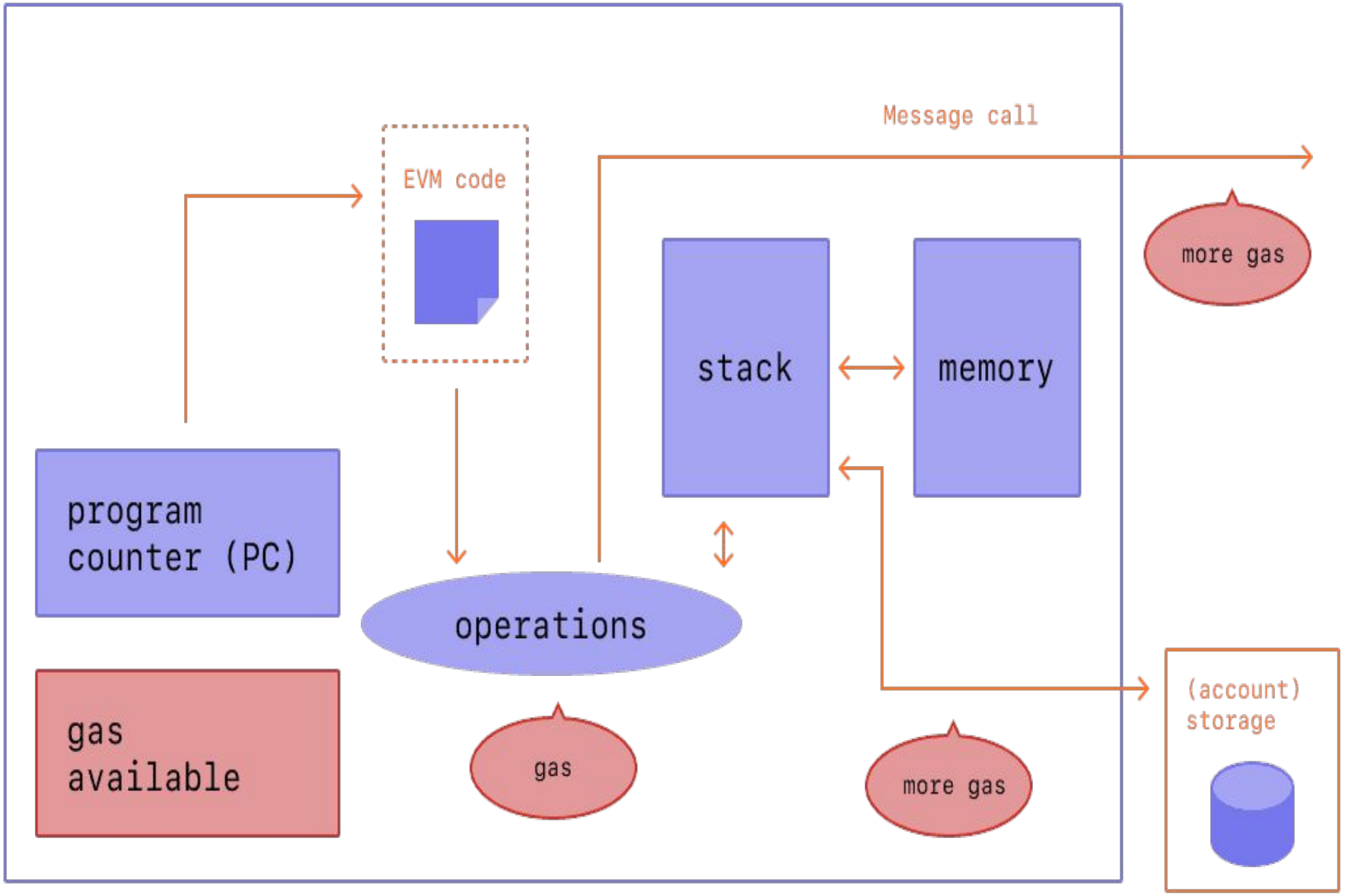
Ethereum virtual machine (EVM)

- EVM is a simple stack-based execution machine that runs bytecode instructions in order to transform the system state from one state to another. The word size of the virtual machine is set to 256-bit.
- The stack size is limited to 1024 elements and is based on the **LIFO (Last in First Out)** queue.
- EVM is a Turing-complete machine but is limited by the amount of gas that is required to run any instruction.
- This means that infinite loops that can result in denial of service attacks are not possible due to gas requirements.

- EVM also supports exception handling in case exceptions occur, such as not having enough gas or invalid instructions, in which case the machine would immediately halt and return the error to the executing agent.
- EVM is a fully isolated and sandboxed runtime environment.
- The code that runs on the EVM does not have access to any external resources, such as a network or filesystem.
- EVM is a stack-based architecture.
- EVM is big-endian by design and it uses 256-bit wide words.
- This word size allows for Keccak 256-bit hash and elliptic curve cryptography computations.
- There are two types of storage available to contracts and EVM.
- The first one is called memory, which is a byte array.
- When a contract finishes the code execution, the memory is cleared.

- It is similar to the concept of RAM.
- The other type, called storage, is permanently stored on the blockchain.
- It is a key value store.
- Memory is unlimited but constrained by gas fee requirements.
- The storage associated with the virtual machine is a word addressable word array that is nonvolatile and is maintained as part of the system state.
- Keys and value are 32 bytes in size and storage.
- The program code is stored in a **virtual read-only memory (virtual ROM)** that is accessible using the CODECOPY instruction.
- The CODECOPY instruction is used to copy the program code into the main memory.

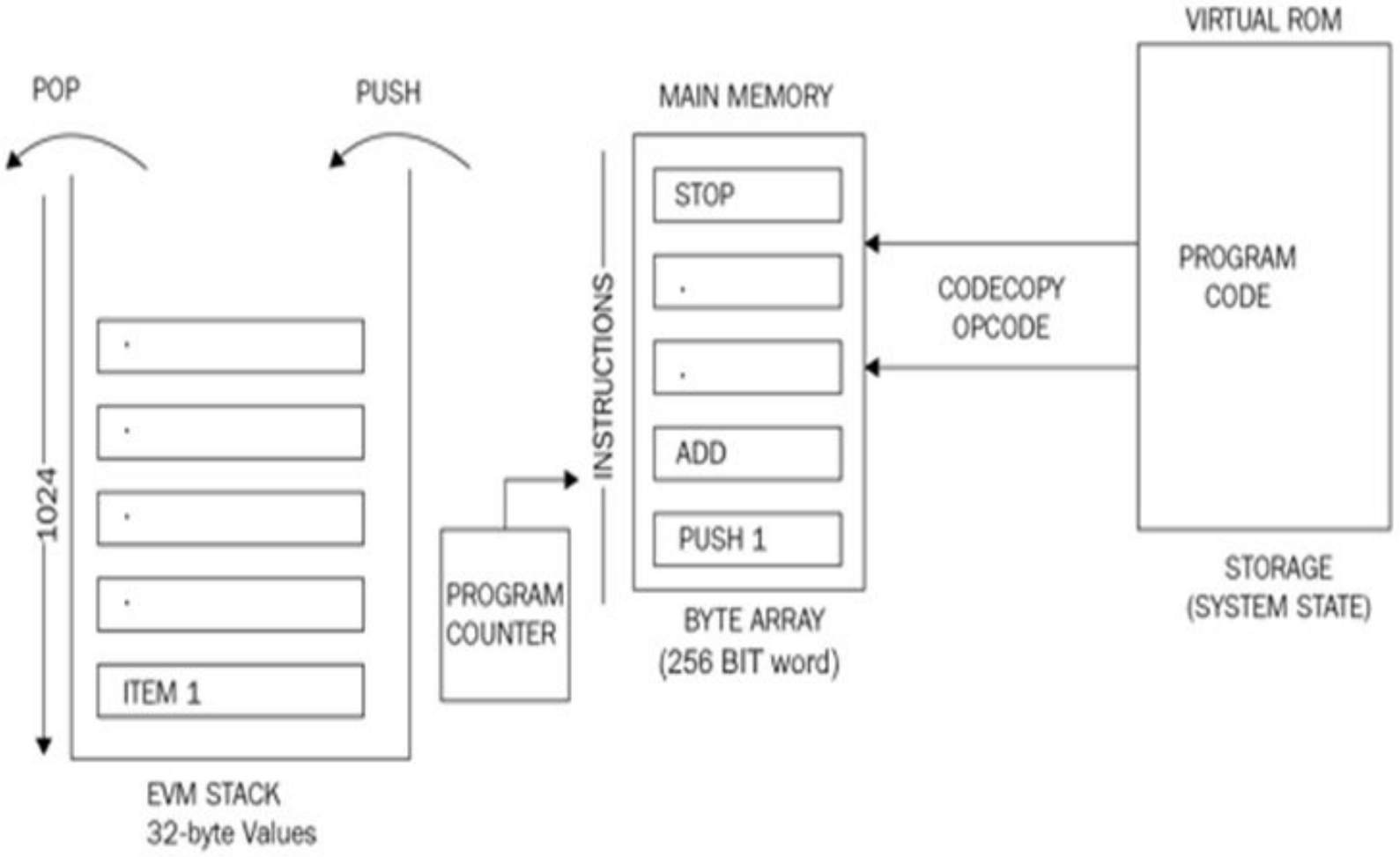
- Initially, all storage and memory is set to zero in the EVM.
- The following diagram shows the design of the EVM where the virtual ROM stores the program code that is copied into main memory using **CODECOPY.**
- The main memory is then read by the EVM by referring to the program counter and executes instructions step by step.
- The program counter and EVM stack are updated accordingly with each instruction execution.


Message call
EVM code
more gas
stack
memory
program
counter (PC)
operations
gas
available
gas
more gas
(account)
storage

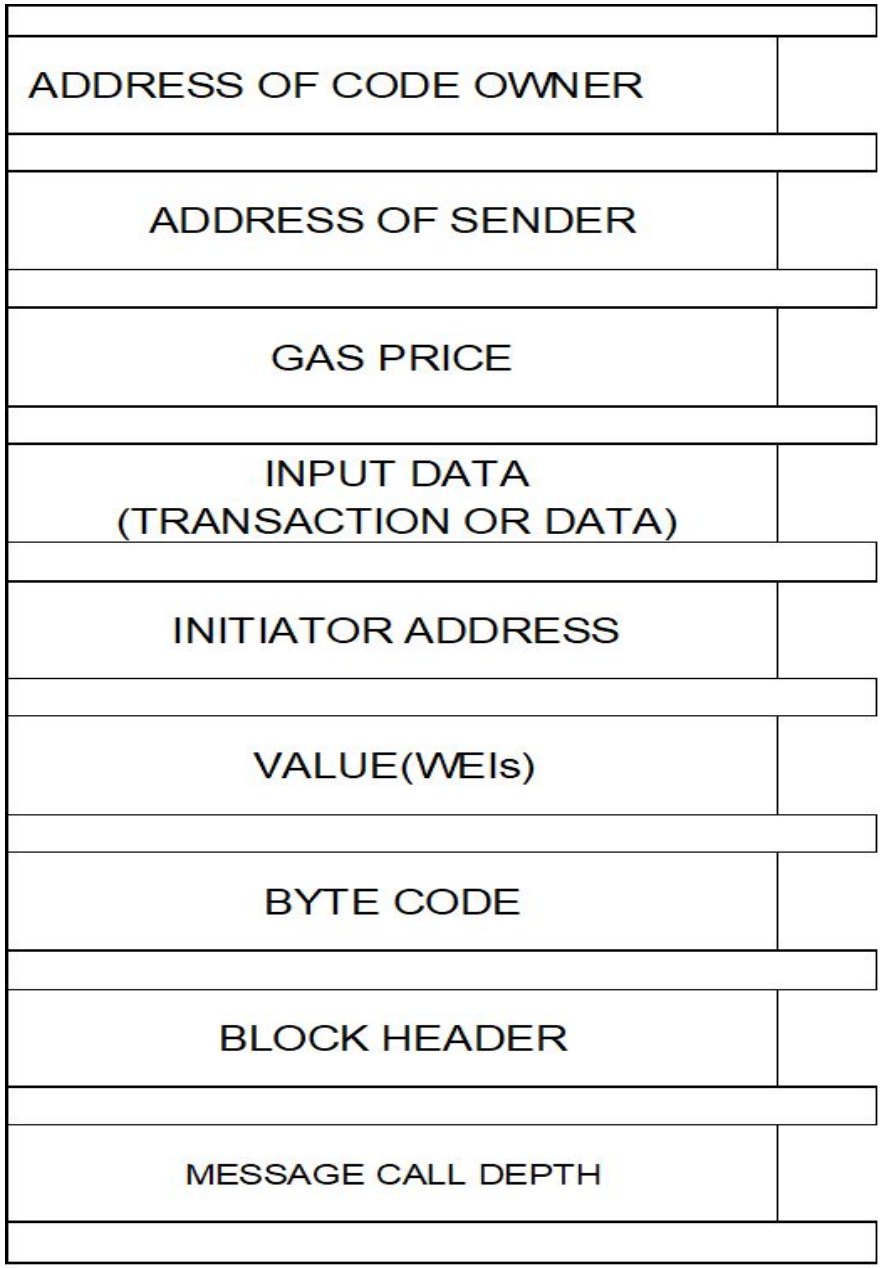
EVM operation

- **Web assembly (WASM)** is developed by Google, Mozilla, and Microsoft and is now being designed as an open standard by the W3C community group.
- The aim of WASM is to be able to run machine code in the browser that will result in execution at native speed.
- Similarly, the aim of EVM 2.0 is to be able to run the EVM instruction set (Opcodes) natively in CPUs, thus making it faster and efficient.

EVM operation



Execution environment Tuple



- In addition to the previously mentioned nine fields, system state and the remaining gas are also provided to the execution environment.
- The execution results in producing the resulting state, gas remaining after the execution, self-destruct or suicide set (described later), log series (described later), and any gas refunds.
- **Machine state**
- Machine state is also maintained internally by the EVM.
- Machine state is updated after each execution cycle of EVM.

- An iterator function runs in the virtual machine, which outputs the results of a single cycle of the state machine.
- Machine state is a tuple that consist of the following elements:
    - Available gas
    - The program counter, which is a positive integer up to 256
    - Memory contents
    - Active number of words in memory
    - Contents of the stack

EXECUTION ENVIRONMENT

There are some key elements that are required by the execution environment in order to execute the code.

1. The address of the account that owns the executing code.

2. The address of the sender of the transaction and the originating address of this execution.
3. The gas price in the transaction that initiated the execution.
4. Input data or transaction data depending on the type of executing agent. This is a byte array; in the case of a message call, if the execution agent is a transaction, then the transaction data is included as input data.
5. The address of the account that initiated the code execution or transaction sender. This is the address of the sender in case the code
execution is initiated by a transaction; otherwise, it's the address of the account.
6. The value or transaction value. This is the amount in Wei. If the execution agent is a transaction, then it is the transaction value.
7. The code to be executed presented as a byte array that the iterator function picks up in each execution cycle.
8. The block header of the current block
9. The number of message calls or contract creation transactions currently in execution. In other words, this is the number of CALLs or CREATES currently in execution.

The EVM is designed to handle exceptions and will halt (stop execution) in case any of the following exceptions occur: •

- Not having enough gas required for execution
- Invalid instructions
- Insufficient stack items
- Invalid destination of jump opcodes
- Invalid stack size (greater than 1024)

Machine state can be viewed as a tuple shown in the following diagram:

- Machine State
- Available Gas
- Program Counter
- Memory Contents
- Number of Words
- Stack Contents

Machine state tuple

The virtual machine is also able to halt in normal conditions if STOP or SUICIDE or RETURN Opcodes are encountered during the execution cycle.

- Code written in a high-level language such as serpent, LLL, or Solidity is converted into the byte code that EVM understands in order for it to be executed by the EVM.
- Solidity is the high-level language that has been developed for Ethereum with JavaScript such as syntax to write code for smart contracts.
- Once the code is written, it is compiled into bytecode that's understandable by the EVM using the Solidity compiler called sole.
- **LLL (Lisp-like Low-level language)** is another language that is used to write smart contract code.
- Serpent is a Python-like high-level language that can be used to write smart contracts for Ethereum.

- For example, a simple program in solidity is shown as follows:

```
pragma solidity ^0.4.0; contract TestI {
uint x=2;
function additionI(uint x) returns (uint y) { y=x+2;
```

This program is converted into bytecode

OPCODES:

Arithmetic operations

- All arithmetic in EVM is modulo 2^A256. This group of opcodes is used to perform basic arithmetic operations. The value of these operations starts from 0x00 up to 0x0b.

Logical operations

- Logical operations include operations that are used to perform comparisons and Boolean logic operations. The value of these operations is in the range of 0x10 to Oxla.

Environmental information

- The opcodes for instructions are used to provide information related to addresses, runtime environments, and data copy operations.

Push operations

- These operations include PUSH operations that are used to place items on the stack. The range of these instructions is from 0x60 to 0x7f. There are 32 PUSH operations available in total in the EVM. PUSH operation, which reads from the byte array of the program code.

Duplication operations

- Duplication operations are used to duplicate stack items. The range of values is from 0x80 to 0x8f. There are 16 DUP instructions available in the EVM. Items placed on the stack or removed from the stack also change incrementally with the mnemonic used; for example, DUP1 removes one item from the stack and places two items on the stack, whereas DUP16 removes 16 items from the stack and places 17 items.

Exchange operations

- SWAP operations provide the ability to exchange stack items. There are 16 SWAP instructions available and with each instruction, the stack items are removed and placed incrementally up to 17 items depending on the type of Opcode used.

Logging operations

- Logging operations provide opcodes to append log entries on the sub-state tuple's log series field. There are four log operations available in total and they range from value 0x0a to 0xa4

System operations

- System operations are used to perform various system-related operations, such as account creation, message calling, and execution control.

Precompiled contracts

There are four precompiled contracts in Ethereum.

The elliptic curve public key recovery function

ECDSARECOVER (Elliptic curve DSA recover function) is available at address 1.

- It is denoted as ECREC and requires 3000 gas for execution.
- If the signature is invalid, then no output is returned by this function.
- Public key recovery is a standard mechanism by which the public key can be derived from the private key in elliptic curve cryptography.

The ECDSA recovery function is shown as follows:

$$ECDSARECOVER(H, V, R, S) = Public\ Key$$

It takes four inputs: H, which is a 32 byte hash of the message to be signed and *V, R*, and S, which represent the ECDSA signature with the recovery ID and produce a 64 byte public key. *V, R,* and S.

The SHA-256 bit hash function

- The SHA-256 bit hash function is a precompiled contract that is available at address 2 and produces a SHA256 hash of the input.
- It is almost like a pass-through function.
- Gas requirement for SHA-256 (SHA256) depends on the input data size.
- The output is a 32 byte value.

The RIPEMD-160 bit hash function

- The RIPEMD-160 bit hash function is used to provide RIPEMD 160-bit hash and is available at address 3.
- The output of this function is a 20-byte value.
- Gas requirement, similar to SHA-256, is dependent on the amount of input data.

The identity function

- The identity function is available at address 4 and is denoted by the ID.
- Whatever input is given to the ID function, it will output the same value.
- Gas requirement is calculated by a simple formula: *15 + 3 [lc/32]* where I_d is the input data.
- This means that at a high level, the gas requirement is dependent on the size of the input data albeit with some calculation performed.

Accounts

Accounts are one of the main building blocks of the Ethereum blockchain. The state is created or updated as a result of the interaction between accounts. Operations performed between and on the accounts represent state transitions. State transition is achieved using Ethereum state transition function as follows:

1. Confirm the transaction validity by checking the syntax, signature validity, and nonce.
2. Transaction fee is calculated and the sending address is resolved using the signature. Furthermore, sender's account balance is checked and subtracted accordingly and nonce is incremented. An error is returned if the account balance is not enough.
3. Provide enough ether (gas price) to cover the cost of the transaction. This is charged per byte incrementally according to the size of the transaction.
4. In this step, the actual transfer of value occurs. The flow is from the sender's account to receiver's account. The account is created automatically if the destination account specified in the transaction does not exist yet. Moreover, if the destination account is a contract, then the contract code is executed. This also depends on the amount of gas available. If enough gas is available, then the contract code will be executed fully; otherwise, it will run up to the point where it runs out of gas.
5. In cases of transaction failure due to insufficient account balance or gas, all state changes are rolled back with the exception of fee payment, which is paid to the miners.
6. Finally, the remainder (if any) of the fee is sent back to the sender as change and fee is paid to the miners accordingly. At this point, the function returns the resulting state.

Types of accounts

There are two types of accounts in Ethereum:

- Externally owned accounts
- Contract accounts

E**xternally owned accounts (EOAs)** are controlled by a private key in bitcoin. Contract accounts are the accounts that have code associated with them along with the private key. An EOA has ether balance, is able to send transactions, and has no associated code.

A **Contract Account (CA)** has ether balance, associated code, and the ability to get triggered and execute code in response to a transaction or a message. Due to the Turing-completeness property of the Ethereum blockchain, the code within contract accounts can be of any level of complexity. The code is executed by EVM by each mining node on the Ethereum network. In addition, contract accounts are able to maintain their own permanent state and can call other contracts. In the serenity release, the distinction between externally owned accounts and contract accounts may be eliminated.

Block

Blocks are the main building blocks of a blockchain. Ethereum blocks consist of various components,

- The block header
- The transactions list
- The list of headers of Ommers or Uncles

The transaction list is a list of all transactions included in the block. In addition, the list of headers of Uncles is also included in the block.

Block header

Block headers are the most critical and detailed components of an Ethereum block. The header contains valuable information.

PARENT HASH

This is the Keccak 256-bit hash of the parent (previous) block's header.

OMMERS HASH

This is the Keccak 256-bit hash of the list of Ommers (Uncles) blocks included in the block.

BENEFICIARY

- Beneficiary field contains the 160-bit address of the recipient that will receive the mining reward once the block is successfully mined.

STATE ROOT

- The state root field contains the Keccak 256-bit hash of the root node of the state trie.
- It is calculated after all transactions have been processed and finalized.

TRANSACTIONS ROOT

- The transaction root is the Keccak 256-bit hash of the root node of the transaction trie.
- Transaction trie represents the list of transactions included in the block.

RECEIPTS ROOT

- The receipts root is the keccak 256 bit hash of the root node of the transaction receipt trie.
- This trie is composed of receipts of all transactions included in the block.
- Transaction receipts are generated after each transaction is processed and contain useful post-transaction information.

LOGS BLOOM

- The logs bloom is a bloom filter that is composed of the logger address and log topics from the log entry of each transaction receipt of the included transaction list in the block.

DIFFICULTY

- The difficulty level of the current block.

NUMBER

- The total number of all previous blocks; the genesis block is block zero.

GAS LIMIT

- The field contains the value that represents the limit set on the gas consumption per block.

GAS USED

- The field contains the total gas consumed by the transactions included in the block.

TIMESTAMP

- Timestamp is the epoch Unix time of the time of block initialization.

EXTRA DATA

- Extra data field can be used to store arbitrary data related to the block.

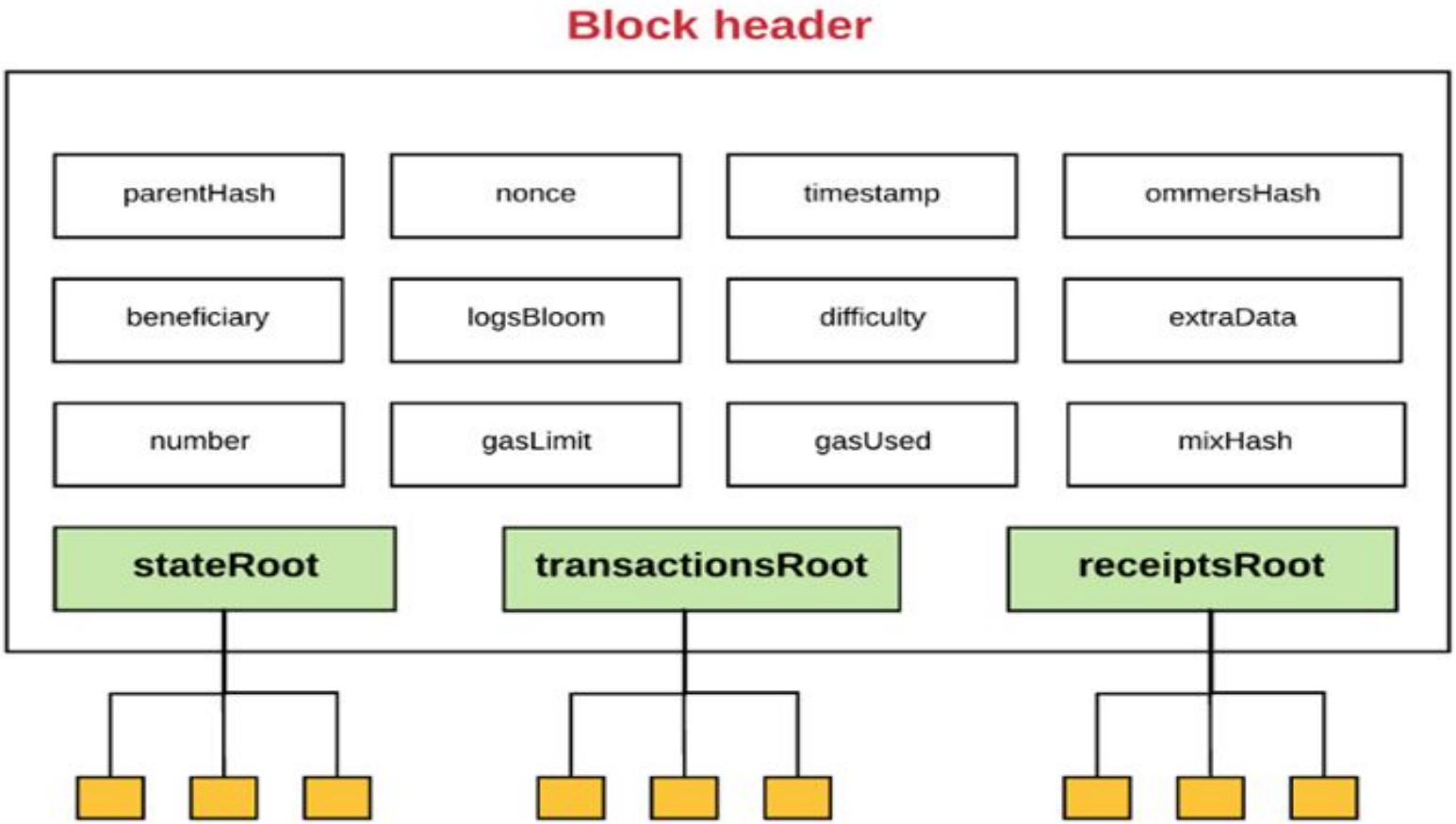
NONCE

- Nonce is a 64-bit hash (a number) that is used to prove, in combination with the mixhash field, that adequate computational effort has been spent in order to create this block.

The genesis block

- The genesis block varies slightly with regard to the data it contains and the way it has been created from a normal block.

Ethereum block structure with block header



Transaction receipts

- Transaction receipts are used as a mechanism to store the state after a transaction has been executed.
- These structures are used to record the outcome of the transaction execution.
- It is produced after the execution of each transaction.
- All receipts are stored in an index-keyed trie.
- Hash (Keccak 256-bit) of the root of this trie is placed in the block header as the receipts root It is composed of four elements

MIXHASH

- Mixhash field contains a 256-bit hash that once combined with the nonce is used to prove that adequate computational effort has been spent in order to create this block.

THE POST-TRANSACTION STATE

- This item is a trie structure that holds the state after the transaction has executed.
- It is encoded as a byte array.

GAS USED

- This item represents the total amount of gas used in the block that contains the transaction receipt.
- The value is taken immediately after the transaction execution is completed. The total gas used is expected to be a nonnegative integer.

SET OF LOGS

- This field shows the set of log entries created as a result of transaction execution.
- Log entries contain the logger's address, a series of log topics, and the log data.

Transaction validation and execution

Transactions are executed after verifying the transactions for validity.

Initial tests are listed as follows:

- A transaction must be well-formed and RLP-encoded without any additional trailing bytes
- The digital signature used to sign the transaction is valid
- Transaction nonce must be equal to the sender's accounts current nonce
- Gas limit must not be less than the gas used by the transaction
- The sender's account contains enough balance to cover the execution cost

THE TRANSACTION SUB STATE

A transaction sub-state is created during the execution of the transaction that is processed immediately after the execution completes.

This transaction sub-state is a tuple that is composed of three items.

Suicide set

This element contains the list of accounts that are disposed of after the transaction is executed.

Log series

- This is an indexed series of checkpoints that allow the monitoring and notification of contract calls to the entities external to the Ethereum environment, such as application frontends.
- It works like a trigger mechanism that is executed every time a specific function is invoked or a specific event occurs.
- Logs are created in response to events occurring in the smart contract.
- It can also be used as a cheaper form of storage.

Refund balance

- This is the total price of gas in the transaction that initiated the execution.
- Refunds are not immediately executed; instead, they are used to partially offset the total execution cost.
- The following diagram describes the transaction substate tuple:
  - Substrate
  - Suicide Set
  - Log Series
  - Refund Balance

- **The block validation mechanism**

  An Ethereum block is considered valid if it passes the following checks:

  - Consistent with Uncles and transactions. This means that all Ommers (Uncles) satisfy the property that they are indeed Uncles and also if the Proof of Work for Uncles is valid.
  - If the previous block (parent) exists and is valid.
  - If the timestamp of the block is valid. This basically means that the current block's timestamp must be higher than the parent block's timestamp. Also, it should be less than 15 minutes into the future. All block times are calculated in epoch time (Unix time).
  - If any of these checks fails, the block will be rejected.

BLOCK FINALIZATION

Block finalization is a process that is run by miners in order to validate the contents of the block and apply rewards. It results in four steps being executed.

Ommers validation

Validate Ommers (stale blocks also called Uncles). In the case of mining, determine Ommers. The validation process of the headers of stale blocks checks whether the header is valid and the relationship of the Uncle with the current block satisfies the maximum depth of six blocks. A block can contain a maximum of two Uncles.

Transaction validation

Validate transactions. In the case of mining, determine transactions. The process involves checking whether the total gas used in the block is equal to the final gas consumption after the final transaction.

Reward application

Apply rewards, which means updating the beneficiary's account with a reward balance. In Ethereum, a reward is also given to miners for stale blocks, which is 1/32 of the block reward. Uncles that are included in the blocks also receive 7/8 of the total block reward. The current block reward is 5 Ether. A block can have a maximum of two Uncles.

State and nonce validation

Verify the state and nonce. In the case of mining, compute a valid state and nonce.

BLOCK DIFFICULTY

- Block difficulty is increased if the time between two blocks decreases, whereas it increases if the block time between two blocks decreases.
- This is required to maintain a roughly consistent block generation time.
- The difficulty adjustment algorithm in Ethereum's homestead release is shown as follows:

```
block_diff = parent_diff + parent_diff // 2048 *
max(1 - (block_timestamp -parent_timestamp) // 10, -99) +
int(2**((block.number // 100000) - 2))
```

- If the time difference between the generation of the parent block and the current block is less than 10 seconds, the difficulty goes up.
- If the time difference is between 10 to 19 seconds, the difficulty level remains the same.
- Finally, if the time difference is 20 seconds or more, the difficulty level decreases.
- This decrease is proportional to the time difference.

- In addition to timestamp-difference-based difficulty adjustment, there is also another part that increases the difficulty exponentially after every 100,000 blocks.
- This is the so called *difficulty time bomb* or *Ice age* introduced in the Ethereum network, which will make it very hard to mine on the Ethereum blockchain at some point in the future.
- This will encourage users to switch to Proof of Stake as mining on the POW chain will eventually become prohibitively difficult.
- According to the latest update and estimates based on the algorithm, the block generation time will become significantly high during the second half of the year 2017 and in 2021, it will become so high that it will be virtually impossible to mine on the POW chain.

Ether

- Ether is minted by miners as a currency reward for the computational effort they spend in order to secure the network by verifying and with validation transactions and blocks.
- Ether is used within the Ethereum blockchain to pay for the execution of contracts on the EVM.
- Ether is used to purchase gas as crypto fuel, which is required in order to perform computation on the Ethereum blockchain.
- Fees are charged for each computation performed by the EVM on the blockchain.

- Gas is required to be paid for every operation performed on the ethereum blockchain.
- This is a mechanism that ensures that infinite loops cannot cause the whole blockchain to stall due to the Turing-complete nature of the EVM.
- A transaction fee is charged as some amount of Ether and is taken from the account balance of the transaction originator.
- A fee is paid for transactions to be included by miners for mining.
- If this fee is too low, the transaction may never be picked up; the more the fee, the higher are the chances that the transactions will be picked up by the miners for inclusion in the block.
- Conversely, if the transaction that has an appropriate fee paid is included in the block by miners but has too many complex operations to perform, it can result in an out-of-gas exception if the gas cost is not enough.
- In this case, the transaction will fail but will still be made part of the block and the transaction originator will not get any refund.

- Transaction cost can be estimated using the following formula:

  *Total cost = gasUsed * gasPrice*

- Here, *gasUsed* is the total gas that is supposed to be used by the transaction during the execution and *gasPrice* is specified by the transaction originator as an incentive to the miners to include the transaction in the next block.
- This is specified in Ether.
- Each EVM opcode has a fee assigned to it.
- It is an estimate because the gas used can be more or less than the value specified by the transaction originator originally.
- For example, if computation takes too long or the behavior of the smart contract changes in response to some other factors, then the transaction execution may perform more or less operations than originally intended and can result in consuming more or fewer gas.
- If the execution runs out of gas, everything is immediately rolled back; otherwise, if the execution is successful and there is some remaining gas, then it is returned to the transaction originator.

Each operation costs some gas; a high level fee schedule of a few operations

Operation Name	Gas Cost
step	1
stop	0
suicide	0
sha3	30
sload	20
txdata	5
transaction	500
contract creation	53000

Based on the preceding fee schedule and the formula, an example calculation of the SHA3

operation can be calculated as follows:

- SHA3 costs 30 gas
- Current gas price is 25 GWei, which is 0.000000025 Ether
- Multiplying both: ***0.000000025 *30 = 0.00000075*** Ether

In total, 0.00000075 Ether is the total gas that will be charged.

Fee schedule

Gas is charged in three scenarios as a prerequisite to the execution of an operation:

- The computation of an operation
- For contract creation or message call
- Increase in the usage of memory

Messages

- Messages are the data and value that are passed between two accounts.
- A message is a data packet passed between two accounts.
- This data packet contains data and value (amount of ether).
- It can either be sent via a smart contract (autonomous object) or from an external actor (externally owned account) in the form of a transaction that has been digitally signed by the sender.
- Contracts can send messages to other contracts.
- Messages only exist in the execution environment and are never stored.
- Messages are similar to transactions; however, the main difference is that they are produced by the contracts, whereas transactions are produced by entities external (externally owned accounts) to the Ethereum environment.

A message consists of the components:

1. Sender of the message
2. Recipient of the message
3. Amount of Wei to transfer and message to the contract address
4. Optional data field (Input data for the contract)
5. Maximum amount of gas that can be consumed

- Messages are generated when CALL or DELEGATECALL Opcodes are executed by the contracts.

Calls

- A call does not broadcast anything to the blockchain; instead, it is a local call to a contract function and runs locally on the node. It is almost like a local function call.

- It does not consume any gas as it is a read-only operation.
- It is akin to a dry run. Calls are executed locally on a node and generally do not result in any state change.
- This is the act of passing a message from one account to another.
- If the destination account has an associated EVM code, then the virtual machine will start upon the receipt of the message to perform the required operations.
- If the message sender is an autonomous object, then the call passes any data returned from the virtual machine operation.
- State is altered by transactions.
- These are created by external factors and are signed and then broadcasted to the Ethereum network.

Mining

Mining is the process by which new currency is added to the blockchain. This is an incentive for the miners to validate and verify blocks made up of transactions. The mining process helps secure the network by verifying computations.

A miner performs the following functions:

1. Listens for the transactions broadcasted on the Ethereum network and determines the transactions to be processed.
2. Determines stale blocks called Uncles or Ommers and includes them in the block.
3. Updates the account balance with the reward earned from successfully mining the block.
4. Finally, a valid state is computed and block is finalized, which defines the result of all state transitions.

- The current method of mining is based on Proof of Work, which is similar to that of bitcoin.
- When a block is deemed valid, it has to satisfy not only the general consistency requirements, but it must also contain the Proof of Work for a given difficulty.
- The Proof of Work algorithm is due to be replaced with the Proof of Stake algorithm with the release of serenity.

- An Algorithm named Casper replace the existing Proof of Work algorithm in Ethereum.
- This is a security deposit based on the economic protocol where nodes are required to place a security deposit before they can produce blocks.
- Nodes have been named bonded validators in Casper, whereas the act of placing the security deposit is named bonding.

Ethash

- Ethash is a memory-hard algorithm, which makes it difficult to be implemented on specialized hardware.

- As in bitcoin, ASICs have been developed, which have resulted in mining centralization over the years, but memory-hard Proof of Work algorithms are one way of thwarting this threat and Ethereum implements Ethash to discourage ASIC development for mining.
- This algorithm requires choosing subsets of a fixed resource called **DAG (Directed Acyclic Graph)** depending on the nonce and block headers.
- DAG is around 2 GB in size and changes every 30000 blocks.
- Mining can only start when DAG is completely generated the first time a mining node starts.
- The time between every 30000 blocks is around 5.2 days and is called epoch.
- This DAG is used as a seed by the Proof of Work algorithm called Ethash.
- According to current specifications, the epoch time is defined as 30,000 blocks.

- The current reward scheme is 5 Ether for successfully finding a valid nonce.
- In addition to receiving 5 Ethers, the successful miner also receives the cost of the gas consumed within the block and an additional reward for including stale blocks (Uncles) in the block.
- A maximum of two Uncles are allowed per block and are rewarded 7/8 of the normal block reward.
- In order to achieve a 12 second block time, block difficulty is adjusted at every block.
- The rewards are directly proportional to the miner's hash rate, which basically means how fast a miner can hash.
- Mining can be performed by simply joining the Ethereum network and running an appropriate client.
- The key requirement is that the node should be fully synced with the main network before mining can start.

Various types of mining

CPU mining

- Even though not profitable on the main net, CPU mining is still valuable on the test network or even a private network to experiment with mining and contract deployment.
- Geth can be started with mine switch in order to start mining:

```
geth --mine --minerthreads <n>
```

- CPU mining can also be started using the web 3 geth console.
- Geth console can be started by issuing the following command:

```
geth attach
```

- The miner can be started by issuing the following command, which will return true if successful, or false otherwise.

  Miner.start(4) True

- The preceding command will start the miner with four threads.

  Miner.stop True

- The preceding command will stop the miner.
- The command will return true if successful.

GPU mining

- At a basic level, GPU mining can be performed easily by running two commands:

```
geth --rpc
```

- Once geth is up and running and the blockchain is fully downloaded, Ethminer can be run in order to start mining.
- Ethminer is a standalone miner that can also be used in the farm mode to contribute to mining pools.
- Once the graphics cards are installed and configured correctly, the process can be started by issuing the ethminer -G command.

```
ethminer -G
```

- Running with G switch assumes that the appropriate graphics card is installed and configured correctly.
- If no appropriate graphics cards are found, ethminer will return an error

- The GPU device to be used can also be specified in the command line:
  - $ ethminer -M -G --opencl-device 1

- As GPU mining is implemented using OpenCL AMD, chipset-based GPUs tend to work faster as compared to NVidia GPUs.
- Due to the high memory requirements (DAG creation), FPGAs and ASICs will not provide any major advantage over GPUs.
- This is done on purpose in order to discourage the development of specialized hardware for mining.

Mining rigs

- As difficulty increased over time for mining Ether, mining rigs with multiple GPUs were starting to be built by the miners.
- A mining rig usually contains around five GPU cards, and all of them work in parallel for mining, thus improving the chances of finding valid nonces for mining.
- Mining rigs can be built with some effort and are also available commercially from various vendors.

MOTHERBOARD

- A specialized motherboard with multiple PCI-E xl or xl6 slots, for example, BIOSTAR Hi-Fi or ASRock H81, is required

SSD HARD DRIVE

An SSD hard drive is required. The SSD drive is recommended because of its much faster performance over the analog equivalent. This will be mainly used to store the blockchain.

GPU

The GPU is the most important component of the rig as it is the main workhorse that will be used for mining. For example, it can be a Sapphire AMD Radeon R9 380 with 4 GB RAM.

Linux Ubuntu's latest version is usually chosen as the operating system for the rig. There is also another variant of Linux available, called EthOS, that is especially built for Ethereum mining and supports mining operations natively.

- Finally, mining software such as Ethminer and geth are installed.
- Additionally, some remote monitoring and administration software is also installed so that rigs can be monitored and managed remotely, if required.
- It is also important to put appropriate air conditioning or cooling mechanisms in place as running multiple GPUs can generate a lot of heat.
- This also necessitates the need for using an appropriate monitoring software that can alert users if there are any problems with the hardware, for example, if the GPUs are overheating.

Mining pools

- There are many online mining pools that offer Ethereum mining.
- Ethminer can be used to connect to a mining pool using the following command.
- Each pool publishes its own instructions, but generally, the process of connecting to a pool is similar.

```
ethminer -C -F http://ethereumpool.co/?
```

Clients and wallets

- The following is a list of all main components, client software, and tools that are available with Ethereum.

Geth

- This is the Go implementation of the Ethereum client.

Eth

- This is the C++ implementation of the Ethereum client.

Pyethapp

- This is the Python implementation of the Ethereum client.

Parity

- This implementation is built using Rust and developed by EthCore. EthCore is a company that works on the development of the parity client.

Light clients

- SPV clients download only a small subset of the blockchain.
- This allows low resource devices, such as mobile phones, embedded devices, or tablets, to be able to verify the transactions.
- A complete ethereum blockchain and node are not required in this case and SPV clients can still validate the execution of transactions. SPV clients are also called light clients.
- This idea is similar to bitcoin SPV clients.
- There is a wallet available from Jaxx which can be installed on iOS and Android, which provides the **SPV (Simple Payment Verification)** functionality.

Installation

- The following installation procedure describes the installation of various Ethereum clients on Ubuntu systems.
- **Geth client** can be installed by using the following command on an Ubuntu system:

```
> sudo apt-get install -y software-properties -common
>   sudo add-apt-repository -y ppa:ethereum/ethereum
>   sudo apt-get update
>   sudo apt-get install -y ethereum
```

- After installation is completed.
- Geth can be launched simply by issuing the geth command at the command prompt, as it comes preconfigured with all the required parameters to connect to the live Ethereum network (main net):

ETH INSTALLATION

- Eth is the C++ implementation of the Ethereum client and can be installed using the following command on Ubuntu:

```
> sudo apt-get install cpp-ethereum
```

MIST BROWSER

- Mist browser is a user-friendly interface for end users with a feature-rich graphical user interface that is used to browse DAPPS and for account management and contract management.
- When Mist is launched for the first time, it will initialize geth in the background and will sync with the network.
- It can take from a few hours to a few days depending on the speed and type of the network to fully synchronize with the network.
- If TestNet is used, then syncing completes relatively faster as the size of TestNet (Ropsten) is not as big as MainNet.

- Mist browser is not a wallet; it is a browser of DAPPS and provides a user-friendly user interface for the creation and management of contracts, accounts and browsing decentralized applications.
- Ethereum wallet is a DAPP that is released with Mist.
- Wallet is a generic program that can store private keys and associated accounts based on the addresses stored within it, it can compute the existing balance of Ether associated with the addresses by querying the blockchain.
- Other wallet is MyEtherWallet, which is an open source ether wallet developed in JavaScript.

- MyEtherWallet runs in the client browser.
- Icebox is developed by Consensys.
- This is a cold storage browser that provides secure storage of Ether.
- It depends on whether the computer on which Icebox is run is connected to the Internet or not.
- Various wallets are available for ethereum for desktop, mobile, and web platforms.

Parity installation

- Parity is another implementation of the Ethereum client.
- It has been written using the Rust programming language.
- The main aim behind the development of parity is high performance, small footprint, and reliability.
- Parity can be installed using the following commands on an Ubuntu or Mac system:
  - bash <(curl https://get.parity.io -Lk)
- This will initiate the download and installation of the parity client.
- After the installation of parity is completed, the installer will also offer the installation of the netstats client.
- The netstat client is a daemon that runs in the background and collects important statistics and displays them on stats.ethdev.com.

- Once the installation is completed successfully, the following message is displayed. Ethereum parity node can then be started using parity - j.
- If compatibility with geth is required in order to use Ethereum wallet (Mist browser) with parity, then the parity -geth command should be used to run parity.
- This will run parity in compatibility mode with the geth client and will consequently allow Mist to run on top of parity.
- All connected clients are listed with relevant attributes, such as the node name, node type, latency, mining status, number of peers, number of pending transactions, last block, difficultly, block transactions, and number of Uncles.
- Parity also offers a user-friendly web interface from where various tasks, such as account management, address book management, DAPP management, contract management, and status and signer operations, can be managed.

The Ethereum network

- The Ethereum network is a peer-to-peer network where nodes participate in order to maintain the blockchain and contribute to the consensus mechanism.
- Networks can be divided into three types, based on requirements and usage.

MainNet

- MainNet is the current live network of ethereum.
- The current version of MainNet is homestead.

TestNet

- TestNet is also called Ropsten and is the test network for the Ethereum blockchain.
- This blockchain is used to test smart contracts and DApps before being deployed to the production live blockchain.
- Moreover, being a test network, it allows experimentation and research.

Private net(s)

- This is the private network that can be created by generating a new genesis block. T
- This is the case in distributed ledger networks, where a private group of entities start their own blockchain and use it as a permissioned blockchain.

Supporting protocols

- There are various supporting protocols that are in development in order to support the complete decentralized ecosystem.
- This includes whisper and Swarm protocols.
- In addition to the contracts layer, which is the core blockchain layer, there are additional layers that need to be decentralized in order to achieve a complete decentralized ecosystem.
- This includes decentralized storage and decentralized messaging.
- Whisper, being developed for ethereum, is a decentralized messaging protocol, whereas Swarm is a decentralized storage protocol.

WHISPER

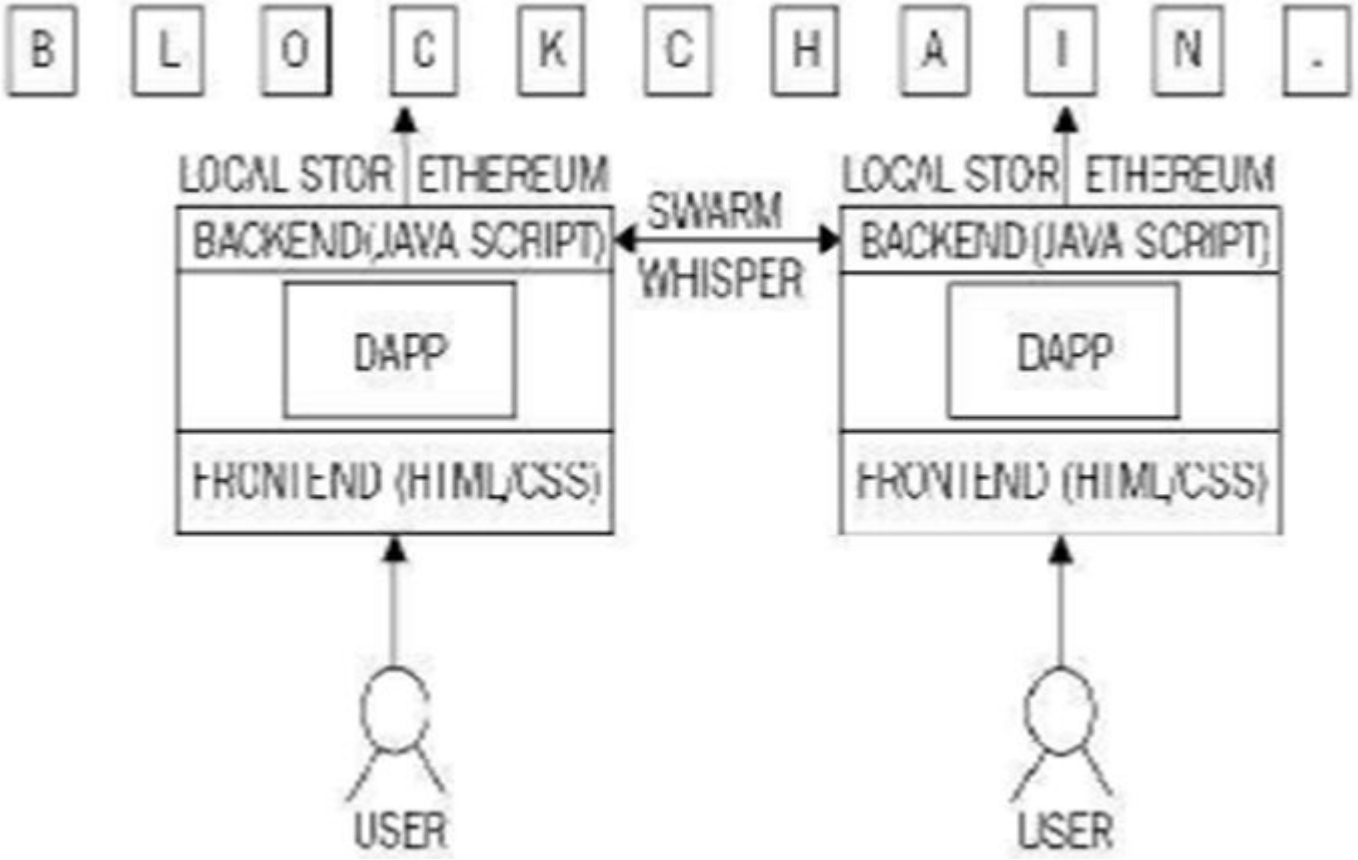
- Whisper provides decentralized peer-to-peer messaging capabilities to the ethereum network.
- Whisper is a communication protocol that nodes use in order to communicate with each other.
- The data and routing of messages are encrypted within whisper communications.
- Moreover, it is designed to be used for smaller data transfers and in scenarios where real-time communication is not required.
- Whisper is also designed to provide a communication layer that cannot be traced and provides "dark communication" between parties.

- Blockchain can be used for communication, but that is expensive and consensus is not really required for messages exchanged between nodes.
- Therefore, whisper can be used as a protocol that allows.
- Whisper is already available with geth and can be enabled using the - - shh option while running the geth ethereum client.

SWARM

- Swarm is being developed as a distributed file storage platform.
- It is a decentralized, distributed, and peer-to-peer storage network.
- Files in this network are addressed by the hash of their content.
- This is in contrast to the traditional centralized services, where storage is available at a central location only.
- This is developed as a native base layer service for the Ethereum web 3.0 stack.
- Swarm is integrated with DevP2P, which is the multiprotocol network layer of Ethereum.
- Swarm is envisaged to provide a DDOS (Distributed Denial of service)-resistant and fault-tolerant distributed storage layer for Ethereum Web 3.0.

Swarm and whisper fit together and work with blockchain



Ethereum Development

Setting up a development environment

- Test Net is called Ropsten and is used by developers or users as a test platform to test smart contracts and other blockchain-related proposals.
- The Private Net option in Ethereum allows the creation of an independent private network that can be used as a distributed ledger between participating entities and for the development and testing of smart contracts.
- While there are other clients available for Ethereum, such as Parity.

Test Net (Ropsten)

- The Ethereum Go client, geth, can be connected to the test network using the following command:

```
$ geth --TestNet
```

Setting up a Private Net

- Private Net allows the creation of an entirely new blockchain.
- This is different from Test Net or Main Net in the sense that it uses its on-genesis block and Network ID.
- In order to create Private Net, three components are needed:

1. Network ID.
2. Genesis file.
3. Data directory to store blockchain data.

- Even though data directory is not strictly required to be mentioned, if there is more than one blockchain already active on the system, then data directory should be specified so that a separate directory is used for the new blockchain.
- Private Net allows the creation of an entirely new blockchain.
- This is different from Test Net or Main Net in the sense that it uses its own unique genesis block and Network ID.

- On Main Net, geth knows about peers by default and connects automatically, but on Private Net, geth needs to be configured by specifying appropriate flags and configuration in order for it to be able to be discoverable by other peers or to discover other peers.
- It is desirable to disable **node discovery** so that other nodes on the Internet cannot discover your private network and is truly private.
- If other networks happen to have the same genesis file and Network ID, they may connect to your Private Net.
- The chance of having the same Network ID and genesis block is very low.

NETWORK ID

- Network ID can be any positive number except 1 and 3, which are already in use by Ethereum MainNet and Test Net (Ropsten), respectively.

THE GENESIS FILE

- The genesis file contains necessary fields required for a custom genesis block.
- This is the first block in the network and does not point to any previous block.
- The Ethereum protocol performs rigorous checking in order to ensure that no other node on the Internet can participate in the consensus mechanism, unless they have the same genesis block.

Data Directory

- This is the directory where the blockchain data for the private Ethereum network will be saved.
- For example, in the following example, it is -/.ethereum/privatenet.
- In the geth client, a number of parameters are specified in order to launch, further fine-tune the configuration and launch the private network.

These flags are ,

Flags and their meaning

- --nodiscover: This flag ensures that the node is not automatically discoverable if it happens to have the same genesis file and Network ID.
- - -maxpeers: This flag is used to specify the number of peers allowed to be connected to the private net. If it is set to 0, then no one will be able to connect, which might be desirable in a few scenarios, such as private testing.
- - - rpc: This is used to enable the RPC interface in geth.
- - - rpcapi: This flag takes a list of APIs to be allowed as a parameter. For example, eth ,web3 will enable the web3 and eth interface over RPC.
- - - rpcport: This sets up the TCP RPC port; for example: 9999.
- - - rpccorsdomain: This flag specifies the URL that is allowed to connect to the private geth node and perform RPC operations.
- - - port: This specifies the TCP port that will be used to listen to the incoming connections from other peers.
- - - identity: This flag is a string that specifies the name of a private node.

STATIC NODES

- If there is a need to connect to a specific set of peers, then these nodes can be added to a file where the chaindata and keystore files are saved, for example, in the -/. ethereum/privatenet directory.
- The filename should be static-nodes.json. This can be valuable in a private network.

RUNNING MIST ON PRIVATE NET

- It provides all features, such as wallet, account management, and contract deployment on Private Net via Mist.

DEPLOYING CONTRACTS USING MIST

Mist provides an interface where contracts can be written in solidity and then deployed on the network.

Languages

Contracts can be programmed in a variety of languages.

- **Mutan:** This is a Go-style language, which was deprecated in early 2015 and is no longer used.
- **LLL:** This is a Lisp-like language, hence the name LLL. This is also not used anymore.
- **Serpent:** This is a simple and clean Python-like language. It is actively used for contract development.
- **Solidity:** This language has become a standard for contract writing for Ethereum.

Compilers

- Compilers are used to convert high-level contract source code into the format that the Ethereum execution environment understands.
- The solidity compiler is the most common one in use.

SOLC

- The solidity compiler converts from a high-level solidity language into Ethereum Virtual Machine (EVM) bytecode so that it can be executed on the blockchain by EVM.

- The solidity compiler on a Linux Ubuntu operating system can be installed using the following command:
- $ sudo apt-get install sole

INTEGRATED DEVELOPMENT ENVIRONMENTS (IDES)

Browser solidity

- Browser solidity is the web-based environment for the development and testing of contracts using solidity.
- It does not run on a live blockchain; in fact, it is a simulated environment in which contracts can be deployed, tested, and debugged.

- Various features, such as transaction interaction, options to connect to JavaScript VM, configuration of execution environment, debugger, formal verification, and static analysis, are available.
- They can be configured to connect to execution environments such as JavaScript VM, injected Web3--where Mist or a similar environment has provided the execution environment--or Web3 provider, which allows connection to the locally running Ethereum client (for example, geth) via I PC or RPC over HTTP (web3 provider endpoint).

Remix

- Remix is a browser-based IDE that is under heavy development currently and only the debugger part of it is available as of now.
- This debugger is very powerful and can be used to perform detailed level tracing and analysis of the EVM byte code.
- Remix is also available as part of browser solidity.
- It can be connected to the local Private Net by providing the web3 provider endpoint.

ETHEREUMJS

- It is not possible to test on the Test Net and Main Net is not a place to test the contracts.
- Private Net can be time consuming to set up at times.
- EthereumJS testrpc comes in handy when quick testing is required and no proper test net is available.
- It uses EthereumJS to simulate the Ethereum geth client behavior and allows for faster development testing.
- Testrpc is available via npm as a node package.
- Before installing testrpc, Node.js should already have been installed and the npm package manager should also be available.
- Testrpc can be installed using this command:

```
npm install -g ethereumjs-testrpc
```

- In order to start testrpc, run below command and open another terminal to work on contracts.

Stestrpc

Contract development and deployment

- There are four steps in order to develop and deploy the contracts. writing, testing, verification, and deployment.
- After deployment, the next step is to create the user interface and present it to the end users via a web server.
- The writing step is concerned with writing the contract source code in solidity.
- This can be done in any text editor.

- There are various plugins and add-ons available for Vim in Linux, Atom, and other editors that provide syntax highlighting and formatting for solidity source code.
- Testing is usually performed by automated means.
- Truffle uses the Mocha framework to test contracts.
- However, manual testing can be performed as well.
- Once the contract is verified, working, and tested on a simulated environment (for example, EthereumJS testrpc) or on Private Net, it can be deployed to Ropsten Test Net and finally to live blockchain (Homestead).

Hyperledger

- Hyperledger is not a blockchain, but it is a project that was initiated by Linux foundation in December 2015 to advance blockchain technology.
- To build an open source distributed ledger framework that can be used to develop and implement cross-industry blockchain applications and systems.
- The key focus is to build and run platforms that support global business transactions.
- The project also focuses on improving the reliability and performance of blockchain systems.

- Projects under Hyperledger undergo various stages of development, starting from **proposal** to **incubation** and graduating to an **active** state.
- Projects can also be **deprecated** or in **End of Life** state where they are no longer actively developed.
- In order for a project to be able to move into incubation stage, it must have a fully working code base along with an active community of developers.

Projects

- Currently, there are six projects under the Hyperledger umbrella: Fabric, Iroha, Sawtooth lake, blockchain explorer, Fabric chaintool, and Fabric SDK Py.
- Corda is the recent addition expected to be added to the Hyperledger project.
- The Hyperledger project currently has 100 members and is very active with more than 120 contributors, with regular meet-ups and talks being organized around the globe.

Fabric

- Fabric is a blockchain project that was proposed by IBM and **DAH (Digital Asset Holdings).**
- This is intended to provide a foundation for the development of blockchain solutions and is based on pluggable architecture where various components, such as consensus algorithm, can be plugged into the system as required.

Sawtooth lake

- Sawtooth lake is a blockchain project proposed by Intel in April 2016 with some key innovations focusing on **decoupling** of ledgers from transactions, flexible usage across multiple business areas using *transaction families,* and **pluggable consensus.**

- The *transactions* are decoupled from the *consensus layer* by making use of a new concept called *Transaction families.*
- Instead of transactions being individually coupled with the ledger, transaction families are used, which allows for more flexibility, rich semantics and unrestricted design of business logic.
- Transactions follow the patterns and structures defined in the transaction families.
- Intel has also introduced a novel consensus algorithm, proof of elapsed time (PoET), which makes use of **Intel Software Guard Extensions (Intel's SGX)** architecture's **trusted execution environment (TEE)** in order to provide a safe and random leader election process.
- It also supports permissioned and permissionless setups.

Iroha

- Iroha is aiming to build a library of reusable components that users can choose to run on their Hyperledger-based distributed ledgers.
- Iroha's main goal is to complement other Hyperledger projects by providing reusable components written in C++ with an emphasis on mobile development.
- This project has also proposed a novel consensus algorithm called Sumeragi, which is a chain based Byzantine fault tolerant consensus algorithm.
- Various libraries have been proposed and are being worked on by Iroha, including but not limited to a digital signature library (ed25519), an SHA-3 hashing library, a transaction serialization library, a P2P library, an API server library, an iOS library, an Android library, and a JavaScript.

Blockchain explorer

- This project aims to build a blockchain explorer for Hyperledger that can be used to view and query the transactions, blocks, and associated data from the blockchain.
- It also provides network information and the ability to interact with chain code.
- Currently there are two other projects that are in incubation: Fabric chaintool, and Fabric SDK Py. These projects are aimed at supporting Hyperledger Fabric.

Fabric chaintool

- Hyperledger chaincode compiler is being developed to support Fabric chaincode development.
- The aim is to build a tool that reads in a high-level Google protocol buffer structure and produces a chaincode.
- Additionally, it packages the chaincode so that it can be deployed directly.
- It is envisaged that this tool will help developers in various stages of development, such as compiling, testing, packaging, and deployment.

Fabric SDK Py

- The aim of this project is to build a python based SDK library that can be used to interact with the blockchain (Fabric).

Corda

- Corda is the latest project that has been contributed by R3 to the Hyperledger project.
- Corda is heavily oriented towards the financial services industry and has been developed in collaboration with major banks and organizations in the financial industry.
- Corda is not a blockchain but has key features similar to those of a blockchain, such as consensus, validity, uniqueness, immutability, and authentication.

Hyperledger as a protocol

- Hyperledger is aiming to build a new blockchain platform that is driven by industry use cases.
- As there have been number of contributions made to the Hyperledger project by the community, Hyperledger blockchain platform is evolving into a protocol for business transactions.
- Hyperledger is also evolving into a specification that can be used as a reference to build blockchain platforms as compared to earlier blockchain solutions that address only a specific type of industry or requirement.

Reference architecture

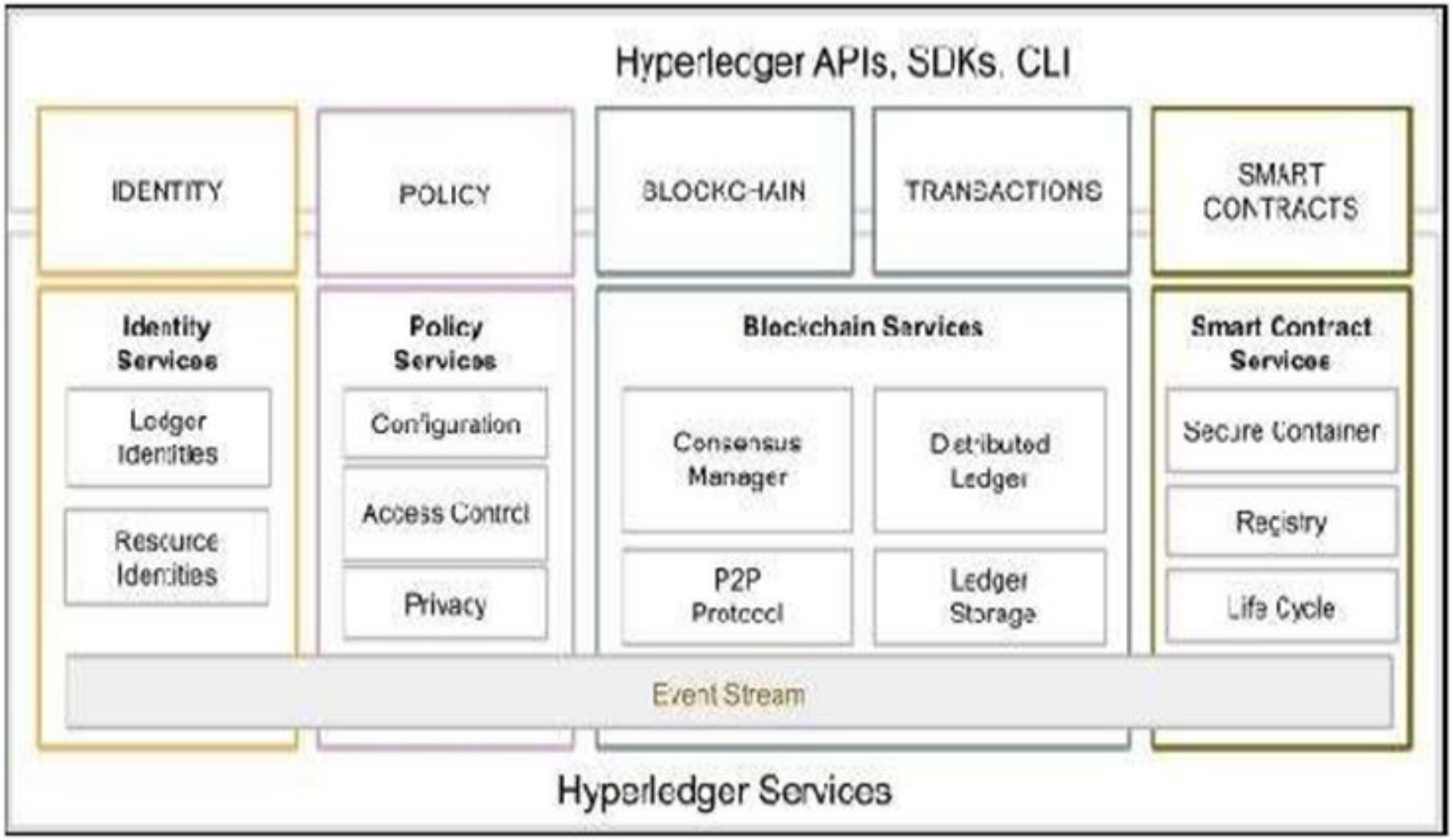
- Hyperledger with reference architecture can serve as a guideline to build permissioned distributed ledgers.
- The reference architecture consists of two main components:
  - ❖ Hyperledger services and Hyperledger APIs, SDKs, and CLI.
- Hyperledger services provide various services such as identity services, policy services, blockchain services, and smart contract services.
- Hyperledger APIs, SDKs, and CLIs provide an interface into blockchain services via appropriate application programming interfaces, software development kits, or command line interfaces.

- Moreover, an event stream, which is a gRPC channel, runs across all services.
- It can receive and send events.
- Events are either pre-defined or custom.
- Validating peers or chaincode can emit events to which external application can respond or listen to.

Requirements

- There are certain requirements of a blockchain service.
- The reference architecture is driven by the needs and requirements raised by the participants of the Hyperledger project and after studying the industry use cases.

Hyperledger architecture



MODULAR APPROACH

- The main requirement of Hyperledger is a modular structure.
- It is expected that, as a cross-industry fabric (blockchain), it will be used in many business scenarios.
- Functions related to storage, policy, chaincode, access control, consensus and many other blockchain services should be pluggable.
- The modules should be plug and play and users should be able to easily remove and add a different module that meets the requirements of the business.

- For example, if a business blockchain needs to be run only between already trusted parties and performs very basic business operations, then there is no need to have advanced cryptographic support for confidentiality and privacy and therefore users should be able to remove that functionality (module) or replace that with a more appropriate module that suits their needs.
- Similarly, if users need to run a cross-industry blockchain, then confidentiality and privacy can be of paramount importance.
- In this case, users should be able to plug an advanced cryptographic and access control mechanism (module) into the blockchain (fabric).

PRIVACY AND CONFIDENTIALITY

- Privacy and confidentiality of transactions and contracts is of utmost importance in a business blockchain.
- Hyperledger's vision is to provide a wide range of cryptographic protocols and algorithms and it is expected that users will be able to choose appropriate modules according to their business requirements.
- The fabric should be able to handle complex cryptographic algorithms without compromising performance.

IDENTITY

- In order to provide privacy and confidentiality services, a flexible PKI model that can be used to handle the access control functionality is also required.
- The strength and type of cryptographic mechanisms is also expected to vary according to the needs and requirements of the users.
- In certain scenarios it might be required for a user to hide their identity, and as such the Hyperledger is expected to provide this functionality.

AUDITABILITY

- Auditability is another requirement of a Hyperledger Fabric.
- An immutable audit trail of all identities, related operations and any changes is kept.

INTEROPERABILITY

- Currently there are many blockchain solutions available, but they cannot communicate with each other and this can be a limiting factor in the growth of a blockchain based global business ecosystem.
- It is envisaged that many blockchain networks will operate in the business world for specific needs, but it is important that they are able to communicate with each other.
- There should be a common set of standards that all blockchains can follow in order to allow communication between different ledgers.
- A protocol will be developed that allow the exchange of information between many Fabrics.

PORTABILITY

- The portability requirement is concerned with the ability to run across multiple platforms and environments without the need to change anything at code level.
- Hyperledger is envisaged to be portable, not only at infrastructure level but also at code, libraries, and API levels so that it can support uniform development across various implementations of Hyperledger.

Fabric

- Fabric can be defined as a collection of components providing a foundation layer that can be used to deliver a blockchain network.
- There are various types and capabilities of a fabric network, but all fabrics share common attributes such as immutability and are consensus driven.
- Some fabrics can provide modular approach towards building blockchain networks.
- In this case the blockchain network can have multiple pluggable modules to perform various function on the network.
- For example, consensus algorithms can be a pluggable module in a blockchain network where, depending on the requirements of the network, an appropriate consensus algorithm can be chosen and *plugged* into the network.

- The modules can be based on some particular specification of the fabric and can include APIs, access control, and various other components.
- Fabrics can also be designed either to be private or public and can allow the creation of multiple business networks.
- As an example, bitcoin is an application that runs on top of its fabric (blockchain network). blockchain can either be permissioned or permissionless and the same is true for fabric in Hyperledger terminology.
- Fabric is also the name given to the code contribution made by IBM to the Hyperledger foundation and is formally called Hyperledger Fabric. IBM also offers blockchain as a service (IBM Blockchain) via its Bluemix cloud service.

Hyperledger Fabric

- Fabric is the contribution originally made by IBM to the Hyperledger project.
- The aim of this contribution is to enable a modular, open and flexible approach towards building blockchain networks.
- Various functions in the fabric are pluggable, and it also allows use of any language to develop smart contracts.
- This is possible because it is based on container technology which can host any language.
- Chaincode (smart contract) is sandboxed into a secure container which includes a secure operating system, chaincode language, runtime environment and SDKs for Go, Java, and Node.js.
- Other languages can be supported too if required.

- Smart contracts are called chaincode in the Fabric.
- This is a very powerful feature compared to domain specific languages in Ethereum, or the very limited scripted language in bitcoin.
- It is a permissioned network which address issues such as scalability, privacy, and confidentiality.
- The key idea behind this is modular technology, which would allow for flexibility in design and implementation.
- This result in achieving scalability, privacy and other desired attributes.
- Transactions in fabric are private, confidential and anonymous for general users, but they can still be traced and linked to the users by authorized auditors.
- As a permissioned network, all participants are required to be registered with the membership services in order to access the blockchain network.
- This ledger also provided auditability functionality in order to meet the regulatory and compliance needs.

Fabric architecture

- The Fabric is logically organized into three main categories based on the type of service provided.
- These include membership services, blockchain services, and chaincode services.

MEMBERSHIP SERVICES

- These services are used to provide access control capability for the users of the fabric network. Function it provides are,

1. User identity validation.
2. User registration.
3. Assign appropriate permissions to the users depending on their roles.

- Membership services makes use of **Public Key Infrastructure (PKI)** in order to support identity management and authorization operations.

Membership services are made up of various components:

- **Registration authority (RA):** A service that authenticates the users and assesses the identity of the fabric participants for issuance of certificates.
- **Enrolment certificate authority: Enrolment certificates (Ecerts)** are long term certificates issued by ECA to registered participants in order to provide identification to the entities participating on the network.
- **Transaction certificate authority:In** order to send transactions on the networks, participants are required to hold a transaction certificate. TCA is responsible for issuing transaction certificates to holders of Enrolment certificates and is derived from Ecerts.
- **TLS certificate authority:**In order to secure the network level communication between nodes on the Fabric, TLS certificates are used. TLS certificate authority issues TLS certificates in order to ensure security of the messages being passed between various systems on the blockchain network.

BLOCKCHAIN SERVICES

- Blockchain services are at the core of the Hyperledger Fabric. Components which are,

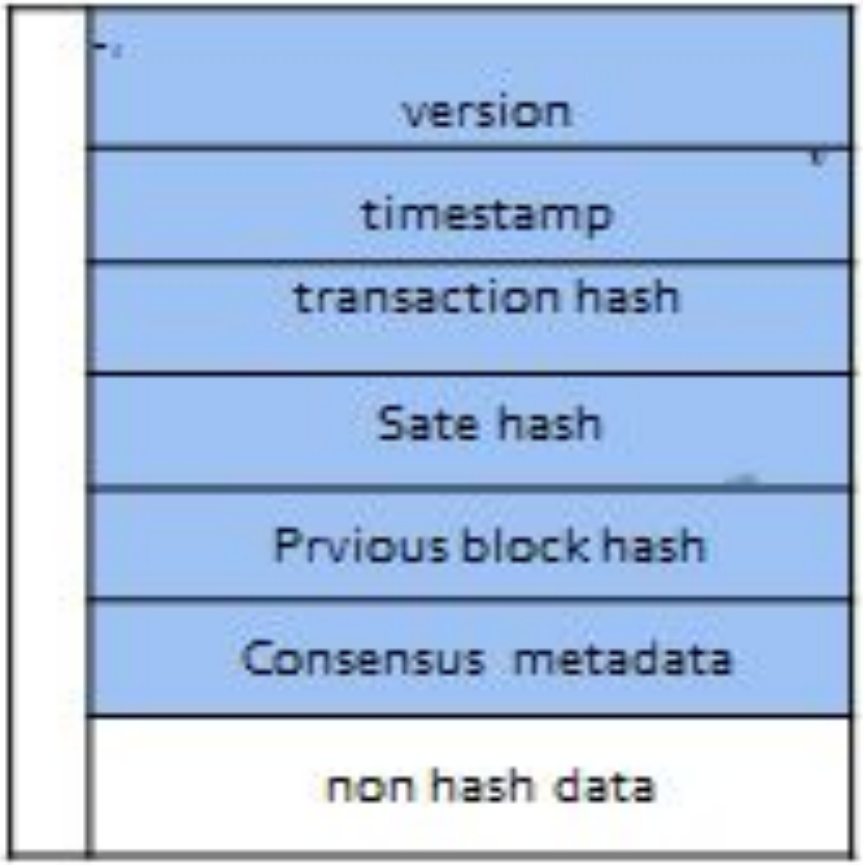
Consensus manager

- Consensus manager is responsible for providing the interface to the consensus algorithm.
- This serves as an adapter that receives the transaction from other Hyperledger entities and executes them under criteria according to the type of algorithm chosen.
- Consensus is pluggable and currently there are three types of consensus algorithm available in Fabric, namely the batch PBFT protocol, SIEVE algorithm, and NOOPS.

Distributed ledger

- Blockchain and world state are two main elements of the distributed ledger.
- Blockchain is simply a linked list of blocks and world ledger is a key-value database.
- This database is used by smart contracts to store relevant states during execution by the transactions.
- The blockchain consists of blocks that contain transactions.
- These transactions contain chaincode, which runs transactions that can result in updating the world state.
- Each node saves the world state on disk in RocksDB.

Block structure fields



- **Version:** Used for keeping track of changes in the protocol.
- **Timestamp:** Timestamp in UTC epoch time, updated by block proposer.
- **Transaction hash:** This field contains the Merkle root hash of the transactions in the block.
- **State hash:** This is the Merkle root hash of the world state.

- **Previous hash:** This is the previous block's hash, which is calculated after serializing the block message and then creating the message digest by applying the SHA3 SHAKE256 algorithm.
- **Consensus metadata:** This is an optional field that can be used by the consensus protocol to provide some relevant information about the consensus.
- **Non-Hash data:** This is some metadata that is stored with the block but is not hashed. This feature makes it possible to have different data on different peers. It also provides the ability to discard data without any impact on the blockchain.

Peer to Peer protocol

- P2P protocol in the Hyperledger Fabric is built using **google RPC (gRPC).**
- It uses protocol buffers to define the structure of the messages.
- Messages are passed between nodes in order to perform various functions.
- There are four main types of messages in Hyperledger Fabric: Discovery, transaction, synchronization and consensus.
- Discovery messages are exchanged between nodes when starting up in order to discover other peers on the network.
- Transaction messages can be divided into two types: Deployment transactions and Invocation transactions.
- The former is used to deploy new chaincode to the ledger, and the latter is used to call functions from the smart contract.
- Transactions can be public, confidential, and confidential chaincode transactions.
- Public transactions are open and available to all participants.
- Confidential transactions are allowed to be queried only by transaction owners and participants.

- Confidential chaincode transactions have encrypted chaincode and can only be decrypted by validating nodes.
- Validating nodes run consensus, validate the transactions and maintain the blockchain.
- Non-validating nodes on the other hand, provide transaction verification, stream server, and REST services.
- They also act as a proxy between the transactors and the validating nodes.
- Synchronization messages are used by peers to keep the blockchain updated and in synch with other nodes.
- Consensus messages are used in consensus management and broadcasting payloads to validating peers.
- These are generated internally by the consensus framework.

Ledger storage

- In order to save the state of the ledger, RocksDB is used, and it is stored at each peer. RocksDB is a high performance database

CHAINCODE SERVICES

- These services allow the creation of secure containers that are used to execute the chaincode. Components include,

  - Secure container: Chaincode is deployed in Docker containers that provide a locked down sandboxed environment for smart contract execution. Currently Golang is supported as the main smart contract language, but any other mainstream language can be added and enabled if required.
  - Secure registry: This provides a record of all images containing smart contracts.

EVENTS

- Events on the blockchain can be triggered by validator nodes and smart contracts.
- External applications can listen to these events and react to them if required via event adapters.

APIS AND CLIS

- An application programming interface provides an interface into the fabric by exposing various REST APIs.
- Additionally, command line interfaces that provide a subset of REST APIs and allow for quick testing and limited interaction with the blockchain are also available.

Components of the Fabric

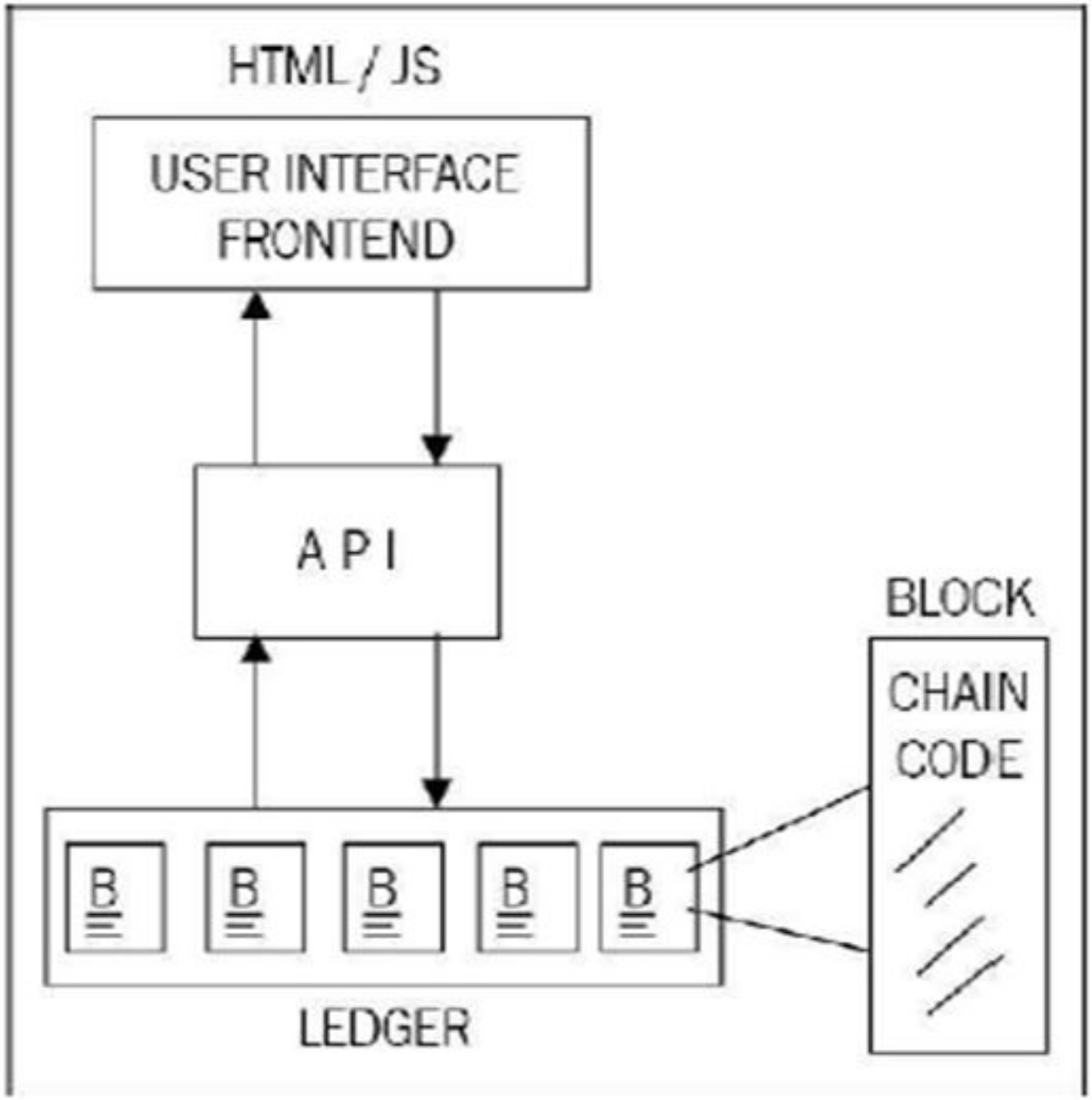
- There are various components that can be part of the blockchain.
- These components include but are not limited to the ledger, chaincode, consensus mechanism, access control, events, system monitoring and management, wallets and system integration components.

PEERS OR NODES

- There are two main types of peers that can be run on a fabric network: Validating and non-validating.
- A validating node runs consensus, creates and validates a transaction, and contributes towards updating the ledger and maintaining the chaincode.
- A non-validating peer does not execute transactions and only constructs transactions that are then forwarded to validating nodes.
- Both nodes manage and maintain user certificates that have been issued by membership services.

APPLICATIONS ON BLOCKCHAIN

- A typical application on Fabric is simply composed of a user interface, usually written in JavaScript/HTML, that interacts with the backend chaincode (smart contract) stored on the ledger via an API layer.
- Hyperledger provides various APIs and command line interfaces to enable interaction with the ledger.
- These APIs include interfaces for identity, transactions, chaincode, ledger, network, storage, and events.


HTML / JS
USER INTERFACE
FRONTEND
API
BLOCK
CHAIN
CODE
B
B
B
B
B
LEDGER

Chaincode implementation

- Chaincode is usually written in Golang or Java.
- Chaincode can be public, confidential or access controlled.
- These codes serve as a smart contract that users can interact with via APIs.
- Users can call functions in the chaincode that result in a state change, and consequently updates the ledger.
- There are also functions that are only used to query the ledger and do not result in any state change.
- Chaincode implementation is performed by first creating the chaincode shim interface in the code.
- It can either be in Java or Golang code.

The following four functions are required in order to implement the chaincode:

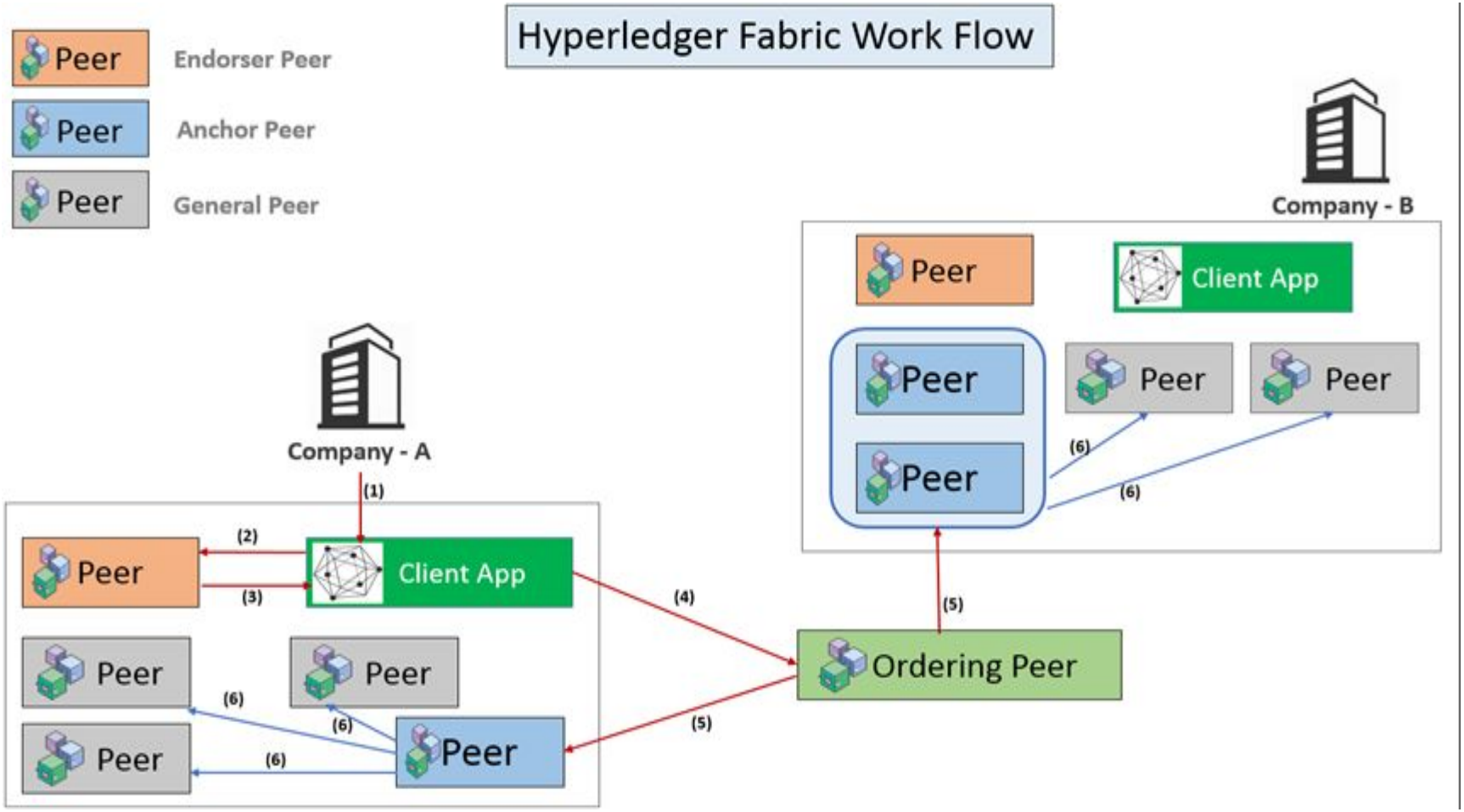
- Init(): This function is invoked when chaincode is deployed onto the ledger. This initializes the chaincode and results in making a state change, which accordingly updates the ledger.
- Invoke(): This function is used when contracts are executed. It takes a function name as parameters along with an array of arguments. This function results in a state change and writes to the ledger.
- Query (): This function is used to query the current state of a deployed chaincode. This function does not make any changes to the ledger.
- Main(): This function is executed when a peer deploys its own copy of the chaincode. The chaincode is registered with the peer using this function.

- Any blockchain application for Hyperledger Fabric follows MVC-B architecture.
- This is based on the popular MVC design pattern. Components in this model are Model, View, Control, and Blockchain:

  - **View logic:** This is concerned with the user interface. It can be a desktop, web application or mobile frontend.
  - **Control logic:** This is the orchestrator between user interface, data model, and APIs.
  - **Data model:** This model is used to manage the off-chain data.
  - **Blockchain logic:** This is used to manage the blockchain via the controller and the data model via transactions.

- Moreover, the IBM Bluemix service offers sample applications for blockchain under its blockchain as a service offering.
- This service allows users to create their own blockchain networks in an easy to use environment.

Hyperledger Fabric Application model



Sawtooth lake

- Sawtooth lake can run in both permissioned and non-permissioned modes.
- It is a distributed ledger that proposes two novel concepts:
- The first is the introduction of a new consensus algorithm called **Proof of Elapsed Time (PoET);** and the second is the idea of transaction families.

PoET

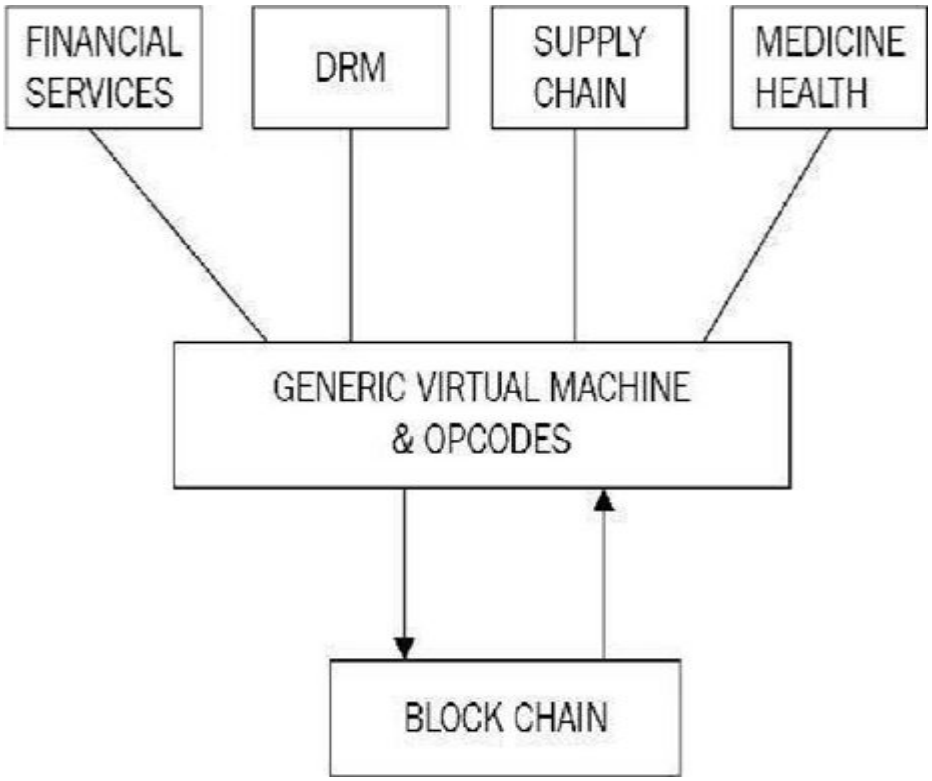
- PoET is a novel consensus algorithm that allows a node to be selected randomly based on the time that the node has waited before proposing a block.
- This is in contrast to other leader election and lottery based proof of work algorithms, where an enormous amount of electricity and computer resources are used in order be elected as a block proposer, for example in the case of bitcoin.

- PoET is a type of Proof of Work algorithm but, instead of spending computer resources, it uses a trusted computing model to provide a mechanism to fulfill Proof of Work requirements.
- PoET makes use of Intel's SGX architecture to provide a trusted execution environment to ensure randomness and cryptographic security of the process.
- It should be noted that the current implementation of Sawtooth lake does not require real hardware SGX based TEE, as it is simulated for experimental purposes only and as such should not be used in production environments.

Transaction families

- A traditional smart contract paradigm provides a solution that is based on a general purpose instruction set for all domains.
- For example, in the case of Ethereum, a set of opcodes has been developed for the **Ethereum virtual machine (EVM)** that can be used to build smart contracts to address any type of requirements for any industry.
- Whilst this model has its merits, it is becoming clear that this approach is not very secure as it provides a single interface into the ledger with a powerful and expressive language, which potentially offers a larger attack surface for malicious code.
- This complexity and generic virtual machine paradigm has resulted in several vulnerabilities that were found and exploited recently by hackers.

- A recent example is the DAO hack and further **Denial of Services (DoS)** attacks that exploited limitations in some EVM opcodes.
- A model shown in the following figure describes the traditional smart contract model, where a generic virtual machine has been used to provide the interface into the blockchain for all domains:

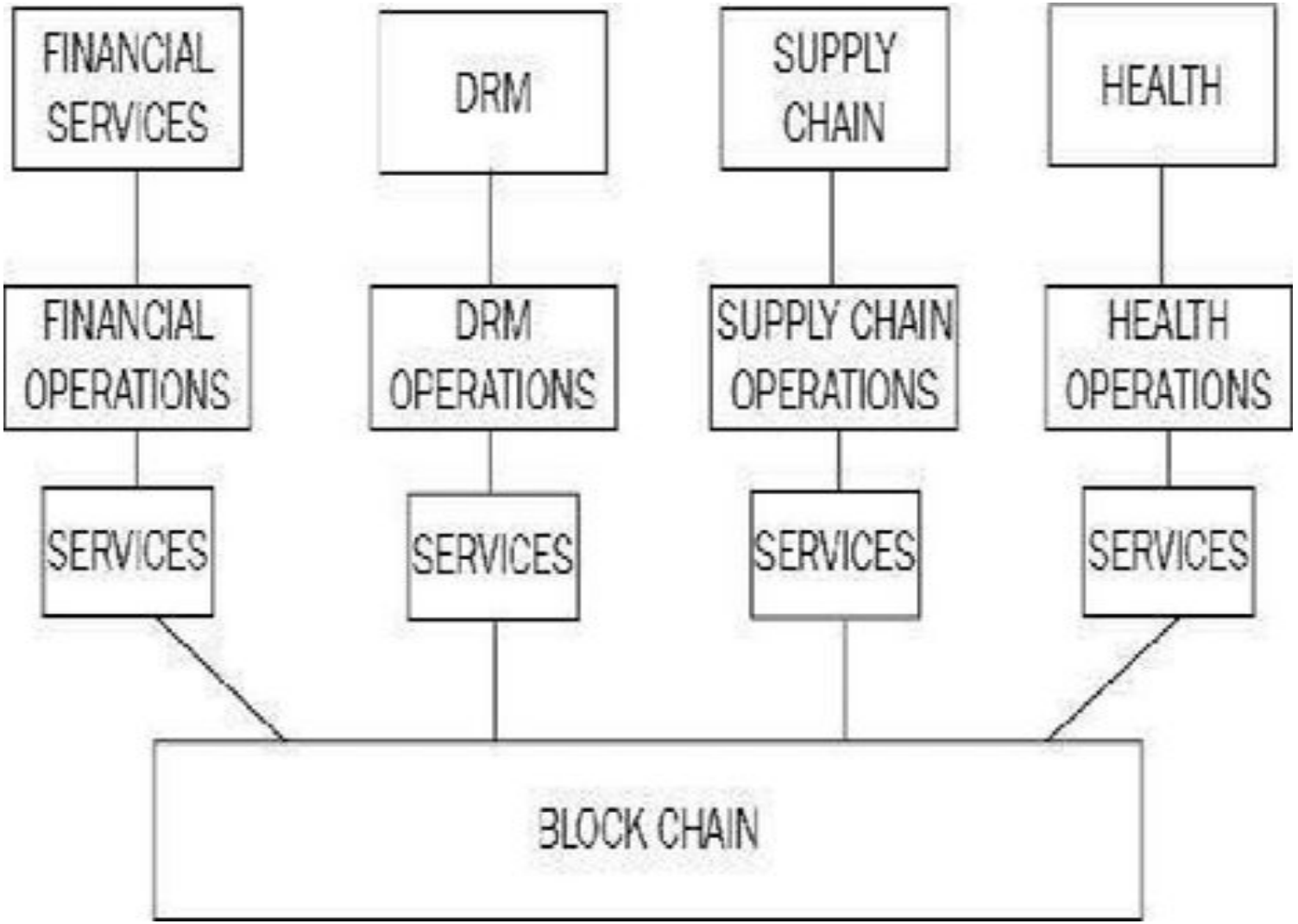


Traditional smart contract paradigm

- In order to address this issue, Sawtooth lake has proposed
- A transaction family is created by decomposing the logic layer into a *set of rules* and a *composition layer* for a specific domain.
- The key idea is that business logic is composed within *transaction families,* which provides a more secure and powerful way to build smart contracts.
- Transaction families contain the domain-specific rules and another layer that allows for creating transactions for that domain.
- Another way of looking at it is that transaction families are a combination of a data model and a transaction language that implements a logic layer for a specific domain.
- The data model represents the current state of the blockchain (ledger) whereas the transaction language modifies the state of the ledger.
- Users will build their own transaction families according to their business requirements.

- The following diagram represents this model, where each specific domain, like financial services, **digital rights management (DRM),** supply chain, and the health industry, has its own logic layer comprised of operations and services specific to that domain.
- This makes the logic layer both restrictive and powerful at the same time.
- Transaction families ensure that operations related to only the required domain are present in the control logic, thus removing the possibility of executing needless, arbitrary and potentially harmful operations.

Sawtooth (transaction families) smart contract paradigm



Intel has provided three transaction families with Sawtooth:

Endpoint registry, Integerkey, and Marketplace.

1. **Endpoint registry** is used for registering ledger services.
2. **Integerkey** is used for testing deployed ledgers.
3. **Marketplace** is used for selling, buying and trading operations and services.

Sawtooth_bond has been developed as a proof of concept to demonstrate a bond trading platform.

Consensus in Sawtooth

- Sawtooth has two types of consensus mechanisms based on the choice of network.
- PoET is a trusted executed environment based lottery function that elects a leader randomly based on the time a node has waited for block proposal.
- There is another consensus type called quorum voting, which is an adaptation of consensus protocols built by Ripple and Stellar.
- This consensus algorithm allows instant transaction finality, which is usually desirable in permissioned networks.

Corda

- Corda is not a blockchain.
- Traditional blockchain solutions have the concept of transactions that are bundled together in a block and each block is linked back cryptographically to its parent block, which provides an immutable record of transactions.
- This is not the case with Corda: Corda has been designed entirely from scratch with a new model for providing all blockchain benefits, but without a traditional blockchain.
- It has been developed purely for the financial industry to solve issues arising from the fact that each organization manages their own ledgers and thus have their own view of *truth,* which leads to contradictions and operational risk.
- Moreover, data is also duplicated at each organization which results in an increased cost of managing individual infrastructures and complexity.

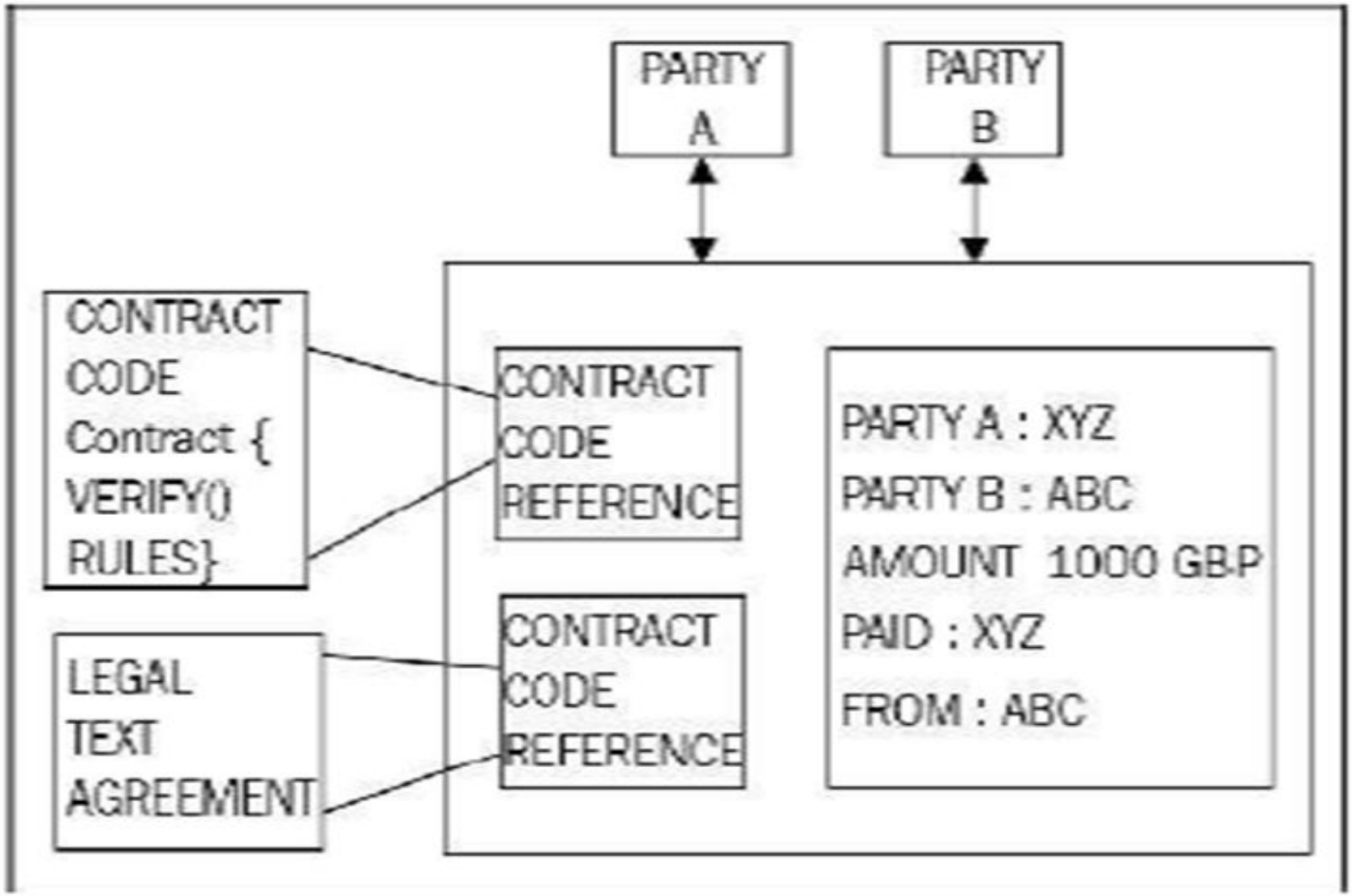
- It is written in a language called Kotlin, which is a statically typed language targeting the **Java Virtual Machine (JVM).**

Architecture

- The main components of the Corda platform include state objects, contract code, legal prose, transactions, consensus, and flows.

STATE OBJECTS

- State objects represent the smallest unit of data that represent a financial agreement.
- They are created or deleted as a result of a transaction execution.
- They refer to **contract code** and **legal prose.**
- Legal prose is optional and provides legal binding to the contract.
- However, contract code is mandatory in order to manage the state of the object.



An example state object

- State objects contain a data structure that represent the current state of the object.
- For example, in the following diagram, a state object represents the current state of the object.
- In this case, it is a simple mock agreement between **Party A** and **Party B** where **Party ABC** has paid **Party** XYZ **1,000 GBP.**
- This represents the current state of the object; however the referred contract code can change the state via transactions.
- State objects is considered as a state machine, which are consumed by transactions in order to create updated state objects.

TRANSACTIONS

- Transactions are used to perform transitions between different states.
- For example, the state object shown in the preceding diagram is created as a result of a transaction.
- Corda uses a bitcoin-style UTXO based model for its transaction processing.
- The concept of state transition by transactions is same as in bitcoin.
- Similar to bitcoin, transactions can have none, single or multiple inputs, and single or multiple outputs.
- All transactions are digitally signed.
- Moreover, Corda has no concept of mining because it does not use blocks to arrange transactions in a blockchain.
- Instead, notary services are used in order to provide temporal ordering of transactions.
- In Corda, new transaction types can be developed using JVM bytecode, which makes it very flexible and powerful.

CONSENSUS

- The consensus model in Corda is quite simple and is based on notary services.
- The general idea is that the transactions are evaluated for their uniqueness by the notary service and, if they are unique, they are signed as valid.
- There can be single or multiple clustered notary services running on a Corda network.
- Various consensus algorithms like PBFT or Raft can be used by notaries to reach consensus.
- There are two main concepts regarding consensus in Corda: Consensus over state validity, and consensus over state uniqueness.
- The first concept is concerned with the validation of the transaction, ensuring that all required signatures are available and states are appropriate.
- The second concept is a means to detect double-spend attack and ensures that a transaction has not been already been spent and is unique.

FLOWS

- Flows in Corda are a novel idea that allow the development of decentralized workflows.
- All communication on the Corda network is handled by these flows.
- These are transaction-building protocols that can be used to define any financial flow of any complexity using code.
- Flows run as an asynchronous state machine and they interact with other nodes and users.
- During the execution, they can be suspended or resumed as required.

Components

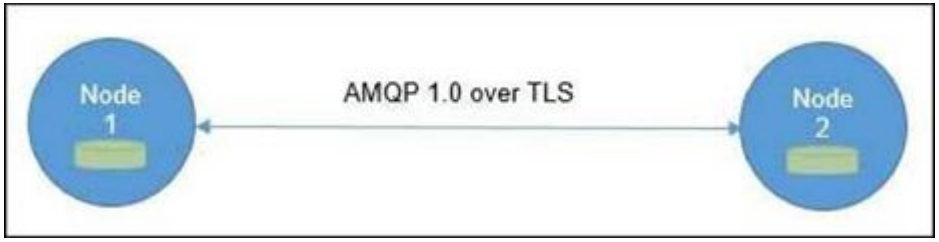
The Corda network has multiple components.

NODES

Nodes in a Corda network operated under a trust-less model and run by different organizations. Nodes run as part of an authenticated peer-to-peer network. Nodes communicate directly with each other using the **Advanced Message Queuing Protocol (AMQP),** which is an approved international standard (ISO/IEC 19464) and ensures that messages across different nodes are transferred safely and securely. AMQP works over **Transport Layer Security (TLS)** in Corda, thus ensuring privacy and integrity of data communicated between nodes.

Nodes also make use of a local relational database for storage. Messages on the network are encoded in a compact binary format. They are delivered and managed by using the **Apache Artemis message broker (Active MQ).** A node can serve as a network map service, notary, Oracle, or a regular node.

The following diagram shows a high-level view of two nodes communicating with each other:



Two nodes communicating in a Corda network

In the preceding diagram, **Node 1** is communicating with **Node** 2 over a TLS communication channel using the AMQP protocol, and the nodes have a local relational database for storage.

PERMISSIONING SERVICE

- A Permissioning service is used to provision TLS certificates for security.
- In order to participate on the network, participants are required to have a signed identity issued by a root certificate authority.
- Identities are required to be unique on the network and the Permissioning service is used to sign these identities.
- The naming convention used to recognise participants is based on the X.500 standard.
- This ensures the uniqueness of the name.

Network map service

- Oracle services either sign a transaction containing a fact, if it is true, or can themselves provide factual data. They allow real world feed into the distributed ledgers.

Transactions

- Transactions in a Corda network are never transmitted globally, but in a semi-private network.
- They are shared only between a subset of participants who are related to the transaction.
- This is in contrast to traditional blockchain solutions like Ethereum and bitcoin, where all transactions are broadcasted to the entire network globally.
- Transactions are digitally signed and either consume state(s) or create new state(s).

Transactions on a Corda network are composed of the following elements: •

- Input references: This is a reference to the states the transaction is going to consume and use as an input.
- Output states: These are new states created by the transaction.
- Attachments: This is a list of hashes of attached zip files. Zip files can contain code and other relevant documentation related to the transaction. Files themselves are not made part of the transaction, instead, they are transferred and stored separately.
- Commands: A command represents the information about the intended operation of the transaction as a parameter to the contract. Each command has a list of public keys which represents all parties that are required to sign a transaction.
- Signatures: This represents the signature required by the transaction. The total number of signatures required is directly proportional to the number of public keys for commands.
- Type: There are two types of transactions namely, Normal or Notary changing. Notary changing transactions are used for reassigning a notary for a state.
- **Timestamp:** This field represents a bracket of time during which the transaction has taken place. These are verified and enforced by notary services. Also, it is expected that if strict timings are required, which is desirable in many financial services scenarios, notaries should be synched with an atomic clock.
- **Summaries:** This is a text description that describes the operations of the transaction.

Vaults

- Vaults run on a node and are akin to the concept of wallets in bitcoin.
- As the transactions are not globally broadcast, each node will have only that part of data in their vaults that is considered relevant to them.
- Vaults store their data in a standard relational database and as such can be queried by using standard SQL.
- Vaults can contain both on ledger and off ledger data, meaning that it can also have some part of data that is not on ledger.

CorDapp

- The core model of Corda consists of state objects, transactions and transaction protocols, which when combined with contract code, APIs, wallet plugins, and user interface components results in constructing a **Corda distributed application (CorDapp).**
- Smart contracts in Corda are written using Kotlin or Java.
- The code is targeted for JVM.
- JVM has been modified slightly in order to achieve deterministic results of execution of JVM bytecode.

There are three main components in a Corda smart contract as follows:

1. Executable code that defines the validation logic to validate changes to the state objects.
2. State objects represent the current state of a contract and either can be consumed by a transaction or produced (created) by a transaction.
3. Commands are used to describe the operational and verification data that defines how a transaction can be verified.

Development environment

The development environment for Corda can be set up easily using the following steps.

Required software includes the following:

1. JDK 8 which is available at http://www.oracle.com/technetwork/java/javase/downloads/index.ht ml.
2. IntelliJ IDEA community edition which is free and available at https://www.jetbrains.com/idea/download .
3. H2 database platform independent zip, and is available at http://www.h2database.com/html/download.html.
4. Git, available at https://git-scm.com/downloads.
5. Kotlin language, which is available for IntelliJ, and more information can be found at https://kotlinlang.org/.
6. Gradle is another component that is used to build Corda.

Once all these tools are installed, smart contract development can be started.

Blockchain-Outside of Currencies

- Digital currencies were the first ever application of blockchain technology, arguably without realizing its true potential.
- With the invention of bitcoin the concept of blockchain was introduced for the very first time, but it wasn't until 2013, with the advent of *Blockchain 2.0* that the real benefits of blockchain were realized with its possible application in many different industries.
- Since then a number of use cases of blockchain technology in different industries, have been proposed including but not limited to finance, the Internet of Things, digital rights management, government, and law.
- Four main industries namely the **Internet of Things (IoT),** government, health, and finance are discussed.

Internet of Things

- The Internet of Things has recently gained much traction due to its potential for transforming business applications and everyday life.
- IoT can be defined as a network of computationally intelligent physical objects that are capable of connecting to the Internet, sensing real-world events or environments, reacting to those events, collecting relevant data, and communicating it over the Internet.
- This simple definition has huge implications and has led to exciting concepts, such as wearable's, smart homes, smart grids, smart connected cars, and smart cities, that are all based on this basic concept of an IoT device.
- There are four functions that come to light as being performed by an IoT device.
- These include sensing, reacting, collecting, and communicating.

- All these functions are performed by using various components on the IoT device.
- Sensing is performed by sensors.
- Reacting or controlling is performed by actuators, collection is a function of various sensors, and communication is performed by chips that provide network connectivity.
- One thing to note is that all these components are accessible and controllable via the Internet in the IoT.
- An IoT device on its own is useful to some extent but if it is part of a larger IoT ecosystem it is more valuable.
- An IoT can consist of many physical objects connecting with each other and to a centralized cloud server.

IoT network



- Elements of IoT are spread across multiple layers and various reference architectures exist that can be used to develop IoT systems.
- Generally, a five layer model can be used to describe IoT, which contains a physical object layer, device layer, network layer, services layer, and application layer.
- Each layer or level is responsible for various functions and includes various components.
- **Physical object layer**
- These include any physical real-world objects includes people, animals, cars, trees, fridges, trains, factories, homes, and in fact anything that is required to be monitored and controlled can be connected to the IoT.
- **Device layer**
- This layer contains things that make up the IoT such as sensors, transducers, actuators, smart phones, smart devices, and **Radio Frequency Identification tags (RFIDs).**

- There can be many categories of sensors such as body sensors, home sensors, and environmental sensors based on the type of work they perform.
- This is the core of an IoT ecosystem where various sensors are used to sense real-world environments.
- This includes sensors that can monitor temperature, humidity, liquid flow, chemicals, air, pressure, and much more.
- An **Analog to Digital Converter (ADC)** is required on a device in order to turn the real-world analog signal into a digital signal that a microprocessor can understand.
- Actuators in this layer provide the means to enable control of external environments, for example, starting a motor or opening a door.
- These components also require digital to analog converters in order to convert a digital signal into analogue.
- This is especially relevant when control of a mechanical component is required by the IoT device.

Network layer

- This layer is composed of various network devices that are used to provide Internet connectivity between devices and to the cloud or servers that are part of the IoT ecosystem.
- These devices can include gateways, routers, hubs, and switches.
- This layer can include two types of communication.
- First is the horizontal means of communication, which includes radio, Bluetooth, WiFi, Ethernet, LAN, ZigBee, and PAN and can be used to provide a communication between IoT devices.
- Second, we have communicating to the next layer, which is usually through the Internet and provides communication between machines and people or other upper layers.
- The first layer can optionally be included in the device layer as it physically is residing on the device layer where devices can communicate with each other at the same layer.

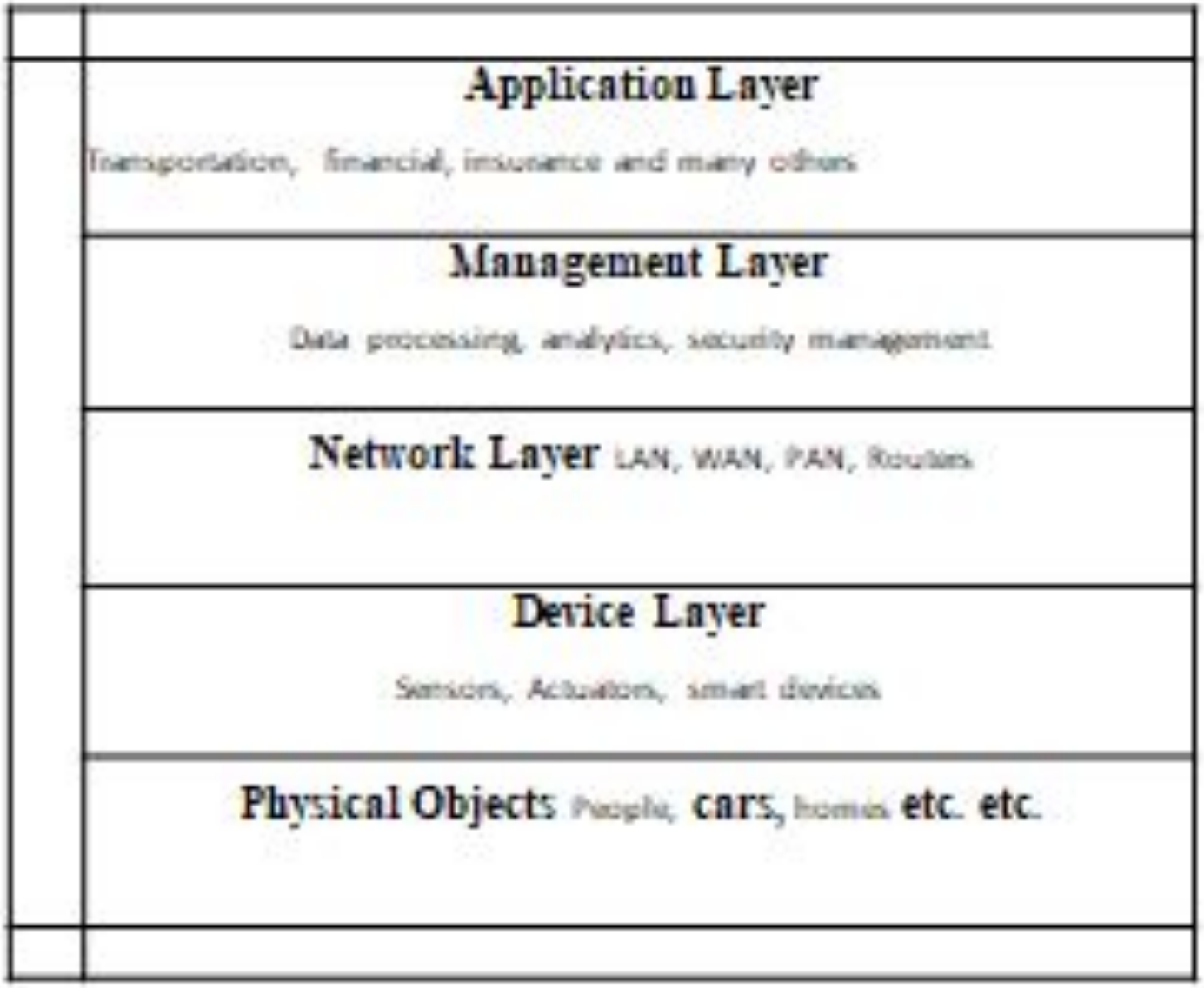
Management layer

- This layer provides the management layer for the IoT ecosystem.
- This includes platforms that enable processing of data gathered from the IoT devices and turn that into meaningful insights.
- Also, device management, security management, and data flow management are included in this layer. It also manages communication between the device and application layers.

Application layer

- This layer includes applications running on top of the IoT network.
- This can include a number of applications depending on the requirements such as transportation, healthcare, financial, insurance, or supply chain management.

IoT five-layer model



- With the availability of cheap sensors, hardware, and bandwidth, IoT has gained popularity in recent years and currently has applications in many different areas including healthcare, insurance, supply chain management, home automation, industrial automation, and infrastructure management.
- Moreover, advancements in technology such as the availability of IPv6, smaller and powerful processors, and better Internet access have also played a vital role in the popularity of IoT.
- The benefits of IoT range from cost saving to enabling businesses to make vital decisions and thus improve performance based on the data provided by the IoT devices.
- Raw data from millions of things (IoT devices) is analyzed and provides meaningful insights that help in making timely and effective business decisions.

- The normal IoT model is based on a centralized paradigm where IoT devices usually connect with a cloud infrastructure or central servers in order to report and process the relevant data back.
- This centralization poses certain possibilities of exploitation including hacking and data theft.
- Moreover, not having control of personal data on a single, centralized service provider also increases the possibility of security and privacy issues.
- Whilst there are methods and techniques to build a highly secure IoT ecosystem based on the normal IoT model there are certain much more desirable benefits that blockchain can bring to IoT.
- A blockchain-based IoT model differs from the traditional IoT network paradigm.
- According to IBM, blockchain for IoT can help to build trust, reduce costs, and accelerate transactions.

- Also the peer-to-peer communication model provided by blockchain can help to reduce costs because there is no need to build high-cost centralized data centres or implementation of complex public key infrastructure for security.
- Devices can communicate with each other directly or via routers.
- As an estimate from various researchers and companies, by 2020 there will be roughly 22 billion devices connected to the Internet.
- With this explosion of billions of devices connecting to the Internet, it is hard to imagine that centralized infrastructures will be able to cope with the high demands of bandwidth, services, and availability without incurring excessive expenditure.
- Blockchain-based IoT will be able to solve scalability, privacy, and reliability issues in the current IoT model.

- Blockchain enables *things* to communicate and transact with each other directly and with the availability of smart contracts negotiation and financial transactions can also occur directly between the devices instead of requiring a middleman, authority, or human intervention.
- For example, if a room in a hotel is vacant, it can rent itself out, negotiate the rent, and can open the door lock for a human who has paid the right amount of funds.
- Another example could be that if a washing machine runs out of detergent, it could order it online after finding the best price and value based on the logic programmed in its smart contract.

- The above mentioned five-layer IoT model can be adapted to a blockchain-based model by adding a blockchain layer on top of the network layer.
- This layer will run smart contracts, and provide security, privacy, integrity, autonomy, scalability, and decentralization services to the IoT ecosystem.
- The management layer consist of only software related to analytics and processing, and security and control can be moved to the blockchain layer.

Blockchain-based IoT model

Application Layer

Transportation, financial, insurance and many others

Management Layer Data processing, analytics Blockchain Layer

Security, P2P (M2M) autonomous transactions, decentralization, smart contracts

Network Layer

LAN, WAN, PAN, Routers

Device Layer

Sensors, Actuators, smart devices Physical Objects People, cars, homes etc. etc.

- All devices are communicating and negotiating with each other without a central command and control entity.
- It can also result in cost saving which is due to easier device management by using a blockchain based decentralised approach.
- The IoT network can be optimized for performance by using blockchain.
- In this case there will be no need to store IoT data centrally for millions of devices because storage and processing requirements can be distributed to all IoT devices on the blockchain.
- This can result in completely removing the need for large data centres for processing and storing the IoT data.

- Blockchain-based IoT can also thwart denial of service attacks where hackers can target a centralized server or data centre more easily but with blockchain's distributed and decentralized nature, such attacks are no longer possible.
- Additionally, if as estimated there will be billions of devices connected to the Internet in the near future, it will become almost impossible to manage security and updates of all those devices from traditional centrally-owned servers.
- Blockchain can provide a solution to this problem by allowing devices to communicate with each other directly in a secure manner and even request firmware and security updates from each other.
- On a blockchain network these communications can be recorded immutably and securely which will provide auditability, integrity, and transparency to the system.
- This is not possible with traditional P2P systems.

- For example, IBM Blue Horizon and IBM Bluemix are IoT platforms supporting blockchain IoT platforms.
- Various start-ups such as Filament have already proposed novel ideas on how to build a decentralised network that enables devices on IoT to transact with each other directly and autonomously driven by smart contracts.
- This IoT device is connected to the Ethereum blockchain and is used to open a door (in this case the door lock is represented by an LED) when the appropriate amount of funds are sent by a user on the blockchain.
- This is a simple example and requires a more rigorously-tested version in order to implement it in production but it demonstrates how an IoT device can be connected, controlled, and responded to in response to certain events on an Ethereum blockchain.

Government

- E-government or electronic government is a paradigm where information and communication technology is used to deliver public services to citizens.
- The concept is not new and has been implemented in various countries around the world but with blockchain a new avenue of exploration has opened up.
- Many governments are researching the possibility of using blockchain technology for managing and delivering public services.
- Transparency, auditability, and integrity are attributes of blockchain that can go a long way in effectively managing various government functions.

Border control

- Automated border control systems have been in use for decades now in order to thwart illegal entry into countries and prevent terrorism and human trafficking.
- Machine-readable travel documents and specifically biometric passports have paved the way for automated border control; however current systems are limited to a certain extent and blockchain technology can provide solutions.
- A **Machine-readable Travel Document (MRTD)** standard is defined in document ICAO 9303 by the **International Civil Aviation Organization (ICAO)** and has been implemented by many countries around the world.

- Each passport contains various security and identity attributes that can be used to identify the owner of the passport and also circumvent attempts at tampering with the passports.
- These include biometric features such as retina scan, finger prints, facial recognition, and standard ICAO specified features including **Machine Readable Zone (MRZ)** and other text attributes that are visible on the first page of the passport.
- One key issue with current border control systems is centralization whereby the systems are controlled by a single entity and the fact that data is not readily shared between law enforcement agencies.
- This makes it difficult to track suspected individuals.

- Another issue is related to the immediate implementation of blacklisting of a travel document, for example, when there is an immediate need to track and control suspected travel documents.
- Currently, there is no mechanism available to immediately blacklist or revoke a suspected passport.
- Blockchain can provide a solution to this problem by maintaining a blacklist in a smart contract which can be updated as required and any changes will be immediately visible to all agencies and border control points thus enabling immediate control over the movement of a suspected travel document.
- It could be argued that traditional mechanisms like PKIs and P2P networks can also be used for this purpose but they do not provide the benefits that a blockchain can provide.
- With blockchain the whole system can be simplified without the requirement of complex networks and PKI setups which will also result in cost reduction.
- Moreover blockchain based systems will provide cryptographically guaranteed immutability which helps with auditing and discourages any fraudulent activity.

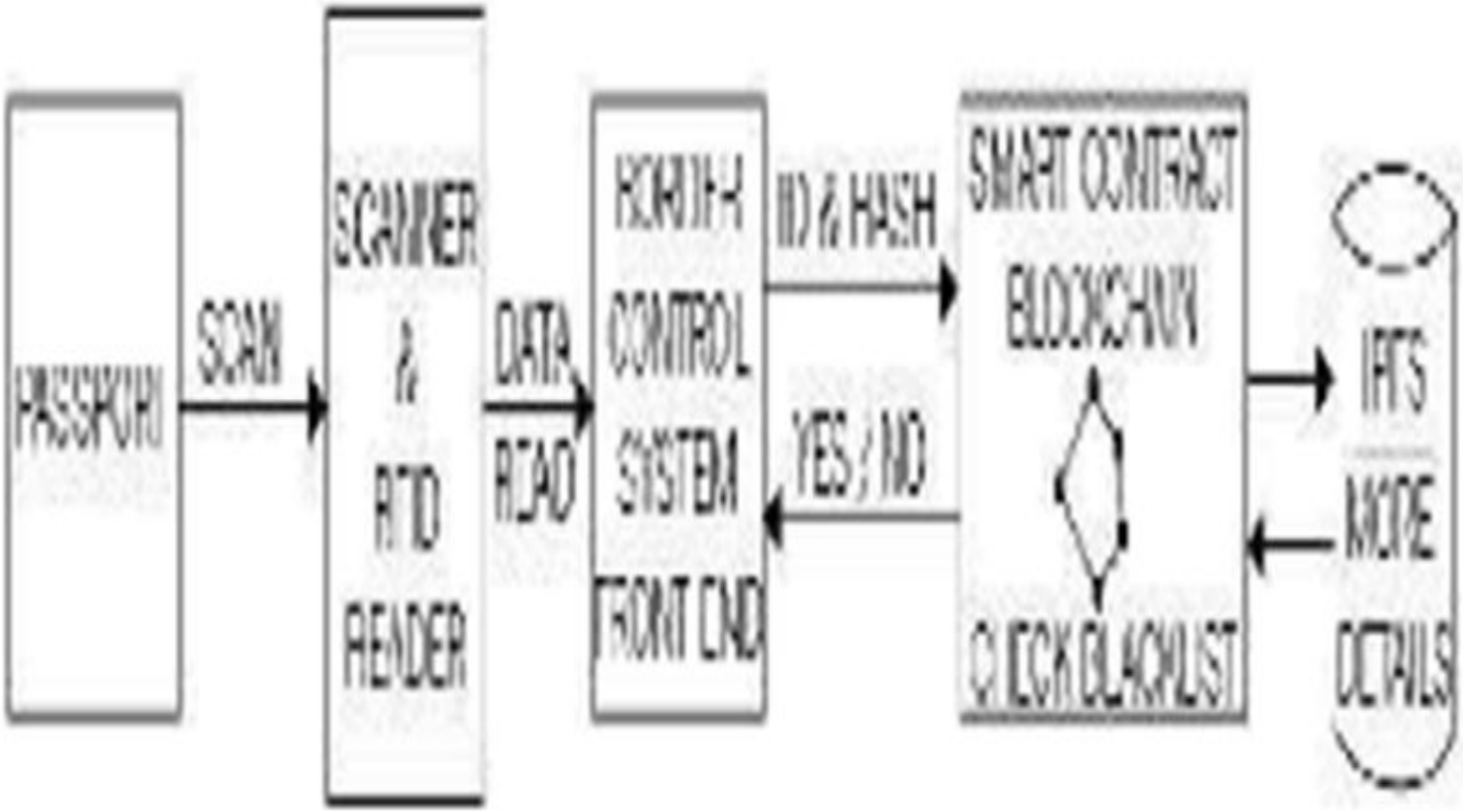
- The full database of all travel documents perhaps cannot be stored on the blockchain currently due to scalability issues but a backend distributed database such as BigChainDB, IPFS, or Swarm can be used for that purpose.
- In this case, a hash of the travel document with the biometric ID of an individual can be stored in a simple smart contract and a hash of the document can then be used to refer to the detailed data available on the distributed file system such as IPFS.
- When a travel document is blacklisted anywhere on the network, that information will be available immediately with the cryptographic guarantee of its authenticity and integrity throughout the distributed ledger.
- This functionality can also provide effective support in antiterrorism activities, thus playing a vital role in the homeland security function of a government.

- A simple contract in solidity can have an array defined for storing identities and associated biometric records.
- This array can be used to store the identifying information about a passport.
- The identity can be a hash of **Machine readable zone (MRZ)** of the passport or travel document concatenated with the biometric record from the RFID chip.
- A simple boolean field can be used to identify blacklisted passports.
- Once this initial check passes, further detailed biometric verification can be performed by traditional systems and eventually when a decision is made regarding the entry of the passport holder that decision can be propagated back to the blockchain, thus enabling all participants on the network to immediately share the outcome of the decision.

- A high-level approach to building a blockchain-based border control system can be visualized as shown in the following figure.
- In this scenario, the passport is presented for scanning to an RFID and page scanner which reads the data page and extracts machine-readable information along with a hash of the biometric data stored in the RFID chip.
- At this stage, a live photo and retina scan of the passport holder is also taken.
- This information is then passed on to the blockchain where a smart contract is responsible for verifying the legitimacy of the travel document by first checking its own list of blacklisted passports and then requesting more data from the backend IPFS database for comparison.

- Note that the biometric data such as photo or retina scan is not stored on the blockchain, instead only a reference to this data in the backend (IPFS or BigChainDB) is stored in the blockchain.
- If the data from the presented passport matches with what is held in the IPFS as files or in BigChainDB and also passes the smart contract logical check then the border gate can be opened.
- After verification this information is propagated throughout the blockchain and is instantly available to all participants on the border control blockchain.
- These participants can be a worldwide consortium of homeland security departments of various nations.

Automated Border Control using Blockchain



Voting

- Voting in any government is a key function and allows citizens to participate in the democratic election process.
- Whilst voting has evolved over time into a much more mature and secure process, it still has limitations that need to be addressed in order to achieve a desired level of maturity.
- Usually, the limitations in current voting systems revolve around fraud, weaknesses in operational processes, and especially transparency.
- Over the years, secure voting mechanisms have been built which make use of specialized voting machines that promised security and privacy but they still had vulnerabilities that could be exploited in order to subvert the security mechanisms of those machines.
- This can lead to serious implications for the whole voting process and can result in mistrust in the government by the public.

- Blockchain-based voting systems can resolve these issues by introducing end-to-end security and transparency in the process.
- Security is provided in the form of integrity and authenticity of votes by using public key cryptography which comes as standard in a blockchain.
- Moreover, immutability guaranteed by blockchain ensures that votes cast once cannot be cast again.
- This can be achieved through a combination of biometric features and a smart contract maintaining a list of votes already cast.
- For example a smart contract can maintain a list of already casted votes with the biometric ID (for example a fingerprint) and can use that to detect and prevent double casting.
- Secondly, zero knowledge proofs can also be used on the blockchain to protect voters' privacy on the blockchain.

Citizen identification (ID cards)

- Electronic IDs or national ID cards are issued by various countries around the world at present.
- These cards are secure and possess many security features that thwart duplication or tampering attempts.
- However, with the advent of blockchain technology there are several improvements that can be made to this process.
- Digital identity is not only limited to just government-issued ID cards, it is a concept that is applicable in online social networks and forums too.
- There can be multiple identities used for different purposes.
- A blockchain-based online digital identity allows control over personal information sharing.

- Users can see who used their data and for what purpose and can control access to it.
- This is not possible with the current infrastructures which are centrally controlled.
- The key benefit is that a single identity issued by the government can be used easily and in a transparent manner for multiple services via a single government blockchain.
- In this case, the blockchain serves as a platform where government is providing various services such as pensions, taxation, or benefits and a single ID is being used for accessing all these services.
- Blockchain in this case provides an immutable record of every change and transaction made by a digital ID, thus ensuring integrity and transparency of the system.

- Also citizens can notarize birth certificates, marriages, deeds, and many other documents on the blockchain tied with their digital ID as a proof of existence.
- Currently, there are successful implementations of identity schemes in various countries that work well and there is an argument that perhaps blockchain is not really required in identity management systems.
- Although, there are several benefits such as privacy and control over the usage of identity information but due to the current immaturity of blockchain technology perhaps it is not ready for use in real-world identity systems.
- However, research is being carried out by various governments to explore the usage of blockchain for identity management.

- Moreover, laws such as the right to be forgotten can be quite difficult to incorporate in to blockchain due to its immutable nature.

Miscellaneous

- Other government functions where blockchain technology can be implemented in order to improve cost and efficiency include collection of taxes, benefits management and disbursement, land ownership record management, life event registration (marriages, births), motor vehicle registration, and licenses.
- This is not an exhaustive list and over time many functions and processes of a government can be adapted to a blockchain-based model.
- The key benefits of blockchain such as immutability, transparency and decentralization can help to bring improvements to most of the traditional government systems.

Health

- The health industry has been identified as another major industry that can benefit by adapting blockchain technology.
- Blockchain provides an immutable, auditable, and transparent system that traditional P2P networks cannot.
- In addition blockchain provides a cost-effective, simpler infrastructure as compared to traditional complex PKI networks.
- In healthcare, major issues such as privacy compromises, data breaches, high costs, and fraud can arise from lack of interoperability, overly complex processes, transparency, auditability, and control.
- Another burning issue is counterfeit medicines; especially in developing countries, this is a major cause of concern.

- With the adaptability of blockchain in the health sector, several benefits can be realized, ranging from cost saving, increased trust, faster processing of claims, high availability, no operational errors due to complexity in the operational procedures, and preventing the distribution of counterfeit medicines.
- From another angle, blockchains that are providing a digital currency as an incentive for mining can be used to provide processing power to solve scientific problems that can help to find cures for certain diseases.
- Examples include FoldingCoin, which rewards its miners with FLDC tokens for sharing their computer's processing power for solving scientific problems that require particularly large calculations.

Finance

- Blockchain has many applications in the finance industry.
- Blockchain in finance is the hottest topic in the industry currently and major banks and financial organizations are researching to find ways to adapt blockchain technology especially due to its highly-desired potential to cost-save.

Insurance

- In the insurance industry, blockchain technology can help to stop fraudulent claims, increase the speed of claim processing, and enable transparency.
- Imagine a shared ledger between all insurers that can provide a quick and efficient mechanism for handling inter-company claims.
- Also with the convergence of IoT and blockchain, an ecosystem of smart devices can be imagined where all these things are able to negotiate and manage their own insurance policies controlled by smart contracts on the blockchain.

- Blockchain can reduce the overall cost and effort required to process claims.
- Claims can be automatically verified and paid via smart contracts and the associated identity of the insurance policy holder.
- For example a smart contract with the help of Oracles and possibly loT can make sure that when the accident occurred, it can record related telemetry data and based on this information can release payment.
- It can also withhold payment if the smart contract after evaluating conditions of payment concludes that payment should not be released.
- For example in a scenario where the vehicle was not repaired by an authorized workshop or was used outside a designated area and so on and so forth.
- There can be many conditions that a smart contract can evaluate to process claims and choice of these rules depend on the insurer, but the general idea is that smart contracts in combination with loT and Oracles can automate the entire vehicle insurance industry.

- Several start-ups such as Dynamis have proposed smart contract-based peer-to-peer insurance platforms that run on Ethereum blockchain.
- This is initially proposed to be used for unemployment insurance and does not require underwriters in the model.

Post trade settlement

- This is the most sought-after application of blockchain technology.
- Currently, many financial institutions are exploring the possibility of using blockchain technology to simplify, automate, and speed up the costly and time-consuming post-trade settlement process.
- A trade lifecycle contains three steps: execution, clearing, and settlement.

- Execution is concerned with the commitment of trading between two parties and can be entered into the system via front office order management terminals or exchanges.
- Clearing is the next step whereby the trade is matched between the seller and buyer based on certain attributes such as price and quantity.
- At this stage, accounts that are involved in payment are also identified. Finally, settlement is where eventually the security is exchanged for payment between the buyer and seller.
- In the traditional trade lifecycle model, a central clearing house is required in order to facilitate trading between parties which bears the credit risk of both parties.

- The current scheme is somewhat complicated, whereby a seller and buyer have to take a complex route in order to trade with each other.
- This comprises of various firms, brokers, clearing houses, and custodians but with blockchain a single distributed ledger with appropriate smart contracts can simplify this whole process and can enable buyers and sellers to talk directly to each other.
- Particularly, the post trade settlement process takes two to three days and has dependency on central clearing houses and reconciliation systems.
- With the shared ledger approach, all participants on the blockchain can immediately see a single version of truth regarding the state of the trade.
- Moreover, peer-to-peer settlement is possible, which results in the reduction of complexity, cost, risk, and the time it takes to settle the trade.
- Finally, intermediaries can be totally eliminated by making use of appropriate smart contracts on the blockchain.

Financial crime prevention

- **Know your customer (KYC)** and **Anti Money laundering (AML)** are the key enablers for the prevention of financial crime.
- In the case of KYC, currently each institution maintains their own copy of customer data and performs verification via centralized data providers.
- This can be a time-consuming process and can result in delays in on-boarding a new client.
- Blockchain can provide a solution to this problem by securely sharing a distributed ledger between all financial institutions that contains verified and accurate identities of customers.
- This distributed ledger can only be updated by consensus between the participants thus providing transparency and auditability.
- This can not only reduce costs but also enable meeting regulatory and compliance requirements in a better and consistent manner.

- In the case of AML, due to the immutable, shared, and transparent nature of blockchain, regulators can easily be granted access to a private blockchain where they can fetch data for relevant regulatory reporting.
- This will also result in reducing complexity and costs related to the current regulatory reporting paradigm where data is fetched from various legacy and disparate systems and aggregated and formatted together for reporting purposes.
- Blockchain can provide a single shared view of all financial transactions in the system that are cryptographically secure, authentic, and auditable, thus reducing the costs and complexity associated with the currently employed regulatory reporting methods.